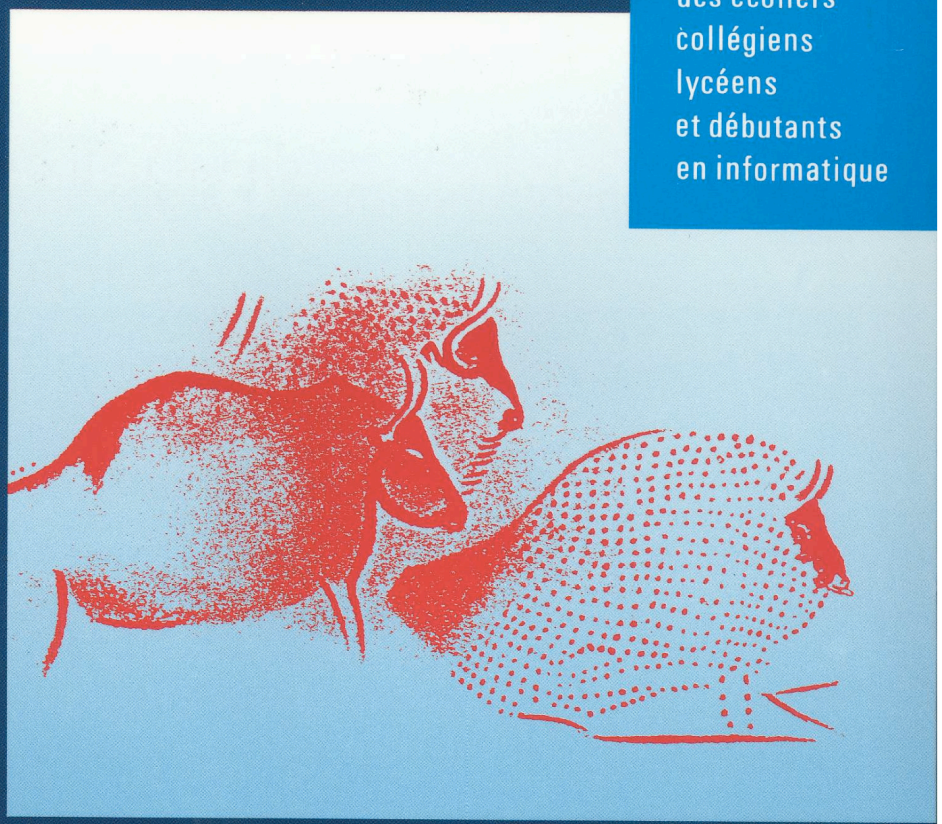
Premières leçons de programmation en Python

Laurent, Patrick, Radhia et Thibault Cousot

A l'usage
des écoliers
collégiens
lycéens
et débutants
en informatique



À Lucie, Maël, Margot, Maxime, et Sacha

Premières leçons de programmation en Python

Laurent, Patrick, Radhia et Thibault Cousot

© Cousot 2026

PREMIÈRES LEÇONS DE PROGRAMMATION EN PYTHON

Laurent COUSOT **Patrick COUSOT**
Radhia COUSOT **Thibault COUSOT**

© 2026

A l'usage des écoliers,
collégiens,
lycéens,
et débutants en informatique.

Table des matières

Introduction **1**

1 Spécification **5**

2 Interaction **13**
2.1 Le robot 13
2.2 Pilotage du robot 15
2.2.1 Position initiale du robot 16
2.2.2 Manœuvrer le crayon du robot 16
2.2.3 Faire pivoter le robot sur place d'un quart de tour . . . 17
2.2.4 Faire pivoter le robot sur place d'un huitième de tour . 17
2.2.5 Faire avancer le robot 18
2.2.6 Faire virer le robot 19
2.2.7 Impossibilité de sortir du cadre 20
2.2.8 Corriger les erreurs de pilotage 20
2.2.9 Terminer et reproduire le dessin 20
2.2.10 Dessiner un point, une croix, la grille des points d'arrêt du robot 21
2.2.11 Dessiner en couleur 21
2.2.12 Déplacer rapidement le robot avec la souris 21

3 Programmation **29**

4 Écriture, interprétation et exécution de programmes **37**
4.1 Structure des programmes 37
4.2 Commentaires 42
4.3 Interprétation des programmes 44
4.4 Erreurs syntaxiques 44

4.5 Exécution des programmes 46
4.6 Erreurs logiques 48
4.7 Comment localiser les erreurs logiques dans le programme . . 49

5 Procédures **57**
5.1 Recopie de texte 57
5.2 Exemples de procédures 60
5.3 Déclaration de procédures 62
5.4 Identificateurs 64
5.5 Appel de procédures 65
5.6 Commandes du robot définies par des procédures 66

6 Sauts du robot au bord et au centre du cadre **71**

7 Boucles “for” **75**
7.1 Répétitions 75
7.2 Le compteur de boucle 76
7.3 La boucle “for” 77
7.4 Utilisation de la boucle “for” dans une procédure 79
7.5 Boucles “for” imbriquées (indépendantes) 80
7.6 Boucles “for” imbriquées (dépendantes) 82
7.7 Invariants de boucles “for” 83

8 Expressions entières **97**

9 Appel de procédures avec paramètres **111**
9.1 Faire avancer et reculer le robot 111
9.2 Épaisseur du crayon 113
9.3 Vitesse du robot 114

10 Grille de déplacement du robot **119**
10.1 Grille carrée 119
10.2 Grille rectangulaire 120

11 Constantes **135**

12 Peinture **141**
12.1 Noir et blanc . . . 141
12.2 Couleur . . . 142

13 Déclarations de procédures avec paramètres **153**

14 Repère cartésien **161**

15 Test “if” **171**

16 Expressions booléennes **181**
16.1 Parité . . . 181
16.2 Comparaison d’entiers . . . 182
16.3 Négation, conjonction et disjonction . . . 184
16.4 Expressions booléennes . . . 186

17 Boucles “while” **195**
17.1 Le robot est-il au bord du cadre ? . . . 195
17.2 Boucle “while” . . . 196

18 Codage **205**
18.1 Code Morse . . . 205
18.2 Numération . . . 206
18.3 Codage binaire . . . 207
18.3.1 Entiers naturels . . . 207
18.3.2 Caractères (code ASCII) . . . 209
18.3.3 Texte . . . 210
18.3.4 Chaînes de caractères . . . 210
18.4 Musique . . . 212
18.4.1 Figures de notes . . . 213
18.4.2 Mouvement . . . 213
18.4.3 Notes . . . 215
18.4.4 Silences . . . 216
18.4.5 Nuances . . . 216

19 Expressions rationnelles **227**
19.1 Nombres rationnels . . . 227
19.2 Paramètres de type “float” . . . 228
19.3 Expressions rationnelles . . . 229

19.4 Erreurs d'arrondi 230

20 Rotations et translations **235**
20.1 Dimensions de la grille de déplacement du robot en coordonnées polaires 235
20.2 Rotations 236
20.3 Translations 238

21 Récursivité **253**
21.1 Récursivité simple 253
21.2 Invariants 258
21.3 Non-terminaison 260
21.4 Récursivité mutuelle 260

22 Variables **279**
22.1 Variables et affectation 279
22.2 Sauvegarde de l'état du robot 280
22.3 Compter 282
22.4 Entrée d'une valeur au clavier 283
22.5 Affichage de la valeur d'une expression 284

Bibliographie **294**

Résumé des commandes du robot **299**

Index **311**

L'usage des programmes contenus dans ce livre et dans le dossier qui les accompagne se fait aux risques et périls des utilisateurs en ce qui concerne leur qualité ou leur fonctionnement. Les auteurs, l'éditeur et les distributeurs déclinent toute responsabilité concernant l'usage des programmes contenus dans ce livre et dans le dossier qui les accompagne. Ils ne donnent aucune garantie explicite ou tacite que ces programmes ne contiennent pas d'erreurs, qu'ils satisfont à de quelconques standards académiques ou commerciaux ou qu'ils sont conformes à de quelconques spécifications requises pour une application quelconque, en particulier celles pouvant entraîner directement ou indirectement une violation des lois d'un pays quelconque. Ils ne pourront en aucun cas être tenus pour responsables des préjudices directs ou indirects, de quelque nature que ce soit, résultant d'une erreur dans les programmes ou le livre, même s'ils ont été avisés de la présence de telles erreurs pouvant entraîner de tels préjudices.

Python est un langage de programmation libre dont le développement est soutenu par la Python Software Foundation, qui détient la marque « Python ». Macintosh© est une marque déposée d'Apple Inc., Windows© est une marque déposée de Microsoft Corporation. La bibliothèque des commandes du Robot est couverte par le droit d'auteur.

Introduction

L'usage de l'ordinateur se généralise dans tous les domaines de l'activité économique. La plupart des utilisateurs emploient des logiciels d'application pour accomplir une ou plusieurs tâches spécifiques : édition de texte, comptabilité, jeux, etc. Par désir de tirer tout le parti de la puissance de la machine, par plaisir ludique ou par simple envie de comprendre le principe de fonctionnement des ordinateurs, de nombreux informaticiens en herbe ont envie d'apprendre, parfois seuls, la programmation. La tâche se révèle souvent longue, difficile voire rébarbative. Nous voudrions en faire une activité créatrice, formatrice pour l'esprit et passionnante.

De nombreuses introductions à la programmation reposent sur la connaissance préalable de l'architecture des ordinateurs ou utilisent des notions mathématiques qui ne sont pas à la portée des enfants ou des adultes n'ayant pas une formation scientifique [10] [1]. (Les petits chiffres surélevés renvoient à une note de bas de page expliquant le mot qui précède. Par exemple, la note qui se trouve en bas de cette page explique l'usage des chiffres entre crochets comme [10].) Nous proposons de procéder autrement.

Nous utilisons le dessin, qui est à la portée de tous. Lors de la première publication du livre en 1991, cette approche était relativement nouvelle dans l'apprentissage de l'informatique, les progrès saisissants de la technologie qui ont permis de mettre à la portée du grand public des micro-ordinateurs dotés d'écrans graphiques très performants ne datant que des années 1980. Il est plaisant de noter que les origines de notre activité graphique remontent à

1. Les scientifiques ont l'habitude de 'citer leurs sources' en donnant les références aux articles de revues scientifiques et aux livres des auteurs dont ils utilisent les idées. La méthode moderne pour citer les sources est d'utiliser un numéro placé entre crochets (comme par exemple [10]) qui renvoie à une liste de références, placée à la fin du livre, que l'on appelle *bibliographie* (voir page 294).

la préhistoire (plus précisément à la période de l'art pariétal [2] entre 22000 et 13000 ans av. J.-C.). L'informatique graphique qui est née pour le grand public dans les années 1980 va donc nous permettre à 20000 ans d'intervalle de dessiner avec le même plaisir que nos lointains ancêtres.

Ce livre s'adresse à tous. Les enfants des classes maternelles sont capables de faire des dessins stylisés à la main et avec un peu d'imagination, de les reproduire sur l'écran de l'ordinateur en utilisant un programme de dessin interactif (chapitres 1 et 2). Les enfants des classes primaires peuvent écrire des programmes pour réaliser des dessins figuratifs (chapitres 3 à 5). Très vite ils découvriront la facilité avec laquelle ils peuvent programmer l'ordinateur pour faire des dessins géométriques simples basés sur des motifs répétitifs qu'ils n'auraient pas la patience ni la minutie d'exécuter à la main (chapitres 6 et 7). Les collégiens mettront en œuvre leurs connaissances rudimentaires d'algèbre et de géométrie analytique pour écrire des programmes de dessin utilisant des calculs élémentaires et des repères cartésiens (chapitres 8 à 17). Ils trouveront une base intuitive pour mettre en œuvre des concepts qui peuvent paraître abstraits. Les lycéens sont à même d'assimiler des notions plus informatiques comme le codage, voire des notions moins élémentaires comme la récursivité, qui deviennent évidentes par le dessin (chapitres 17 à 22). Les étudiants trouveront du plaisir à comprendre simplement la programmation qui paraît parfois obscure quand elle est présentée trop rapidement en premier cycle universitaire.

Les langages de programmation n'offrant pas les possibilités graphiques souhaitables, nous avons ajouté une bibliothèque au langage Python [3] qui nous permet d'obtenir de façon simple des effets simples (comme les arcs de cercles) ou spectaculaires (comme la peinture) sans être liés à la technologie des écrans graphiques actuels et sans faire appel à des notions avancées de programmation comme les structures de données. Cette bibliothèque est dans le répertoire `LeRobot_Python_FR` ainsi que tous les exemples et programmes solutions des exercices proposés dans le livre sont fournis dans le dossier

2. Peinture sur les parois d'une grotte.

3. Python est un langage de programmation conçu par Guido van Rossum aux Pays-Bas au début des années 1990. Il est utilisé aussi bien pour l'enseignement de la programmation que pour écrire de grands logiciels. Nos programmes utilisent Python (version 3) et sa bibliothèque graphique `tkinter` ; nous les avons essayés sur Macintosh, où la fenêtre du $\mathcal{R}^{o}\!\!\not{b}^{o}t$ est une fenêtre ordinaire de macOS.

`LeRobot_Python_FR/livre/` [4]. Elle offre de nombreuses possibilités qui ne sont pas explorées dans le livre, comme par exemple pour les spécialistes, de créer des programmes interactifs d'apprentissage comme ceux qui sont proposés pour l'addition, les coordonnées cartésiennes et la translation.

Ce livre devrait être lu par étapes successives, en prenant le temps et le plaisir de faire de nombreuses expériences personnelles sur ordinateur, en résolvant des exercices, en modifiant les programmes proposés en solution et en recherchant de nouveaux thèmes de dessin.

4. Le répertoire `LeRobot_Manual_Python` contient le manuel d'installation `LeRobot_Install.pdf` et un manuel d'utilisation `LeRobot_Manual.pdf` du $\mathcal{R}^{o}\!\!\smile^{o}t$. Les programmes du livre se trouvent dans le dossier `livre`, rangés par chapitre. Pour exécuter un programme, par exemple `Carre.py`, il suffit de taper la commande `./robot.sh livre/03_programmation/Carre.py` dans une fenêtre de terminal ouverte dans le dossier `LeRobot_Python_FR`. Une autre solution consiste à faire glisser (avec la souris bouton (gauche) enfoncé (ne pas le relâcher) ou le pavé tactile (trackpad en anglais) avec deux doigts) l'icône du fichier contenant le programme sur l'icône du robot `LeRobotPython.app`.

Addenda

Nous donnons en addenda à la fin de chaque chapitre des notes additionnelles destinées aux lecteurs déjà familiers de l'informatique.

Nous introduisons dans ce livre les activités de base de la programmation comme la saisie du texte, l'interprétation, l'exécution et la mise au point de programmes. Les notions de programmation sont présentées en Python : structuration des programmes en procédures (fonctions), itérations bornées, expressions entières, déclarations de constantes, paramètres (passés par valeur), tests, expressions booléennes, itérations non bornées, codages de l'information, expressions rationnelles (flottantes), récursivité, variables et affectations. Notre approche basée sur le dessin permet de donner une intuition graphique à ces notions et évite les problèmes plus complexes de spécification et de correction des programmes (qui sont des évidences visuelles). Les structures de données (listes, tuples, dictionnaires et objets en Python) sont volontairement laissées pour des apprentissages ultérieurs plus élaborés, mais sont illustrés graphiquement.

L'apprentissage de la programmation à un niveau très élémentaire comme celui qui est proposé dans ce livre permet d'acquérir le sens du passage du concret à l'abstrait, l'intuition des structures formelles, le raisonnement inductif et la capacité à formaliser qui sont à la base de la programmation mais également le point de départ de toute démarche scientifique.

Remerciements : Nous remercions nos collègues qui ont utilisé le manuscrit de la première version de ce livre dans les années 1990 pour initier leurs enfants à la programmation et tout particulièrement Guy Cousineau, Michèle Soria, Denis et Marion ; Colette et Jean-Marc Steyaert, Florent et Benoît ; Jean Vuillemin et Thomas.

1

Spécification

Nous allons apprendre à programmer un ordinateur en lui faisant faire des dessins. Pour commencer simplement nous utilisons du papier quadrillé à petits carreaux. La **règle de dessin** à respecter est que *nos dessins ne doivent comporter que des traits droits et des quarts de cercle joignant obligatoirement deux extrémités d'un même côté horizontal, d'un même côté vertical ou d'une même diagonale d'un carreau du quadrillage*. Cette règle de dessin nous offre donc la possibilité de tracer :

- les segments horizontaux _ correspondant aux côtés horizontaux d'un carreau du quadrillage ;
- les segments verticaux | correspondant aux côtés verticaux d'un carreau du quadrillage ;
- les segments obliques / et \ correspondant aux diagonales d'un carreau du quadrillage ;
- les quarts de cercles ⌣ et ⌢ joignant les extrémités d'un même côté horizontal d'un carreau du quadrillage ;
- les quarts de cercles (et) joignant les extrémités d'un même côté vertical d'un carreau du quadrillage ;
- les quarts de cercles ◜ , ◞ , ◝ et ◟ joignant les extrémités d'une même diagonale d'un carreau du quadrillage.

Les petits dessins de la figure 1.1 (page 6) sont des exemples simples tracés en respectant notre règle de dessin sur papier quadrillé. Bien que cette règle ne laisse pas une totale liberté à notre imagination, nous pouvons faire des dessins figuratifs comme ceux du stégosaure ou du château fort de la page 6.

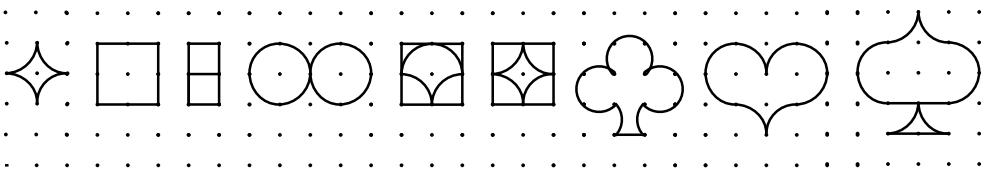


FIG. 1.1 – *Quelques dessins géométriques simples*

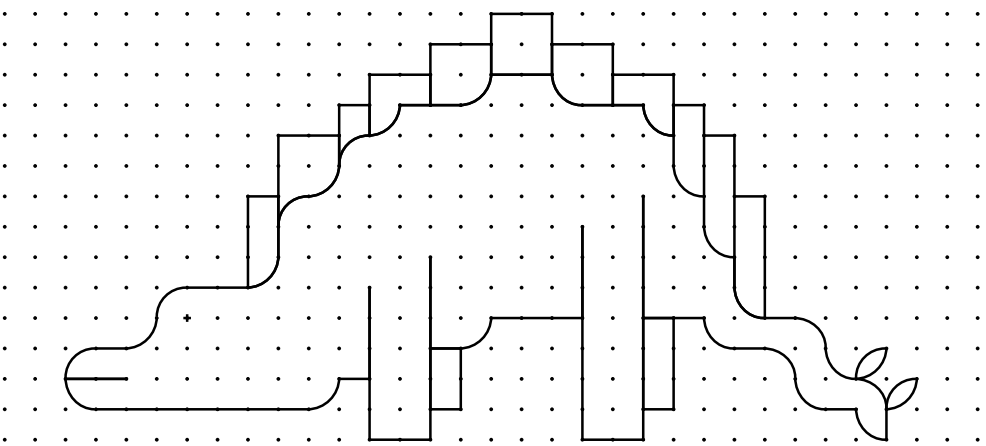


FIG. 1.2 – *Stégosaure*

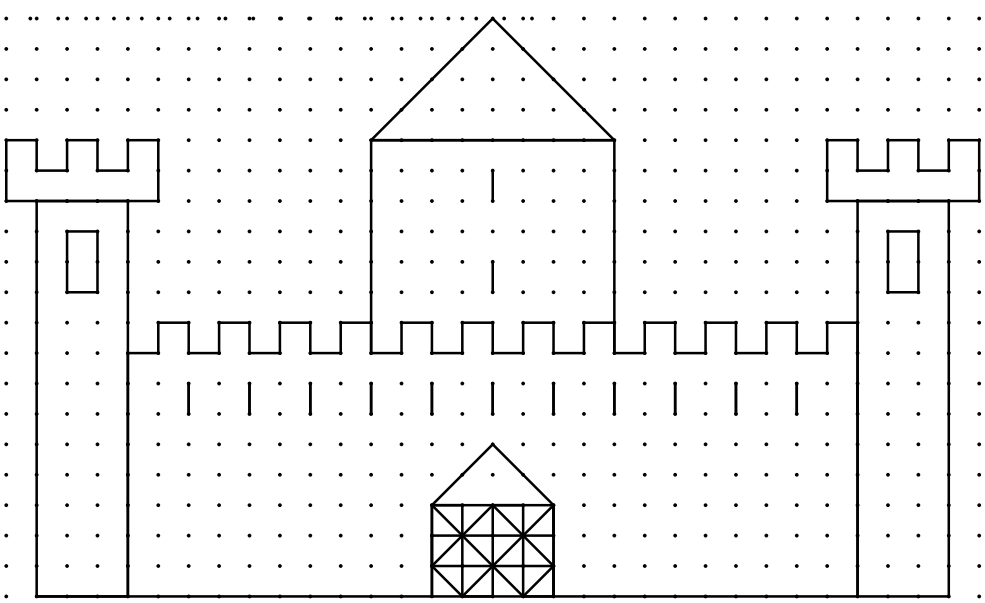


FIG. 1.3 – *Château fort*

Exercice 1

– *Compléter les dessins de la figure 1.4 ci-dessous, en n'utilisant que des traits horizontaux* _, *verticaux* | *ou obliques* ／ *et* ＼ *joignant obligatoirement deux sommets du quadrillage de numéros successifs, sommets qui sont matérialisés sur cette figure par de petits points.*

– *Réaliser des dessins géométriques sur papier quadrillé à petits carreaux respectant la règle de dessin. Voici quelques idées (ces dessins seront ensuite réalisés sur ordinateur) : une marelle, un trapèze, un parallélogramme, un rectangle, un carré, une maison, une fleur stylisée, une cocotte en papier, un bonhomme, la lettre H majuscule en relief, les chiffres d'une montre digitale, l'alphabet (en lettres majuscules), un hélicoptère, un camion, un train, une péniche, etc.* □ [1]

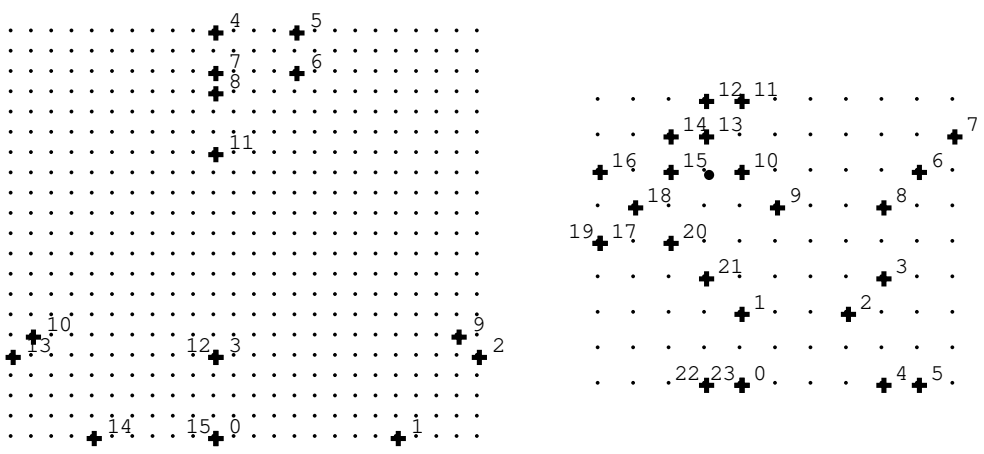


FIG. 1.4 – *Dessins à compléter*

1. La boîte □ marque la fin du texte d'un exemple, d'un exercice, d'un corrigé, etc. Elle permet au lecteur qui ne voudrait pas lire ce texte de trouver rapidement où reprendre pour continuer sa lecture.

Corrigé 1

_ *Une fois complétés, les dessins de la figure 1.4 (page 7) représentent un bateau et un chien.*

_ *Des exemples de dessins sont donnés aux pages 8, 9 et 10.* □ [2]

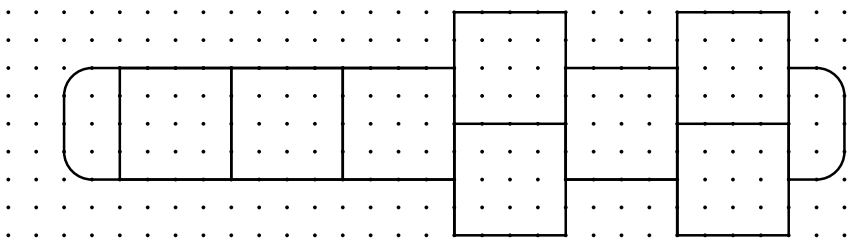


FIG. 1.5 – *Marelle*

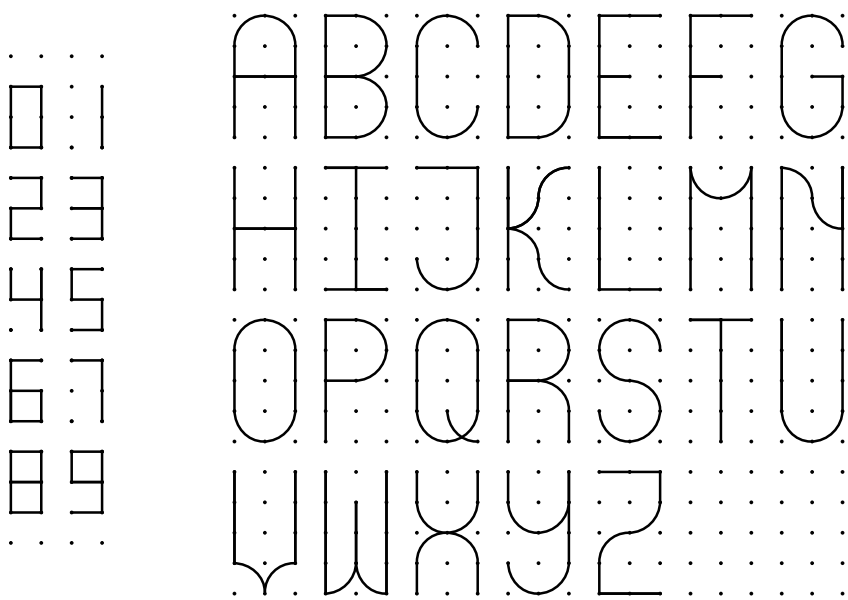


FIG. 1.6 – *Chiffres digitaux et alphabet (en lettres majuscules)*

2. Nous proposons les solutions des exercices quelques pages après en avoir donné les énoncés. Nous conseillons vivement au lecteur de lire ce livre par petites étapes en prenant le soin de faire quelques exercices après chaque chapitre.

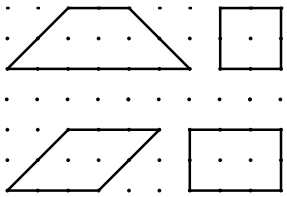


FIG. 1.7 – *Quadrilatères*

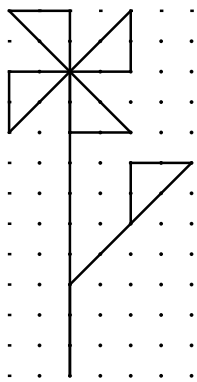


FIG. 1.8 – *Fleur stylisée*

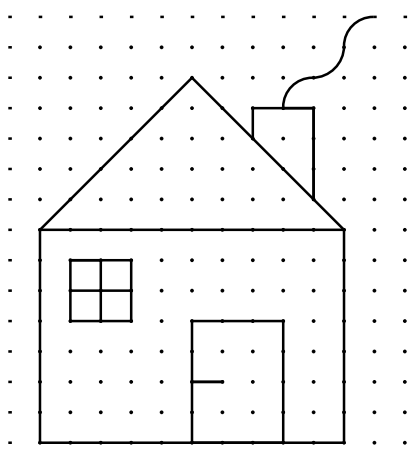


FIG. 1.9 – *Maison*

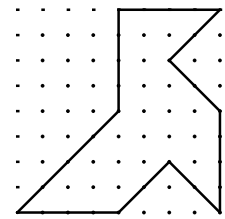


FIG. 1.10 – *Cocotte en papier*

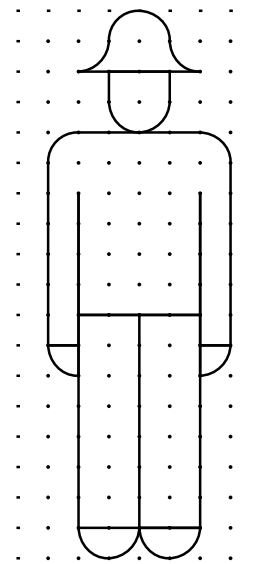


FIG. 1.11 – *Bonhomme*

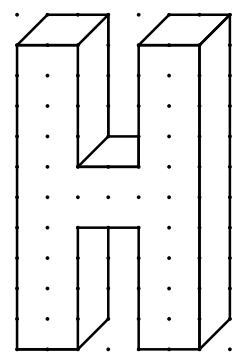


FIG. 1.12 – *Lettre H majuscule en relief*

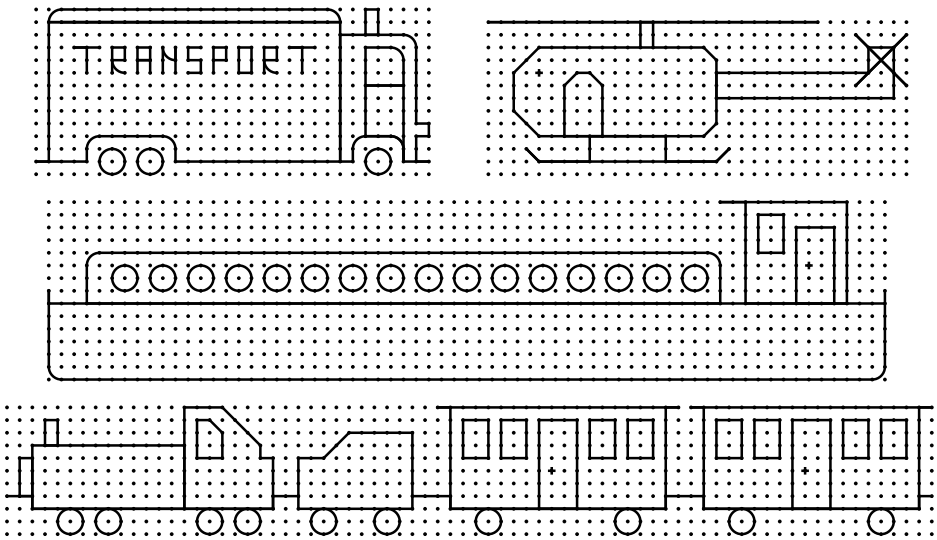


FIG. 1.13 – *Moyens de transport*

Addenda

Avant d'écrire un programme informatique pour résoudre un problème, il faut commencer par spécifier avec précision la tâche que devra accomplir l'ordinateur. Dans notre approche graphique pour expliquer la programmation des ordinateurs, cette *spécification* est un dessin, l'ordinateur devant réaliser ce dessin.

Le dessin à la main en respectant la contrainte de n'utiliser que des traits de la forme _, |, /, \, ◜, ◞, ◝, ◟, ⌣, ⌢, (ou) est difficile pour les très jeunes enfants. Pour assurer une progression du simple au complexe, on commence par leur faire recopier des dessins uniquement constitués de traits horizontaux et verticaux sur du papier à grands carreaux puis on introduit les segments obliques, les quarts de cercles sur les diagonales et enfin les quarts de cercles sur les côtés d'un carré du quadrillage. Ces dessins géométriques constituent une excellente préparation à l'apprentissage de l'écriture. Ensuite, les enfants ont plaisir à inventer leurs propres dessins. Ils doivent alors apprendre à modéliser leurs idées dans le cadre assez strict qui leur est fixé par les règles de dessin. Cette notion de *modèle*, au sens de représentation simplifiée d'un processus ou d'un système complexe est à la base de la *démarche scientifique*.

Ces dessins géométriques construits en n'utilisant que quelques formes primitives constituent également un premier pas vers la programmation qui consiste à construire un édifice parfois complexe à partir des briques élémentaires que sont les instructions du langage de programmation.

Dans le chapitre suivant, il s'agit non plus de faire des dessins à la main mais de les dessiner sur l'écran de l'ordinateur en pilotant un robot avec le clavier de cet ordinateur. Ceci permet de découvrir le fonctionnement de l'ordinateur en *mode interactif* et d'entrer directement dans le monde de l'*infographie*, c'est-à-dire la création d'images sur ordinateur. Par construction, le robot dessine en respectant les règles de dessins qui ont été fixées. Il permet de corriger ses erreurs facilement, de conserver son dessin sur disque, d'en faire des réductions ou des agrandissements et d'en imprimer autant d'exemplaires que l'on désire.

2

Interaction

2.1 Le robot

Pour réaliser nos dessins géométriques sur ordinateur nous allons piloter un petit robot. Son nom est $\mathcal{R}\!\mathit{ob}^{o}t$ [1]. Pour dessiner avec le $\mathcal{R}\!\mathit{ob}^{o}t$, double cliquer sur son icône

LeRobot.app

Notre $\mathcal{R}\!\mathit{ob}^{o}t$ porte un crayon qu'il peut lever ou baisser pendant ses déplacements sur l'écran de l'ordinateur. Quand le $\mathcal{R}\!\mathit{ob}^{o}t$ se déplace avec son crayon baissé, il laisse une trace sur l'écran comme le ferait un escargot sur une feuille. Quand il se déplace avec son crayon levé, il ne laisse aucune trace de son passage. Pour dessiner, le $\mathcal{R}\!\mathit{ob}^{o}t$ doit suivre exactement les lignes du dessin avec son crayon baissé. Il peut également aller d'un point à un autre du dessin qui ne sont pas reliés par un trait en levant son crayon. Pour guider le $\mathcal{R}\!\mathit{ob}^{o}t$ nous utilisons le clavier de l'ordinateur qui sert de télécommande. Nous pouvons donc piloter le $\mathcal{R}\!\mathit{ob}^{o}t$ comme s'il s'agissait d'une petite voiture conduite à distance par radiocommande.

Sur l'écran de l'ordinateur notre $\mathcal{R}\!\mathit{ob}^{o}t$ est représenté par une petite flèche ⬆ qui indique dans quelle direction de la rose des vents il est tourné (voir la figure 2.1, page 14). Il peut s'orienter au nord ⬆, au nord-est ⬈, à l'est ➡, au

1. La calligraphie de son nom ne laisse aucun doute sur son caractère humanoïde et sympathique.

sud-est ⬊, au sud ⬇, au sud-ouest ⬋, à l'ouest ⬅ ou au nord-ouest ⬉. La flèche est pleine quand le *Robot* a baissé son crayon et évidée (⇧, ⇗, ⇨, ⇘, ⇩, ⇙, ⇦ ou ⇖) quand le crayon est levé.

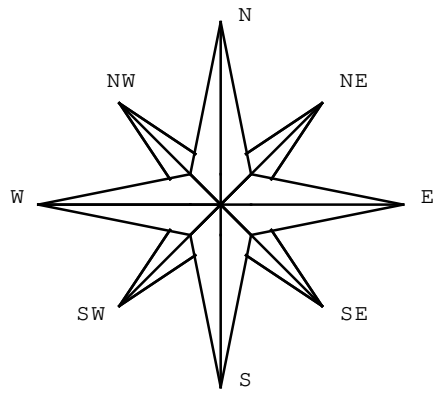


FIG. 2.1 – *Rose des vents à huit directions*

Comme les trains sur leurs rails, notre *Robot* ne peut pas aller n'importe où sur l'écran. Il doit impérativement se déplacer sur les traits ou arcs joignant deux points successifs d'une grille dont les dimensions sont fixées par la taille de l'écran de l'ordinateur. Une petite partie de cette grille est représentée ci-dessous à la figure 2.2.

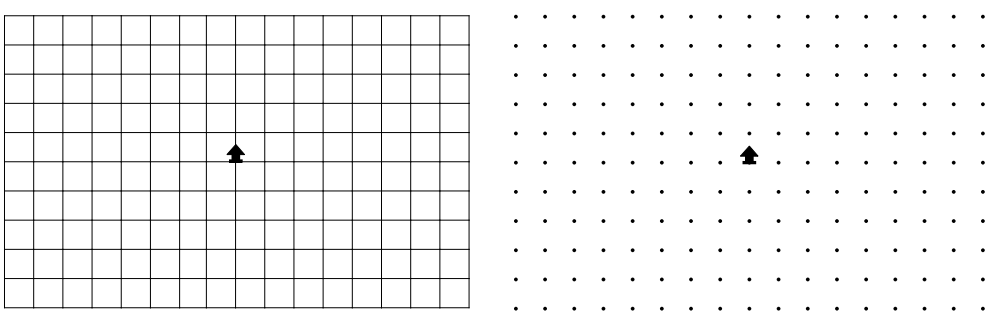


FIG. 2.2 – *Grille et points d'arrêt du Robot*

Cette grille imaginaire correspond au papier quadrillé que nous avons utilisé pour faire nos dessins à la main, sauf que les carreaux ne sont pas visibles sur l'écran de l'ordinateur. Après un déplacement, le *Robot* s'arrête toujours

à l'intersection d'une ligne horizontale et d'une ligne verticale de la grille imaginaire. La figure 2.3 (page 15) montre toutes les trajectoires possibles du *Robot* entre deux points d'arrêt quelconques sur la grille. La figure 2.4

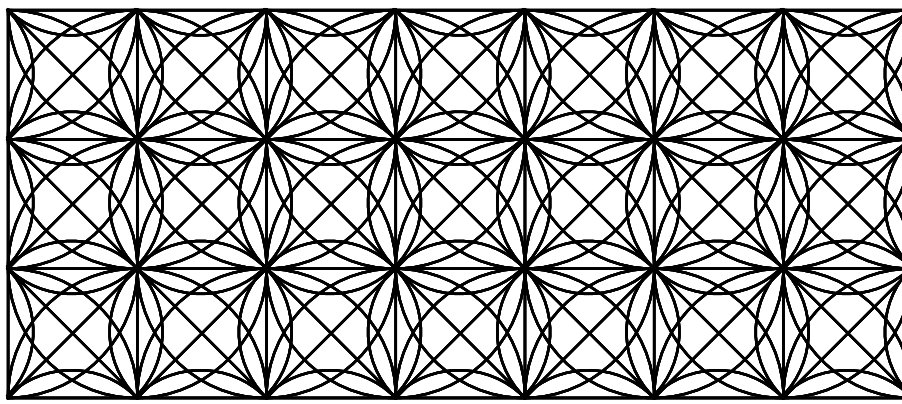


FIG. 2.3 – *Trajectoires du Robot (grossissement* × 4,5*)*

FIG. 2.4 – *Le Robot arrêté (grossissement* × 2,5*)*

(page 15) décrit la position exacte du *Robot* quand il s'arrête sur la grille. Il place la pointe de son crayon située au milieu et à l'arrière de la flèche exactement sur un point d'arrêt de la grille.

2.2 Pilotage du robot

Pour piloter le *Robot* sur l'ordinateur, il faut "*exécuter le programme* `Dessiner`" qui fait partie de la bibliothèque `LeRobot_Python_FR` accompagnant ce livre [2].

2. Il suffit de double cliquer sur l'icône du *Robot* `LeRobotPython.app`. On peut aussi, dans une fenêtre de terminal placée dans le dossier `LeRobot_Python_FR`, taper `python3 Dessiner.py`. Sur le Macintosh, la fenêtre du *Robot* est une fenêtre ordinaire, qui vient au premier plan ; si les touches restent sans effet, cliquer dans cette fenêtre.

2.2.1 Position initiale du robot

Quand l'exécution du programme `Dessiner` commence, l'écran de l'ordinateur se présente comme on le voit en réduction à la figure 2.5 (page 16). Au départ le *Robot* se trouve donc arrêté au milieu de la grille imaginaire,

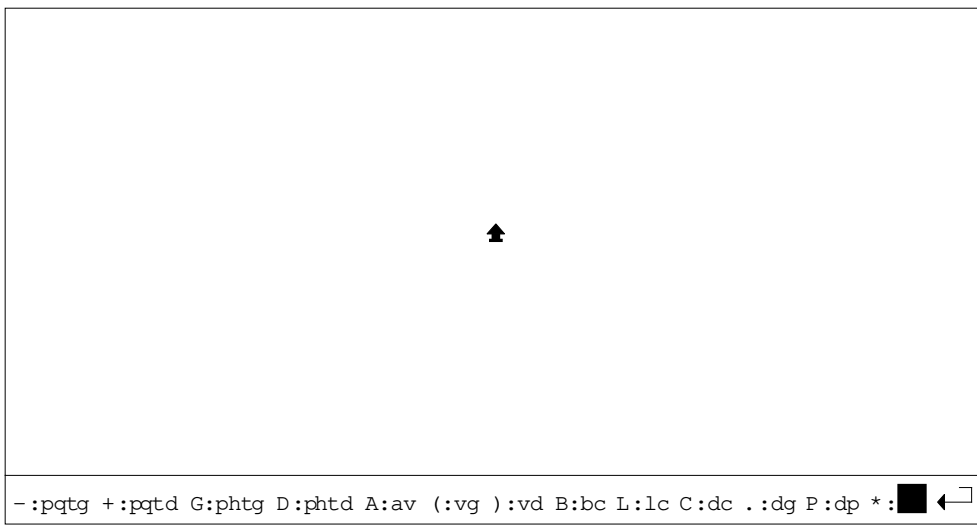


FIG. 2.5 – *Écran de l'ordinateur au début* de l'exécution du programme `Dessiner`

crayon baissé, orienté vers le nord ⬆. Pour piloter le *Robot*, nous utilisons les touches du clavier de l'ordinateur qui sert de télécommande. Nous allons maintenant voir sur quelles touches appuyer pour lever ou baisser le crayon, avancer, pivoter sur place, tourner, etc.

2.2.2 Manœuvrer le crayon du robot

En tapant sur la touche marquée [L] du clavier, le *Robot* lève son crayon ⇑. En tapant sur la touche [B], il le baisse ⬆. Faute de disposer de beaucoup de place sur l'écran de l'ordinateur pour permettre de rappeler en détail le mode d'emploi du *Robot*, nous utilisons des abréviations mnémotechniques [3]

3. Les abréviations mnémotechniques permettent d'aider la mémoire par association d'idées.

placées en bas de l'écran comme on peut le voir à la figure 2.5 ci-dessus. Par exemple :

- `l:lc` signifie "la touche [L] permet de lever le crayon" ;
- `b:bc` signifie "la touche [B] permet de baisser le crayon".

Les lettres se tapent en minuscules, sans appuyer sur la touche majuscules (une lettre majuscule n'est pas une commande du *Robot*).

L'effet des touches [L] et [B] de manœuvre du crayon du *Robot* est résumé dans la table 2.1 (page 17).

[L] (`lc`)	Avant	↑	↖	→	↙	↓	↗	←	↘
	Après	↑	↗	→	↘	↓	↙	←	↖
[B] (`bc`)	Avant	↑	↗	→	↘	↓	↙	←	↖
	Après	↑	↖	→	↙	↓	↗	←	↘

TAB. 2.1 – *Manœuvres du crayon du Robot*

2.2.3 Faire pivoter le robot sur place d'un quart de tour

La touche [-] fait pivoter le *Robot* sur place d'un quart de tour à gauche (`pqtg` en abrégé). Par exemple si le *Robot* est orienté dans la direction nord ↑, la touche [-] le fait pivoter sur place dans la direction ouest ←. S'il est orienté à l'est →, il se place dans la direction nord ↑. La touche [+] fait pivoter le *Robot* sur place d'un quart de tour à droite (`pqtd` en abrégé).

Quand on parle du côté gauche du *Robot*, il faut s'imaginer à la place du *Robot*, le regard tourné dans la direction de la flèche. Par exemple, la gauche de → est le haut de cette page tandis que la gauche de ↓ est à droite de cette feuille !

L'effet des commandes [-] (`pqtg`) et [+] (`pqtd`) est résumé dans la table ci-dessous dans le cas où le crayon est baissé. L'effet est le même quand le crayon est levé.

2.2.4 Faire pivoter le robot sur place d'un huitième de tour

La touche [G] fait pivoter le *Robot* sur place d'un huitième de tour à gauche (`phtg` en abrégé). Par exemple, s'il est dans la direction nord ↑, le

– (pqtg)	Avant	↑	↖	→	↙	↓	↗	←	↘
	Après	←	↘	↑	↖	→	↙	↓	↗
+ (pqtd)	Avant	↑	↖	→	↙	↓	↗	←	↘
	Après	→	↙	↓	↗	←	↘	↑	↖

TAB. 2.2 – *Pivotements du Robot d'un quart de tour sur lui-même*

Robot se place dans la direction nord-ouest ↖. En continuant à taper sur la touche G, le Robot tourne à chaque fois d'un huitième de tour en passant par les positions successives ← puis ↗, ↓, ↙, →, ↖, ...La touche D fait pivoter le Robot sur place d'un huitième de tour à droite (`phtd` en abrégé). L'effet des touches G (`phtg`) et D (`phtd`) est résumé ci-dessous :

G (phtg)	Avant	↑	↖	←	↗	↓	↙	→	↘
	Après	↖	←	↗	↓	↙	→	↘	↑
D (phtd)	Avant	↑	↖	→	↙	↓	↗	←	↘
	Après	↖	→	↙	↓	↗	←	↘	↑

TAB. 2.3 – *Pivotements du Robot d'un huitième de tour sur lui-même*

2.2.5 Faire avancer le robot

La touche A fait avancer le Robot tout droit, de la taille d'un côté d'un carreau du quadrillage. L'abréviation mnémotechnique de cette commande du Robot est `av`. Si le crayon est baissé, ce déplacement permet de tracer un petit trait. Si le crayon est levé, ce déplacement ne laisse aucune trace sur le dessin. Dans les deux cas le Robot avance tout droit dans la direction où il se trouve sans changer d'orientation :

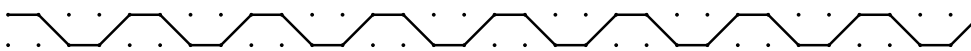
A (av)	Avant	· ↑ ·	· ↖ ·	· → ·	↙ · · ·	· ↓ ·	· · ↗	· ← ·	· · ↘
	Après	↑	↗	→	↘	↓	↙	←	↖

TAB. 2.4 – *Avancées du Robot dans les huit directions*

Exemple 1 (Frise crantée) En tapant successivement sur les touches [A] (`av`), [+] (`pqtd`), [A] (`av`), [+] (`pqtd`), [A] (`av`), [−] (`pqtg`), [A] (`av`), [−] (`pqtg`), [A] (`av`), [+] (`pqtd`), [A] (`av`), [+] (`pqtd`), [A] (`av`), [−] (`pqtg`), [A] (`av`), [−] (`pqtg`), ..., on obtient la frise suivante :

□

Exemple 2 (Frise crénelée) En tapant successivement sur les touches [D] (`phtd`), [D] (`phtd`), [A] (`av`), [D] (`phtd`), [A] (`av`), [G] (`phtg`), [A] (`av`), [G] (`phtg`), [A] (`av`), [D] (`phtd`), [A] (`av`), [D] (`phtd`), [A] (`av`), ..., on obtient la frise suivante :



□

2.2.6 Faire virer le robot

La touche [(] permet de faire virer le $\mathcal{R}^{o}\underline{b}^{o}t$ à gauche (`vg` en abrégé) tandis que la touche [)] permet de le faire virer à droite (`vd`). Ces touches ont été choisies parce que les parenthèses évoquent l'idée de virage et que la parenthèse gauche [(] qui se trouve à gauche sur le clavier correspond au virage à gauche tandis que la parenthèse droite [)] qui correspond au virage à droite se trouve à droite sur le clavier. Si le crayon est baissé, ce déplacement permet de tracer un petit arc de cercle. Si le crayon est levé, ce déplacement ne laisse aucune trace sur le dessin. Dans les deux cas le $\mathcal{R}^{o}\underline{b}^{o}t$ se déplace sur la grille et change d'orientation comme indiqué dans la table ci-dessous :

[(] (`vg`)	Avant								
	Après								
[)] (`vd`)	Avant								
	Après								

TAB. 2.5 – *Virages du $\mathcal{R}^{o}\underline{b}^{o}t$*

Exemple 3 (Frise d'accolades) En tapant successivement sur les touches) (vd), ((vg), + (pqtd), + (pqtd), ((vg),) (vd), + (pqtd), + (pqtd),) (vd), ((vg), + (pqtd), + (pqtd), ..., on obtient la frise suivante :

□

2.2.7 Impossibilité de sortir du cadre

En tapant sur la touche A plusieurs fois de suite le $\mathcal{R}^{o}_{\smile}{}^{o}t$ ▲ avance tout droit en traçant un trait jusqu'à rencontrer le bord du cadre qui délimite son champ d'action. Si l'on continue à taper sur la touche A pour essayer de le faire sortir du cadre c'est une *erreur* en ce sens que le $\mathcal{R}^{\check{o}}_{\smile}{}^{\check{o}}t$ refuse de le faire et émet un « bip-bip » de protestation [4] (il écrit aussi « `** SORTIE INTERDITE **` » dans la fenêtre du terminal) !

2.2.8 Corriger les erreurs de pilotage

Il arrive souvent que l'on fasse des erreurs de pilotage du $\mathcal{R}^{\check{o}}_{\smile}{}^{\check{o}}t$. Par exemple, au lieu de le faire tourner à droite avec une parenthèse droite) (vd), on tape une parenthèse gauche ((vg) et le $\mathcal{R}^{\check{o}}_{\smile}{}^{\check{o}}t$ tourne à gauche ! La touche d'*effacement* marquée d'une flèche orientée à gauche ← ou delete permet de corriger les erreurs en revenant en arrière et en effaçant les derniers tracés dans l'ordre inverse où ils ont été dessinés par le $\mathcal{R}^{o}_{\smile}{}^{o}t$.

2.2.9 Terminer et reproduire le dessin

Pendant l'exécution du programme `Dessiner`, il est possible, à chaque instant, de terminer le dessin en tapant sur la touche de retour à la ligne ↩ ou return, également marquée `Entrée` ou en anglais `Ret` sur certains claviers d'ordinateurs. Ceci est rappelé en bas à droite de l'écran de l'ordinateur par un symbole ↩ qui clignote.

Le dessin qui vient d'être terminé n'est pas perdu, il est possible de le reproduire et de l'imprimer [5].

4. La calligraphie de son nom ne laisse aucun doute sur son mécontentement.
5. Après la touche de retour à la ligne, le $\mathcal{R}^{o}_{\smile}{}^{o}t$ demande dans la ligne du bas de

2.2.10 Dessiner un point, une croix, la grille des points d'arrêt du robot

La touche P (`dp`) permet de dessiner un point, sans déplacer le $\mathcal{R}$obot. On utilise la touche C (`dc`) pour dessiner une petite croix `+`. La touche . (`dg`) permet de dessiner la grille des points d'arrêt du $\mathcal{R}$obot comme on la voit par exemple sur la figure 1.13 (page 10).

2.2.11 Dessiner en couleur

Le $\mathcal{R}$obot a la possibilité de dessiner avec des crayons de différentes couleurs. Pendant l'exécution du programme `Dessiner`, la couleur du crayon utilisé par le $\mathcal{R}$obot figure dans un petit carré placé en bas à droite de l'écran (voir la figure 2.5 page 16). Pour changer la couleur du crayon, il suffit de taper une ou plusieurs fois sur la touche * pour faire défiler toutes les couleurs possibles les unes après les autres.

Pour peindre une zone fermée par un trait, on place le $\mathcal{R}$obot de sorte que son arrière soit dans la zone, puis on tape sur la touche W (`peindre`) : la zone prend la couleur du crayon (voir le chapitre 12).

2.2.12 Déplacer rapidement le robot avec la souris

Pour déplacer le $\mathcal{R}$obot sans rien dessiner, on peut lever le crayon L (`lc`) puis avancer une ou plusieurs fois A (`av`) en changeant éventuellement de direction (D (`phtd`) ou G (`phtg`)) et enfin baisser le crayon B (`bc`). Pour aller plus vite, on peut utiliser la souris pour déplacer le $\mathcal{R}$obot sur un point quelconque de la grille des points d'arrêt sans rien tracer : il suffit de cliquer sur ce point (ou d'y amener la souris bouton enfoncé), ou taper sur les flèches.

---

l'écran `Voulez-vous sauver dans mon_dessin.py ? (y/n)`. En tapant sur la touche Y, le dessin est enregistré sous la forme d'un programme Python dans le fichier `mon_dessin.py`, qui commence par **from** `LeRobot` **import** `*` ; on le reproduit en tapant `make mon_dessin` (ou `python3 mon_dessin.py`) dans le dossier `LeRobot_Python_FR`. Le prochain dessin sauvé remplacera ce fichier : pour garder définitivement le fichier, il faut changer son nom puis l'exécuter en le faisant glisser sur l'application `LeRobotPython.app`.

Résumé des commandes interactives de pilotage du robot

Les commandes de pilotage du $\mathcal{R}\text{o}\mathcal{b}\text{o}t$ pour dessiner interactivement avec le programme `Dessiner` sont résumées dans la table 2.6 (page 23).

Touche :	*Commande :*	*Effet :*
Grille des points d'arrêt du $\mathcal{R}$obot :		
.	`dg`	dessine la grille des points d'arrêt du $\mathcal{R}$obot.
Pivotements du $\mathcal{R}$obot sur place :		
D	`phtd`	fait pivoter le $\mathcal{R}$obot sur place d'un huitième de tour à droite.
+	`pqtd`	fait pivoter le $\mathcal{R}$obot sur place d'un quart de tour à droite.
G	`phtg`	fait pivoter le $\mathcal{R}$obot sur place d'un huitième de tour à gauche.
–	`pqtg`	fait pivoter le $\mathcal{R}$obot sur place d'un quart de tour à gauche.
Déplacements du $\mathcal{R}$obot :		
A	`av`	fait avancer le $\mathcal{R}$obot tout droit jusqu'au prochain point d'arrêt du quadrillage.
)	`vd`	fait tourner le $\mathcal{R}$obot par un virage d'un quart de cercle à droite.
(	`vg`	fait tourner le $\mathcal{R}$obot par un virage d'un quart de cercle à gauche.
← → ↑ ↓		déplace le $\mathcal{R}$obot d'un point d'arrêt, sans tracer.
Manœuvres du crayon du $\mathcal{R}$obot :		
L	`lc`	lève le crayon du $\mathcal{R}$obot, pour éviter de dessiner lors de ses déplacements.
B	`bc`	baisse le crayon du $\mathcal{R}$obot, pour dessiner lors de ses déplacements.
P	`dp`	dessine un point, sans déplacer le $\mathcal{R}$obot.
C	`dc`	dessine une croix, sans déplacer le $\mathcal{R}$obot.
Couleur et peinture :		
*	`cc`	change la couleur du crayon (couleur suivante).
W	`peindre`	peint la zone fermée derrière le $\mathcal{R}$obot avec la couleur du crayon.
Erreurs de pilotage du $\mathcal{R}$obot :		
← ou delete		efface, dans l'ordre inverse, les derniers tracés du $\mathcal{R}$obot.
Arrêt du $\mathcal{R}$obot :		
↩ ou return		termine le dessin ; la touche Y le sauve alors dans `mon_dessin.py` (il peut ensuite être reproduit et imprimé).

Exercice 2 *Supposons que le Robot soit initialement orienté vers le nord avec son crayon baissé ⬆. Parmi les chiffres d'une montre digitale représentés à la figure 1.6 (page 8), quel est celui obtenu en donnant au robot la* séquence de commandes[6] *ci-dessous :*

```
pqtd() ; av() ; pqtg() ; av() ; av() ; pqtg()
av() ; pqtg() ; av() ; pqtg() ; av()
```

c'est-à-dire en tapant successivement :

+ A − A A − A − A − A ? □

Exercice 3 *Quelle lettre majuscule de l'alphabet de la figure 1.6 (page 8) obtient-on en donnant au Robot ⬆ la séquence de commandes suivante :*

```
av() ; av() ; pqtd() ; av() ; pqtd() ; pqtd() ; av()
pqtd() ; av() ; av() ; pqtd() ; av() ; av()
```

c'est-à-dire en tapant :

A A + A + + A + A A + A A ? □

Exercice 4 *Quelle est la suite de commandes à donner au Robot ⬆ pour qu'il dessine la lettre* `T` *majuscule représentée à la figure 1.6 (page 8) ?* □

Exercice 5 *Quelle est la suite de commandes à donner au Robot ⬆ pour qu'il dessine le chiffre 4 représenté à la figure 1.6 (page 8) ?* □

Exercice 6 *Quelle est la séquence de commandes à donner au Robot pour dessiner l'octogone de la figure 2.6 ci-dessous ?* □

Exercice 7 *Quelle est la séquence de commandes à donner au Robot pour dessiner les trois cercles tangents de la figure 2.6 ci-dessous ?* □

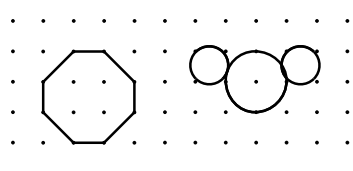


FIG. 2.6 – *Octogone et cercles tangents de tailles différentes*

6. Une séquence de commandes est une succession ou suite de commandes données dans un certain ordre.

Corrigé 2 *En essayant sur l'ordinateur on trouve le chiffre neuf.* □

Corrigé 3 *La lettre F.* □

Corrigé 4 *La lettre T majuscule de la figure 1.6 (page 8) est obtenue en donnant successivement au* Robot ♠ *les commandes suivantes :*

```
av() ; av() ; av() ; av() ; pqtd() ; av()
pqtd() ; pqtd() ; av() ; av()
```

c'est-à-dire en tapant successivement :

A A A A + A + + A A. □

Corrigé 5 *Le chiffre 4 d'une montre digitale est obtenu en donnant au* Robot ♠ *la séquence de commandes suivante :*

```
av() ; av() ; pqtd() ; pqtd() ; av()
pqtd() ; av() ; pqtd() ; av()
```

c'est-à-dire en tapant successivement :

A A + + A + A + A. □

Corrigé 6 *L'octogone de la figure 2.6 (page 24), est obtenu en donnant au* Robot *la séquence de commandes ci-dessous :*

```
av() ; phtd() ; av() ; phtd() ; av() ; phtd() ; av() ; phtd()
av() ; phtd() ; av() ; phtd() ; av() ; phtd() ; av()
```

c'est-à-dire en tapant successivement :

A D A D A D A D A D A D A D A. □

Corrigé 7 *Les cercles tangents de la figure 2.6 (page 24), sont obtenus en donnant au* Robot *la séquence de commandes ci-dessous :*

```
phtd()
vd() ; vd() ; vd() ; vd() ; vd() ; vd()
phtg()
vg() ; vg() ; vg() ; vg() ; vg() ; vg()
phtg()
vd() ; vd() ; vd() ; vd()
```

c'est-à-dire en tapant successivement :

D
))))))
G
((((((
G
)))) . □

Addenda

Dans ce chapitre nous avons appris à piloter le $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ de manière interactive. Le *mode interactif* est le mode de fonctionnement de nombreux programmes d'ordinateur que l'on appelle des *interpréteurs*. Ces programmes engagent un dialogue avec l'utilisateur en utilisant un langage de communication que l'on appelle le *langage de commande*. L'utilisateur dirige le travail de l'interpréteur en lui donnant des commandes au moyen du clavier de l'ordinateur, de la souris, etc. Les exemples sont très nombreux. Citons les éditeurs de texte, les tableurs, le 'Finder' du Macintosh ou l'interpréteur de commandes des systèmes d'exploitation [a] (comme Microsoft `Windows` sur PC ou Unix sur les stations de travail scientifiques), les systèmes transactionnels d'interrogation de bases de données (comme celles qui sont interrogées par minitel). L'avantage évident de cette interaction avec l'ordinateur est que l'utilisateur peut corriger ses erreurs dès qu'elles se produisent. L'inconvénient majeur est que les tâches répétitives deviennent très vite fastidieuses.

Pour notre $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$, le langage de commande se réduit à quelques commandes très simples données au clavier (A, B, C, D, G, L, P, +, −, (,), ., *, W, ←↩ ou return) sans oublier la souris qui est un moyen très rapide pour désigner un point précis sur l'écran. Le $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ ne communique avec l'utilisateur que par des « bips-bips » lui signalant qu'il ne doit pas le faire sortir du cadre. Les erreurs de dessin peuvent être corrigées par la touche d'effacement ← ou delete qui annule l'effet des dernières commandes données au $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$.

Pendant le dessin interactif, le $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ engendre un programme Python (le fichier `mon_dessin.py`) qui peut ensuite être exécuté pour reproduire et imprimer le dessin. Ceci peut être comparé aux robots de peinture de l'industrie automobile. Un ouvrier spécialisé peint une voiture au pistolet. Tous ses mouvements sont enregistrés sur ordinateur puis utilisés pour engendrer un programme. En exécutant ce programme, l'ordinateur peut ensuite commander un robot disposant d'un bras manipulateur portant un pistolet pour reproduire fidèlement tous ces mouvements et peindre indéfiniment des voitures. Il s'agit d'un exemple rudimentaire d'*apprentissage*, un thème de recherche actif en *intelligence artificielle* où l'ordinateur engendre son propre programme pour réaliser certaines tâches à partir d'exemples qui lui sont fournis en plus ou moins grand nombre.

[a] Les systèmes d'exploitation sont des programmes qui gèrent l'organisation interne et l'affectation des ressources des ordinateurs.

Le dessin interactif sur ordinateur avec le $\mathcal{R}^{o}_{b}{}^{o}t$ n'est pas plus difficile et est certainement plus formateur que n'importe quel jeu vidéo. Il apprend aux enfants à utiliser un *formalisme* simple. Il demande un effort d'imagination pour s'orienter dans le plan puisque les commandes ne sont pas relatives à l'observateur mais au $\mathcal{R}^{o}_{b}{}^{o}t$. Cependant la tâche devient vite pénible pour les dessins géométriques répétitifs. Seule la programmation, que nous abordons dans le prochain chapitre permet d'obtenir de tels dessins sans peine.

3

Programmation

Jusqu'à présent nous avons utilisé l'ordinateur pour piloter le $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ en mode *interactif* ou *conversationnel*. Dans ce mode de fonctionnement nous tapons sur une touche du clavier de l'ordinateur, qui transmet une commande au $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ que celui-ci exécute immédiatement à moins que par erreur nous tentions de le faire sortir du cadre auquel cas il répond par un « bip-bip ». Nous recommençons ainsi, en corrigeant les fautes de pilotage au fur et à mesure que le dessin se construit, jusqu'à ce que nous indiquions à l'ordinateur que le dessin est terminé.

Alors que la commande interactive du $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ correspond à l'exécution immédiate d'une tâche, la commande programmée du $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ consiste à séparer la description de la tâche de ses exécutions ultérieures.

Plus généralement, l'idée de la *programmation* est la suivante : pour faire quelque chose, le *programmeur* écrit un *programme* (dans un certain *langage*) qui décrit avec précision comment effectuer cette chose. Ensuite il demande à un *ordinateur* d'exécuter le programme, en suivant exactement les instructions données par le programme. Autrement dit, au lieu de faire la chose lui-même tout de suite, le programmeur explique à un ordinateur comment le faire à sa place plus tard ! On évite ainsi d'avoir à réaliser des tâches longues et répétitives, à condition de savoir décrire ces tâches convenablement.

S'il s'est trompé, le programmeur doit corriger ses fautes en modifiant le programme. C'est ce qu'on appelle *déboguer* le programme. Le *déboguage* du programme consiste à trouver puis à éliminer les *bogues* c'est-à-dire les erreurs se manisfestant à l'exécution par des anomalies de fonctionnement (comme par exemple `pqtg()` au lieu de `pqtd()` ou `st()` oublié à la fin du

programme) [1]. Le principe de fonctionnement des ordinateurs commandé par un programme a des analogies avec de nombreuses activités humaines comme le montre la table ci-dessous :

Programmeur	*Programme*	*Ordinateur*	*Dessin*
Chef-cuisinier	Recette de cuisine	Cuisiniers	Repas
Compositeur	Œuvre musicale	Musiciens	Concert
Auteur	Texte d'une pièce de théâtre	Acteurs	Représentation théâtrale
Chorégraphe	Chorégraphie	Danseurs	Ballet
Réalisateur	Film	Projecteur cinématographique	Séance de cinéma
Tisserand	Cartes perforées	Métier à tisser jacquard	Tissu jacquard
Chef de piste	Plan du parcours	Cavalier et cheval	Parcours de saut d'obstacles

TAB. 3.1 – *Exemples d'activités programmées*

Nous allons maintenant apprendre à programmer les ordinateurs en Python. Le pilotage du $\mathcal{R}^{o}\!\!\smile^{o}t$ pour faire des dessins sera notre principale application. Pour faire nos dessins dans ce mode de fonctionnement programmé, nous procédons comme suit :

1. Nous commençons par écrire un programme Python où nous indiquons quelle est la séquence de commandes que le $\mathcal{R}^{o}\!\!\smile^{o}t$ devra exécuter pour réaliser le dessin que nous avons en tête ou que nous avons préalablement dessiné sur une feuille de papier quadrillé. Ce programme est écrit avec l'aide de l'ordinateur (avec un éditeur de texte) qui le conserve sur disque dans un fichier dont le nom se termine par `.py`, par exemple `Carre.py`.
2. Ensuite nous demandons à un programme de l'ordinateur, l'*interpréteur* Python, de lire ce programme, de vérifier qu'il est correctement

1. Nous utilisons le vocabulaire recommandé par la commission ministérielle de terminologie [7]. La bogue est la traduction homonymique en français du mot américain *bug* qui désigne les petits insectes nuisibles et plus familièrement les microbes et autres virus aussi bien que les petites choses qui clochent comme les erreurs dans les programmes. La bogue désignant en français l'enveloppe piquante de la châtaigne, l'idée de désagrément est conservée. L'enveloppe fait également penser à la coquille typographique. La traduction ne garde pas l'idée de petit grain de sable aux conséquences dramatiques du mot américain.

écrit et de le traduire dans le *langage machine* de l'ordinateur (ou dans une forme qui en est très proche). On dit que l'interpréteur *interprète* le programme. Le programme traduit et le programme Python original sont équivalents en ce sens qu'ils décrivent exactement la même séquence de commandes pour le $\mathcal{R}^{o}_{b}{}^{o}t$. Ils sont simplement écrits dans des langages différents.

3. Enfin l'interpréteur fait *exécuter* le programme traduit par l'ordinateur. Ce faisant, l'ordinateur pilote le $\mathcal{R}^{o}_{b}{}^{o}t$ en lui demandant d'exécuter les commandes dans l'ordre où nous les avons inscrites dans le programme Python original.

Les deux dernières étapes se font avec une seule commande, tapée dans une fenêtre de terminal (voir le chapitre suivant) : `./robot.sh Carre.py` fait interpréter le programme `Carre.py`, c'est-à-dire le fait vérifier, traduire puis exécuter. On peut aussi faire glisser l'icône du fichier `Carre.py` sur celle de l'application `LeRobotPython.app`.

Exemple 4 (Carré) Soit à dessiner le carré ci-contre. Le programme Python sera le suivant :

```
from LibRobot import *

# Dessiner un carré
av(); av(); pqtd(); av(); av(); pqtd()
av(); av(); pqtd(); av(); av(); pqtd()
st()
```

Quand il exécute ce programme, l'ordinateur transmet au $\mathcal{R}^{o}_{b}{}^{o}t$ l'ordre d'exécuter la séquence de commandes `av()` puis `av()` puis `pqtd()` puis `av()` ... et enfin `pqtd()` exactement comme si nous avions tapé au clavier [A] puis successivement [A], [+], [A], [A], [+], [A], [A], [+], [A], [A] et enfin [+]. La dernière commande `st()` indique à l'ordinateur qu'il doit attendre que nous tapions sur la touche d'échappement [Esc] pour terminer l'exécution du programme (une autre touche fait redessiner le dessin). Ceci nous laisse le temps de contempler à loisir le dessin du $\mathcal{R}^{o}_{b}{}^{o}t$ avant qu'il ne disparaisse. Le dessin n'est pas perdu puisqu'il suffit d'exécuter à nouveau le programme pour le reproduire. □

On constate sur ce premier exemple que le langage Python, bien que conçu aux Pays-Bas par Guido van Rossum au Cwi (Centre de mathématiques et d'informatique d'Amsterdam), utilise des mots anglais comme '**from**' (qui signifie 'de', 'depuis') et '**import**' (qui signifie 'importer') : la première ligne du

programme importe depuis la bibliothèque `LibRobot` toutes ses commandes, ce que désigne l'étoile '`*`'. L'informatique conserve les traces de son origine anglo-saxonne !

Pour l'instant il nous suffit de savoir que l'ordinateur qui exécute ce programme demande au *Robot* d'exécuter les commandes figurant après la ligne '**from** `LibRobot` **import** `*`', les unes après les autres, ligne après ligne, et de gauche à droite sur chaque ligne. Chaque commande est suivie de `()`, qui veut dire qu'elle ne demande aucune précision supplémentaire pour être exécutée. Les points-virgules qui séparent les commandes d'une même ligne peuvent être compris comme signifiant 'puis'. Il n'y a pas de point-virgule à la fin d'une ligne : le passage à la ligne suivante suffit à dire 'puis'. La ligne qui commence par le signe '`#`' est un commentaire, que l'ordinateur ignore (nous y reviendrons au chapitre suivant).

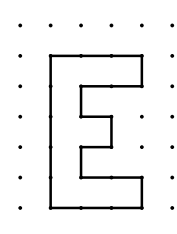
Exercice 8 *Exécuter le programme* `Dessiner` *de commande interactive du Robot et lui donner les ordres décrits par le programme Python suivant (la correspondance entre les touches du clavier et les commandes du Robot est donnée par la table 2.6 page 23) :*

```
from LibRobot import *

phtd(); av(); av(); av(); av()
phtg(); av(); av(); av(); av()
pqtd(); av(); av(); av(); av()
pqtd(); phtd(); av(); av(); pqtg(); av(); av()
phtd(); av(); av(); av(); av()
pqtd(); phtd(); av(); av(); pqtg(); av(); av()
phtd(); av(); av(); av(); av()
st()
```

Quel dessin avez-vous demandé au Robot de réaliser ? □

Exercice 9 *Nous voudrions que le programme ci-dessous dessine la lettre* `E` *majuscule ci-contre :*



```
from LibRobot import *

# Dessiner une lettre E majuscule épaisse
av(); av(); av(); av(); av(); pqtd()
av(); av(); av(); pqtd(); av(); pqtd()
av(); av(); pqtd()
av(); pqtg()
av(); pqtd(); av(); pqtd(); av(); pqtg()
av(); pqtg()
```

```
av(); av(); pqtd(); av(); pqtd()
av(); av()
st()
```

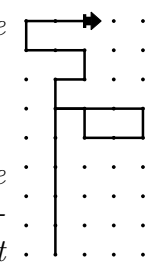
Ce programme erroné contient deux bogues. Pouvez-vous les trouver ? □

Exercice 10 *Exécuter le programme Python* `Stegosaure` *figurant dans le dossier* `livre` *qui accompagne ce livre.* □

Corrigé 8 *Une cocotte en papier.* □

Corrigé 9 *Le programme* `LettreEavecBogues` *de la page 32 contient deux bogues :*

1. *la commande* `pqtd()` *de la troisième ligne devrait être une commande* `pqtg()`,
2. *il manque une commande* `av()` *à l'avant-dernière ligne.*



Le résultat de l'exécution de ce programme est reproduit ci-contre (le Robot étant placé dans sa position finale). Le programme débogué qui dessine correctement la lettre `E` *épaisse est le suivant (les corrections sont soulignées) :*

```
from LibRobot import *

# Dessiner une lettre E majuscule épaisse
av(); av(); av(); av(); av(); pqtd()
av(); av(); av(); pqtd(); av(); pqtd()
av(); av(); pqtg()
av(); pqtg()
av(); pqtd(); av(); pqtd(); av(); pqtg()
av(); pqtg()
av(); av(); pqtd(); av(); pqtd()
av(); av(); av()
st()
```

□

Corrigé 10 *Le programme Python* `Stegosaure` *est rangé dans le fichier* `Stegosaure.py` *du dossier* `livre/04_ecriture_compil_execution`. *Dans une fenêtre de terminal, placée dans le dossier* `LeRobot_Python_FR`, *il suffit de taper la commande*

```
./robot.sh livre/04_ecriture_compil_execution/Stegosaure.py
```

qui fait interpréter le programme Python `Stegosaure.py` *: la fenêtre du Robot s'ouvre et le stégosaure est dessiné. On obtient le même résultat en faisant glisser l'icône du fichier* `Stegosaure.py` *sur celle de l'application* `LeRobotPython.app`.

□

Addenda

Nous devons à Seymour Papert, l'auteur de LOGO [31], la diffusion de l'idée d'utiliser le dessin pour apprendre les concepts de base de la programmation. Cependant l'introduction d'un quadrillage nous évite l'usage des angles et des longueurs comptées en pixels [a]. Le quadrillage permet de tracer très facilement des arcs de cercles, ce qui n'est pas immédiat en LOGO.

L'idée d'utiliser un quadrillage pour faire des dessins géométriques n'est pas nouvelle. On la connaît à l'école depuis longtemps (voir [41] par exemple). On la retrouve, à la base d'exercices de programmation graphique, dans de nombreux ouvrages informatiques de haut niveau comme en TEX [b] par exemple (voir [22] pages 389–390). Le quadrillage élimine les problèmes de représentation discrète sur un écran fini de concepts mathématiques continus. Par conséquent nos segments ressemblent à des segments tels qu'on les dessine sur le papier (et non pas à des traits juxtaposés en escalier), nos cercles ressemblent à des cercles (et non pas à des carrés comme cela arrive sur les écrans graphiques pour des cercles trop petits). Certes l'emploi de grands carreaux limite les possibilités de dessin. La programmation n'en est que plus facile si nous tirons avantage des formes géométriques pour induire la structure des programmes. Une fois que les notions de base seront acquises nous pourrons faire varier la taille de la grille de sorte qu'en passant à la limite fixée par les pixels de l'écran nous retrouverons toutes les possibilités offertes par les écrans graphiques actuels à haute résolution.

Dans le chapitre suivant nous commençons à écrire nos premiers programmes Python. Les algorithmes [c] restent très simples puisqu'ils consistent à codifier le mouvement de la main qui dessine.

---

[a] Le *pixel* est le plus petit élément de l'écran graphique qui peut être allumé (blanc), éteint (noir) ou colorié avec des luminosités et des couleurs plus ou moins nombreuses selon la qualité de l'écran.

[b] TEX ou plutôt sa variante LATEX [23] est le langage de programmation que nous avons utilisé pour composer ce livre.

[c] Un *algorithme* est la description d'un enchaînement d'opérations sur des objets pour accomplir une tâche. Ce mot a pour origine la version latine du nom du mathématicien et astronome persan al-Khwarizmi du IX$^{\text{ème}}$ siècle de notre ère. Son traité *Al-Jabr wa l-muqabala* enseigne comment résoudre algorithmiquement les équations du premier et second degré à cœfficients numériques. Le terme d'*algèbre* dérive du titre même de ce traité.

4

Écriture, interprétation et exécution de programmes

4.1 Structure des programmes

De même que pour écrire correctement en français il faut respecter des règles précises d'orthographe et de grammaire, l'écriture de programmes corrects est soumise à un certain nombre de *règles de syntaxe* heureusement plus simples que pour le français. La différence est que des phrases françaises comportant des fautes d'orthographe ou de grammaire peuvent rester compréhensibles par des francophones alors qu'un programme comportant des *erreurs syntaxiques* ne peut pas être traduit dans le langage de l'ordinateur, ni donc exécuté. Cette traduction est effectuée par un programme (appelé *interpréteur*) prévu pour indiquer nos erreurs mais pas pour les corriger. Pour corriger nos erreurs de manière satisfaisante, l'interpréteur devrait pouvoir comprendre nos intentions, un problème bien difficile.

Le programme `Cocotte` ci-dessous est un exemple de programme Python correct.

Exemple 5 (Programme syntaxiquement correct)

```
from LibRobot import *
# Dessiner une cocotte en papier tournée vers l'ouest
# Arrière
phtd(); av(); av(); av(); av()
phtg(); av(); av(); av(); av()
# Dessus
```

```
pqtd(); av(); av(); av(); av(); pqtd(); phtd()
# Avant
av(); av(); pqtg(); av(); av()
phtd(); av(); av(); av(); av()
# Dessous
pqtd(); phtd(); av(); av(); pqtg(); av(); av()
phtd(); av(); av(); av(); av()
# Arrêt du robot
st()
```

□

Tous nos programmes Python de pilotage du $\mathcal{R}\!\mathit{obot}$ ont la même forme :

Programme Python :

from ␣ `LibRobot` ␣ **import** ␣ `*`
`#` Commentaire expliquant ce que fait le programme
... Instructions du programme (une par ligne, ou séparées par des ';') ...

Un programme Python est rangé dans un fichier dont le nom se termine par `.py`. Le nom du programme est celui de ce fichier, sans le `.py` : le programme `Cocotte` est rangé dans le fichier `Cocotte.py`.

Ce nom est choisi par le programmeur à sa convenance, de façon à évoquer ce à quoi sert son programme. La seule règle à respecter est que le nom du programme ne doit comporter que des lettres minuscules ou majuscules non accentuées[1], des chiffres ou des soulignés '_'[2] et doit commencer par une lettre. Par exemple `S`, `LeStegosaure` ou `Le_Stegosaure` sont des noms corrects de programmes alors que `1S`, `s !`, `Le Stegosaure` ou `Le-Stégosaure` ne respectent pas la règle d'écriture des noms de programmes.

Un programme Python commence par la ligne '**from** `LibRobot` **import** `*`'. Les mots anglais **from** et **import** signifient 'de' et 'importer' : cette ligne importe de la *bibliothèque* `LibRobot` toutes (c'est le sens de l'étoile '`*`') les commandes du $\mathcal{R}\!\mathit{obot}$, sans quoi l'ordinateur ne les connaîtrait pas. Elle ouvre aussi la fenêtre dans laquelle le $\mathcal{R}\!\mathit{obot}$ dessine.

Il y a obligatoirement un blanc entre le mot **from** et le nom `LibRobot`, ainsi qu'entre `LibRobot` et le mot **import**. Dans un livre il y a des blancs

1. Les accents n'existent pas en anglais (à de très rares exceptions près, comme dans les mots 'naïve', 'protégé' ou 'rôle' empruntés au français).
2. Le souligné '_' ne doit pas être confondu avec le tiret '-'.

partout et même, en y regardant bien, plus de blanc que toute autre chose. Mettre un blanc dans un livre pour dire qu'il faut taper un blanc est donc idiot parce que la plupart des lecteurs ne s'intéressent pas aux blancs dans les livres mais simplement à ce qui y est écrit. Nous utiliserons donc le symbole '␣' pour écrire nos blancs noir sur blanc ! Autrement dit nous écrivons ␣ pour dire qu'il faut taper sur la grande barre située en bas du clavier de l'ordinateur qui s'appelle la *barre d'espacement* pour obtenir un blanc ' ' dans le programme. Nous ne le faisons pas dans les programmes donnés en exemple qui sont présentés comme ils apparaissent à l'écran. De même, à la fin d'une ligne il faut taper sur la touche de retour à la ligne ↩ ou return pour passer au début de la ligne suivante. Nous ne l'écrivons pas dans les livres car tous les lecteurs savent qu'il faut passer au début de la ligne suivante quand ils sont arrivés en fin de ligne.

Après le commentaire qui explique ce que fait le programme viennent les instructions du programme. Elles sont écrites dans l'ordre où elles doivent être exécutées par l'ordinateur, en commençant au début de la ligne (sans blanc devant : nous verrons au chapitre suivant que les blancs en début de ligne ont un sens précis en Python). On peut écrire une instruction par ligne ou bien plusieurs instructions sur une même ligne en les séparant par des points-virgules, sans point-virgule après la dernière de la ligne. Les instructions du programme peuvent être des commandes du *Robot* : `av`, `vg`, `vd`, `pqtd`, `pqtg`, `phtd`, `phtg`, `lc`, `bc`, `dc`, `dp`, `dg` ou `st`, chacune suivie de `()`. En général les programmes de pilotage du *Robot* se terminent par la commande `st()` qui permet d'attendre que nous tapions sur une touche avant que le dessin ne disparaisse à la fin de l'exécution du programme.

Les textes qui commencent par le signe `#` et se poursuivent jusqu'à la fin de la ligne sont des *commentaires* ignorés par l'ordinateur.

Le programme `CocotteErronee` ci-dessous contient plusieurs erreurs très courantes chez les débutants.

Exemple 6 (Programme syntaxiquement incorrect)

```
"""Dessiner une cocotte en papier tournée vers l'ouest

# Arrière
phtd(); av(); av(); av(); av()
pgtg(); av(); av(); av(); av()
# Dessus
pqtd(); av(); av(); av(); av(); pqtd(); phtd()
# Avant
av(); av(); pqtg(); av(); av()
```

```
phtd(); av(); av(); av(); av()
# Dessous
pqtd(); phtd(); av(); av(); pqtg(); av(); av()
phtd(); av(); av(); av(); av()
# Arrêt du robot
st(
```

Les erreurs dans ce programme sont les suivantes :

1. *la ligne '*`from LibRobot import *`*' a été oubliée ;*
2. *le texte qui explique ce que fait le programme a été écrit entre triples guillemets* `"""` *(c'est une* chaîne de documentation*, que l'on peut utiliser comme un commentaire de plusieurs lignes) mais les triples guillemets fermants* `"""` *ont été oubliés ;*
3. *l'instruction* `pgtg()` *n'est pas une commande du* $\mathcal{R}\breve{o}\underset{\frown}{b}\breve{o}t$ *;*
4. *la parenthèse ouverte de la dernière commande* `st(` *n'est pas fermée.*

□

Un exemple compliqué de programme Python syntaxiquement correct est donné ci-dessous :

Exemple 7 (Le stégosaure) Le programme Python de pilotage du $\mathcal{R}\breve{o}\underset{\frown}{b}\breve{o}t$ pour dessiner le stégosaure représenté à la figure 1.2 (page 6) est le suivant :

```
from LibRobot import *
# Rejoindre l'extrémité de la queue
# En bas
lc(); pqtd(); pqtd(); av(); av(); av(); av(); av()
av(); av()
# A droite
pqtg(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); pqtg(); bc()
# Dessus de la queue
av(); pqtd(); vg(); pqtg(); vg(); pqtd(); pqtd()
# 1ère épine caudale
vg(); pqtd(); pqtd(); vg(); pqtg(); vg(); pqtd()
# 2ème épine caudale
vd(); vg(); av()
# Dessiner les 11 plaques osseuses dorsales d'arrière en avant
# 1
vd(); av(); av(); av(); pqtd(); av(); pqtd(); av()
av(); av(); av(); pqtd(); vd(); av()
# 2
```

```
pqtg(); vd(); av(); av(); av(); pqtd(); av()
pqtd(); av(); av(); pqtd(); lc(); av(); bc()
# 3
vd(); av(); av(); pqtd(); av(); pqtd(); av()
pqtd(); lc(); av(); bc()
# 4
vd(); pqtg(); av(); pqtd(); av(); pqtd(); av()
av(); pqtd(); av(); av(); pqtd(); vd(); pqtg()
av()
# 5
av(); vd(); av(); pqtd(); av(); av(); pqtd(); av()
av(); pqtd(); av(); vd(); pqtg()
# 6
av(); av(); pqtd(); av(); av(); pqtd(); av(); av()
pqtd(); av(); av(); pqtd(); av(); av(); pqtg()
# 7
vd(); av(); pqtd(); av(); av(); pqtd(); av(); av()
pqtd(); av(); vd(); av()
# 8
av(); pqtg(); vd(); pqtd(); av(); av(); pqtd()
av(); av(); pqtd(); av(); pqtd(); av(); pqtg()
vd()
# 9
vg(); pqtd(); pqtd(); av(); av(); pqtd(); av()
pqtd(); av(); pqtd(); vg()
# 10
vd(); vg(); pqtd(); pqtd(); av(); av(); av()
pqtd(); av(); av(); pqtd(); av(); vd(); vg()
# 11
av(); vd(); pqtd(); av(); av(); av(); pqtd(); av()
pqtd(); av(); av(); vd()
# Dessiner la tête
# Crane
av(); av()
# Oeil
lc(); pqtg(); av(); dc(); pqtd(); pqtd(); av()
pqtg(); bc()
# Front
vg(); vd()
# Dessus du museau
av(); vg()
# Gueule
pqtg(); av(); av(); pqtd(); pqtd(); av(); av()
pqtg()
# Dessous de la tête
```

```
vg(); av(); av(); av(); av(); av(); av(); av()
vg(); pqtd(); av()
# Dessiner les pattes antérieures
# Patte gauche
pqtg(); av(); av(); av(); pqtd(); pqtd(); av()
av(); av(); av(); av(); pqtg(); av(); av(); pqtg()
av(); av(); av(); av(); av(); av(); pqtd(); pqtd()
av(); av(); av()
# Patte droite
av(); av(); pqtg(); av(); pqtg(); av(); av()
pqtg(); av(); pqtd(); pqtd(); av()
# Dessiner le ventre
vg(); pqtd(); av(); av(); av()
# Dessiner les pattes postérieures
# Patte gauche
pqtg(); av(); av(); av(); pqtd(); pqtd(); av()
av(); av(); av(); av(); av(); av(); pqtg(); av()
av(); pqtg(); av(); av(); av(); av(); av(); av()
av(); av(); pqtd(); pqtd(); av(); av(); av(); av()
av(); av()
# Patte droite
av(); pqtg(); av(); pqtg(); av(); av(); av()
pqtg(); av(); pqtd(); pqtd(); av()
# Compléter le dessous de la queue
av(); pqtd(); vg(); av(); vd(); vg(); av(); pqtd()
vg(); pqtd(); st()
```

□

4.2 Commentaires

Un programme Python peut comporter des *commentaires* qui sont des textes français quelconques écrits après le signe `# ...`, jusqu'à la fin de la ligne. Un commentaire peut occuper toute une ligne ou bien suivre les instructions d'une ligne, comme dans `vg()  # ERREUR : vd`. Un texte de plusieurs lignes peut aussi être placé entre triples guillemets `""" ... """` : c'est une *chaîne de documentation*, que l'ordinateur lit mais qui ne commande rien au $\mathcal{R}\text{o}\underset{\smile}{\text{b}}\text{o}t$. L'ordinateur ne tient pas compte de ces commentaires qui sont uniquement destinés à expliquer l'usage et le principe de fonctionnement du programme aux lecteurs de ce programme. Si l'on enlève tous les commentaires d'un programme, il devient illisible pour un humain mais rien n'est changé pour l'ordinateur.

Exemple 8 (Le stégosaure illisible) En enlevant tous les commentaires du programme `Stegosaure` de la page 40, ainsi que les blancs et les passages à la ligne inutiles, on obtient un programme Python totalement illisible mais qui dessine parfaitement bien le stégosaure représenté à la figure 1.2 (page 6) :

```
from LibRobot import *
lc(); pqtd(); pqtd(); av(); av(); av(); av(); av(); av(); av()
pqtg(); av(); av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); pqtg(); bc(); av(); pqtd(); vg(); pqtg()
vg(); pqtd(); pqtd(); vg(); pqtd(); pqtd(); vg(); pqtg(); vg()
pqtd(); vd(); vg(); av(); vd(); av(); av(); av(); pqtd(); av()
pqtd(); av(); av(); av(); av(); pqtd(); vd(); av(); pqtg(); vd()
av(); av(); av(); pqtd(); av(); pqtd(); av(); av(); pqtd(); lc()
av(); bc(); vd(); av(); av(); pqtd(); av(); pqtd(); av(); pqtd()
lc(); av(); bc(); vd(); pqtg(); av(); pqtd(); av(); pqtd(); av()
av(); pqtd(); av(); av(); pqtd(); vd(); pqtg(); av(); av(); vd()
av(); pqtd(); av(); av(); pqtd(); av(); av(); pqtd(); av(); vd()
pqtg(); av(); av(); pqtd(); av(); av(); pqtd(); av(); av()
pqtd(); av(); av(); pqtd(); av(); av(); pqtg(); vd(); av()
pqtd(); av(); av(); pqtd(); av(); av(); pqtd(); av(); vd(); av()
av(); pqtg(); vd(); pqtd(); av(); av(); pqtd(); av(); av()
pqtd(); av(); pqtd(); av(); pqtg(); vd(); vg(); pqtd(); pqtd()
av(); av(); pqtd(); av(); pqtd(); av(); pqtd(); vg(); vd(); vg()
pqtd(); pqtd(); av(); av(); av(); pqtd(); av(); av(); pqtd()
av(); vd(); vg(); av(); vd(); pqtd(); av(); av(); av(); pqtd()
av(); pqtd(); av(); av(); vd(); av(); av(); lc(); pqtg(); av()
dc(); pqtd(); pqtd(); av(); pqtg(); bc(); vg(); vd(); av(); vg()
pqtg(); av(); av(); pqtd(); pqtd(); av(); av(); pqtg(); vg()
av(); av(); av(); av(); av(); av(); av(); vg(); pqtd(); av()
pqtg(); av(); av(); av(); pqtd(); pqtd(); av(); av(); av(); av()
av(); pqtg(); av(); av(); pqtg(); av(); av(); av(); av(); av()
av(); pqtd(); pqtd(); av(); av(); av(); av(); av(); pqtg(); av()
pqtg(); av(); av(); pqtg(); av(); pqtd(); pqtd(); av(); vg()
pqtd(); av(); av(); av(); pqtg(); av(); av(); av(); pqtd()
pqtd(); av(); av(); av(); av(); av(); av(); av(); pqtg(); av()
av(); pqtg(); av(); av(); av(); av(); av(); av(); av(); av()
pqtd(); pqtd(); av(); av(); av(); av(); av(); av(); av(); pqtg()
av(); pqtg(); av(); av(); av(); pqtg(); av(); pqtd(); pqtd()
av(); av(); pqtd(); vg(); av(); vd(); vg(); av(); pqtd(); vg()
pqtd(); st()
```

Cet exemple montre également que la disposition des instructions du programme est importante. L'espacement entre instructions et les alignements convenables facilitent grandement la lecture des programmes. □

4.3 Interprétation des programmes

L'*interprétation* d'un programme Python consiste à le traduire dans le langage de l'ordinateur appelé *langage machine* et à le faire exécuter par l'ordinateur. Elle est faite par un programme que l'on appelle l'*interpréteur* Python [3]. L'interpréteur procède en deux temps :

- il lit d'abord *tout* le programme pour vérifier qu'il respecte les règles de syntaxe de Python et le traduire ; s'il trouve une erreur syntaxique, il le signale et s'arrête sans rien exécuter ;
- puis, si le programme est syntaxiquement correct, il fait exécuter ses instructions par l'ordinateur, les unes après les autres.

Certaines erreurs ne peuvent être découvertes que pendant ce second temps, au moment où l'ordinateur arrive à l'instruction fautive, par exemple l'appel d'une commande qui n'existe pas. L'exécution s'arrête alors, mais le $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ a déjà fait tout le dessin demandé par les instructions précédentes. Bien évidemment le programme Python et sa traduction pilotent le $\mathcal{R}\mathcal{o}\mathcal{b}\mathcal{o}t$ de la même façon. Cette manière de procéder nous évite d'avoir à apprendre le langage machine. Il est bien difficile de comprendre ce langage machine car tous les programmes s'écrivent uniquement avec des 0 et des 1 !

4.4 Erreurs syntaxiques

Pour pouvoir exécuter un programme Python, il ne faut pas qu'il contienne d'erreurs syntaxiques violant les règles d'écriture des programmes. En cas d'erreur l'interpréteur explique brièvement, en anglais, la cause de l'erreur et désigne l'endroit du programme où l'erreur a été trouvée (le numéro de la ligne, par exemple `line 15`, puis la ligne elle-même et, en dessous, un ou plusieurs accents circonflexes `^` sous le texte fautif). Le message commence par le nom de l'erreur, par exemple `SyntaxError` (erreur de syntaxe). Il convient alors de corriger l'erreur et de relancer le programme.

Une erreur découverte pendant l'exécution est signalée de la même façon, mais le message commence par `Traceback (most recent call last):`

3. La commande `./robot.sh Cocotte.py`, tapée dans une fenêtre de terminal placée dans le dossier `LeRobot_Python_FR`, appelle l'interpréteur `python3` pour faire exécuter le programme `Cocotte.py`. Faire glisser l'icône de `Cocotte.py` sur celle de l'application `LeRobotPython.app` fait la même chose.

(historique, l'appel le plus récent en dernier) et se termine par le nom de l'erreur, par exemple `NameError` (erreur de nom).

Exemple 9 (Programme syntaxiquement incorrect) Le programme `CocotteErronee` de la page 39 contient plusieurs des erreurs les plus fréquentes pour un programme aussi simple. Quand on lance ce programme `CocotteErronee`, l'interpréteur commence par indiquer '`SyntaxError: unterminated triple-quoted string literal (detected at line 15)`' (chaîne entre triples guillemets non terminée, détectée à la ligne 15) en désignant le `"""` de la première ligne : les triples guillemets fermants ont été oubliés à la fin du texte qui explique ce que fait le programme, si bien que tout le reste du programme, jusqu'à la dernière ligne 15, fait partie de ce texte ! Ayant ajouté les `"""` oubliés à la fin de la première ligne, une nouvelle tentative indique '`SyntaxError: '(' was never closed`' (la parenthèse '`(`' n'a jamais été fermée) en désignant la parenthèse de `st(` à la ligne 15. Nous la fermons en écrivant `st()`. Le programme est maintenant syntaxiquement correct et son exécution peut commencer. Elle s'arrête aussitôt, sur la première instruction, avec le message '`NameError: name 'phtd' is not defined`' (le nom `phtd` n'est pas défini). Ayant oublié la ligne '**from** `LibRobot` **import** `*`', l'interpréteur ignore la signification des commandes du $\mathcal{R}\breve{o}\breve{b}\breve{o}t$, et la fenêtre du $\mathcal{R}o\breve{b}ot$ ne s'est même pas ouverte. Après avoir ajouté cette ligne au début du programme, nous pouvons le relancer. La fenêtre s'ouvre, le $\mathcal{R}o\breve{b}ot$ dessine le premier trait de l'arrière de la cocotte, puis tout s'arrête avec le message '`NameError: name 'pgtg' is not defined. Did you mean: 'phtg'?`' (le nom `pgtg` n'est pas défini. Vouliez-vous dire `phtg` ?) qui désigne la ligne où se trouve `pgtg()`. C'est une faute de frappe que nous corrigeons en remplaçant `pgtg` par `phtg`. Nous obtenons ainsi laborieusement le programme `Cocotte` de la page 37 qui est correct (au commentaire près, écrit avec `#` dans `Cocotte`). L'interpréteur s'arrête à la première erreur qu'il trouve : il faut donc relancer le programme après chaque correction. On remarquera que les deux erreurs de syntaxe ont été signalées avant toute exécution, alors que les deux noms inconnus n'ont été découverts qu'au cours de l'exécution, au moment où l'ordinateur est arrivé à l'instruction fautive. □

4.5 Exécution des programmes

L'*exécution* d'un programme consiste à faire réaliser par l'ordinateur les actions définies par les instructions du programme. Comme dans notre cas les instructions sont des commandes du *Robot*, l'exécution du programme permet de piloter le *Robot*.

Quand le *Robot* a terminé son dessin, la commande `st()` affiche en bas de la fenêtre les touches qui permettent de continuer :

- diminuer (en tapant une ou plusieurs fois sur la touche moins [-]) ou augmenter (en tapant une ou plusieurs fois sur la touche plus [+]) la vitesse du *Robot* (qui peut varier de 0 à 10) ;
- faire redessiner tout le dessin par le *Robot*, à la vitesse choisie, en tapant sur une autre touche, par exemple la touche de retour à la ligne [↩] ou [return] ; quand le *Robot* dessine trop rapidement pour pouvoir observer le déroulement du dessin, il suffit de diminuer sa vitesse puis de lui faire redessiner le dessin ;
- arrêter définitivement l'exécution du programme en tapant sur la touche d'échappement [Esc] (ou sur l'une des touches [§] ou [#]), ce qui donne l'écran de la figure 4.2 (page 47).

Pendant l'exécution d'un programme de pilotage du *Robot*, nous pouvons aussi arrêter le dessin du *Robot* à tout moment en tapant sur la touche d'échappement [Esc] (ou sur [§] ou [#]). Le dessin reste alors affiché tel qu'il était à cet instant, jusqu'à ce que nous tapions sur une touche. Par exemple si nous interrompons l'exécution du programme `Stegosaure` de la page 40 au moment où le *Robot* termine le dessin de la patte avant gauche, nous obtenons un écran semblable à celui de la figure 4.1 de la page 47.

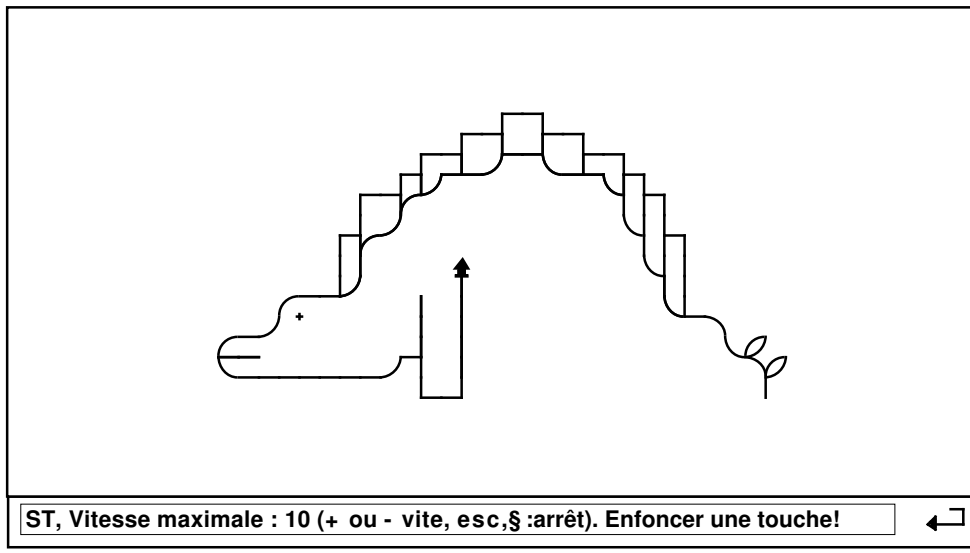


FIG. 4.1 – *Écran de l'ordinateur après un stop*

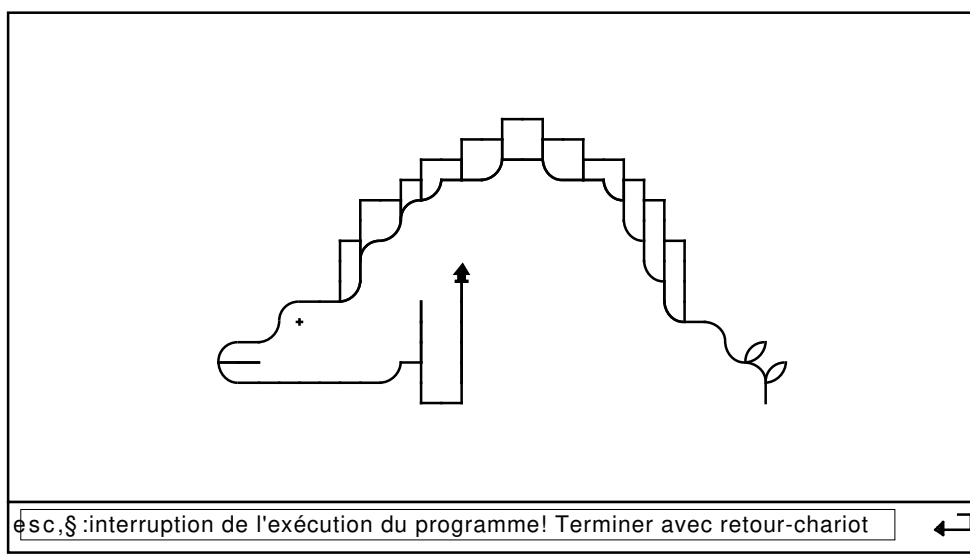


FIG. 4.2 – *Écran de l'ordinateur avant l'arrêt définitif*

4.6 Erreurs logiques

Après avoir éliminé toutes les erreurs syntaxiques c'est-à-dire les erreurs de programmation qui relèvent de la violation des règles de grammaire du langage Python, nous ne sommes pas encore au bout de nos peines avant de pouvoir contempler le dessin ! Il reste encore les *erreurs logiques* que nous découvrirons à l'exécution du programme.

La première erreur possible est que la succession de commandes que notre programme demande à l'ordinateur de donner au $\mathcal{R}o\!\!\!/o t$ conduise celui-ci en dehors du cadre. C'est le cas du programme suivant :

```
from LibRobot import *

# Programme conduisant à une erreur à l'exécution parce qu'il fait
# sortir le robot du cadre.
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av()
```

Ce programme donne le triste résultat ci-dessous, accompagné dans la fenêtre de terminal du message '`Erreur ligne 8 de SortirDuCadre.py : le robot sortirait du cadre (av)`' :

La deuxième erreur possible est que nous nous soyons trompés dans le programme et que le dessin obtenu ne ressemble pas du tout à ce que nous avions imaginé. Par exemple, le programme `StegosaureErrone` ci-dessous est une version logiquement erronée mais syntaxiquement correcte du programme `Stegosaure` de la page 40.

```
from LibRobot import *

...
# 10
vg()  # ERREUR : vd
vg(); pqtd(); pqtd(); av(); av(); av()
pqtd(); av(); av(); pqtd(); av(); vd(); vg()
...
# Dessiner le ventre
vg(); pqtg()  # ERREUR : pqtd
av(); av(); av()
...
st()
```

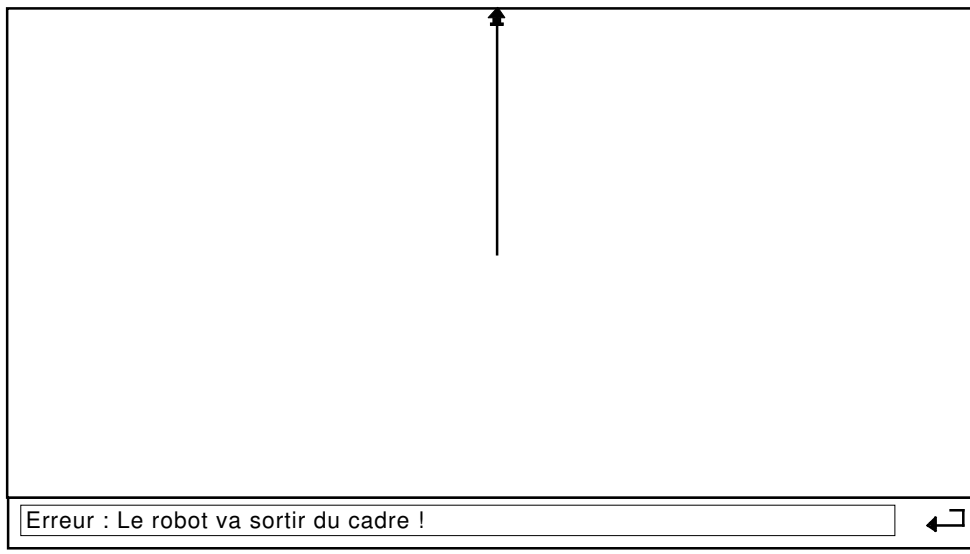


FIG. 4.3 – *Écran de l'ordinateur quand le robot sort du cadre*

L'auteur de ce programme `StegosaureErrone` a par deux fois confondu sa gauche et sa droite. Le résultat de l'exécution de ce programme tel que représenté ci-dessous (figure 4.4) étonnerait plus d'un paléontologue !

Une troisième erreur, propre aux débutants en Python, est d'oublier les parenthèses `()` après une commande du $\mathcal{R}^{o}_{\smile}{}^{o}t$, par exemple '`av ; pqtd()`' au lieu de '`av() ; pqtd()`'. Le programme est syntaxiquement correct mais le $\mathcal{R}^{o}_{\smile}{}^{o}t$ n'avance pas car `av` sans `()` désigne la commande sans demander de l'exécuter. L'interpréteur Python ne signale rien du tout : ni erreur, ni même un *avertissement*. Seul le dessin, auquel il manque un trait, révèle l'erreur. Il faut donc toujours vérifier que chaque commande est bien suivie de ses parenthèses.

4.7 Comment localiser les erreurs logiques dans le programme

Pour trouver les erreurs logiques dans le programme il faut s'imaginer que l'on exécute le programme à la place du $\mathcal{R}^{o}_{\smile}{}^{o}t$. Quand le programme est très long ceci n'est pas facile. Le mieux est donc de suivre l'exécution du

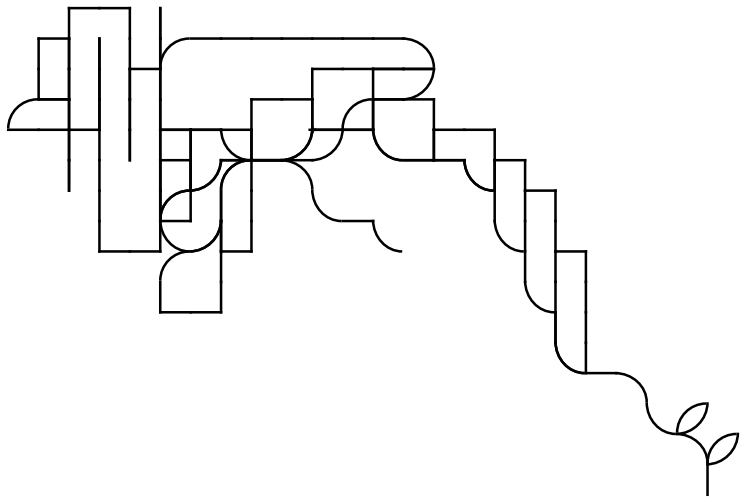


FIG. 4.4 – *Stégosaure plutôt raté*

programme en mode *pas à pas*, ce qui permet de suivre les commandes transmises par l'ordinateur au $\mathcal{R}^{\check{o}}\underset{\smile}{b}^{\check{o}}t$ pour localiser l'erreur avec précision. Quand l'erreur est localisée, il faut arrêter l'exécution du programme pour pouvoir la corriger dans le programme. Une fois que c'est fait, on peut relancer le programme pour tenter une nouvelle exécution.

- Pour passer en mode pas à pas nous pouvons :
  - à la fin du dessin, réduire la vitesse à zéro en tapant plusieurs fois sur la touche [-] puis faire redessiner le dessin en tapant sur la touche de retour à la ligne [↩] ou [return] ;
  - ou insérer une instruction `pp()` dans le programme Python, un peu avant l'endroit supposé de l'erreur. Cette instruction `pp()` fait passer l'ordinateur en mode d'exécution pas à pas.
- En mode pas à pas, l'ordinateur attend que nous lui donnions l'ordre d'exécuter chaque commande du $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$ en tapant sur une touche, par exemple la touche de retour à la ligne [↩] ou [return]. On peut donc suivre le déroulement de l'exécution du programme instruction après instruction, ce qui permet en général de trouver assez facilement l'erreur.
- Quand l'erreur est localisée avec précision dans le programme, on peut arrêter l'exécution du programme en tapant sur la touche d'échappement [Esc] (ou sur [§] ou [#]).

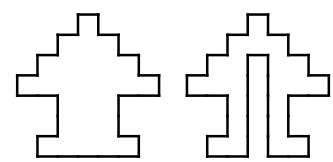
Il est également possible de programmer le retour à une vitesse d'exécution normale en insérant une instruction `ex()` dans le programme.

Il faut presque toujours plusieurs essais avant que le programme soit logiquement correct. Quand il l'est, on est souvent tenté de l'améliorer. Le plus difficile est alors de savoir quand s'arrêter !

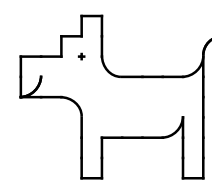
`st()`	stoppe l'exécution du programme à la fin du dessin. La vitesse peut être diminuée par [-] (minimum 0) ou augmentée par [+] (maximum 10). Toute autre touche fait redessiner le dessin à la vitesse indiquée. L'exécution s'arrête définitivement en tapant sur [esc] ou [§] ou [#].
`pp()`	exécute la suite du programme en mode pas à pas (vitesse 0) : chaque commande du *Robot* s'exécute en tapant sur une touche.
`ex()`	exécute la suite du programme en mode exécution normale (vitesse 10).

TAB. 4.1 – *Commandes de contrôle du mode de pilotage du Robot*

Exercice 11



– *Écrire, mettre au point et exécuter des programmes Python de dessin du Robot tel qu'il est stylisé ci-contre avec son crayon baissé et son crayon levé.*



– *Écrire, mettre au point et exécuter un programme Python qui pilote le Robot pour lui faire dessiner le chien représenté ci-contre.*

– *Programmer les dessins de la page 9.*

– *Programmer le dessin de vos initiales en lettres majuscules dans le style de la lettre* `H` *en relief représentée à la figure 1.12 (page 9).*

□

Corrigé 11 *Nous donnons un programme, les autres se trouvant dans le dossier* `livre`.

```
from LibRobot import *
# Profil arrière de la patte avant
av(); av(); pqtd()
# Ventre
av(); av(); av(); vg()
# Patte arrière
pqtd(); pqtd(); av(); av(); av(); pqtg(); av()
pqtg(); av(); av(); av(); av()
# Queue
av(); av(); vd(); pqtd(); pqtd(); vg(); vd()
# Dos
av(); av(); av(); vd()
# Tête
# Oreille
av(); av(); pqtg(); av(); pqtg(); av()
# Oeil
lc(); av(); dc(); pqtd(); pqtd(); av(); bc()
pqtg(); av(); pqtg(); av()
# Museau
pqtd(); av(); av(); pqtg(); av(); av(); pqtg()
# Gueule
vg(); pqtd(); pqtd(); vd(); pqtd(); pqtd(); av()
av(); vd()
# Profil avant de la patte avant
av(); av(); av(); pqtg(); av(); st()
```

□

Addenda

Le cycle écriture, mise au point syntaxique, traduction, mise au point logique et exécution des programmes s'est considérablement raccourci ces dernières années grâce aux progrès fantastiques de l'architecture des ordinateurs qu'il convient de comprendre très schématiquement. Un micro-ordinateur comprend :

- une unité arithmétique et logique pour faire des calculs et manipuler des données,
- une mémoire pour conserver temporairement des données et des programmes (son contenu est perdu quand on coupe l'électricité),

- un ou plusieurs disques durs magnétiques ou à état solid (SSD) pour conserver en permanence des fichiers contenant des données ou des programmes,
- des prises (USB) pour brancher des clés de mémoire qui permettent de conserver en permanence des fichiers contenant des données ou des programmes ou les échanger avec d'autres ordinateurs,
- un clavier pour entrer des données ou des programmes en mémoire,
- un écran graphique pour visualiser des textes et des images,
- une souris pour désigner des points sur l'écran graphique,
- un bus pour permettre des échanges rapides de données et de programmes entre ces différents organes.

Quand on met l'ordinateur en marche, il est cablé pour charger dans sa mémoire et exécuter un programme rangé en permanence sur disque que l'on appelle le *système d'exploitation*. Ce programme a pour tâche essentielle de gérer tous les organes de l'ordinateur et de dialoguer avec l'utilisateur grâce à un *interpréteur de commandes* qui va attendre que l'utilisateur lui donne des ordres par l'intermédiaire du clavier et les exécuter immédiatement les uns après les autres jusqu'à ce que l'utilisateur lui donne l'ordre d'éteindre l'ordinateur. Parmi les commandes de l'interpréteur, citons par exemple la suppression d'un fichier rangé sur disque ou le lancement de l'exécution d'un programme écrit en langage machine (généralement appelé *programme d'application*) comme par exemple un *éditeur de texte* pour créer un texte et le ranger dans un fichier ou un *interpréteur* pour traduire un programme Python en langage machine et le faire exécuter.

Les premières interfaces entre l'homme et la machine étaient purement textuelles. On tape sur le clavier et on voit le résultat sur une imprimante vite remplacée par un écran avec seulement des lignes de texte de longueur maximale 80 caractères. Les écrans en noir et blanc furent remplacés pat des écrans couleur (comme on voit encore dans l'application `terminal`). Puis sont apparus les écrans graphiques constitués de très petits pixels de couleur. Le fenêtrage permet de donner l'illusion de plusieurs écrans graphiques sur un seul. Puis vint l'idée d'icônes pour représenter les fichiers et des applications sur lesquels on peut double cliquer pour lancer une application. Par exemple, un double click sur l'icône de `LeRobotPython.app` lance le programme de dessin interactif tandis que faire glisser l'icône d'un programme du $\mathcal{R}\text{o}\underset{\frown}{\text{b}}\text{o}t$ sur cette icône de l'application `LeRobotPython.app` exécute ce programme en faisant apparaître la fenêtre de dessin et le terminal. Noter qu'il faut cliquer sur l'une ou l'autre des fenêtres pour l'utiliser. On peut arrêter le programme par Control+C ou ctrl+C (que les informaticiens notent ^C) en sélectionnant la fenêtre terminal ou en fermant cette fenêtre ou encore avec la touche d'échappement (`esc`) dans la fenêtre graphique.

Les ordinateurs sont *binaires* en ce sens que toutes les données, en particulier les programmes, sont représentés par des suites de 0 et de 1. L'ordinateur fonctionne en transférant une instruction du programme depuis une case de sa mémoire dans son unité arithmétique et logique (via le bus) et en l'exécutant (ce qui en général modifie le contenu de la mémoire, du disque, de l'écran, etc.) puis en recommençant ainsi avec l'instruction rangée dans la case suivante de sa mémoire. Certaines instructions dites *instructions de branchement* permettent d'indiquer que l'instruction suivante est rangée dans une case mémoire de numéro donné.

Pour exécuter un programme écrit en langage machine rangé sur disque, le système d'exploitation demande à l'unité arithmétique et logique de charger ce programme en mémoire c'est-à-dire de recopier le programme dans un endroit de la mémoire qui était inoccupé puis d'exécuter une instruction de branchement à la première instruction de ce programme. Le programme doit se terminer par une instruction de branchement en retour vers le système d'exploitation. Les ordinateurs modernes possèdent également la faculté d'interrompre l'exécution du programme en cours puis de la reprendre ultérieurement comme si rien ne s'était passé. Entre temps, c'est le système d'exploitation qui s'exécute pour réagir en fonction de la cause de l'interruption. Par exemple si l'utilisateur a bougé la souris, ses nouvelles coordonnées sont rangées en mémoire. Si l'horloge sonne, ce qui arrive tous les soixantièmes de seconde par exemple, l'heure est mise à jour, etc. En particulier l'utilisateur dispose de boutons ou de touches (Esc, §, # pour notre $\mathcal{R}^{o}\underset{\smile}{b}^{o}t$) permettant d'interrompre l'exécution de son programme s'il constate une erreur.

Parmi les applications que peut exécuter le système d'exploitation, il y en a une qui nous intéresse particulièrement, c'est l'interpréteur Python.

Les environnements de programmation modernes (comme `IDLE`, fourni avec Python) se chargent de l'édition du texte du programme, c'est-à-dire de sa création et de ses modifications par interaction avec l'utilisateur. Le programme Python est affiché sur l'écran et rangé en mémoire au fur et à mesure que l'utilisateur le tape au clavier. Il est prévu qu'il puisse le modifier en utilisant diverses touches de fonctions et la souris. La possibilité lui est également offerte de sauvegarder son texte sur le disque dur dans un fichier dont il doit donner le nom.

Quand l'édition de texte est terminée, l'environnement peut faire traduire le programme en un programme équivalent écrit en langage machine. Pour les langages dits *compilés*, comme OCaml ou C, cette traduction est faite une fois pour toutes par un *compilateur* : le programme compilé est rangé sur disque, puis l'utilisateur retourne au niveau du système d'exploitation pour charger en mémoire et exécuter son programme compilé comme expliqué précédemment.

Pendant la phase de débogage du programme, il est plus rapide de ranger le programme traduit directement en mémoire, le traducteur se chargeant lui-même de son exécution puis reprenant la main quand cette exécution est terminée ou interrompue. C'est ainsi que procède l'interpréteur Python, à chaque fois qu'on lui demande d'exécuter un programme.

On comprend les progrès considérables qui ont été faits depuis les premiers ordinateurs pour lesquels il fallait modifier le cablage pour changer le programme [25] :

- le programme étant rangé en mémoire (selon l'idée de Von Neumann [a]) et la mémoire reliée à des disques rapides, il est possible de changer le programme de la machine presque instantanément ;
- l'usage de langages de haut niveau comme Python ou OCaml et d'interpréteurs ou de compilateurs performants, permet d'éviter d'avoir à programmer en un langage machine ingrat ;
- l'écran alphanumérique permet à l'utilisateur d'éditer son programme de manière interactive au clavier, ce qui fait gagner un temps considérable par rapport aux manipulations de cartes perforées en carton encore utilisées dans les années 1970 ;
- la puissance de calcul, liée à la densité des transitors sur les puces informatiques (qui varie de quelques milliards à plusieurs centaines de milliards de transistors en 2016) permet de gérer des écrans graphiques directement reliés à la mémoire, ce qui permet des modifications très rapides et donc l'usage de fenêtres, souris, etc ;
- la taille de la mémoire (quelques milliards de caractères, Go, Gigaoctets) et des disques (quelques milliers de milliards de caractères, To, Teraoctets) permet d'envisager des programmes et des données de plus en plus grands et donc des applications de plus en plus sophistiquées.

[a] John von Neumann, 1903–1957, mathématicien américain.

La taille des plus grands programmes que l'on peut actuellement écrire se chiffre maintenant en dizaines ou centaines de millions de lignes, ce qui est considérable et ne va pas sans difficultés car on atteint les limites de la compréhension humaine. Une formation de l'esprit, une nouvelle rigueur, plus précise encore que la logique des mathématiciens, devient la qualité première qui, en plus de l'imagination et du sens de l'esthétique, fait les bons informaticiens.

Un peu de méthode aide également beaucoup. Toute l'histoire des langages de programmations est remplie d'inventions permettant de formuler plus clairement la façon d'organiser les données en mémoire ou les calculs de l'ordinateur. A chaque fois les capacités humaines, donc la taille limite des plus grands programmes, ont été repoussées. Une idée très ancienne de structuration des calculs, due à Von Neumann et Turing [a] est celle de *procédure*, on disait autrefois *routine* ou *sous-programme*. Nous commençons à l'introduire dans le chapitre suivant.

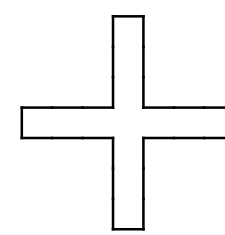
[a] Alan Turing, 1912–1954, mathématicien anglais.

5

Procédures

5.1 Recopie de texte

Dans le programme `CroixGrecque_1` ci-dessous on retrouve quatre fois la même séquence de commandes pour dessiner les quatre branches identiques de la croix et tourner. Chaque branche se décompose en deux cotés égaux de longueur 3 et une extrémité de longueur 1 (l'unité étant la longueur d'un côté d'un carreau du quadrillage).



```
from LibRobot import *
# Dessiner une croix grecque
av(); av(); av(); pqtg(); av(); pqtg()
av(); av(); av(); pqtd()
av(); av(); av(); pqtg(); av(); pqtg()
av(); av(); av(); pqtd()
av(); av(); av(); pqtg(); av(); pqtg()
av(); av(); av(); pqtd()
av(); av(); av(); pqtg(); av(); pqtg()
av(); av(); av(); pqtd()
st()
```

Cette situation où l'on trouve la même séquence de commandes du $\mathcal{R}^{o}\!\!\smile^{o}t$ plusieurs fois de suite dans un programme est très fréquente car les dessins géométriques sont souvent composés de morceaux identiques. Il est alors très fastidieux de taper plusieurs fois le même texte. On peut l'éviter en utilisant les possibilités offertes par l'ordinateur de copier un texte déjà enregistré dans sa mémoire et de le recopier ailleurs dans le programme. La manière de

procéder est expliquée pour les PC (sous Windows ou Linux) à la figure 5.1 (page 58) et pour le Macintosh à la figure 5.2 (page 59).

Sélectionner et recopier un texte sur PC (Windows ou Linux)

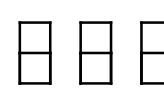
1. Cliquer avec la souris au début du texte à recopier pour y placer le curseur d'édition clignotant.
2. Maintenir la touche majuscules ⇧ enfoncée et cliquer à la fin du texte à recopier, de sorte que le texte à recopier apparaisse en *surbrillance* (on peut aussi faire glisser la souris du début à la fin du texte en maintenant son bouton enfoncé).
3. Maintenir la touche de contrôle Ctrl enfoncée le temps de taper sur la touche C pour *copier* le texte sélectionné.
4. Cliquer avec la souris à l'endroit où le texte doit être recopié.
5. Maintenir la touche de contrôle Ctrl enfoncée le temps de taper sur la touche V pour *coller* le texte copié.

L'opération de recopie (Ctrl V) en un endroit quelconque du programme peut être répétée autant de fois que nécessaire. En cas d'erreur, Ctrl Z annule la dernière modification.

FIG. 5.1 – *Copier-coller sur PC (Windows ou Linux)*

Faire beaucoup de recopies à la main est long. De plus on imagine bien que dans un programme de plusieurs pages, le lecteur a du mal à retrouver les parties de texte identiques. Ce phénomème est illustré dans le programme `Nombre888_1` ci-dessous. La difficulté serait évidemment encore plus grande si les parties de texte identiques se trouvaient sur des pages différentes.

```
from LibRobot import *
# Dessiner le nombre 888 dans le style
# des montres digitales.
# Dessiner le 8 des centaines
av(); pqtd(); av(); pqtg(); av(); pqtg(); av()
pqtg(); av(); pqtg(); av(); pqtd(); av(); pqtd()
av()
# Prévoir un espace et réorienter le robot vers le nord
```



Sélectionner et recopier un texte sur Macintosh

1. Placer le curseur d'édition clignotant au début du texte à recopier en le déplaçant à l'aide des flèches de défilement (↑, ↓, ←, →) ou bien en cliquant avec la souris à l'endroit désiré.
2. Maintenir la touche majuscules ⇧ ou shift enfoncée, sans la relâcher.
3. Placer le curseur d'édition clignotant à la fin du texte à recopier en le déplaçant à l'aide des flèches de défilement (↑, ↓, ←, →) ou bien en cliquant avec la souris à l'endroit désiré, de sorte que le texte à recopier passe en *vidéo-inverse* c'est-à-dire en blanc sur fond noir (si par erreur vous tapez du texte au clavier, il remplace le texte sélectionné en vidéo-inverse ; la touche d'effacement ← supprime le texte sélectionné en vidéo-inverse ; dans les deux cas, choisir l'option `Annuler` du menu `Edition` pour rétablir l'original).
4. Relâcher la touche majuscules ⇧ ou shift.
5. Sélectionner l'option `Copier` du menu `Edition` (ou de manière équivalente maintenir la touche commande ⌘ enfoncée et taper sur la touche C).
6. Placer le curseur d'édition clignotant à l'endroit où le texte doit être recopié en le déplaçant à l'aide des flèches de défilement (↑, ↓, ←, →) ou bien en cliquant à l'endroit désiré avec la souris.
7. Sélectionner l'option `Coller` du menu `Edition` (ou de manière équivalente maintenir la touche commande ⌘ enfoncée et taper sur la touche V).

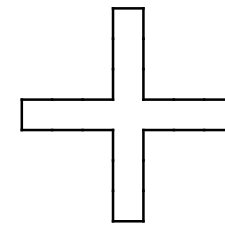
L'opération de recopie (`Coller`) en un endroit quelconque du programme peut être répétée autant de fois que nécessaire.

FIG. 5.2 – *Copier-coller sur le Macintosh*

```
lc(); pqtd(); pqtd(); av(); av(); pqtg(); bc()
# Dessiner le 8 des dizaines
av(); pqtd(); av(); pqtg(); av(); pqtg(); av()
pqtg(); av(); pqtg(); av(); pqtd(); av(); pqtd()
av()
# Prévoir un espace et réorienter le robot vers le nord
lc(); pqtd(); pqtd(); av(); av(); pqtg(); bc()
# Dessiner le 8 des unités
av(); pqtd(); av(); pqtg(); av(); pqtg(); av()
pqtg(); av(); pqtg(); av(); pqtd(); av(); pqtd()
av()
st()
```

5.2 Exemples de procédures

Plutôt que de faire des recopies de textes à la main soi-même, on peut demander à l'interpréteur Python de le faire à notre place en utilisant des *procédures*. Commençons par réécrire le programme `CroixGrecque_1` (page 57) en utilisant des procédures pour mettre en évidence la décomposition de la croix grecque en quatre branches identiques, chacune d'elles se décomposant en deux cotés égaux de longueur 3 et une extrémité de longueur 1.



```
from LibRobot import *

# Dessiner une croix grecque
def cote():
   av(); av(); av()

def extremite():
   pqtg(); av(); pqtg()

# Dessiner le côté gauche, l'extrémité puis le côté
# droit d'une branche, et orienter le robot en
# direction de la branche suivante.
def branche():
   cote(); extremite(); cote(); pqtd()

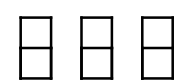
def croix():
   branche(); branche(); branche(); branche()

croix(); st()
```

L'emploi d'une procédure comprend :

- la *déclaration de procédure* qui définit une séquence d'instructions (appelée *corps de la procédure*) et lui donne un nom (appelé *nom de la procédure*) ;
- l'*appel de procédure* qui indique que la séquence d'instructions constituant le corps de la procédure doit être recopiée à la place du nom de la procédure.

Pour le lecteur du programme, l'utilisation de procédures est préférable aux recopies de textes. A chaque fois qu'il voit le nom d'une procédure, il comprend qu'il s'agit de la copie d'un texte qu'il peut consulter dans la déclaration de procédure. S'il rencontre plusieurs appels d'une même procédure, il n'a pas à faire d'effort pour comprendre immédiatement qu'il s'agit du même texte qui est recopié. Le programme est donc plus clair. N'est-ce pas le cas du programme `Nombre888_2` ci-dessous qui est plus facilement compréhensible que le programme `Nombre888_1` ?



```
from LibRobot import *

# Dessiner le nombre 888 comme sur les
# montres digitales.

# Dessiner le chiffre 8, en partant en bas à gauche du
# 8 avec le robot orienté au nord et en arrivant en bas
# à gauche du 8 avec le robot orienté à l'ouest.
def huit():
   av(); pqtd(); av(); pqtg(); av(); pqtg(); av()
```

```
    pqtg(); av(); pqtg(); av(); pqtd(); av(); pqtd()
    av()
# Partant avec le robot orienté à l'ouest en bas à
# gauche du 8, reculer en bas à droite du 8 puis d'un
# espace et réorienter le robot au nord.
def espace():
    lc(); pqtd(); pqtd(); av(); av(); pqtg(); bc()
huit(); espace()  # le 8 des centaines
huit(); espace()  # le 8 des dizaines
huit()  # le 8 des unités
st()
```

5.3 Déclaration de procédures

Les déclarations des procédures d'un programme se placent entre la ligne '**from** `LibRobot` **import** `*`' et les instructions du programme. La structure d'un programme Python est donc la suivante :

Programme Python :

from ␣ `LibRobot` ␣ **import** ␣ `*`
`#` Commentaire expliquant ce que fait le programme
... Déclarations des procédures du programme ...
... Instructions du programme [a] (non indentées) ...

[a] Y compris les appels des procédures du programme.

En Python, une déclaration de procédure a la forme suivante (nous utiliserons des formes plus élaborées dans la suite) :

Déclaration de procédure :

`#` Spécification : ce que fait la procédure
def ␣ `nomDeLaProcedure():`
    ... Corps de la procédure [a] (**obligatoirement indenté**) ...

[a] Y compris les appels de procédures précédemment déclarées.

Le mot anglais **def** est l'abréviation de *define* (définir). Le `nomDeLaProcedure` est choisi par le programmeur à sa convenance, de façon à évoquer ce à quoi sert cette procédure. Les parenthèses `()` qui le suivent indiquent que c'est une procédure, qu'il faudra appeler en écrivant `nomDeLaProcedure()`, exactement comme les commandes du *Robot* `av()`, `pqtd()`, etc. D'ailleurs les commandes du *Robot* sont des procédures déclarées dans la bibliothèque `LibRobot`. La ligne **def** se termine obligatoirement par deux points ' :' qui annoncent le corps de la procédure.

Les instructions du corps de la procédure sont écrites sur les lignes suivantes, toutes décalées vers la droite d'un même nombre de blancs, habituellement 4. On dit qu'elles sont *indentées*. Comme dans le programme, on peut écrire une ou plusieurs instructions par ligne, séparées par des points-virgules. C'est l'indentation, et elle seule, qui indique à Python quelles instructions font partie du corps de la procédure : le corps se termine à la première ligne qui n'est plus indentée. Les blancs en début de ligne ne sont donc pas une simple question de présentation : ils font partie de la syntaxe de Python. Si l'on oublie d'indenter la première ligne du corps, Python signale par exemple '`IndentationError: expected an indented block after function definition on line 3`' (erreur d'indentation : un bloc indenté est attendu après la définition de fonction de la ligne 3). Si au contraire on indente par erreur une instruction du programme, il signale '`IndentationError: unexpected indent`' (indentation inattendue). Et si une ligne du corps n'a pas le même nombre de blancs que les précédentes, il signale '`IndentationError: unindent does not match any outer indentation level`' (le retrait ne correspond à aucun niveau d'indentation). Une ligne vide après le corps de chaque procédure, comme dans le programme `Nombre888_2` ci-dessus, n'est pas obligatoire mais rend bien visible la fin de la procédure.

Attention : le corps d'une procédure doit **impérativement** être indenté, et **toutes** ses lignes du même nombre de blancs. Une ligne du corps que l'on oublie d'indenter n'est pas toujours signalée comme une erreur : elle ne fait simplement plus partie de la procédure. Dans l'exemple ci-dessous, `pqtd()` n'est pas indentée : Python l'exécute une seule fois, au début du programme, et non à chaque appel de `coin()`. Au lieu d'un carré, le *Robot* trace un trait droit de huit pas, sans aucun message d'erreur :

```
def coin():
   av(); av()
pqtd()          # Erreur : pas indentée, hors de la procédure
coin(); coin(); coin(); coin()
```

Pour que le Robot tourne à chaque coin, il faut indenter `pqtd()` comme `av() ; av()`, avec les mêmes quatre blancs.

Remarque : en Python, les procédures sont des cas particuliers de ce que l'on appelle des *fonctions*. Nous en reparlerons plus tard.

5.4 Identificateurs

Le nom d'une procédure s'appelle un *identificateur* en Python. En voici une définition précise à laquelle il faudra se référer à chaque fois que nous parlerons d'identificateurs :

Identificateur :

Un *identificateur* est une séquence de lettres minuscules ou majuscules non accentuées, de chiffres ou de soulignés '_' qui ne commence pas par un chiffre et qui est différente des mots anglais réservés par Python comme **from**, **import**, **def**, **for**, **in**, **if**, **while**, **return**, **True**, etc.

Par exemple `cote`, `croixGrecque`, `croix_grecque` ou `cote2` sont des identificateurs alors que `2cote`, `cote-gauche` ou `for` n'en sont pas. Si l'on écrit '**def** `2cote()` :', Python répond '`SyntaxError: invalid decimal literal`' (nombre décimal incorrect) car il croit lire un nombre ; pour '**def** `cote-gauche()` :' ou '**def** `for()` :', il répond simplement '`SyntaxError: invalid syntax`' (syntaxe incorrecte). Les majuscules et les minuscules sont distinguées : `Cote` et `cote` sont deux identificateurs différents. Si l'on déclare '**def** `Cote()` :' puis que l'on appelle `cote()`, Python répond '`NameError: name 'cote' is not defined. Did you mean: 'Cote'?`' (le nom `cote` n'est pas défini. Vouliez-vous dire `Cote` ?). Python permet de commencer un identificateur par une majuscule, mais l'usage est de commencer les noms de procédures par une minuscule. Pour rendre lisible un identificateur formé de plusieurs mots, on met une majuscule au début de chaque mot sauf le premier, comme dans `croixDeSaintAndre` ou `bordInterieurDuVerre` [1].

1. Beaucoup de programmeurs Python préfèrent séparer les mots par des soulignés,

5.5 Appel de procédures

Pour appeler une procédure, il suffit d'écrire son nom suivi de `()`. L'appel `nomDeLaProcedure()` est alors remplacé par le corps de la procédure c'est-à-dire par la liste d'instructions indentées qui suit la ligne '**def** `nomDeLaProcedure() :`' dans sa déclaration.

On notera que si l'on déclare une procédure sans jamais l'appeler, elle ne sert absolument à rien.

On peut appeler une procédure dans le programme ou dans une autre procédure. Par exemple dans le programme `CroixGrecque_2`, on appelle la procédure `croix` qui appelle la procédure `branche` qui elle-même appelle les procédures `cote` et `extremite`.

Il faut impérativement respecter la règle qu'une procédure doit être déclarée avant d'être appelée. Supposons par exemple que l'on ait déclaré une procédure sous le nom `nomProc`. Supposons également que par suite d'une faute de frappe, on ait appelé cette procédure sous le nom `nomProg`. Quand l'exécution arrive à cet appel, l'interpréteur s'arrête en indiquant l'erreur '`NameError: name 'nomProg' is not defined`' (le nom `nomProg` n'est pas défini). Ne sachant pas que nous avons écrit par erreur `nomProg` au lieu de `nomProc`, l'interpréteur suppose que la procédure `nomProc` a été déclarée mais n'a jamais été utilisée. C'est inutile mais autorisé. Il constate également que la procédure `nomProg` est appelée mais n'a pas été déclarée. C'est interdit. Il remarque cependant que les deux noms se ressemblent et suggère '`Did you mean: 'nomProc'?`' (vouliez-vous dire `nomProc` ?). [2] On obtient la même erreur '`NameError`' si le programme appelle une procédure dont la déclaration est placée plus loin : Python exécute les déclarations dans l'ordre où elles sont écrites, et au moment de l'appel il ne connaît pas encore la procédure. Comme toutes les erreurs découvertes pendant l'exécution, ces erreurs n'apparaissent qu'une fois que le $\mathcal{R}^{o}\!\!\smile^{o}t$ a fait la partie du dessin qui précède l'appel fautif.

---

comme dans `croix_de_saint_andre`.

2. Nous parlons souvent des programmes de manière anthropomorphique, c'est-à-dire en leur attribuant des caractères propres à l'homme. En fait nous pensons aux programmeurs qui les ont écrits. Par exemple, l'ordinateur ne se trompe pas ; c'est son constructeur qui s'est trompé ou le programmeur qui a fait une erreur logique dans son programme.

5.6 Commandes du robot définies par des procédures

Certaines commandes du $\mathcal{R}\!\mathit{obot}$ sont équivalentes à une séquence de commandes élémentaires et peuvent donc s'exprimer avec des procédures. Par exemple, pour pivoter d'un quart de tour, il suffit de pivoter deux fois de suite d'un huitième de tour. On a donc :

```
# Fait pivoter le robot sur place d'un quart de tour
#   à droite
def pqtd():
   phtd(); phtd()

# Fait pivoter le robot sur place d'un quart de tour
#   à gauche
def pqtg():
   phtg(); phtg()
```

Voici de nouvelles commandes du $\mathcal{R}\!\mathit{obot}$ qui peuvent s'exprimer à l'aide de procédures en utilisant des commandes que nous connaissons déjà :

```
# Fait pivoter le robot sur place de 3 huitièmes de
#   tour à droite
def p3htd():
   pqtd(); phtd()
# Fait pivoter le robot sur place de 3 huitièmes de
#   tour à gauche
def p3htg():
   pqtg(); phtg()
# Fait pivoter le robot sur place d'un demi-tour
def pdt():
   pqtd(); pqtd()
```

Exercice 12 *Écrire des programmes Python comportant des procédures pour réaliser les croix, l'étoile astroïdale, la loupe ou le double carré cranté ci-dessous.* □

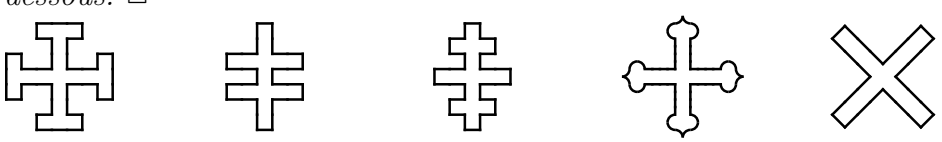


FIG. 5.3 – *Croix potencée, double, papale, tréflée et de Saint André*

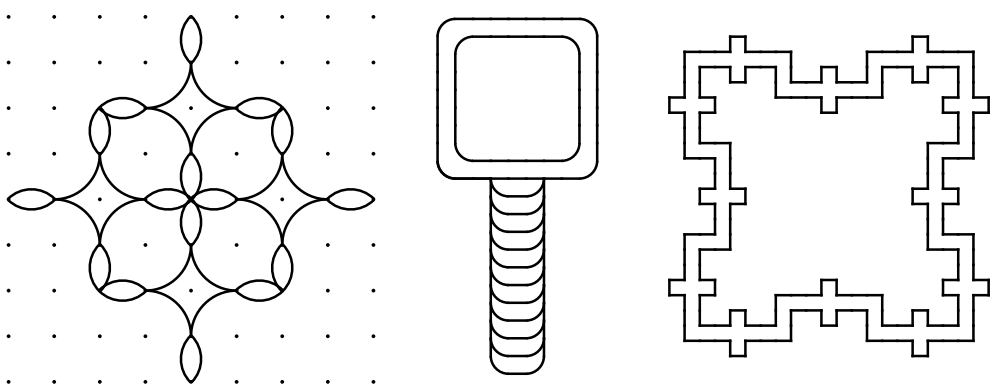


FIG. 5.4 – *Étoile astroïdale, loupe et double carré cranté*

Corrigé 12 *Nous donnons un exemple de programme, les autres se trouvant corrigés dans le dossier* `livre/05_procedures`.

```
from LibRobot import *

# Dessiner une étoile formée de quatre astroïdes
# à boucles
def arc():
   phtg(); vd(); phtg(); vd(); phtg(); vd()

def astroide():
   arc(); p3htd(); arc(); p3htd(); arc(); p3htd()
   arc()

def etoile():
   astroide(); phtd(); astroide(); phtd()
   astroide(); phtd(); astroide()

etoile(); st()
```

□

Addenda

La notion de procédure introduite par Von Neumann et Turing fût utilisée en 1949 sous le nom de *routine* sur l'EDSAC (*Electronic Delay Storage Automatic Computer*) généralement considéré comme le premier ordinateur à programme enregistré ayant fonctionné. Cette machine fût construite dès 1947 à l'université de Cambridge en Angleterre par l'équipe de Wilkes [a] et exécuta ses premiers programmes en Mai 1949 [25].

Les procédures illustrent le concept d'*abstraction* : les propriétés d'un objet sont isolées afin de les utiliser ou de les étudier à part. Plus précisément, une procédure donne un nom à un algorithme de construction d'une partie d'un dessin. Dans le corps de la procédure, on s'intéresse à la façon de construire le dessin. Dans les appels on s'intéresse uniquement au dessin terminé, tout en désirant complètement ignorer comment le dessin est obtenu. On sépare donc l'objet et la façon de le construire, le quoi du comment.

[a] Maurice Wilkes, l'un des inventeurs du radar.

Nous trouvons utile de décrire le dessin réalisé par une procédure dans un commentaire placé au début de la procédure que l'on appelle la *spécification* de la procédure. On remarquera que pour qu'une procédure puisse être utilisée sans jamais consulter son corps, il faut que la spécification de la procédure décrive quel dessin elle fait mais également quelles sont les positions et orientations du $\mathcal{R}o\underline{b}ot$ avant et après la construction du dessin. Il se dégage ainsi une notion d'*invariant* sur laquelle nous reviendrons dans les chapitres suivants sur l'itération.

Les procédures sont à la base des méthodes de structuration des très grands programmes : pour être compréhensibles, les programmes de plusieurs milliers de lignes doivent être découpés en procédures ne dépassant pas quelques pages. Les procédures constituent donc un moyen élégant pour segmenter logiquement un programme. Cependant le concept de procédure est insuffisant pour structurer des programmes de plusieurs millions de lignes. On utilise alors la notion de *module* (on dit également *unité*) et de *bibliothèque* pour regrouper des ensembles de procédures.

Des procédures regroupées dans une *bibliothèque* (en Python on dit aussi *module*) peuvent être fournies aux programmeurs pour les aider dans leurs tâches. Pour utiliser une procédure, le programmeur doit connaître son nom et sa spécification sans avoir besoin de connaître son fonctionnement interne. La ligne '**from** `LibRobot` **import** ∗' au début du programme a pour but d'indiquer à l'interpréteur quelle est la bibliothèque utilisée dans ce programme. Les commmandes du $\mathcal{R}o\underline{b}ot$ sont des procédures regroupées dans des bibliothèques.

L'analyse de dessins géométriques pour en dégager une structure que l'on peut décrire algorithmiquement à l'aide de procédures est très formatrice pour l'esprit. En ce sens l'apprentissage de la programmation peut jouer pour les enfants un rôle formateur analogue à celui que jouait autrefois la géométrie euclidienne.

6

Sauts du robot au bord et au centre du cadre

Notre $\mathcal{R}^{o}_{b}{}^{o}t$ peut sauter au bord ou au centre du cadre (qui délimite la zone de dessin sur l'écran de l'ordinateur), sans changer son orientation, ni rien dessiner. C'est exactement comme si nous pouvions le porter à la main pour le poser au bord ou au centre du cadre comme on peut le faire avec une voiture télécommandée. L'ordinateur le fait à notre place grâce aux commandes données dans la table ci-dessous et illustrées à la figure 6.1 (page 72) :

`ag()`	place le $\mathcal{R}^{o}_{b}{}^{o}t$ à gauche de l'écran, sans changer sa position verticale, ni son orientation, et sans rien dessiner.
`ad()`	place le $\mathcal{R}^{o}_{b}{}^{o}t$ à droite de l'écran, sans changer sa position verticale, ni son orientation, et sans rien dessiner.
`eh()`	place le $\mathcal{R}^{o}_{b}{}^{o}t$ en haut de l'écran, sans changer sa position horizontale, ni son orientation, et sans rien dessiner.
`eb()`	place le $\mathcal{R}^{o}_{b}{}^{o}t$ en bas de l'écran, sans changer sa position horizontale, ni son orientation, et sans rien dessiner.
`ce()`	place le $\mathcal{R}^{o}_{b}{}^{o}t$ au centre de l'écran, sans changer son orientation, et sans rien dessiner.

TAB. 6.1 – *Sauts du $\mathcal{R}^{o}_{b}{}^{o}t$ au bord et au centre du cadre*

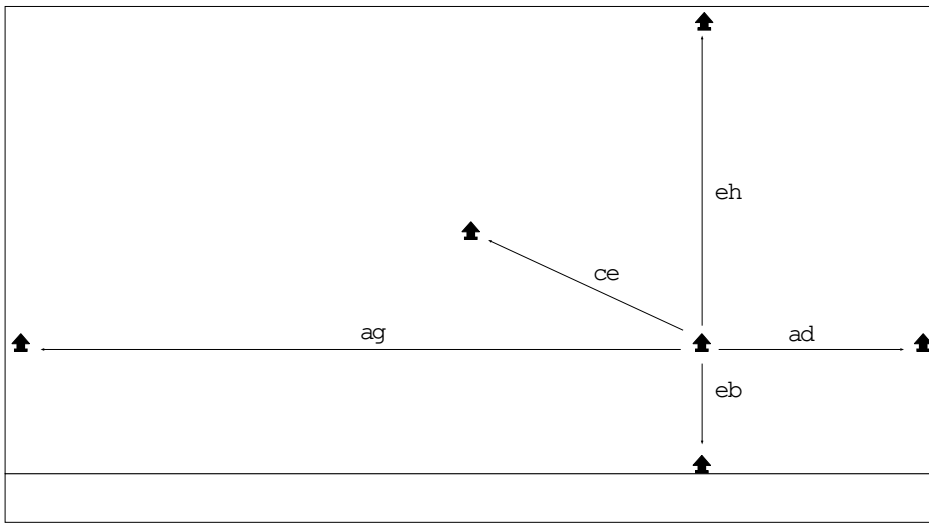
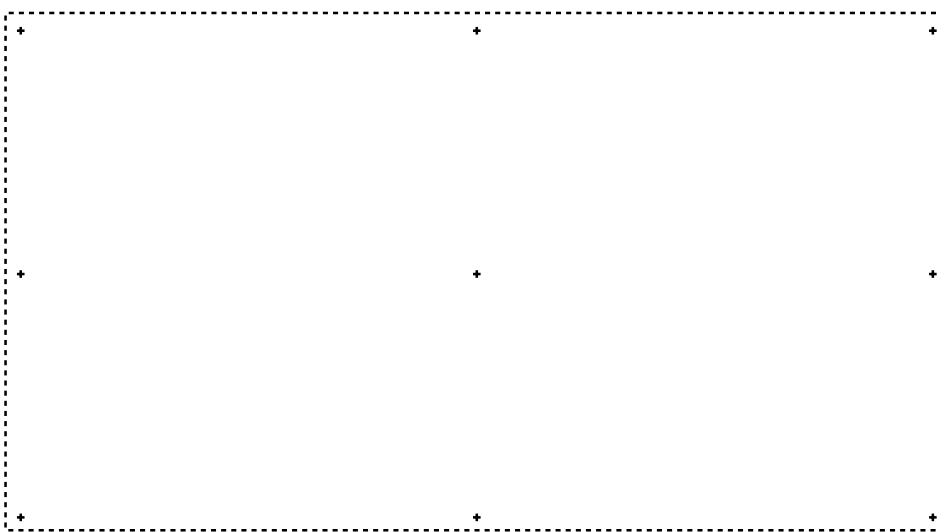


FIG. 6.1 – *Sauts du Robot au bord et au centre du cadre*

Exercice 13 *Écrire un programme Python qui dessine des croix au bord et au centre de l'écran comme le montre la figure ci-dessous.* □



Corrigé 13

```
from LibRobot import *

# Dessiner des croix au bord et au centre du cadre
ag(); dc()  # au milieu du bord gauche
ad(); dc()  # au milieu du bord droit
ce(); eh(); dc()  # au milieu du bord supérieur
eb(); dc()  # au milieu du bord inférieur
eh(); ag(); dc()  # au coin supérieur gauche
ad(); dc()  # au coin supérieur droit
eb(); ag(); dc()  # au coin inférieur gauche
ad(); dc()  # au coin inférieur droit
ce(); dc()  # au centre
st()
```

□

Addenda

Les commandes de déplacement du $\mathcal{R}\text{o}\underbrace{b}\text{o}t$ au bord du cadre seront définies en Python ultérieurement après avoir expliqué les notions de test et d'itération. Nous les introduisons dès maintenant pour pouvoir dessiner aisément des frises. Dans le chapitre suivant ces frises à motifs répétitifs vont nous permettre d'introduire simplement la notion d'itération bornée.

La commande `ce()` de recentrage du $\mathcal{R}\text{o}\underbrace{b}\text{o}t$ sera essentielle dès que nous utiliserons un repère cartésien pour désigner les points de la grille.

FIG. 6.2 – *Croix au bord et au centre du cadre*

7

Boucles “for”

7.1 Répétitions

Nous avons déjà vu quelques exemples de programmes Python dans lesquels une séquence de commandes du $\mathcal{R}\!\mathcal{o}\!\mathcal{b}\!\mathcal{o}t$ ou d'appels de procédures se trouve répétée de nombreuses fois de suite. C'est le cas par exemple dans le programme `FriseDeN_1` dont l'exécution produit la frise ci-dessous :

```
from LibRobot import *
# Dessiner une frise de N
ag()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
av(); av(); pdt(); vg(); vd(); pdt()
```

```
av(); av(); pdt(); vg(); vd(); pdt()
st()
```

Pour éviter les répétitions successives des mêmes séquences d'instructions, nous allons étudier une nouvelle instruction du langage Python qui s'appelle la *boucle "for"* (le mot anglais "for", qui signifie "pour", se prononce comme il s'écrit [1]). Voici le programme `FriseDeN_1` de dessin de la frise ci-dessous réécrit en utilisant une boucle "for" :

```
from LibRobot import *
# Dessiner une frise de N avec une boucle "for"
ag()
for j in range(16):
   av(); av(); pdt(); vg(); vd(); pdt()
st()
```

En français, la boucle '**for** `j` **in** `range(16) :` ...' peut se traduire par 'pour j parmi les 16 premiers nombres faire ...'. Le mot anglais **in**, qui signifie 'dans', se prononce 'inne'. Le mot anglais `range`, qui signifie 'suite' ou 'étendue', se prononce 'rèïndje'.

La séquence d'instructions à répéter s'écrit sur les lignes qui suivent les deux-points ' :', décalées vers la droite. Elle s'appelle le *corps de la boucle*. Le *nombre d'itérations* c'est-à-dire le nombre de fois qu'il faut répéter le corps de la boucle est indiqué entre les parenthèses de `range`. Dans notre exemple, il faut répéter 16 fois de suite le corps de la boucle '`av() ; av() ; pdt() ; vg() ; vd() ; pdt()`'.

7.2 Le compteur de boucle

Juste après le mot **for** nous trouvons le nom `j` d'un *compteur*. Ce compteur va servir à l'ordinateur pour compter le nombre de fois qu'il aura à répéter la séquence d'instructions dans la boucle "for", exactement comme nous pourrions le faire en comptant sur nos doigts ! Pour cela l'ordinateur utilise une case de sa mémoire, qui contient successivement les nombres entiers 0, 1, 2, ..., 15. En effet, `range(16)` désigne la suite des 16 nombres

1. Nous parlons de boucle "for" et non pas de boucle "pour" pour rester dans le contexte du vocabulaire utilisé dans le programme.

entiers qui commence à 0 et s'arrête juste avant 16 (comme les informaticiens, Python compte à partir de 0). On dit que `j` est une *variable* entière puisque sa valeur varie d'une itération à l'autre.

Nous devons donner un nom à ce compteur. C'est un identificateur que nous pouvons choisir comme il nous plaît (ici par exemple `j`), en respectant la règle des identificateurs : il commence par une lettre (ou un souligné '_'), ne contient que des lettres, des chiffres et des soulignés, et ne doit pas être un mot réservé de Python. Si l'on écrit '**for** `1er` **in** ...', Python répond '`SyntaxError: invalid decimal literal`' (nombre décimal incorrect, puisque `1er` commence par un chiffre). Si l'on écrit '**for in in** ...', il répond '`SyntaxError: invalid syntax`' (syntaxe incorrecte).

En Python, il n'est pas nécessaire d'annoncer à l'avance que l'on va utiliser un compteur [2] : le compteur est créé par la boucle "for" elle-même. Python sait que c'est un nombre entier puisqu'il prend les valeurs 0, 1, 2, ... Après la boucle, le compteur garde la dernière valeur qu'il a prise (15 dans notre exemple), mais il vaut mieux ne pas s'en servir : si la boucle n'a fait aucune itération, le compteur n'existe pas et Python indique l'erreur '`NameError: name 'j' is not defined`' (le nom `j` n'est pas défini).

Pour que le compteur prenne les valeurs 1, 2, 3, ..., 16, on écrirait '**for** `j` **in** `range(1, 17) :`'. De manière générale, `range(`a`, `b`)` désigne la suite des nombres entiers de a à $b-1$: elle commence à a et s'arrête juste *avant* b. Ainsi `range(1, 5)` désigne 1, 2, 3, 4 et `range(`n`)`, qui est une abréviation de `range(0, `n`)`, désigne les n nombres 0, 1, ..., $n-1$. Pour que le compteur aille de 1 à n, on écrit donc '**for** `i` **in** `range(1, `n` + 1) :`'.

Le compteur de la boucle du programme `FriseDeN_2` ne sert qu'à compter les itérations. Il est alors plus simple d'écrire `range(16)`. Nous verrons au chapitre suivant que l'on peut aussi utiliser sa valeur dans le corps de la boucle.

7.3 La boucle "for"

Une *boucle "for"* a la forme suivante :

2. D'autres langages comme Pascal obligent à *déclarer* le compteur au début du programme ou de la procédure, en indiquant qu'il désigne un nombre entier.

Boucle “for” :

for ␣ Identificateur ␣ **in** ␣ `range(`Nombre d'itérations`):`
... Instructions du corps de la boucle (décalées) ...

L'identificateur est le nom du compteur d'itérations. La séquence d'instructions qui est répétée s'appelle le *corps de la boucle*. Comme le corps d'une procédure, elle s'écrit sur les lignes qui suivent les deux-points ‘ `:`’, toutes décalées vers la droite du même nombre de blancs (quatre en général), une instruction par ligne ou plusieurs instructions sur une même ligne séparées par des ‘ `;`’. C'est ce décalage, appelé *indentation*, qui indique à Python où commence et où finit le corps de la boucle : la boucle se termine à la première ligne qui n'est plus décalée.

Il ne faut pas oublier les blancs ␣ autour du mot **in**. Si l'on écrit `jin` au lieu de `j` **in**, ou `inrange` au lieu de **in** `range`, Python ne reconnaît plus le mot **in** et signale ‘`SyntaxError: invalid syntax`’. Si l'on oublie les deux-points ‘ `:`’ à la fin de la ligne du **for**, Python indique ‘`SyntaxError: expected ':'`’ (deux-points attendu).

Une boucle “for” est elle-même une instruction. Les instructions qui la suivent s'écrivent sans décalage, au même niveau que le **for**, comme le `st()` du programme `FriseDeN_2`. Si l'on décalait aussi la ligne `st()`, elle ferait partie du corps de la boucle et serait répétée 16 fois ! Si l'on oublie de décaler le corps de la boucle, Python indique ‘`IndentationError: expected an indented block after 'for' statement on line 5`’ (erreur d'indentation : un bloc décalé est attendu après l'instruction **for** de la ligne 5). Si les lignes du corps ne sont pas toutes décalées du même nombre de blancs, Python signale ‘`IndentationError: unexpected indent`’ (décalage inattendu) ou ‘`IndentationError: unindent does not match any outer indentation level`’ (le retour à gauche ne correspond à aucun niveau de décalage précédent).

Si le ‘Nombre d'itérations’ est plus petit que 1 (0 par exemple) les instructions du corps de la boucle ne sont pas exécutées du tout. Il ne faut pas mettre de point-virgule ‘;’ après les deux-points : Python signale alors ‘`SyntaxError: invalid syntax`’ juste après les deux-points.

Quand le corps de la boucle est court, on peut écrire toute la boucle sur une seule ligne, comme dans ‘**for** `i` **in** `range(10) : av()`’. Les blancs supplémentaires à l'intérieur d'une ligne servent seulement à rendre le programme plus lisible, mais ceux du début de ligne sont importants.

Exemple 10 (Frise de pointes) La frise suivante :

est produite par le programme ci-dessous :

```
from LibRobot import *
# Dessiner une frise de pointes
ag(); pqtd()
for j in range(32):
   lc(); av(); bc(); pdt(); vd(); pdt()
   av(); av(); pdt(); vd()
st()
```

□

7.4 Utilisation de la boucle "for" dans une procédure

Le corps d'une procédure peut contenir des boucles "for", comme n'importe quelle autre instruction :

Déclaration de procédure :

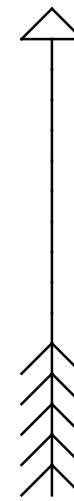
```
def ␣ nomDeLaProcedure():
    ... Instructions du corps de la procédure [a] (décalées) ...
```

[a] Y compris des boucles "for" utilisant des compteurs `i`, `j`, ...

Puisque le compteur d'une boucle ne sert que dans le corps de cette boucle et qu'il repart de sa première valeur à chaque nouvelle boucle, un même identificateur peut servir de compteur dans plusieurs boucles différentes, qu'elles soient dans des procédures différentes ou les unes après les autres dans la même procédure. Ceci est illustré dans le programme `Fleche` ci-dessous où des compteurs de boucles différents, utilisés dans des procédures différentes (`empennage` et `tige`), portent le même nom `i`.

Exemple 11 (Flèche)

```
from LibRobot import *
# Dessiner une flèche verticale
def plume():
    av(); p3htg(); av(); lc(); p3htg(); av(); av()
    p3htg(); bc(); av(); phtd()
def empennage():
    for i in range(5):
        plume()
def tige():
    for i in range(10):
        av()
def pointe():
    pqtg(); av(); p3htd(); av(); pqtd(); av()
    p3htd(); av()
eb(); empennage(); tige(); pointe(); st()
```



□

7.5 Boucles “for” imbriquées (indépendantes)

Le corps d’une boucle “for” :

for ␣ Identificateur1 ␣ **in** ␣ `range(`Nombre d’itérations1`):`

...

peut à nouveau contenir des boucles “for” :

for ␣ Identificateur1 ␣ **in** ␣ `range(`Nombre d’itérations1`):`

...

for ␣ Identificateur2 ␣ **in** ␣ `range(`Nombre d’itérations2`):`

...

...

On dit alors que les boucles “for” sont *imbriquées*. La boucle utilisant le compteur Identificateur2 pour faire Nombre d’itérations2 itérations s’appelle la *boucle intérieure*. La boucle utilisant le compteur Identificateur1 pour faire Nombre d’itérations1 itérations s’appelle la *boucle extérieure*. Le corps de la boucle intérieure est décalé deux fois : une fois parce qu’il fait partie du corps de la boucle extérieure, une deuxième fois parce qu’il est le corps de la boucle intérieure. La boucle intérieure se termine à la première ligne qui revient au

décalage de son **for** ; la boucle extérieure à la première ligne qui revient au décalage du premier **for**. C'est donc le décalage qui indique, sans ambiguïté, à quel **for** appartient chaque instruction. Considérons par exemple le dessin d'un carré :

```
from LibRobot import *
# Dessiner un carré avec des procédures
# Dessiner un côté du carré et pivoter à droite
def cote():
    for j in range(4):
        av()
    pqtd()
# Dessiner un carré
def carre():
    for i in range(4):
        cote()
carre(); st()
```

En remplaçant les appels des procédures par leurs corps, on obtient deux boucles imbriquées :

```
from LibRobot import *
# Dessiner un carré avec des boucles "for" imbriquées
# Dessiner un carré
for i in range(4):
    # Dessiner un côté du carré et pivoter à droite
    for j in range(4):
        av()
    pqtd()
st()
```

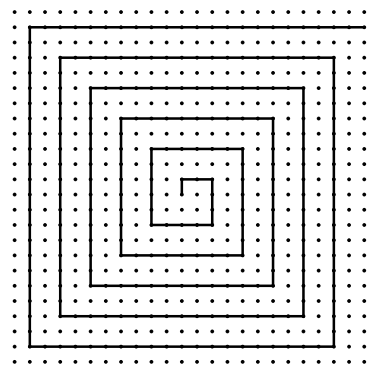
La boucle intérieure sert à construire un côté tandis que la boucle extérieure dessine les quatre côtés.

7.6 Boucles "for" imbriquées (dépendantes)

Dans des boucles "for" imbriquées, le nombre d'itérations dans la boucle intérieure peut être différent à chaque fois que le corps de la boucle extérieure est exécutée. Commençons par considérer l'exemple simple de segments de droite de longueurs 1, 2, 3, ...disposés en spirale (le dessin est représenté ci-dessous à l'échelle $\frac{1}{2}$ avec la grille) :

```
from LibRobot import *

# Dessiner une spirale carrée
for i in range(1, 23):
   for j in range(i):
      av()
   pqtd()
st()
```



Le nombre d'itérations dans la boucle "for" intérieure est égal à la valeur du compteur `i` de la boucle extérieure, c'est-à-dire successivement à 1, 2, 3, ..., 22 (puisque `range(1, 23)` s'arrête juste avant 23). Le programme `SpiraleCarree_1` est donc équivalent au programme `SpiraleCarree_2` ci-dessous où les boucles "for" ont été éliminées pour montrer exactement quelle est la séquence de commandes exécutée par le $\mathcal{R}o\underset{\smile}{b}ot$.

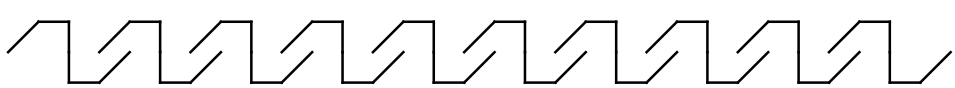
```
from LibRobot import *

# Dessiner une spirale carrée sans boucles "for"
av(); pqtd()
av(); av(); pqtd()
av(); av(); av(); pqtd()
av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av()
pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
```

```
av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av()
pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); pqtd()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); av(); av()
av(); av(); av(); av(); av(); av(); pqtd()
st()
```

7.7 Invariants de boucles “for”

Quand on dessine une frise comme celle-ci :



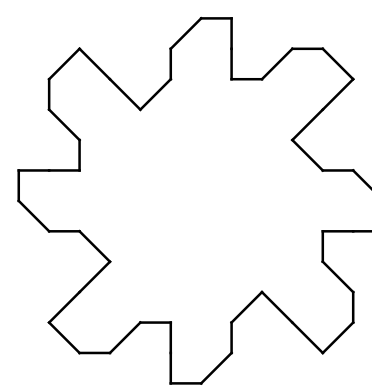
avec une boucle “for”, il faut que le $\mathcal{R}\!\mathit{obot}$ se trouve toujours dans la même direction et dans la même position par rapport au prochain motif qui va être dessiné en exécutant le corps de la boucle. Autrement dit, à la fin du corps de la boucle il faut remettre le $\mathcal{R}\!\mathit{obot}$ dans la position de départ :

```
from LibRobot import *
# Dessiner une frise de tildes
# Dessiner un motif en forme de tilde ~
def tilde():
   phtd(); av(); phtd(); av()
   pqtd(); av(); av(); pqtg()
   av(); phtg(); av()
ag()
for i in range(10):
   tilde()
   # Reculer au début du tilde suivant
   lc(); p3htg(); av(); bc()
   # Orienter le robot au nord
   pqtd()
st()
```

On dit que la position du $\mathcal{R}\!\mathit{obot}$ est un *invariant* de la boucle en ce sens qu’elle doit être la même à chaque itération.

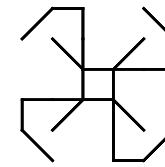
Si dans notre exemple on dessine un motif puis le suivant sans remettre entre-temps le $\mathcal{R}\!\mathit{obot}$ dans la position de départ, on obtient le résultat suivant, dont il faut avouer qu’il n’est pas très ressemblant à l’original :

```
from LibRobot import *
# Dessiner une frise de tildes (première erreur)
# Dessiner un motif en forme de tilde ~
def tilde():
   phtd(); av(); phtd(); av()
   pqtd(); av(); av(); pqtg()
   av(); phtg(); av()
for i in range(10):
   tilde()
st()
```



Si l’on pense à faire reculer le $\mathcal{R}\!\mathit{obot}$ mais que l’on oublie de le remettre dans la bonne direction, le résultat est encore plus étrange :

```
from LibRobot import *
# Dessiner une frise de tildes (deuxième erreur)
# Dessiner un motif en forme de tilde ~
def tilde():
    phtd(); av(); phtd(); av()
    pqtd(); av(); av(); pqtg()
    av(); phtg(); av()
for i in range(13):
    tilde()
    # Reculer au début du tilde suivant
    lc(); p3htg(); av(); bc()
st()
```



Une bonne façon de trouver les erreurs dans une boucle "for" est donc de chercher l'invariant, c'est-à-dire la position et l'orientation dans lesquelles doit se trouver le $\mathcal{R}^{o}\!\!\smile^{o}t$ avant de dessiner le motif en exécutant le corps de la boucle. Il faut alors vérifier que le $\mathcal{R}^{o}\!\!\smile^{o}t$ est convenablement placé quand on arrive pour la première fois dans la boucle et qu'il l'est à nouveau après avoir exécuté le corps de la boucle.

La même notion d'invariant s'applique pour les procédures. Avant chaque appel d'une procédure, il faut que le robot soit dans un ou plusieurs états possibles qui doivent être parfaitement définis par la position levée ou baissée du crayon du $\mathcal{R}^{o}\!\!\smile^{o}t$ et par la position et l'orientation du $\mathcal{R}^{o}\!\!\smile^{o}t$ par rapport à la figure dessinée par la procédure.

Exercice 14

_ *Écrire des programmes Python utilisant des boucles "for" pour dessiner quelques-unes des frises de la figure 7.3 page 87.*

_ *Écrire des programmes Python utilisant des boucles "for" dans des procédures pour dessiner quelques-unes des frises de la figure 7.4 page 88.*

_ *Écrire des programmes Python pour dessiner quelques-unes des rosaces, frises et pavage de la figure 7.5 page 89, en utilisant des boucles "for" imbriquées indépendantes.*

_ *Écrire des programmes Python pour dessiner quelques-unes des spirales, quadrillages et autres tuyaux d'orgues de la figure 7.1 ci-dessous, en utilisant des boucles "for" imbriquées dépendantes.*

_ *Écrire un programme Python pour dessiner le pavage de la figure 7.2 ci-dessous en indiquant avec précision les invariants des procédures et boucles*

"for". □

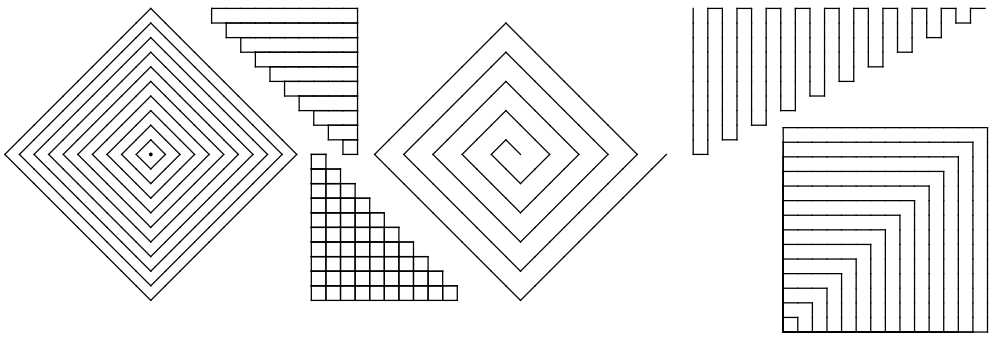


FIG. 7.1 – *Quadrillages, carrés, spirale et tuyaux d'orgues*

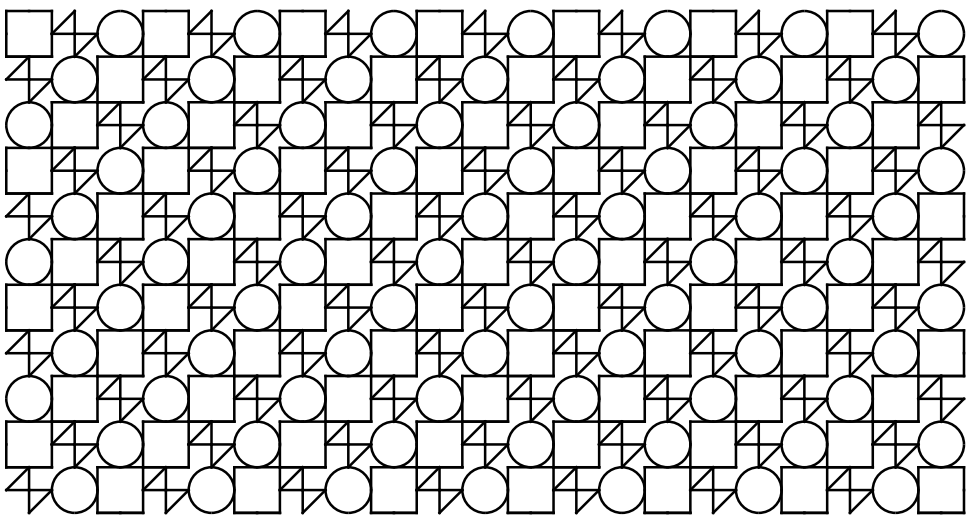


FIG. 7.2 – *Pavage de carrés, triangles et cercles*

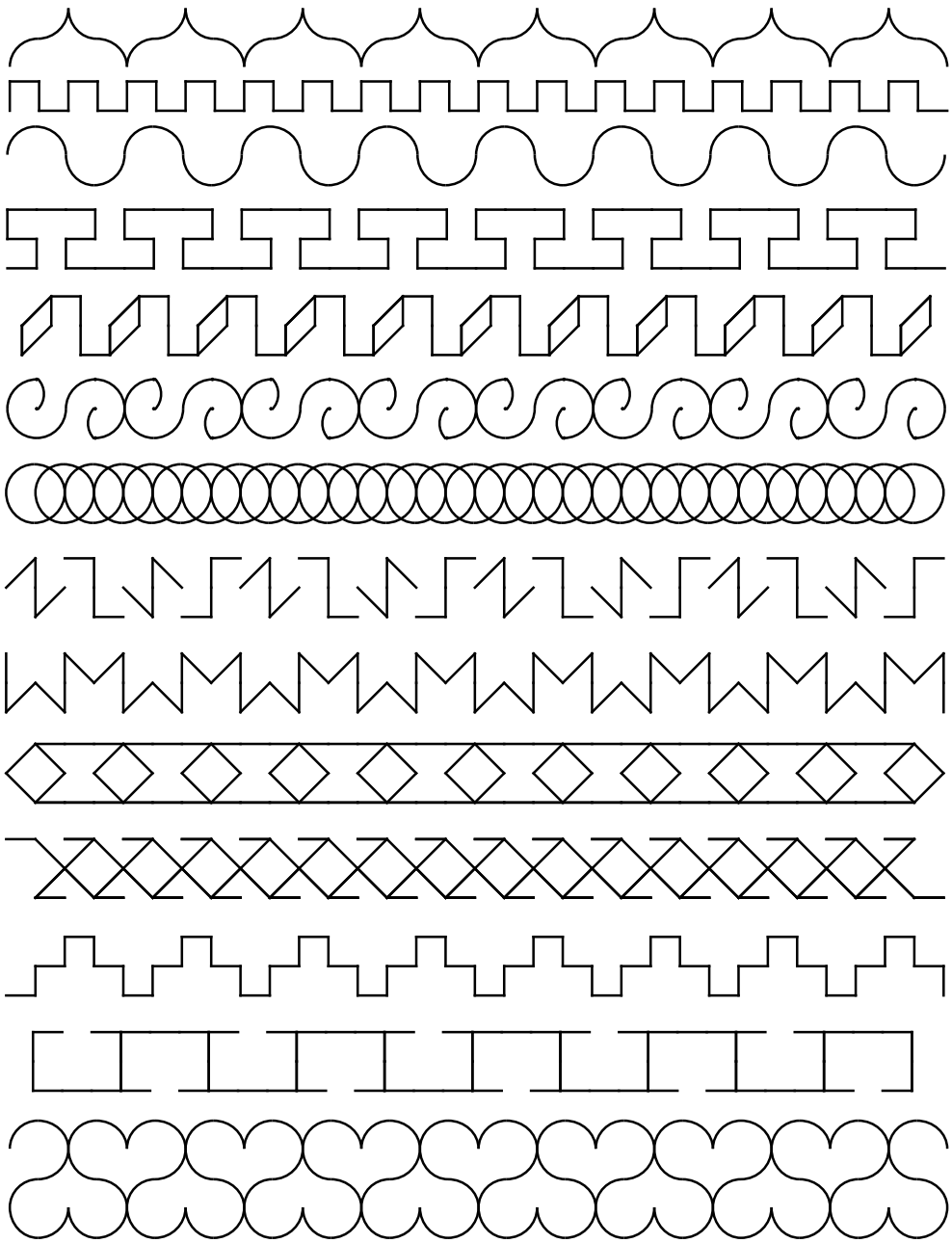


FIG. 7.3 – *Frises (1)*

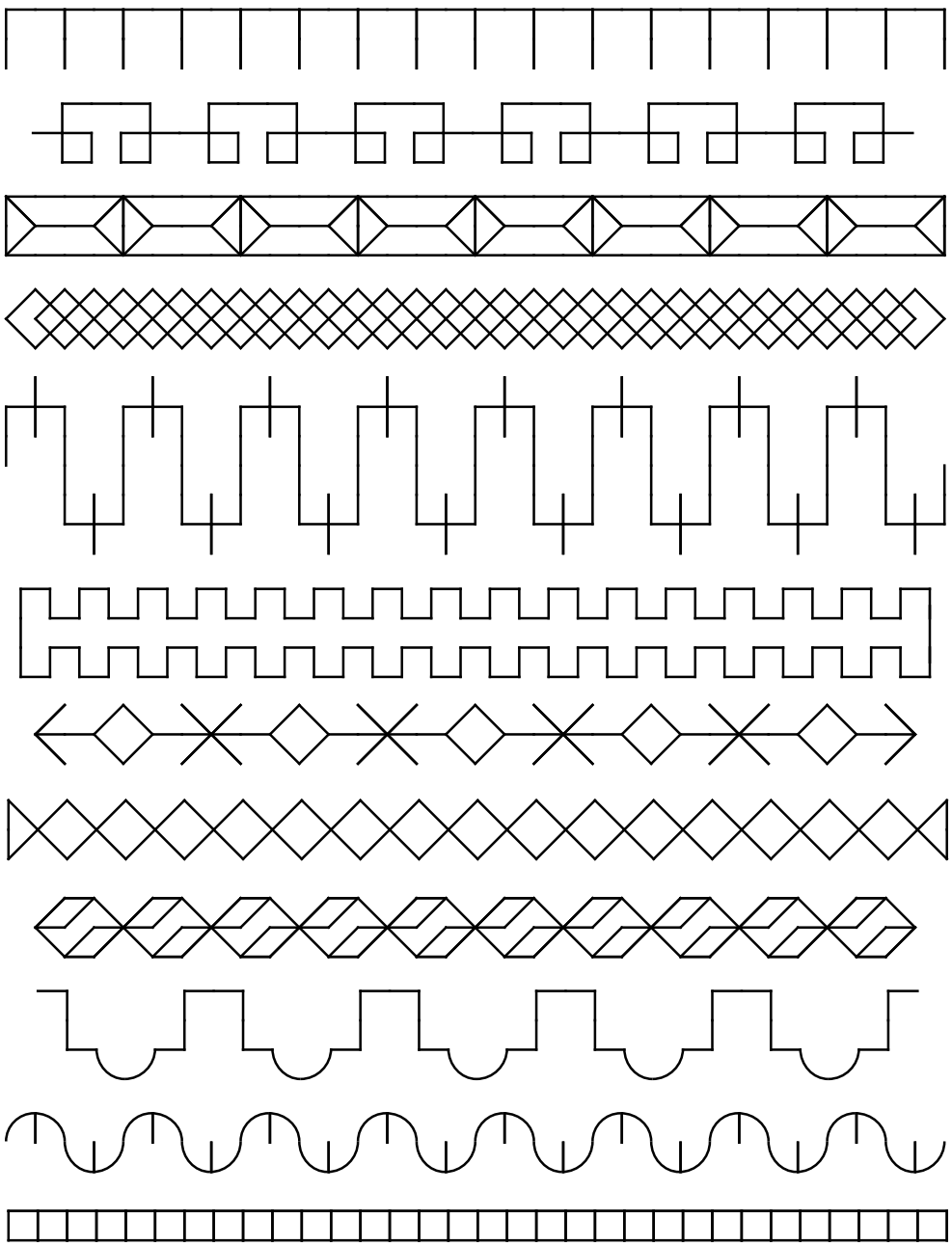


FIG. 7.4 – *Frises (2)*

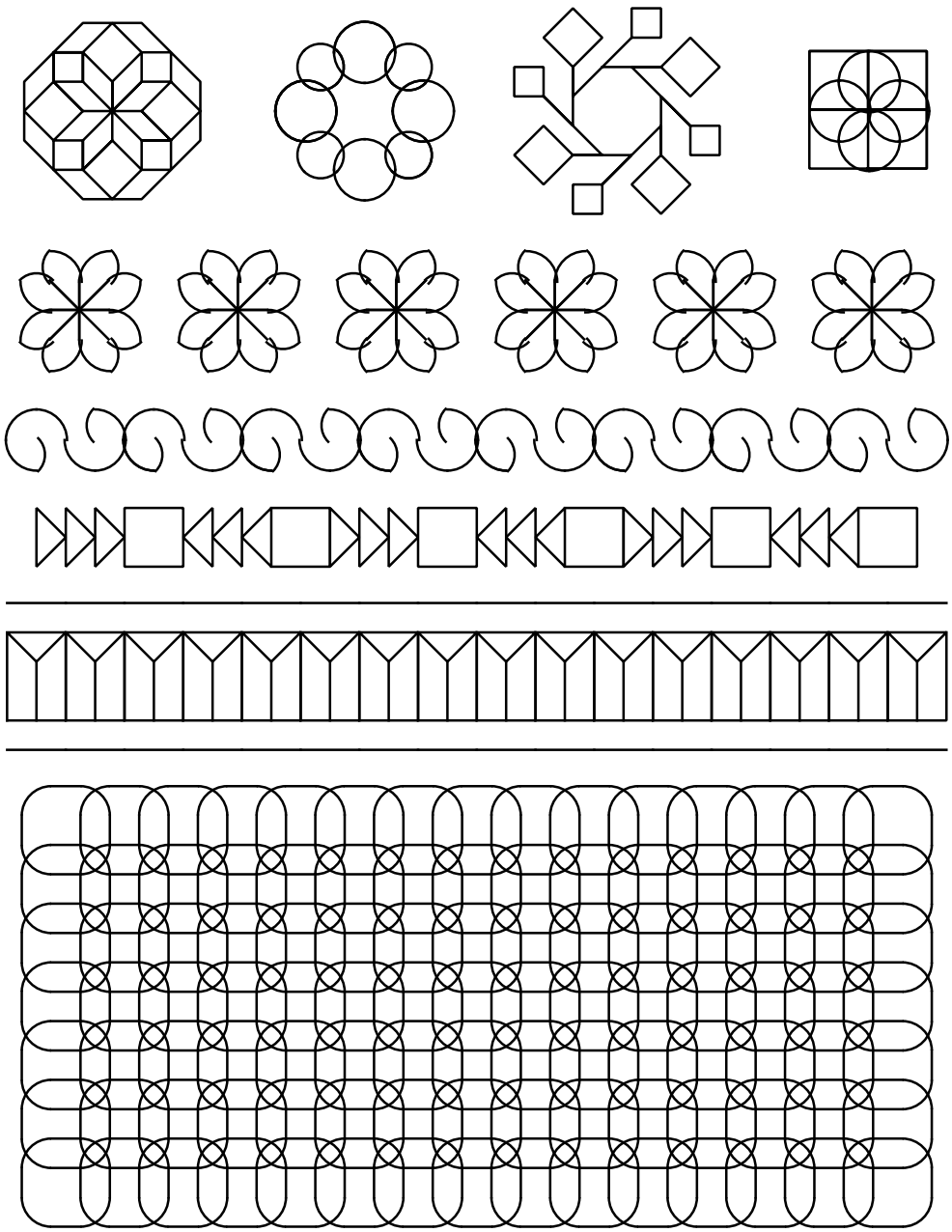


FIG. 7.5 – *Rosaces, frises et pavage*

Corrigé 14 *Nous donnons quelques programmes, les autres se trouvant dans le dossier* `livre/07_for` *:*

- ```
  from LibRobot import *
  # Dessiner une frise crantée

  # Dessiner le motif 'n_'
  def cran():
      av(); pqtd(); av(); pqtd(); av(); pqtg(); av()
      pqtg()

  ag()
  for i in range(20):
      cran()
  st()
  ```

- ```
  from LibRobot import *
  # Dessiner un quadrillage triangulaire
  # Dessiner 10 lignes de carrés juxtaposés
  # horizontalement
  pqtd()
  for i in range(1, 11):
      # Dessiner une ligne de i carrés juxtaposés
      # horizontalement, en partant en haut à gauche et
      # en arrivant en haut à droite, le robot étant
      # dans les deux cas orienté vers l'est.
      for j in range(i):
          # Dessiner un carré, en partant du coin
          # supérieur gauche et en arrivant au coin
          # supérieur droit, le robot étant dans les deux
          # cas orienté vers l'est.
          for k in range(4):
              av(); pqtd()
          av()
      # Revenir, sans dessiner, au début de la ligne
      # suivante, le robot étant orienté vers l'est au
      # départ et à l'arrivée.
      lc(); pdt()
      for j in range(i):
          av()
      pqtg(); av(); pqtg(); bc()
  st()
  ```

- ```
  from LibRobot import *
  # Dessiner un pavage de carrés, triangles et cercles
  ```
  ```

```
# Dessiner un carré de côté de taille 2, le robot
# partant du coin inférieur gauche tourné vers l'est,
# et arrivant crayon baissé au coin inférieur droit
# également orienté à l'est.
def carre():
   # Dessiner le carré.
   av(); av(); pqtg(); av(); av(); pqtg()
   av(); av(); pqtg(); av(); av(); pqtg()
   # Avancer au coin inférieur droit.
   av(); av()

# Dessiner deux triangles opposés par un sommet
# inscrits dans un carré invisible du quadrillage de
# côté 2, en partant du coin inférieur gauche du carré
# tourné vers l'est, et en arrivant au coin inférieur
# droit, crayon baissé et tourné vers l'est.
def triangles():
   lc(); av(); bc()
   pqtg(); av(); av(); p3htg(); av(); p3htg()
   av(); av(); p3htd(); av(); p3htg()
   lc(); av(); bc()

# Dessiner un cercle de rayon 1 inscrit dans un carré
# invisible du quadrillage de côté 2, en partant du
# coin inférieur gauche du carré tourné vers l'est, et
# en arrivant au coin inférieur droit du carré, crayon
# baissé et tourné vers l'est.
def cercle():
   lc(); av(); bc()
   vg(); vg(); vg(); vg()
   lc(); av(); bc()

# Dessiner une ligne constituée d'un motif répété 7
# fois, le motif étant formé d'un carré, de triangles
# et d'un cercle juxtaposés, le robot partant du coin
# inférieur gauche de la ligne orienté vers l'est, et
# arrivant au coin inférieur droit également orienté à
# l'est.
def ligneCarre():
   for i in range(7):
      # Le robot est crayon baissé, à l'est, en bas à
      # gauche du motif
      carre(); triangles(); cercle()

# Comme ligneCarre, avec le motif formé de triangles,
# d'un cercle et d'un carré.
def ligneTriangles():
   for i in range(7):
```

```
      triangles(); cercle(); carre()

# Comme ligneCarre, avec le motif formé d'un cercle,
# d'un carré et de triangles.
def ligneCercle():
   for i in range(7):
      cercle(); carre(); triangles()

# Ramener le robot du coin inférieur droit d'une ligne
# au coin inférieur gauche de la ligne suivante, sans
# dessiner, orienté à l'est et crayon baissé.
def alaLigne():
   lc(); ag(); pqtd(); av(); av(); pqtg(); bc()

# Paver l'écran de 11 lignes de motifs commençant par
# un carré, puis par des triangles, puis par un
# cercle, du coin supérieur droit au coin inférieur
# droit de l'écran, le robot étant orienté à l'est.
def pavage():
   # Dessiner les lignes 1 à 9 de haut en bas.
   for j in range(3):
      alaLigne(); ligneCarre()
      alaLigne(); ligneTriangles()
      alaLigne(); ligneCercle()
   # Dessiner les lignes 10 puis 11.
   alaLigne(); ligneCarre()
   alaLigne(); ligneTriangles()

# Aller au coin supérieur droit, orienté à l'est.
eh(); ad(); pqtd()
# Paver l'écran.
pavage(); st()
```

□

Addenda

La boucle “for” est une des premières notions linguistiques qui soit apparue en informatique puisqu'elle fût inventée à l'ETH de Zürich en 1952 par Heinz Rutishauser [35].

La comparaison des dessins que l'on peut obtenir en utilisant la boucle “for” avec ceux que l'on peut faire en mode interactif donne la mesure de la puissance des ordinateurs quand il s'agit d'effectuer rapidement des tâches répétitives. En cela l'ordinateur se distingue très nettement des calculettes non programmables. On comprend également que cette puissance de calcul n'a rien à voir avec une quelconque forme de pensée ou d'intelligence si ce n'est celle du programmeur de l'ordinateur.

En Python, on utilise une boucle **for** `i` **in** `range(`n_1`,` n_2`)` `:` ... pour répéter $n_2 - n_1$ fois le corps de la boucle (les instructions décalées qui suivent les deux-points) sauf si $n_2 \leq n_1$ auquel cas le corps de la boucle n'est pas exécuté. A chaque itération, le compteur de boucle `i` prend les valeurs successives n_1, n_1+1, n_1+2, ..., n_2-1. Dans la forme **for** `i` **in** `range(`n_1`,` n_2`, -1)` `:` ..., `i` prend les valeurs successives n_1, n_1-1, n_1-2, ..., n_2+1 sauf si $n_2 \geq n_1$ auquel cas la boucle n'est pas exécutée ; par exemple **for** `i` **in** `range(`n`, 0, -1)` `:` fait prendre à `i` les valeurs n, $n-1$, ..., 1 (le troisième nombre, ici -1, est le pas qui est ajouté au compteur à chaque itération). Si le corps de la boucle modifie la valeur du compteur, cela n'a pas d'effet sur les itérations suivantes : à chaque itération, le compteur reprend la valeur suivante de la suite `range`. Quand le compteur ne sert à rien dans le corps de la boucle, les programmeurs Python le nomment souvent ‘`_`’, comme dans **for** `_` **in** `range(16)` `:` ...

L'utilisation de boucles “for” exerce la capacité de percevoir quand un dessin peut se décomposer en un certain nombre de sous-dessins identiques (dans le cas d'une boucle simple) ou similaires (dans le cas de boucles imbriquées). C'est un exercice quotidien pour l'informaticien qui doit savoir décomposer une tâche complexe en sous-tâches simples, ce qui demande parfois énormément d'imagination. Il s'agit d'un premier pas vers l'acquisition du raisonnement inductif des scientifiques qui, à partir d'observations spécifiques, créent une hypothèse générale rendant compte de ces observations, hypothèse qu'il reste à valider dans tous les cas possibles, ici de manière tout à fait expérimentale, en observant le dessin à l'écran. Les mathématiciens appellent *conjecture* une telle hypothèse induite de l'expérience et non encore démontrée.

Ce raisonnement inductif a longtemps été confondu avec le raisonnement par récurrence qui est essentiel en mathématiques. Il a fallu plus d'un millénaire pour que les mathématiciens (Fermat [a], Pascal [b] notamment) comprennent pleinement ce raisonnement par récurrence ([21] page 17).

Nous en sommes pour l'instant aux prémisses, l'absence d'induction correcte étant encore admise dans le raisonnement par récurrence : si le dessin est bien fait pour le motif de base et pour la première répétition du motif alors il est réputé correct, seul le problème du cadrage dans la fenêtre restant à résoudre par simple expérimentation.

La notion d'*invariant* (c'est-à-dire pour notre $\mathcal{R}\text{o}\underbrace{b}\text{o}t$ la description de ses positions et orientations possibles au début du corps des boucles) est due à Turing [c] en 1949, oubliée, puis redécouverte pat Naur [d] en 1966 et Floyd [e] en 1967. Elle repose sur un raisonnement par récurrence : si l'invariant est vrai quand on arrive pour la première fois dans la boucle (c'est-à-dire que la position et l'orientation du $\mathcal{R}\text{o}\underbrace{b}\text{o}t$ est conforme à la description qui en est faite dans l'invariant) et si, en supposant que l'invariant soit vrai au début de l'exécution du corps de la boucle, il reste encore vrai après l'exécution de ce corps de boucle alors l'invariant est toujours vrai au début de chaque itération dans la boucle et l'est encore quand on sort de la boucle. La notion s'applique aussi bien aux procédures : on peut utiliser un *invariant d'entrée* décrivant les positions et orientations du $\mathcal{R}\text{o}\underbrace{b}\text{o}t$ avant l'appel et un *invariant de sortie* décrivant ces positions et orientations après l'appel, de sorte que le corps de la procédure soit compréhensible sans avoir à consulter tous les points d'appel pour savoir dans quelles conditions la procédure est appelée.

[a] Pierre de Fermat, 1601–1665, mathématicien français.
[b] Blaise Pascal, 1623–1662, mathématicien français.
[c] Alan Turing, informaticien anglais.
[d] Peter Naur, informaticien danois.
[e] Robert Floyd, informaticien américain.

L'ordinateur peut être utilisé pour manipuler des programmes, en particulier pour découvrir automatiquement des invariants de forme simple en chaque point d'un programme. C'est l'objet de l'*interprétation abstraite* des programmes [9]. Pour le $\mathcal{R}^o\!\underline{b}^o t$, par l'exemple, une forme simple d'invariant est constituée par l'ensemble de ses orientations possibles, en ignorant la position de son crayon, ses coordonnées sur l'écran, etc. Les orientations possibles du $\mathcal{R}^o\!\underline{b}^o t$ concret peuvent être calculées en exécutant le programme avec un $\mathcal{R}^o\!\underline{b}^o t$ abstrait qui exécute les commandes du $\mathcal{R}^o\!\underline{b}^o t$ concret sans bouger mais en changeant simplement d'orientation. Pour ce $\mathcal{R}^o\!\underline{b}^o t$ abstrait la commande `av` ne fait rien car elle ne change pas l'orientation du $\mathcal{R}^o\!\underline{b}^o t$ concret ; la commande `vd` a le même effet que `pqtd` ; etc. Pendant l'exécution abstraite les orientations possibles du $\mathcal{R}^o\!\underline{b}^o t$ abstrait sont notées comme commentaire en chaque point du programme. L'exécution abstraite du programme s'arrête après stabilisation des résultats. Par exemple, pour le programme `FriseDeN_2` (page 76), on trouve :

```
from LibRobot import *
# Dessiner une frise de N avec une boucle "for"
# Nord
ag()
# Nord
for j in range(16):
   av(); av()  # Nord  # Nord
   pdt(); vg()  # Nord  # Sud
   vd(); pdt()  # Est  # Sud
# Nord
# Nord
st()
# Nord
```

La règle des signes est un exemple d'interprétation abstraite utilisée en mathématiques. Le signe d'une expression peut être calculé en utilisant le signe des variables, qui remplace leurs valeurs, grâce aux opérations abstraites comme $\oplus + \oplus = \oplus$, $\oplus - \ominus = \oplus$, $\oplus \times \ominus = \ominus$, etc. Le prix à payer pour cette simplification est évidemment l'imprécision des résultats, comme pour $\oplus + \ominus$.

8

Expressions entières

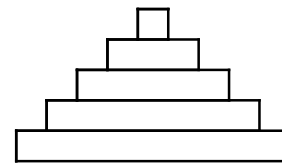
Un ordinateur est capable de faire des *additions* $(15 + 3) = 18$, des *soustractions* $(15 - 3) = 12$, des *multiplications* $(15 * 3) = 45$ et des *divisions* d'entiers $(15 \;//\; 3) = 5$. Si la division ne tombe pas juste, le résultat est le quotient entier. Le reste est donné par l'opération *modulo* notée `%`. Par exemple $(17 \;//\; 3) = 5$ et $(17 \;\%\; 3) = 2$ car la division du dividende 17 par le diviseur 3 donne le quotient 5 et le reste 2. En mathématiques on utilise une croix '$\times$' pour noter la multiplication tandis qu'en Python on utilise l'astérisque $*$. On évite ainsi de confondre le signe multiplié '$\times$' avec la lettre `x` minuscule. De même on utilise deux barres obliques '`//`' pour la division entière plutôt que le signe '$\div$' qui ne figure pas sur les claviers d'ordinateurs. Attention à ne pas écrire une seule barre oblique '`/`' : en Python, elle ne donne pas le quotient entier (17 / 3 ne vaut pas 5). Nous verrons plus tard qu'elle sert à calculer avec des nombres à virgule.

Pour faire un calcul comportant plusieurs opérations, on utilise des parenthèses. Par exemple $((2 * 3) - 1)$ est égal à 5. Le calcul commence par l'évaluation de l'expression entre parenthèses la plus intérieure. Par exemple $((((5+1)*3)-4) \;//\; 2) = (((6*3)-4) \;//\; 2) = ((18-4) \;//\; 2) = (14 \;//\; 2) = 7$.

Dans ces *expressions entières* nous pouvons également utiliser la valeur des compteurs de boucles "for". Par exemple lors des itérations successives dans le corps de la boucle "**for** `i` **in** `range(1, 6) :` ...", le compteur `i` vaut 1, puis 2, puis 3, puis 4 et enfin 5. Dans ce corps de boucle l'expression $((2*\texttt{i}) - 1)$ vaut donc successivement $((2 * 1) - 1) = 1$, $((2 * 2) - 1) = 3$, $((2 * 3) - 1) = 5$, $((2 * 4) - 1) = 7$ et enfin $((2 * 5) - 1) = 9$.

Exemple 12 (Pyramide) *Pour dessiner la pyramide ci-après, nous pou-*

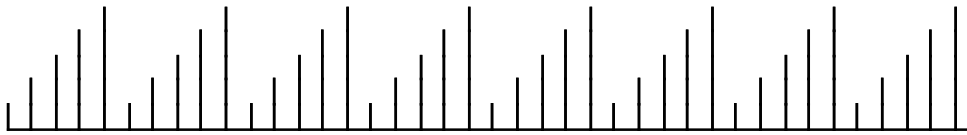
vons commencer par le sommet et dessiner de haut en bas des rectangles successifs de longueurs 1, 3, 5, 7, 9. La longueur du côté du `i`$^{\text{ème}}$ *rectangle à partir du sommet est donc* $((2 * \texttt{i}) - 1)$. *Dans le programme, cette expression est écrite '*`2 * i - 1`*' entre les parenthèses de* `range` *pour indiquer le nombre d'itérations de la boucle qui dessine un côté du rectangle.*



```
from LibRobot import *

# Dessiner une pyramide de 5 étages.
# Orienter le robot à l'ouest.
pqtg()
for i in range(1, 6):
   # Partant du coin inférieur gauche d'un étage avec le
   # robot orienté à l'ouest, dessiner le coin inférieur
   # gauche de l'étage inférieur et pivoter à l'est.
   av(); pqtg(); av(); pqtg()
   # Dessiner le i-ème étage rectangulaire.
   # Côté inférieur
   for j in range(2 * i - 1):
      av()
   # Extrémité droite
   pqtg(); av(); pqtg()
   # Côté supérieur
   for j in range(2 * i - 1):
      av()
   # Redescendre au coin inférieur gauche et orienter
   # le robot à l'ouest.
   pqtg(); av(); pqtd()
st()
```

Exercice 15 *Compléter le programme* `PiquetsEnDentsDeScie`*, en remplaçant les points de suspension '...' par une expression entière, pour obtenir l'alignement de piquets représenté ci-après :*

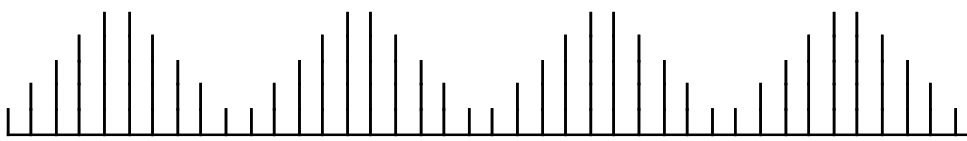


```
from LibRobot import *
# Dessiner un alignement de piquets en dents de scie
ag()
for i in range(1, 41):
   for j in range(2):
      for k in range(...):
         av()
      pdt()
   pqtd(); av(); pqtg()
st()
```

□

Exercice 16 *Écrire des programmes Python de commande du Robot pour réaliser les dessins géométriques de la figure 8.1 (page 100).* □

Exercice 17 (Difficile) *Compléter le programme* `VaguesDePiquets` *pour dessiner l'alignement de piquets représenté ci-dessous :*



```
from LibRobot import *
# Dessiner un alignement de piquets en vagues
ag()
for i in range(1, 9):
   for j in range(1, 6):
      for k in range(2):
         for l in range(...):
            av()
         pdt()
      pqtd(); av(); pqtg()
st()
```

□

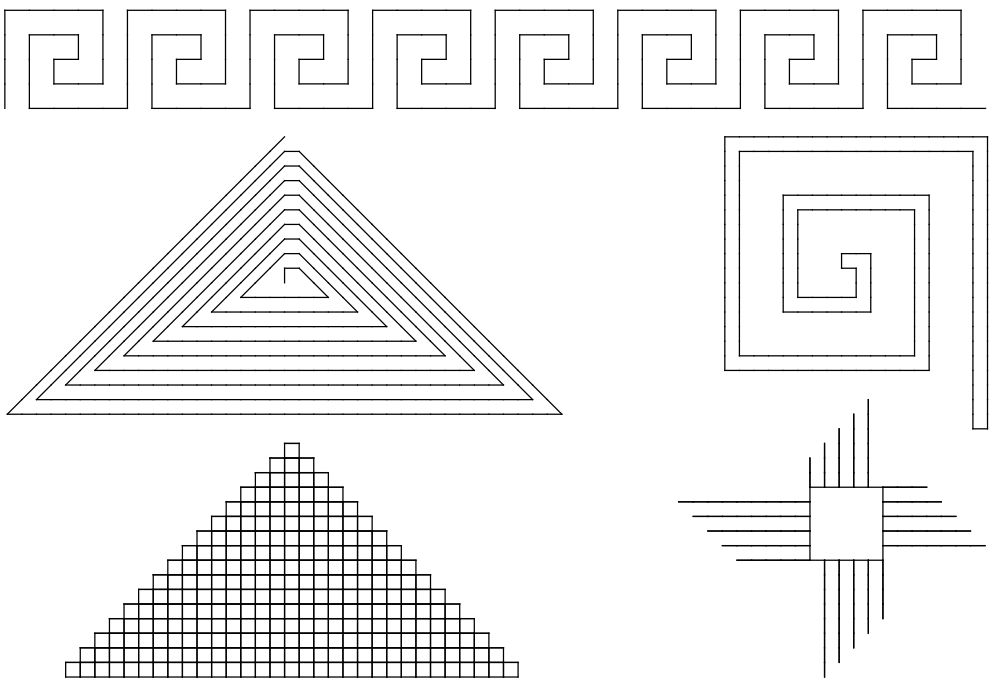


FIG. 8.1 – *Grecque, spirales, pavage triangulaire et carré à franges*

Exercice 18 (Difficile)

1. *Écrire un programme Python pour dessiner le pavage de Nicomaque*[1] *représenté à la figure 8.2 page 102 en utilisant des boucles "for" imbriquées dépendantes. Décrire les invariants de boucles par des commentaires dans le programme. Le pavage de Nicomaque est obtenu par 10 couronnes constituées de carrés dont les longueurs des côtés sont respectivement 1, 2, 3, 4, ..., 10.*
2. *Calculer la surface du carré de deux façons différentes suggérées par la figure 8.3 page 105 :*
   (a) *en calculant la longueur du demi-côté du carré obtenue en additionnant celles des carrés du pavage,*
   (b) *en calculant la surface du quart du grand carré délimité par les diagonales de ce grand carré (voir cette partie en vert à la figure 8.3, page 105), la surface de ce quart de grand carré étant obtenue par addition des surfaces des petits carrés le constituant.*
3. *En déduire l'égalité :*

$$1^3 + 2^3 + 3^3 + 4^3 + \ldots + 10^3 = (1 + 2 + 3 + 4 + \ldots + 10)^2$$

□

Corrigé 15 *L'expression entière manquante dans le programme* `PiquetsEnDentsDeScie` *est :*

$$(1 + ((\texttt{i}-1)\ \%\ 5))$$

qui prend successivement les valeurs 1, 2, 3, 4, 5, 1, 2, 3, ...*quand* `i` *prend les valeurs* 1, 2, 3, 4, 5, 6, 7, 8, ...□

Corrigé 16 *Nous donnons un programme, les autres se trouvant dans le dossier* `livre/08_expressions_entieres` *:*

```
from LibRobot import *

# Dessiner une spirale trapézoïdale
av(); pqtd()
for i in range(1, 10):
    # Dessiner le côté supérieur
    av()
    # Dessiner un côté droit
```

1. Nicomaque de Gérase, mathématicien grecque, env. I^{er} siècle après J.-C.

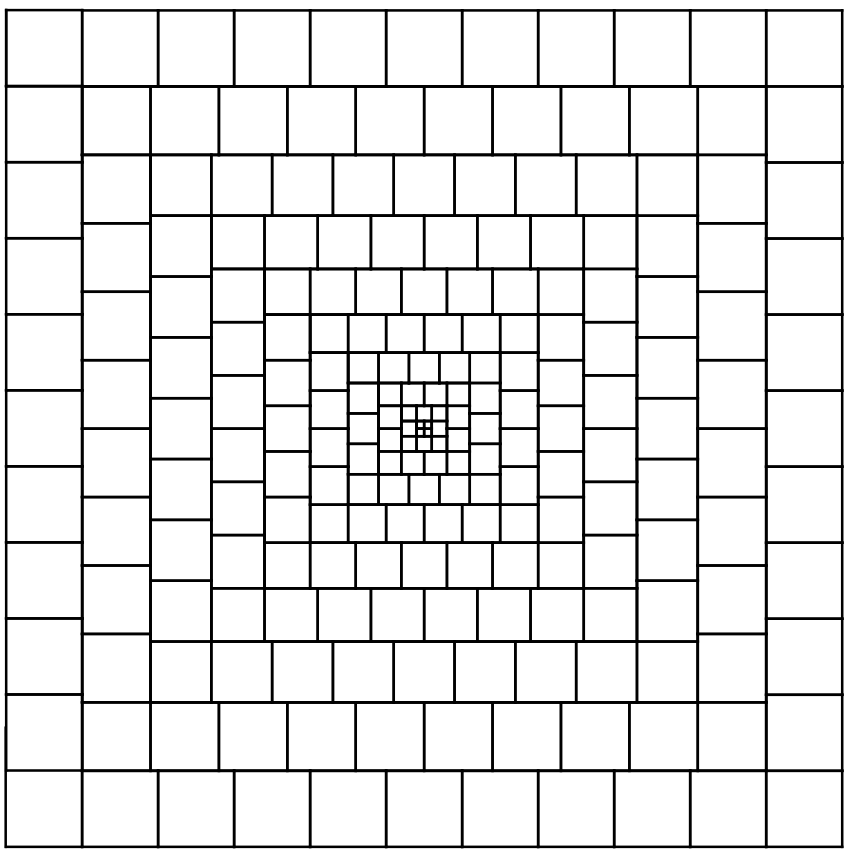


FIG. 8.2 – *Pavage de Nicomaque (Échelle* $\frac{1}{4}$*)*

```
    phtd()
    for j in range(2 * i):
       av()
    # Dessiner le côté inférieur
    p3htd()
    for j in range(4 * i + 2):
       av()
    # Dessiner un côté gauche
    p3htd()
    for j in range(2 * i + 1):
       av()
    phtd()
st()
```

□

Corrigé 17 *L'expression entière manquante dans le programme* `VaguesDePiquets` *est la suivante :*

$$(((6-\texttt{j}) * ((\texttt{i}+1) \,\%\, 2)) + (\texttt{j} * (\texttt{i} \,\%\, 2)))$$

Quand `i` *est impair,* $((\texttt{i}+1) \,\%\, 2) = 0$ *et* $(\texttt{i} \,\%\, 2) = 1$ *donc l'expression vaut* `j`*. Quand* `i` *est pair,* $((\texttt{i}+1) \,\%\, 2) = 1$ *et* $(\texttt{i} \,\%\, 2) = 0$ *donc l'expression vaut* $(6-\texttt{j})$*.* □

Corrigé 18

```
from LibRobot import *
# Paver un carré à l'aide de couronnes successives
# constituées de carrés de tailles 1, 2, 3, ... en
# partant du centre.
for i in range(1, 5):
   # Dessiner les 4 côtés de la i-ème couronne en les
   # pavant avec des carrés de taille i.
   for j in range(4):
      # Dessiner un côté de la i-ème couronne, pavé de
      # i carrés de longueur de côté i, le robot
      # partant du premier carré et arrivant au
      # dernier carré dans le même coin et dans la
      # même direction qu'au départ.
      # Dessiner i - 1 carrés de la i-ème couronne
      for k in range(i - 1):
         # Dessiner un carré de longueur i, le robot
         # partant et arrivant au même coin et dans la
         # même direction.
         for l in range(4):
```

```
            for m in range(i):
               av()
            pqtd()
         # Avancer jusqu'au carré suivant de la i-ème
         # couronne.
         pqtd()
         for l in range(i):
            av()
         pqtg()
      # Dessiner le i-ème carré de longueur i de la
      # i-ème couronne.
      for k in range(4):
         for l in range(i):
            av()
         pqtd()
      # Pivoter d'un quart de tour pour placer le
      # robot au début du côté suivant de la i-ème
      # couronne.
      pqtd()
   # Rejoindre le point de départ de la (i + 1)-ème
   # couronne.
   pqtg()
   for j in range(i):
      av()
   pqtd()
   for j in range(i):
      av()
st()
```

En partageant un côté du grand carré en deux, la longueur de ce demi-côté est la somme $(1+2+\ldots+10)$ *des longueurs des carrés de tailles 1, 2, 3, …, 10 du pavage. Ceci apparaît clairement sur la figure 8.3 page 105 grâce à la réglette que nous avons placée en dessous à droite du grand carré. La surface du grand carré est donc le carré de la longueur du côté soit :*

$$(2 \times (1 + 2 + \ldots + 10))^2 = 4 \times (1 + 2 + \ldots + 10)^2$$

Si l'on partage le grand carré selon les diagonales, on obtient un triangle comme on le voit en vert à la figure 8.3 (page 105). Ce triangle contient 1 carré de longueur de côté 1, 2 carrés de longueur de côté 2, …, 10 carrés de longueur de côté 10. Sa surface est donc $1 \times 1^2 + 2 \times 2^2 + \ldots + 10 \times 10^2$ *soit* $1^3 + 2^3 + \ldots + 10^3$*. En faisant pivoter 4 fois ce triangle, on recouvre entièrement le grand carré. La surface du grand carré est donc :*

$$4 \times (1^3 + 2^3 + \ldots + 10^3)$$

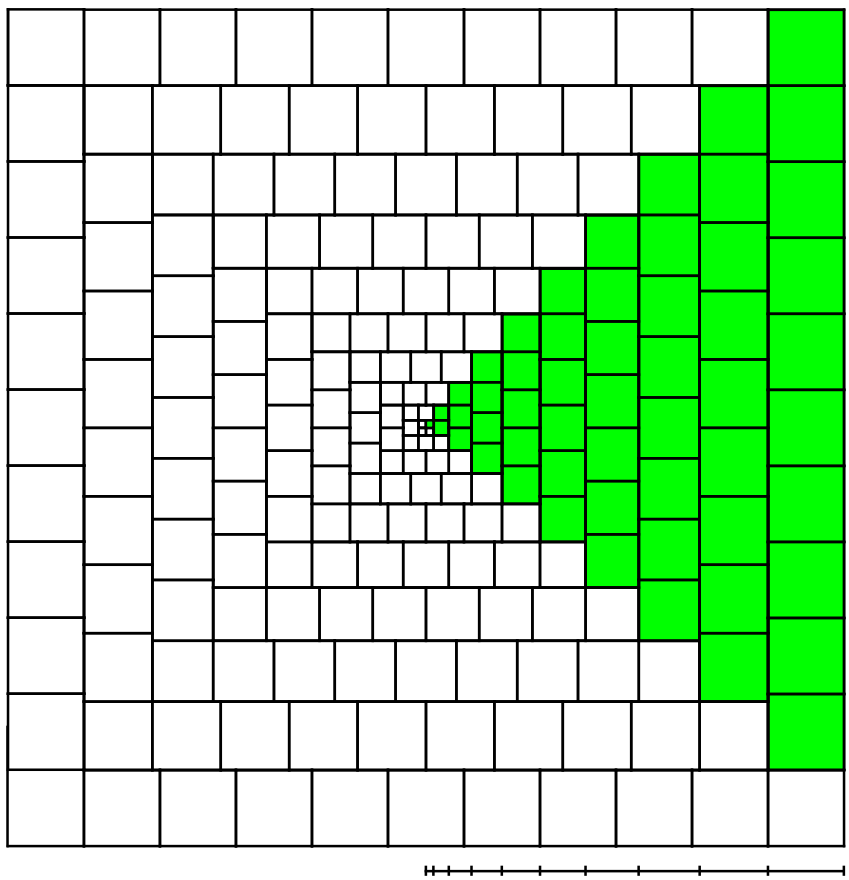


FIG. 8.3 – *Côté et surface du carré de Nicomaque (Échelle $\frac{1}{4}$)*

Le résultat est le même quelle que soit la façon de calculer la surface du carré de Nicomaque. Par conséquent :

$$4 \times (1^3 + 2^3 + \ldots + 10^3) = 4 \times (1 + 2 + \ldots + 10)^2$$

soit :

$$1^3 + 2^3 + \ldots + 10^3 = (1 + 2 + \ldots + 10)^2$$

Une autre démonstration de cette identité a été proposée par Al-Karagi[2] *et repose sur le pavage du carré en équerres comme illustré ci-dessous :*

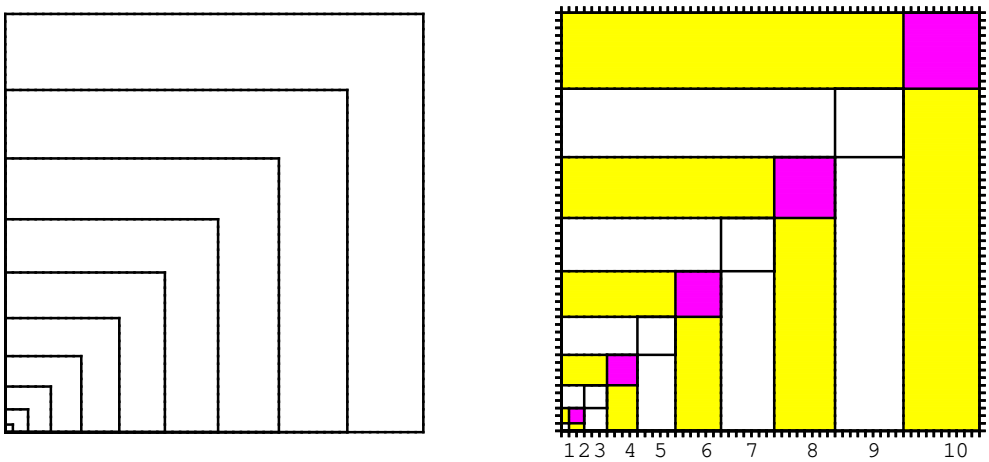


FIG. 8.4 – *Pavage de Al-Karagi*

L'aire de l'équerre extérieure de petit côté 10 vaut :

$$\begin{aligned}(2 \times 10 \times (1 + 2 + \ldots + 10)) - 10^2 &= 2 \times 10 \times \tfrac{10\times(10+1)}{2} - 10^2 \\ &= -10^3 + 10^2 - 10^2 = 10^3\end{aligned}$$

L'aire de l'équerre de petit côté 9 est 9^3*, celle de l'équerre de petit côté 8 est* 8^3*, ..., l'aire du petit carré est* $1 = 1^3$*. Par conséquent, l'aire du grand carré qui est la somme des aires du petit carré et des équerres est* $1^3+2^3+\ldots+10^3$*. D'autre part, la longueur du côté du grand carré est* $(1 + 2 + \ldots + 10)$ *donc sa surface est* $(1 + 2 + \ldots + 10)^2$*. Comme l'aire du grand carré est la même quelque soit la façon de la calculer, on obtient :*

$$1^3 + 2^3 + \ldots + 10^3 = (1 + 2 + \ldots + 10)^2$$

Pour justifier que $(1 + 2 + \ldots + 10) = \frac{10\times(10+1)}{2}$*, on utilise un rectangle de largeur 10 et de longueur (10 +1) dont la moitié est pavée comme suit :*

2. Al-Karagi, fin X$^{\text{ème}}$ – début XI$^{\text{ème}}$ siècle, mathématicien persan.

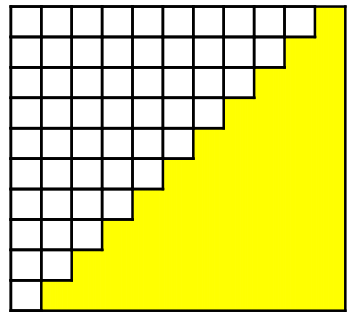


FIG. 8.5 – *Cacul de* $1+2+3+\ldots+10$

Une autre justification géométrique de l'égalité $1^3+2^3+\ldots+10^3=(1+2+\ldots+10)^2$ *proposée par S. Golomb [14] est illustrée ci-dessous :*

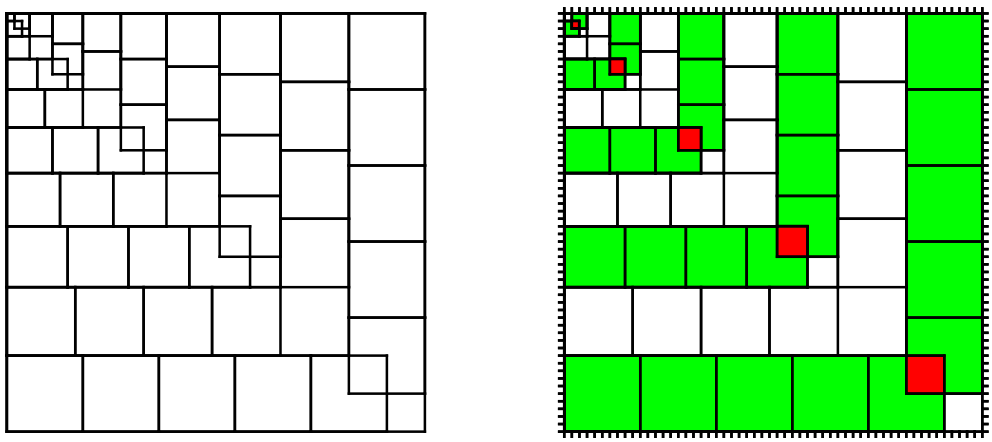


FIG. 8.6 – *Pavage de Golomb*

La longueur du côté du grand carré est bien $(1+2+\ldots+10)$ *donc sa surface est* $(1+2+\ldots+10)^2$*. Cette surface est égale à la somme des surfaces des carrés constituant le pavage soit 1 carré de côté 1, 2 carrés de côté 2, ..., 10 carrés de côté 10. On remarque que pour les carrés de longueur de côté paire montrés en jaune sur la figure, il y a un recouvrement de deux petits carrés figuré en rouge. Mais chacun de ces petits carrés est adjacent à une région non couverte de même taille montrée en blanc. On peut donc enlever un des deux petits carrés qui se recouvrent pour boucher le trou et obtenir un pavage parfait. La surface est donc égale à* $1^3+2^3+\ldots+10^3$*.* □

Addenda

Pour simplifier l'apprentissage de la syntaxe de Python nous écrivons les expressions entières avec des parenthèses. Les expressions entières *unaires* (avec un seul *opérande* E) sont de la forme $(-E)$ où $-$ est l'*opérateur* de signe moins. Les expressions entières *binaires* (avec un *opérande* gauche E_1 et un opérande droit E_2) sont de la forme $(E_1\ \mathbf{op}\ E_2)$ où **op** est l'un des *opérateurs* d'addition $+$, de soustraction $-$, de multiplication $*$, de division entière `//` ou de reste de la division entière `%` (modulo). Les opérandes E, E_1 et E_2 sont une constante entière (positive comme 1, 17345 (sans blancs), ...ou négative comme -10, -5247, ...), une variable (pour l'instant un compteur de boucle "for" comme `i`) ou une *sous-expression* entière de la forme que nous venons de décrire.

Le calcul de la valeur d'une expression s'appelle l'*évaluation* de l'expression. Les règles d'évaluation des expressions complètement parenthésées sont très simples. La valeur d'une constante est l'entier qui est dénoté par cette constante. La valeur d'une variable `i` utilisée comme compteur dans le corps C d'une boucle **for** `i` **in** `range(1,` n `+ 1) :` C (où $n \geq 1$ est une constante entière) est successivement 1 puis 2 puis ...puis n lors des itérations successives dans la boucle. La valeur d'une expression $(-E)$ s'obtient en calculant la valeur v de l'expression E puis en lui appliquant le signe moins. La valeur d'une expression $(E_1\ \mathbf{op}\ E_2)$ s'obtient en calculant d'abord la valeur v_1 de l'expression E_1 puis en calculant la valeur v_2 de l'expression E_2 puis en calculant la valeur $v_1\ \mathbf{op}\ v_2$ du résultat. Les mêmes règles d'évaluation s'appliquent aux sous-expressions E, E_1 et E_2.

Pour éviter les lourdeurs d'écriture, on peut suivre en Python la pratique courante en mathématiques qui consiste à utiliser des *règles de priorité* (on dit aussi *règles de précédence*) permettant d'éliminer certaines parenthèses redondantes. Les règles d'évaluation des expressions entières sont définies et illustrées ci-après :

- En Python comme en mathématiques, l'expression entière $10-10-10$ vaut $((10-10)-10)$ soit -10. De même $2*7\,//\,3\,\%\,2$ vaut $(((2*7)\,//\,3)\,\%\,2)$ soit 0. Dans les deux cas la règle est que des opérateurs de même *priorité* (à savoir $+$ et $-$ d'une part ; $*$, `//` et `%` d'autre part) s'évaluent de gauche à droite.
- L'expression $8+-2*3$ vaut $(8+((-2)*3))$ soit 2. De même $1-5\,//\,2$ vaut $(1-(5\,//\,2))$ soit -1. La règle est qu'en présence d'opérateurs de priorités différentes, on commence par évaluer les opérateurs de plus forte priorité.

En Python les priorités sont les suivantes :

Priorité des opérateurs entiers		
$-$ (unaire)	opérateurs unaires	priorité forte
$*$ // %	opérateurs multiplicatifs	priorité moyenne
$+$, $-$ (binaires)	opérateurs additifs	priorité faible

- L'expression $(8+-2)*3$ vaut $((8+(-2))*3)$ soit 18. De même $(1-5)\,//\,2$ vaut $((1-5)\,//\,2)$ soit -2. Dans les deux cas on commence par l'évaluation des sous-expressions entre parenthèses.
- Quand la division d'un nombre négatif ne tombe pas juste, Python arrondit le quotient vers le bas, c'est-à-dire vers le plus petit entier : $-7\,//\,2$ vaut -4 (et non pas -3 comme en OCaml ou en PASCAL) et le reste -7 % 2 vaut 1 (et non pas -1 comme en OCaml), de sorte que l'on a toujours $((-7\,//\,2)*2)+(-7$ % $2)=-7$. Avec un diviseur positif, le reste % est donc toujours positif ou nul.

De manière équivalente, l'ordre d'évaluation des expressions peut être défini en indiquant à quels opérateurs sont liés les opérandes d'une expression :

- Un opérande figurant entre deux opérateurs de priorités différentes est lié à l'opérateur de plus forte priorité (donc $2+3*5=(2+(3*5))$).
- Un opérande figurant entre deux opérateurs de même priorité est lié à l'opérateur de gauche (donc $2-3-5=((2-3)-5)$).
- Les expressions entre parenthèses sont évaluées avant d'être utilisées comme opérandes.

Les règles d'évaluation des expressions changent d'un langage de programmation à l'autre. Par exemple en APL [18], il n'y a pas de priorités d'opérateurs (car il y en a trop dans le langage pour qu'on puisse espérer s'en souvenir) et les expressions sont associatives à droite de sorte que $10-10-10=(10-(10-10))=(10-0)=10$. Cette règle contraire à la tradition mathématique a l'avantage que l'analyse grammaticale des expressions est grandement simplifiée pour l'ordinateur.

Les langages de programmation plus classiques qu'APL suivent la convention mathématique d'évaluation de gauche à droite avec priorités.

Cependant certains langages possèdent beaucoup d'opérateurs. Par exemple le langage C [20] comprend 43 opérateurs différents répartis en 15 niveaux de priorité. Les priorités des opérateurs ne sont donc pas faciles à retenir.

De plus les règles changent d'un langage à l'autre ce qui rend la vie difficile aux informaticiens pratiquant journellement plusieurs langages. Pour ne pas se tromper ils utilisent des parenthèses redondantes, ce que nous conseillons aux débutants de faire.

Signalons enfin qu'en Python les nombres entiers ne sont pas limités : on peut calculer avec des entiers de plusieurs centaines de chiffres, la seule limite étant la taille de la mémoire de l'ordinateur. Ce n'est pas le cas de tous les langages. En PASCAL sur les premiers ordinateurs personnels, les entiers devaient être compris entre -32768 et 32767. En OCaml, ils sont compris entre `min_int` $= -4611686018427387904$ et `max_int` $= 4611686018427387903$ et au-delà, le calcul donne un résultat faux sans prévenir : (`max_int`$+1$) vaut `min_int` !

9

Appel de procédures avec paramètres

9.1 Faire avancer et reculer le robot

Nous avons vu au chapitre 5 qu'une déclaration de procédure permet de donner un nom à une séquence de commandes que le $\mathcal{R}^{o}_{o}t$ peut ensuite exécuter une ou plusieurs fois quand on appelle la procédure par son nom. Une procédure peut avoir un ou plusieurs *paramètres*. Dans ce cas la procédure peut réaliser des actions semblables mais différentes quand on l'appelle avec des paramètres différents.

Par exemple la procédure `avf` commande au $\mathcal{R}^{o}_{o}t$ d'avancer tout droit une ou plusieurs fois de la taille d'un côté d'un carreau du quadrillage. Elle a un paramètre qui indique le nombre de fois qu'il faut avancer. En Python, pour appeler une procédure avec un paramètre, on écrit le nom de la procédure suivi de la valeur du paramètre entre parenthèses. '`avf(1)`' est équivalent à '`av()`' tandis que '`avf(4)`' est équivalent à '`av() ; av() ; av() ; av()`'. Quand le paramètre est nul, la commande '`avf(0)`' ne fait pas changer le $\mathcal{R}^{o}_{o}t$ de position. Enfin quand le paramètre est un nombre négatif, c'est-à-dire un nombre précédé du signe moins −, le $\mathcal{R}^{o}_{o}t$ recule du nombre de fois indiqué sans changer d'orientation. Par exemple '`avf(−4)`' est équivalent à '`pdt() ; av() ; av() ; av() ; av() ; pdt()`'.

Il ne faut pas oublier les parenthèses. Si l'on écrit '`avf 4`', Python ne comprend pas et signale '`SyntaxError: invalid syntax`'. Si l'on écrit '`avf −4`', Python comprend que l'on veut soustraire 4 de `avf`, ce qui n'a pas

de sens, et signale pendant l'exécution '`TypeError: unsupported operand type(s) for -: 'function' and 'int'`' (on ne peut pas soustraire un entier (`int`) d'une procédure (`function`)). Le paramètre peut aussi être une expression entière, comme dans '`avf(2 * j)`' : Python calcule d'abord la valeur de l'expression `2 * j` puis appelle `avf` avec cette valeur. Si l'on oublie le paramètre, en écrivant '`avf()`', Python signale '`TypeError: avf() missing 1 required positional argument: 'n'`' (il manque un paramètre obligatoire à `avf()`) et si l'on en donne trop, comme dans '`avf(1, 2)`', il signale '`TypeError: avf() takes 1 positional argument but 2 were given`' (`avf()` prend un paramètre mais on lui en a donné deux).

On peut maintenant comprendre les parenthèses vides `()` qui suivent les commandes du *Robot* que nous avons utilisées jusqu'ici, comme `av()` ou `pqtd()` : en Python les parenthèses commandent l'appel de la procédure, et elles sont vides pour les procédures qui n'ont besoin d'aucune information pour s'exécuter. Attention : si l'on oublie ces parenthèses vides, en écrivant `av` au lieu de `av()`, Python ne signale aucune erreur mais la procédure n'est pas appelée et le *Robot* n'avance pas !

Exemple 13 Le programme `StriesOctogonales` ci-dessous dessine les stries triangulaires réparties sur les côtés d'un octogone invisible représentées à la figure 9.1 (page 113) :

```
from LibRobot import *

# Dessiner des stries sur les côtés d'un octogone
lc(); avf(5); bc(); lg(4)
for i in range(8):
   # Dessiner 10 stries de longueurs 2, 4, ..., 20.
   for j in range(1, 11):
      # Dessiner une strie de longueur (2 * j) centrée
      # sur un côté de l'octogone.
      avf(j); avf(-2 * j); avf(j)
      # Passer à la strie suivante (sans tracer le côté
      # de l'octogone).
      lc(); pqtd(); av(); pqtg(); bc()
   # Tourner le robot perpendiculairement au côté
   # suivant.
   phtd()
st()
```

□

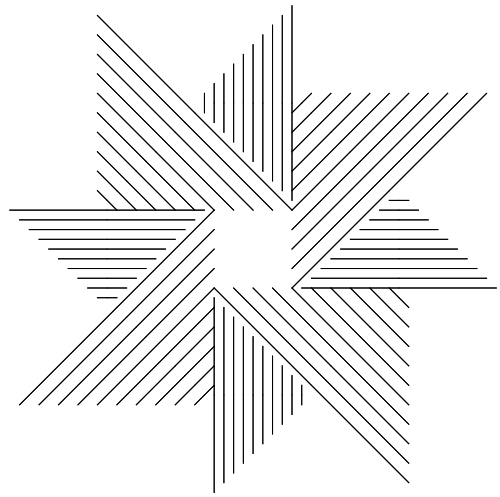
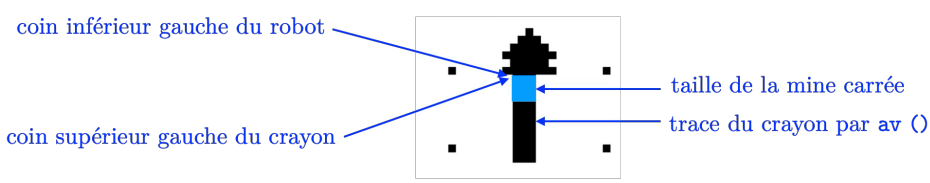


FIG. 9.1 – *Stries sur octogone (Échelle $\frac{1}{3}$)*

9.2 Épaisseur du crayon

La commande `ec` définit l'épaisseur du crayon du $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$. Son paramètre entier indique l'épaisseur du crayon qui est choisie. Jusqu'ici nous avons employé un crayon d'épaisseur 1 qui est celle qu'utilise le $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ par défaut c'est-à-dire quand on ne lui donne pas d'autres précisions. La commande `ec(2)` permet de dessiner avec un trait deux fois plus gros. L'unité de longueur choisie pour mesurer l'épaisseur du crayon est le *pixel* c'est-à-dire, rappelons-le, la taille du plus petit point qui peut être dessiné sur l'écran de l'ordinateur.

Quand on utilise un très gros crayon, il faut savoir que le crayon du $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ a une mine carrée et que le $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ tient cette mine par son coin supérieur gauche.



Exemple 14 (Crayon à grosse mine) Le programme ci-dessous, `EpaisseurCrayon`, permet d'observer l'effet de l'exécution d'une commande `av()` avec une grille standard de 12 pixels quand le $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ utilise des épaisseurs

successives du crayon de 12, 24, 36 et 48 pixels :

□

```
from LibRobot import *
# Effet de av () avec des crayons de plus en plus
# épais
lc(); avf(5); pqtd(); lgX(6); avf(-1); lgX(12)
for i in range(1, 5):
   lc(); pqtd(); avf(i); pqtg(); bc()
   ec(12 * i); av(); lc(); avf(-1)
dg(); st()
```

On observera par exemple que le trait tracé quand le $\mathcal{R}^{o}_{o}t$ exécute une commande `av()` avec un crayon de 48 pixels a une longueur apparente de 5 fois la longueur du côté du quadrillage. Ceci s'explique par le fait qu'en position finale le crayon laisse une trace carrée de côté 48 pixels en bas à droite de son point d'arrêt soit 4 fois la longueur du côté du quadrillage de 12 pixels. □

9.3 Vitesse du robot

Un dernier exemple de procédure avec paramètre est la commande `vt` pour choisir la vitesse du $\mathcal{R}^{o}_{o}t$. Cette vitesse est un nombre entier compris entre 0 et 10. La vitesse 10 correspond à la vitesse maximale par défaut. Donc la commande `vt(10)` est équivalente à la commande `ex()` d'exécution normale. La vitesse `vt(1)` correspond à la vitesse minimale. Elle permet d'observer tous les détails des mouvements du $\mathcal{R}^{o}_{o}t$ qui, à cette vitesse, sont très lents. Enfin la vitesse 0 correspond à l'exécution en mode pas à pas. La commande `vt(0)` est donc équivalente à la commande `pp()`. Le mode pas à pas est utilisé pour trouver les erreurs dans les programmes. Dans ce mode d'exécution, les commandes du $\mathcal{R}^{o}_{o}t$ s'affichent en bas de l'écran et s'exécutent en tapant sur une touche (ou en cliquant sur le bouton de la souris). Les parcours du $\mathcal{R}^{o}_{o}t$ avec le crayon levé sont marqués en pointillés ce qui permet de suivre parfaitement sa trace.

Exercice 19 *Écrire un programme Python pour dessiner :*

_ *la lettre H majuscule en relief de la figure 1.12 (page 9) ;*

_ *le cube en perspective, le cristal à huit branches et la paire de lunettes de la figure 9.2 (page 115) ;*

_ *l'hélicoptère et le train de voyageurs de la figure 1.13 (page 10) ;*

_ *les courbes 'spirolatérales' de la figure 9.3 (page 115), obtenues en faisant avancer le Robot une fois puis deux fois puis trois fois, etc., en tournant à chaque fois d'un quart de tour à droite et en répétant le tout quatre fois de suite ([30]) ;*

_ *l'arabesque et l'échelle de Jacob de la figure 9.4 (page 116), échelle de Jacob que l'on pourra également essayer de réaliser en entrelaçant une ficelle entre ses doigts.*

□

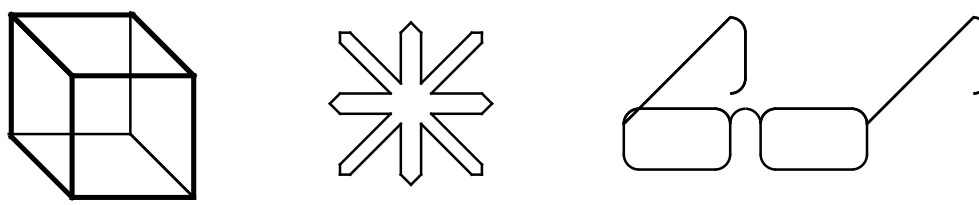


FIG. 9.2 – *Cube en perspective, cristal à huit branches et paire de lunettes*

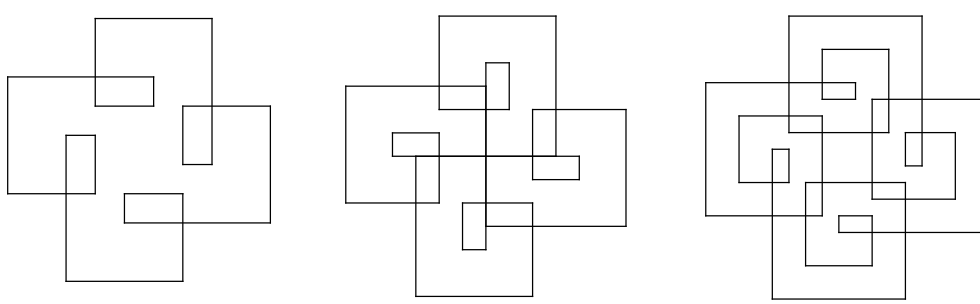


FIG. 9.3 – *Spirolatérales à 90° d'ordre 5, 7 et 9*

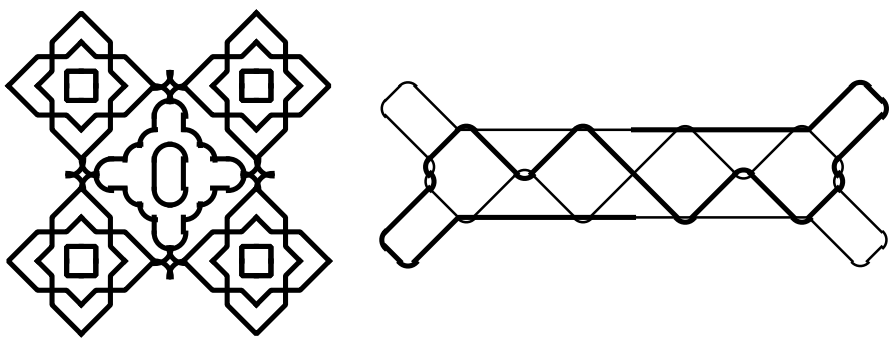


FIG. 9.4 – *Arabesque et échelle de Jacob*

Corrigé 19

```
from LibRobot import *
# Dessiner une courbe spirolatérale de Frank C. Odds
# à 90 degrés d'ordre 9.
lc(); avf(-4); bc()
for i in range(4):
   for j in range(1, 10):
      avf(j); pqtd()
st()
```

□

Addenda

L'usage de paramètres dans les procédures est une des notions de base de la programmation : chaque appel de la procédure dépend d'une ou plusieurs valeurs en fonction desquelles on exprime l'effet de la procédure. Nous ne considérons pour l'instant que le passage de paramètres *par valeur* qui permet de passer une information (un ou plusieurs nombres dans notre cas) à une procédure. Ceci rejoint la notion élémentaire de *fonction* (ou *application*) en mathématiques.

L'emploi de sous-programmes avec paramètres en langage machine est généralement attribué à Alan Turing. Le suisse Heinz Rutishauser [35] et l'italien Corrado Böhm [5] introduisirent les notions de langage de programmation évolué et de compilateur pour traduire les programmes écrits dans ce langage évolué en langage machine (on ne parlait pas à l'époque de compilateur mais de *codification automatique*). Ils incluèrent la notion de sous-programme avec paramètres dans leurs langages sans la dégager très explicitement.

Après ces contributions théoriques liminaires, le premier compilateur qui ait fonctionné sur la machine MARK I à Manchester en Angleterre fût construit par Alick Glennie en 1952. Le language évolué AUTOCODE utilisait des sous-programmes avec paramètres passés dans les registres de la machine.

Ce travail n'eut aucun succès car le travail de programmation était relativement aisé à l'époque comparé aux efforts nécessaires pour adapter les programmes aux dysfonctionnements de la machine. Maurice Wilkes [44] introduisit l'usage de bibliothèques de procédures (comme celles que nous utilisons pour les commandes du $\mathcal{R}^{o}\underline{b}^{o}t$) en 1951.

Le premier langage où ces notions apparurent sous leur forme moderne fut FORTRAN (FORmula TRANslator) conçu en 1954 par une équipe dirigée par John Backus [1]. Grâce à de très nombreuses modifications qui incorporent les nouveaux concepts de programmation, aux extensions qui prennent en compte les architectures récentes de machines et à la richesse des bibliothèques disponibles, ce langage est encore très utilisé pour la programmation scientifique numérique.

10

Grille de déplacement du robot

Jusqu'à présent nous avons tracé nos dessins sur du papier quadrillé à petits carreaux. Le $\mathcal{R}^{o}_{b}{}^{o}t$ a la faculté supplémentaire de changer la taille des carreaux. On peut donc agrandir les dessins en augmentant la taille des carreaux avant de commencer le dessin. Il est également possible de changer la taille des carreaux pendant que le $\mathcal{R}^{o}_{b}{}^{o}t$ dessine. Ceci permet de tracer des segments de longueurs différentes et des arcs de cercles dont le rayon peut varier. En choisissant des carreaux rectangulaires on peut tracer des ellipses et choisir de manière quelconque les angles de déplacement du $\mathcal{R}^{o}_{b}{}^{o}t$.

10.1 Grille carrée

La grille standard représentée à la figure 2.2 (page 14) a une taille de 12 pixels. On peut changer la longueur du côté d'un carreau de la grille avec la commande `lg` qui prend en paramètre la nouvelle longueur de la grille exprimée en pixels. Par exemple `lg(6)` définit une taille de grille deux fois plus petite que la grille habituelle tandis que `lg(24)` double la taille de la grille.

La longueur de la grille peut aussi comporter une partie décimale, comme dans `lg(7.5)` (7 pixels et demi). Un tel nombre, que l'on appelle un *nombre flottant*, s'écrit avec un point (et non pas avec une virgule comme en français) : `6.0` (ou `6.`) est le nombre flottant six tandis que `6` est le nombre entier six. Les commandes `lg`, `lgX` et `lgY` acceptent indifféremment un nombre entier ou un nombre flottant : `lg(6)` et `lg(6.0)` ont le même effet [1]. Quand la longueur

1. Ce n'est pas le cas de tous les langages : en OCaml par exemple, il faut obligatoi-

de la grille est donnée par une expression entière comme (6 ∗ i), on écrit donc simplement `lg(6 * i)`. Nous étudierons les nombres flottants en détail au chapitre qui leur est consacré.

Exemple 15 (Carrés superposés) Le programme ci-dessous permet de dessiner des carrés superposés, le *Robot* partant et arrivant au coin inférieur gauche, orienté vers le nord, la superposition étant obtenue en choisissant des tailles de grille successives de 6, 12, 18, . . ., 84 :

```
from LibRobot import *

# Dessiner des carrés superposés en agrandissant la
# grille
def carre():
   av(); pqtd(); av(); pqtd(); av(); pqtd(); av()
   pqtd()

eb(); ag()
for i in range(1, 15):
   lg(6 * i); carre()
st()
```

□

10.2 Grille rectangulaire

Le *Robot* peut se promener sur une grille rectangulaire dont la longueur du côté horizontal est différente de celle du côté vertical. Ceci lui permet de dessiner des rectangles et des ellipses comme ceux-ci :

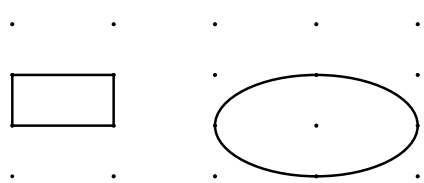


FIG. 10.1 – *Rectangle et ellipse sur un quadrillage* 40 × 20

La grille initiale a une taille 12 × 12 comptée en pixels. La commande `lgX(40)` permet de donner une longueur de 40 pixels au côté horizontal de la grille sans changer la longueur du côté vertical. après cette commande

rement écrire `lg 6.` avec un nombre flottant et convertir les entiers en flottants.

la grille a donc une taille 40×12 pixels. Dans cette notation 40×12 on donne d'abord la longueur du côté horizontal (nommé X) puis la longueur du côté vertical (nommé Y) de la grille. La commande `lgY(20)` permet de donner une longueur de 20 pixels au côté vertical de la grille sans changer la longueur du côté horizontal. après cette commande la grille a donc une taille 40×20 pixels. On peut donc changer indépendamment la taille horizontale (avec `lgX`) ou la taille verticale (avec `lgY`) de la grille. On peut revenir à une grille carrée en utilisant la commande `lg`. Par exemple la commande `lg(12)` permet de revenir à une grille carrée standard 12×12. Le rectangle et l'ellipse ci-dessus sont construits par le programme suivant :

```
from LibRobot import *
# Dessiner un rectangle et une ellipse juxtaposés.
# Taille de la grille
lgX(40); lgY(20); ag()
# Grille
dg()
# Rectangle
av(); pqtd(); av(); pqtd(); av(); pqtd(); av()
pqtd()
# Décalage à droite.
lc(); pqtd(); av(); av(); pqtg(); bc()
# Ellipse
vd(); vd(); vd(); vd()
st()
```

La figure ci-dessous illustre l'effet des commandes `vd()`, `av()` et `vg()` avec une grille rectangulaire de dimensions 30×60.

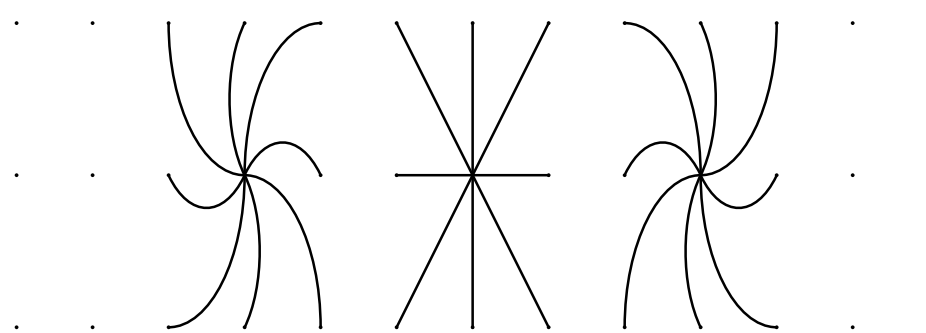


FIG. 10.2 – *Virage à droite, avancer et virage à gauche avec une grille* 30×60

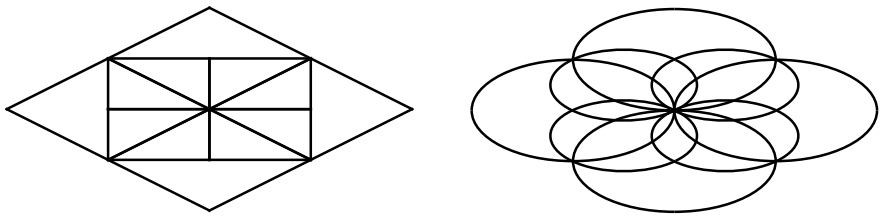


FIG. 10.3 – *Parallélogrammes et ellipses*

Exemple 16 Les divers tracés de segments et d'arcs d'ellipses par les commandes `av()`, `vg()` et `vd()` sont illustrés par les deux programmes ci-dessous qui dessinent des parallélogrammes et des ellipses par pivotement d'un quart de tour autour du centre de la grille, comme on le voit à la figure 10.3 (page 122) :

-
```
from LibRobot import *
# Dessiner des parallélogrammes sur une grille
# rectangulaire
# Définir la taille de la grille 40 x 20.
lgX(40); lgY(20)
# Dessiner huit parallélogrammes en faisant pivoter
# leur coin inférieur gauche d'un quart de tour
# autour du centre de la grille.
for i in range(8):
   av(); pqtd(); av(); pqtd(); av(); pqtd(); av()
   phtd()
st()
```

-
```
from LibRobot import *
# Dessiner des ellipses sur une grille rectangulaire
# Définir la taille de la grille 40 x 20.
lgX(40); lgY(20)
# Dessiner huit ellipses en faisant pivoter leur
# grand axe d'un quart de tour autour du centre de
# la grille.
for i in range(8):
   phtd(); vd(); vd(); vd(); vd()
st()
```

□

Exercice 20 *Utiliser les possibilités offertes par les changements de taille de la grille carrée pour dessiner :*

- *les spirales et cercles concentriques de la figure 10.4 (page 123) ;*
- *la frise de la figure 10.5 (page 124) ;*
- *l'hélice et la balle de la figure 10.6 (page 124) ;*
- *l'escargot de la figure 10.7 (page 124) ;*
- *le stégosaure et son bébé de la figure 10.8 (page 125) ;*
- *la famille de chiens de la figure 10.9 (page 125) ;*
- *le pavage de la figure 10.13 (page 127) inspiré par un motif de van de Vecht ([26], page 26) représenté à la figure 10.10 (page 125).* □

Exercice 21 *Utiliser les possibilités offertes par les grilles rectangulaires pour dessiner :*

- *la frise de losanges se chevauchant de la figure 10.11 (page 125) ;*
- *le vélo de la figure 10.12 (page 126) ;*
- *la rose des vents à huit directions 2.1 (page 14) ;*
- *les pavages des figures 10.14 (page 128), 10.15 (page 129) et 10.16 (page 130) obtenues à partir du pavage de la figure 10.13 (page 127) en diminuant progressivement la taille de la grille sur le côté droit du dessin, sur le côté gauche et dans le dernier cas sur les quatre côtés du rectangle à la manière du graveur hollandais M. C. Escher ([26], [11]).*

□

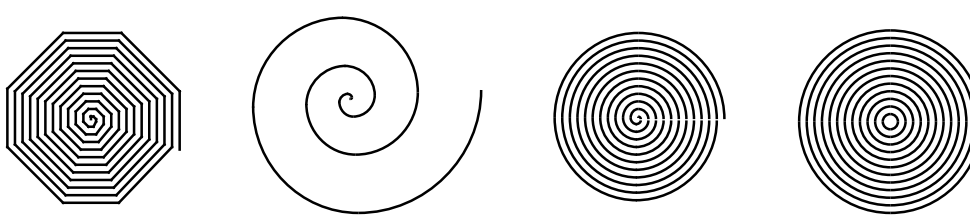


FIG. 10.4 – *Spirales et cercles concentriques*

FIG. 10.5 – *Double frise crantée*

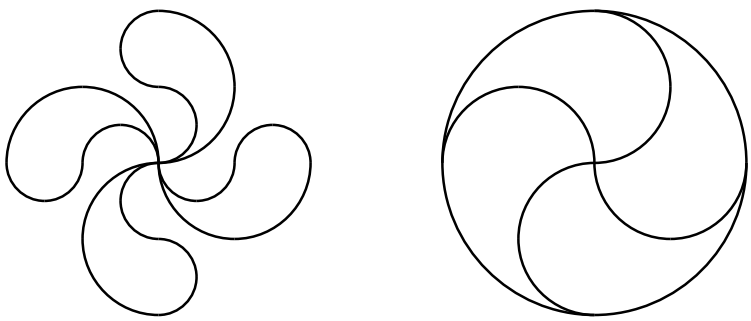


FIG. 10.6 – *Hélice et balle*

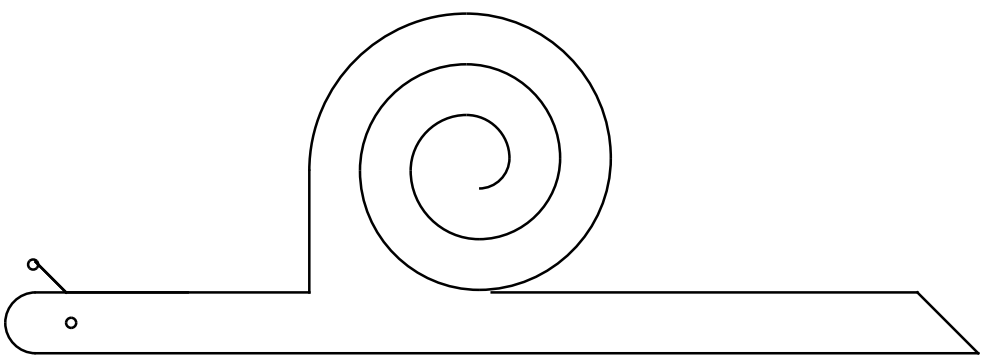


FIG. 10.7 – *Escargot*

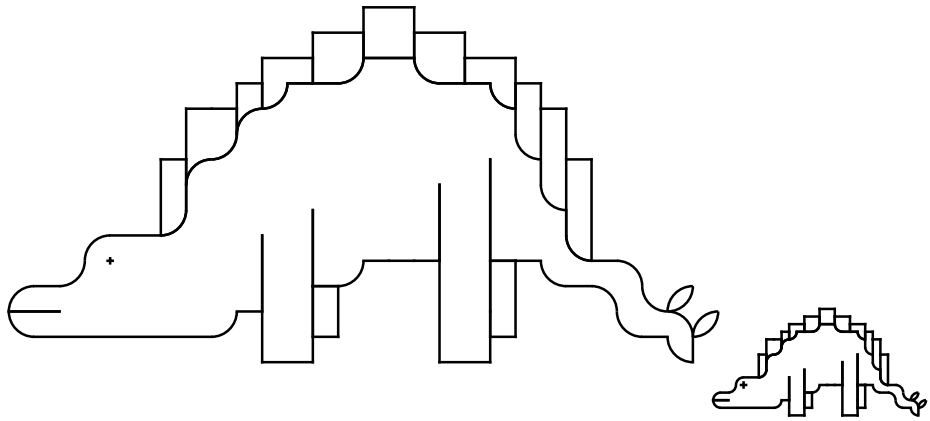


FIG. 10.8 – *Stégosaure et son bébé*

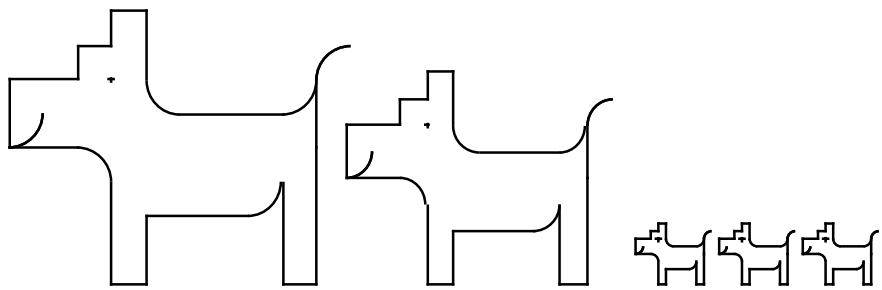


FIG. 10.9 – *Famille de chiens en balade*

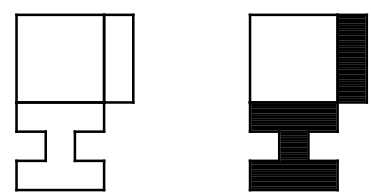


FIG. 10.10 – *Motif ornemental de N. J. van de Vecht*

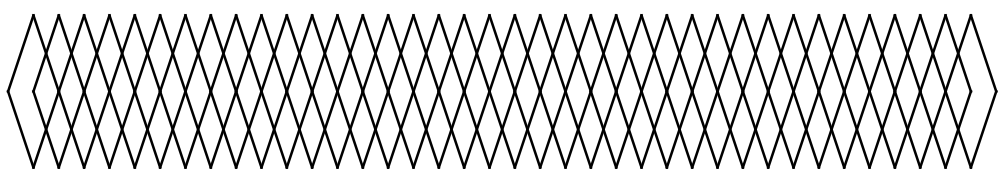


FIG. 10.11 – *Frise de losanges se chevauchant*

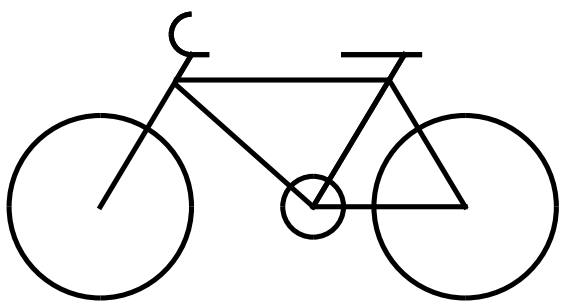


FIG. 10.12 – *Vélo*

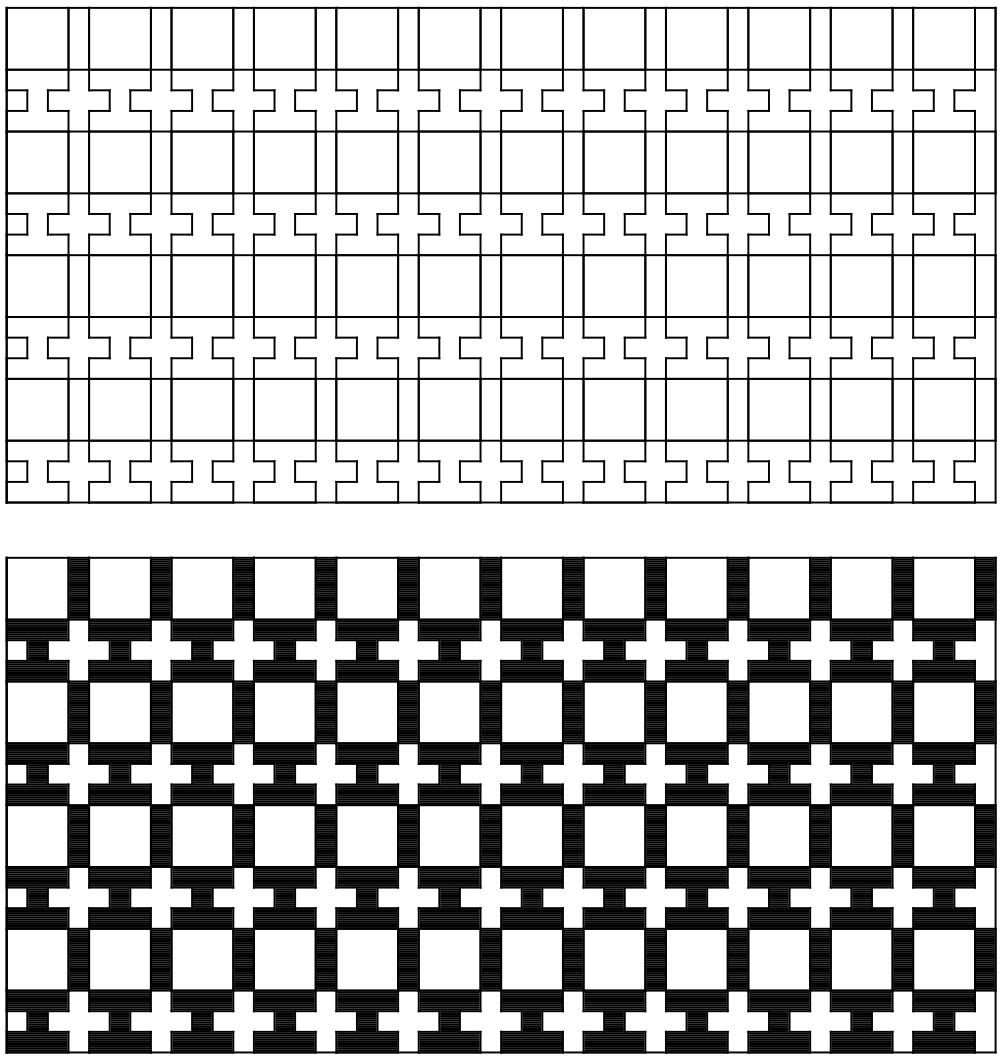


FIG. 10.13 – *Pavage régulier inspiré par un motif ornemental* de N. J. van de Vecht

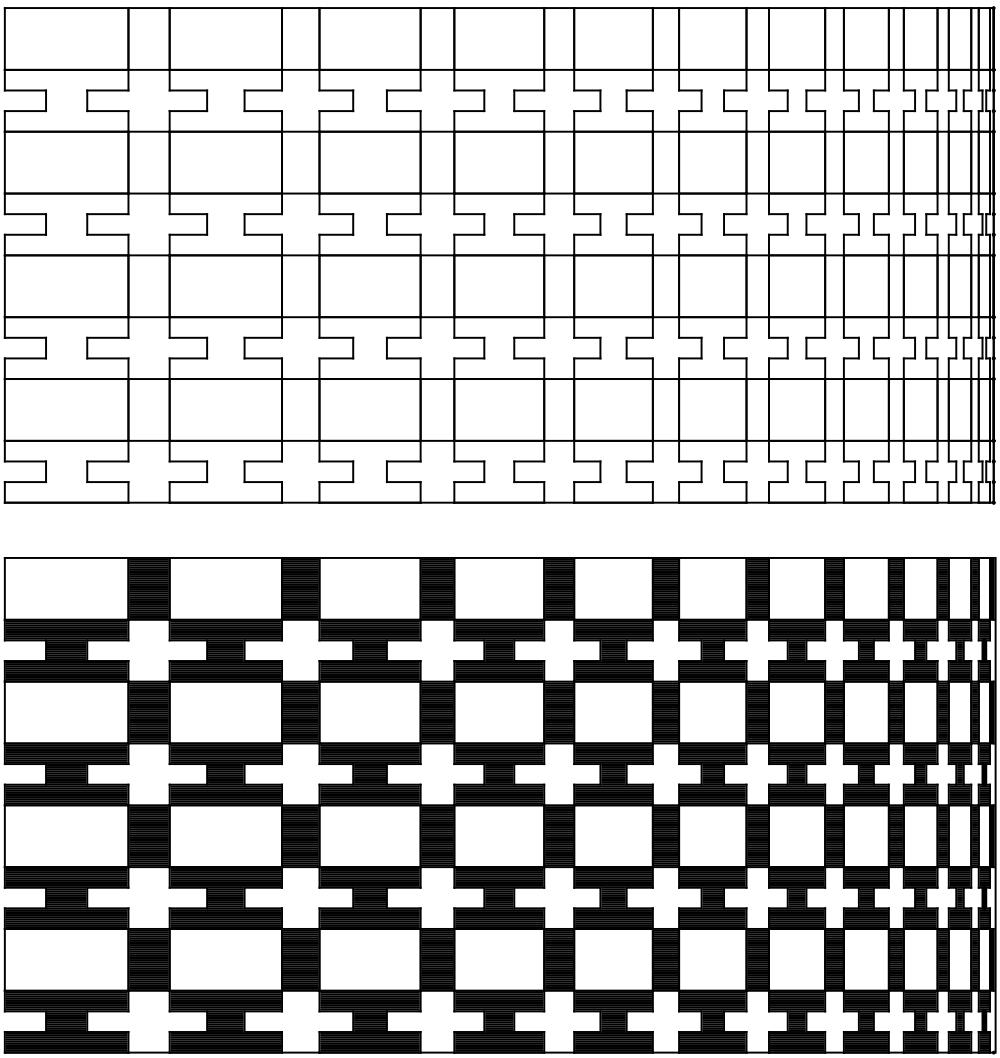


FIG. 10.14 – *Pavage rectangulaire à une limite*

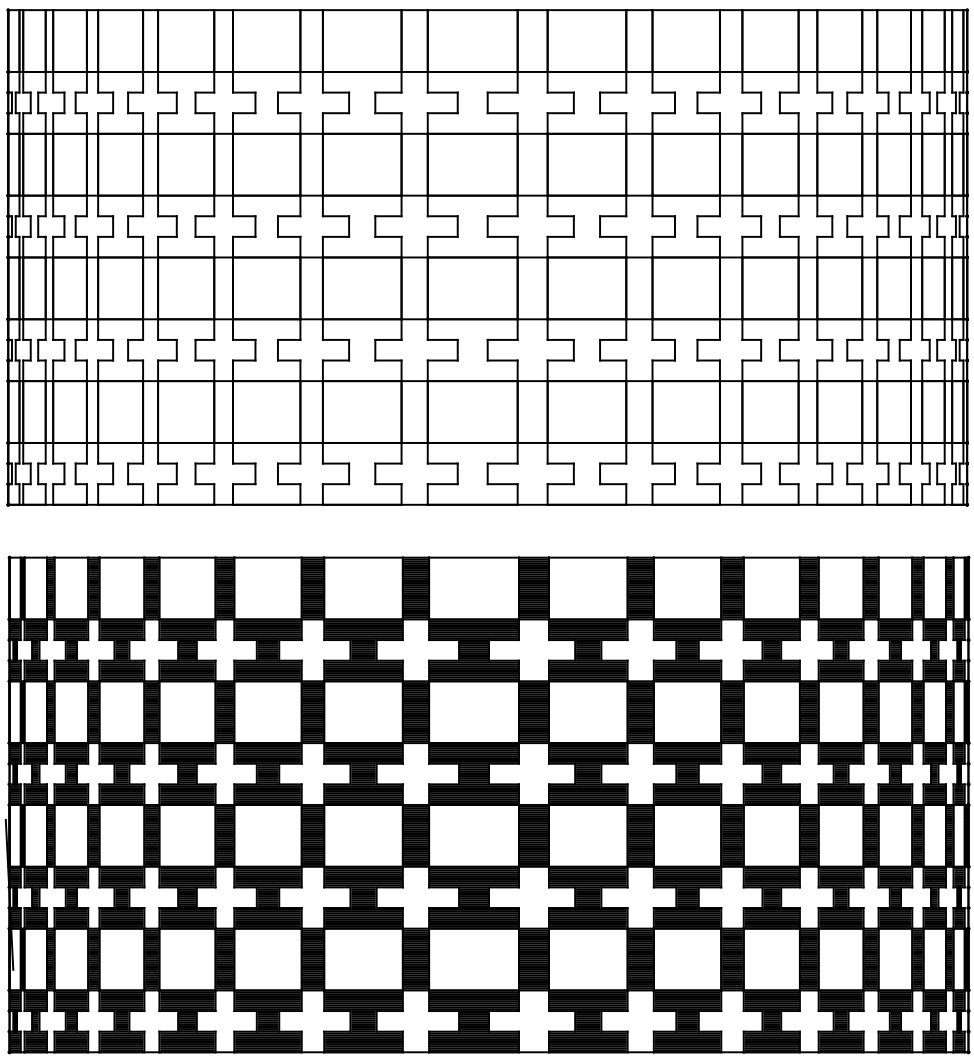


FIG. 10.15 – *Pavage rectangulaire à deux limites*

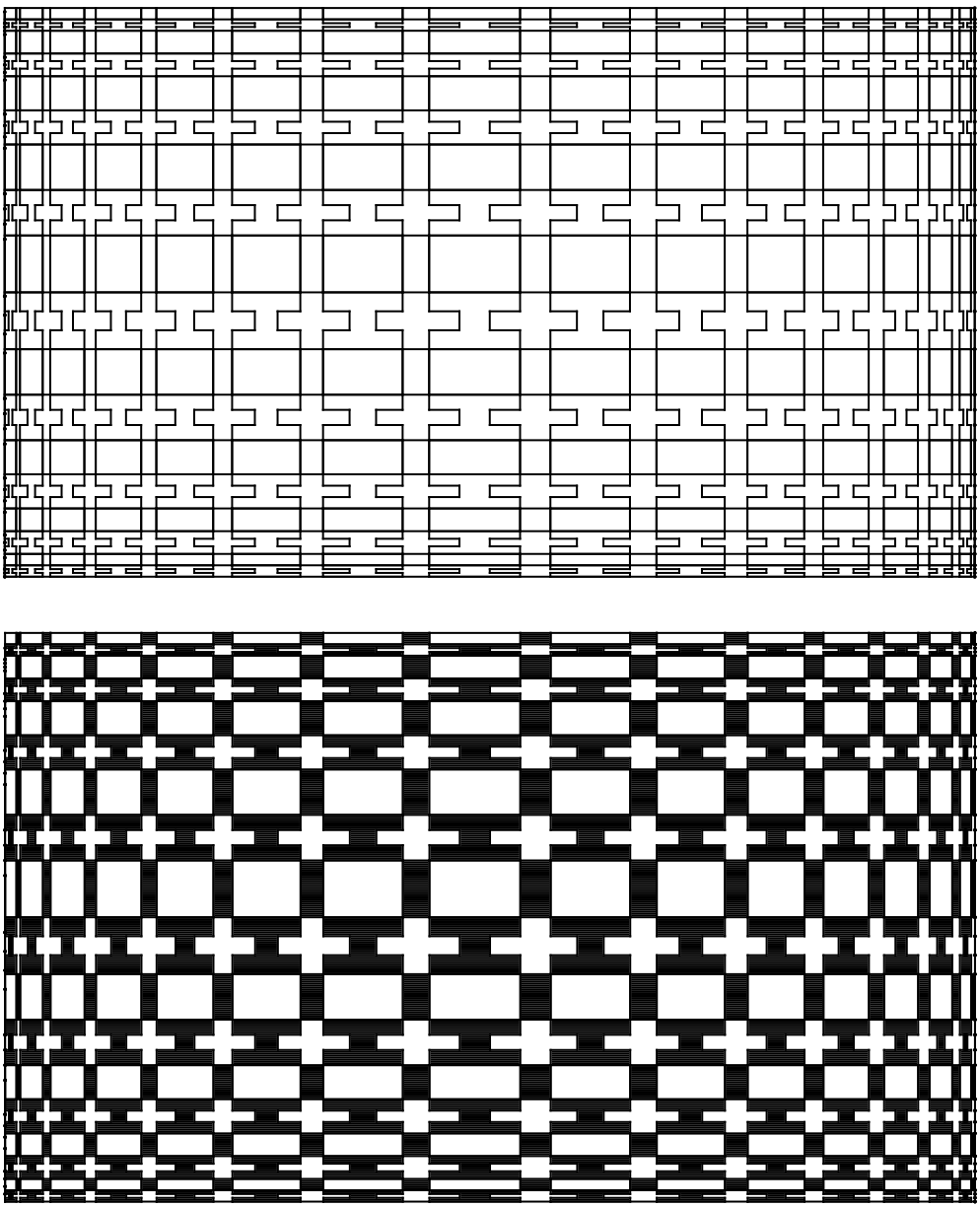


FIG. 10.16 – *Pavage rectangulaire à quatre limites*

Corrigé 20 *Nous donnons quelques programmes, les autres programmes se trouvant dans le dossier* `livre/10_grille_cartesienne` *:*

- ```
  from LibRobot import *
  # Dessiner une spirale d'arcs de cercles de rayons
  # 1, 4, 9, 16, ...
  pdt()
  for i in range(1, 13):
     lg(i * i); vg()
  st()
  ```

- ```
  from LibRobot import *
  # Dessiner un escargot
  # Monter au centre de la coquille
  lc(); avf(4); bc(); pqtd()
  # Coquille
  for i in range(1, 12):
     vg(); lg(12 + 5 * i)
  lg(12); avf(4)
  # Corps
  pqtd(); avf(9); vg(); vg(); avf(31); pdt(); phtd()
  avf(2); phtg(); avf(14)
  # Rejoindre l'antenne
  lc(); avf(10); bc(); avf(4)
  # Antenne
  phtd(); av(); lg(2); vg(); vg(); vg(); vg()
  lg(12); pdt(); av()
  # Oeil
  phtd(); lc(); av(); bc()
  lg(2); vg(); vg(); vg(); vg()
  st()
  ```

□

Corrigé 21 *Nous donnons le programme le plus difficile, à savoir le pavage de la figure 10.16 (page 130) obtenue à partir du pavage de la figure 10.13 (page 127) en diminuant progressivement la taille de la grille sur les quatre côtés du rectangle. Les autres programmes se trouvent dans le dossier* `livre/10_grille_cartesienne` *:*

```
from LibRobot import *
# Pavage irrégulier dans le style de M. C. Escher,
# inspiré d'un motif de N. J. van de Vecht.
# Motif de N. J. van de Vecht, le robot partant en
```

```
# haut à gauche et arrivant en bas à gauche orienté
# vers le nord.
def motif():
   pqtd(); avf(4); pqtd(); avf(3); pqtd(); av(); pqtd()
   avf(3); pdt(); avf(3)
   av(); pqtd(); av(); pqtg(); av(); pqtg(); av()
   pqtd(); av(); pqtd(); avf(3)
   pqtd(); av(); pqtd(); av(); pqtg(); av(); pqtg()
   av(); pqtd(); av(); pqtd(); avf(3)
   pdt(); avf(3); pqtd(); avf(3); lc(); pdt(); avf(6)
   pdt(); bc()

# Dessiner une colonne de motifs de hauteurs 2, 4, 6,
# 8, 10, 8, 6, 4, 2, le robot partant en haut à gauche et
# arrivant en bas à droite (c'est-à-dire en bas à
# gauche de la colonne suivante) orienté vers le nord.
def colonne():
   lgY(1); eh()
   for i in range(1, 6):
      lgY(2 * i); motif()
   for i in range(1, 5):
      lgY(10 - 2 * i); motif()
   pqtd(); avf(4); pqtg()

# Dessiner un pavage irrégulier constitué de colonnes
# de largeurs 0, 2, 4, 6, ..., 14, 16, 14, ..., 2, 0.
def pavage():
   lgX(1); ag(); lgX(0)
   for i in range(1, 9):
      colonne(); lgX(2 * i)
   for i in range(1, 9):
      colonne(); lgX(16 - 2 * i)
   colonne()

pavage(); st()
```

□

Addenda

Si toutes les grilles des points d'arrêt du $\mathcal{R}^{o}\underbrace{b}^{o}t$ étaient alignées sur le centre de l'écran, il se pourrait qu'après un changement de grille le $\mathcal{R}^{o}\underbrace{b}^{o}t$ ne soit plus sur un point de la grille. C'est pourquoi la grille est toujours relative au $\mathcal{R}^{o}\underbrace{b}^{o}t$ et non pas à l'écran. En cas de changement de taille de la grille tous les points de la grille (que l'on peut observer grâce à la commande `dg()`) changent de place, sauf celui sur lequel est placé le $\mathcal{R}^{o}\underbrace{b}^{o}t$. En conséquence aucune commande du $\mathcal{R}^{o}\underbrace{b}^{o}t$ (sauf une, à savoir `ce()`) ne peut lui faire quitter l'un des points de la grille courante. Par exemple les commandes `ag()`, `ad()`, `eh()` et `eb()` déplacent le $\mathcal{R}^{o}\underbrace{b}^{o}t$ horizontalement ou verticalement sur le point de la grille courante (respectivement à gauche, à droite, en haut et en bas) le plus près du bord. Pour pouvoir placer le $\mathcal{R}^{o}\underbrace{b}^{o}t$ n'importe où sur l'écran avec précision, il faut disposer d'un point de repère absolu. Ce point de repère absolu est le centre de l'écran. La commande `ce()` ramène le $\mathcal{R}^{o}\underbrace{b}^{o}t$ toujours au même point exactement situé au centre de l'écran. Cette commande est donc la seule commande du $\mathcal{R}^{o}\underbrace{b}^{o}t$ qui ne s'effectue pas sur la même grille. Si le centre de l'écran n'est pas situé sur la grille courante, la commande `ce()` recentre la grille exactement au centre de l'écran de façon à éviter de faire sortir le $\mathcal{R}^{o}\underbrace{b}^{o}t$ de son quadrillage de déplacement.

Pour éviter d'éventuels problèmes de décalage, il suffit de programmer le dessin comme s'il s'agissait de papier millimétré en choisissant une taille de grille minimale (1 millimètre sur le papier millimétré) de façon à ce que tous les changements de taille de grille soient effectués quand le $\mathcal{R}^{o}\underbrace{b}^{o}t$ est au centre et correspondent à des multiples (5, 10, ...sur le papier millimétré) de la taille de base. On peut également procéder empiriquement par expérimentation, ce qui est quelquefois plus distrayant.

11

Constantes

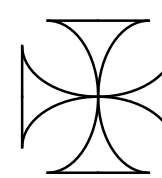
Le nombre 1066 nous fait immédiatement penser à la bataille d'Hastings et la victoire de Guillaume le Conquérant, duc de Normandie ; 1776 à l'indépendance des États Unis ; 1789 à la révolution française. Quand on lit 3.14159 on sait qu'il s'agit d'une valeur approchée de π (écrit avec la convention anglo-saxonne qui utilise un point à la place de notre virgule). Par contre quand on lit le chiffre 2, on ne pense à rien de précis et il est à peu près sûr qu'on se demande « deux quoi ? ». Tout cela pour dire que lorsqu'on voit un nombre dans un programme il n'est pas toujours facile de comprendre immédiatement de quoi il s'agit. Pour le savoir, on peut, en Python, nommer le nombre à l'aide d'une déclaration de constante placée au début du programme :

```
revolutionFrancaise = 1789  # Date de la révolution
```

puis utiliser le nom `revolutionFrancaise` à la place du nombre 1789 dans la suite du programme.

Exemple 17 (Croix de Malte) Dans le programme de dessin d'une croix de Malte ci-dessous, on utilise une constante `longueur` pour définir la longueur d'une branche (c'est-à-dire la distance de sa base au centre de la croix) et une autre `largeur` pour définir la largeur d'une branche (c'est-à-dire la distance entre les extrémités de sa base) :

```
from LibRobot import *
# Dessiner une croix de Malte
largeur = 20  # Largeur d'une branche
longueur = 30  # Longueur d'une branche
```



```
# Dessiner une branche verticale avec le robot
# orienté au nord ou au sud
def brancheVerticale():
   lgX(largeur); lgY(longueur)
   vg(); pdt(); av(); av(); pdt(); vg(); pqtg()

# Dessiner une branche horizontale avec le robot
# orienté à l'est ou à l'ouest
def brancheHorizontale():
   lgX(longueur); lgY(largeur)
   vg(); pdt(); av(); av(); pdt(); vg(); pqtg()

def croixDeMalte():
   for i in range(2):
      brancheVerticale(); brancheHorizontale()

croixDeMalte(); st()
```

Pour choisir la meilleure forme possible pour cette croix, il n'y a qu'un seul endroit dans le programme où il est nécessaire de modifier ces constantes. Par exemple pour obtenir la plus petite croix représentée à la figure 11.1 ci-dessous, il suffit de prendre :

```
largeur = 10  # Largeur d'une branche
longueur = 15  # Longueur d'une branche
```

□

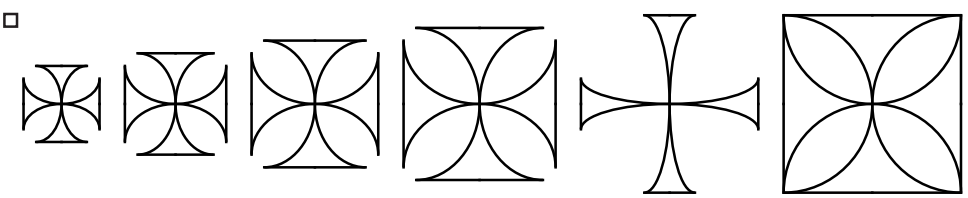


FIG. 11.1 – *Croix de Malte* 10×15, 15×20, 20×25, 25×30, 10×35, 35×35

□

On remarquera que les nombres `20` et `30` (ou `10` et `15`) sont des nombres *entiers*. Ils sont utilisés comme longueurs des côtés des carreaux de la grille dans les commandes `lgX` et `lgY` qui, comme la commande `lg`, acceptent aussi des nombres à virgule, écrits avec un point, comme `20.5`, que l'on appelle des nombres *flottants*. On aurait donc aussi bien pu écrire '`largeur = 20.5`'. Les nombres entiers (comme `20`) et les nombres flottants (comme `20.5` ou `20.0`) sont deux sortes de nombres différentes pour Python, mais il les mélange sans difficulté dans les calculs. Nous verrons pourquoi ils sont différents au chapitre 19 sur les expressions rationnelles.

On indique que l'on utilise une ou plusieurs *constantes nommées* par des *déclarations de constantes* :

Déclaration de constantes :

identificateur1 = nombre1
identificateur2 = nombre2
...

Chaque déclaration de constante s'écrit sur une ligne, au début de la ligne (sans indentation), comme '$x = 3$' en mathématiques. Il est d'usage de mettre un blanc ␣ avant et après le signe =, mais ce n'est pas obligatoire. Comme pour les noms de procédures, l'identificateur doit respecter les règles des identificateurs de Python. L'usage est de le faire commencer par une lettre minuscule. Majuscules et minuscules sont distinguées : si l'on déclare '`Largeur = 20`' puis que l'on utilise `largeur`, Python répond '`NameError: name 'largeur' is not defined. Did you mean: 'Largeur'?`' (le nom `largeur` n'est pas défini, vouliez-vous dire `Largeur` ?). Après ces déclarations, on peut utiliser les noms de constantes 'identificateur1' à la place de 'nombre1' et 'identificateur2' à la place de 'nombre2', etc.

À vrai dire, Python n'a pas de véritables constantes : rien n'interdit de donner plus loin dans le programme une autre valeur au nom `largeur` (nous verrons au chapitre 22 que c'est ainsi que l'on utilise des *variables*). On parle de constante parce que le programme ne donne qu'une seule fois une valeur à ce nom, au début, et ne la change jamais ensuite.

Dans un programme Python, une déclaration définit des constantes (avec le signe =) ou des procédures (avec le mot **def**). Nous verrons plus tard qu'elle peut aussi définir des fonctions et des variables :

Déclaration :

Déclaration de constantes
ou
Déclaration de procédures

Les déclarations de constantes ou de procédures se placent, dans n'importe quel ordre, avant les instructions du programme :

Programme Python :

```
from ␣ LibRobot ␣ import *
# Commentaire expliquant ce que fait le programme
... Déclarations du programme ...
... Instructions du programme [a] (une par ligne ou séparées par des ';') ...
```

[a] Utilisant les constantes et procédures déclarées.

La principale règle à respecter est que tout identificateur doit être déclaré avant d'être utilisé (sinon, quand le $\mathcal{R}^{o}_{o}t$ arrive à l'instruction qui l'utilise, Python s'arrête avec une erreur `NameError`).

Exercice 22 *Écrire des programmes Python comportant des constantes nommées pour dessiner :*

- *l'hélice singulière de la figure 11.2 ci-dessous ;*
- *le canardeau de la figure 11.3 ci-dessous.*

□

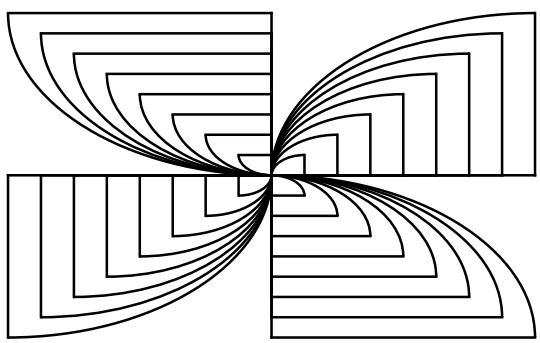


FIG. 11.2 – *Hélice singulière*

Corrigé 22 *Nous donnons un programme, l'autre se trouve dans le dossier* `livre/11_constantes`. *Le côté horizontal des carreaux de la grille de la* i^{e} *feuille d'une pale est* `xbase * i`. *C'est un nombre entier que l'on peut donner directement à la commande* `lgX` *:*

```
from LibRobot import *
# Dessiner une hélice singulière
```

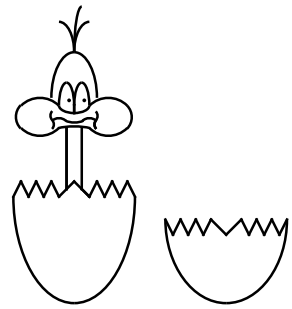


FIG. 11.3 – *Canardeau sortant de son œuf*

```
xbase = 13  # Largeur de la grille de base
ybase = 8  # Hauteur de la grille de base
nombreDeFeuilles = 8  # Feuilles d'une pale
nombreDePales = 4  # Pales de l'hélice

def pale():
    for i in range(1, nombreDeFeuilles + 1):
        # Dessiner un quart d'ellipse, de dimension
        # (xbase * i, ybase * i), sans changer
        # l'orientation initiale du robot
        lgX(xbase * i)
        lgY(ybase * i)
        av(); pqtg(); av(); pqtg(); vg(); pqtg()

for j in range(nombreDePales):
    pale(); pqtd()
st()
```

□

Addenda

Les entiers informatiques sont une approximation des entiers relatifs en mathématiques (mais pas trop grands). Les flottants (plus précisement virgule flottante) sont un sous ensemble des nombres rationnels (et donc des nombres réels) écrits en virgule flottante.

Une *déclaration de constante* `nom = nombre` permet de donner un nom à un nombre particulier. Par la suite le nom, plus significatif, peut être employé à la place du nombre. L'avantage immédiat est que pour changer de nombre il suffit de le faire dans la déclaration de constante alors qu'autrement il faut rechercher toutes ses occurrences dans le programme.

Ceci peut être comparé à la notion de variable en mathématiques c'est-à-dire à l'emploi de lettres x, y, z ...pour désigner des nombres quelconques. La différence est qu'en Python le nom de constante désigne une valeur particulière alors qu'en mathématiques il s'agit d'une valeur quelconque.

Historiquement les premiers problèmes mathématiques traités par les babyloniens (1800 ans av. J.-C.) comportaient une solution algorithmique donnée pour un exemple et des tables numériques séparées pour résoudre d'autres problèmes similaires [34]. Au Moyen Age on utilisait le langage rhétorique comme 'un nombre ajouté à son carré valent 12' pour '$x + x^2 = 12$'. Chuquet [a], auteur du plus ancien traité d'algèbre écrit par un français (1484), utilisait 1^1 pour désigner l'inconnue, 1^2 pour désigner son carré et les symboles $\bar{p}$ et $\bar{m}$ pour l'addition et la soustraction. Il pouvait écrire '$12\ \bar{p}\ 3^1$ egaulx a 4^2' pour '$12 + 3x = 4x^2$'. Viète [b] fit faire un pas important vers la symbolisation en algèbre en introduisant l'usage des lettres pour désigner les quantités inconnues. Il écrivait vers 1590 : '1 QC − 15 QQ + 85 C + 225 Q + 274 N égalent 120', soit si C signifie cube, donc x^3, Q signifie carré, donc x^2, et N signifie nombre, donc x, l'équation $x^5 - 15x^4 + 85x^3 + 225x^2 + 274x = 120$ ([4]). Il fallut attendre quatre mathématiciens français Desargues [c], Descartes [d], Fermat [e] et Pascal [f] (ce dernier ayant été choisi par Wirth pour nommer son langage de programmation PASCAL) pour entrer dans l'ère moderne des mathématiques.

[a] Nicolas Chuquet, mathématicien français, v. 1445-1500.
[b] François Viète, mathématicien français, 1530-1603.
[c] Girard Desargues, mathématicien français, 1591–1661.
[d] René Descartes, mathématicien français, 1596–1650.
[e] Pierre de Fermat, mathématicien français, 1601–1665.
[f] Blaise Pascal, mathématicien français, 1623–1662.

12

Peinture

12.1 Noir et blanc

Le $\mathcal{R}^{o}_{\smile}{}^{o}t$ peut colorier ses dessins. Nous allons commencer par des coloriages en noir et blanc. La commande `peindre()` permet de laisser s'échapper la peinture noire par l'arrière du $\mathcal{R}^{o}_{\smile}{}^{o}t$. La peinture s'étale sur la plus grande partie possible de l'écran délimitée par une ligne noire. Un exemple est donné à la figure 12.1 ci-dessous. On commence par dessiner le pourtour de la zone

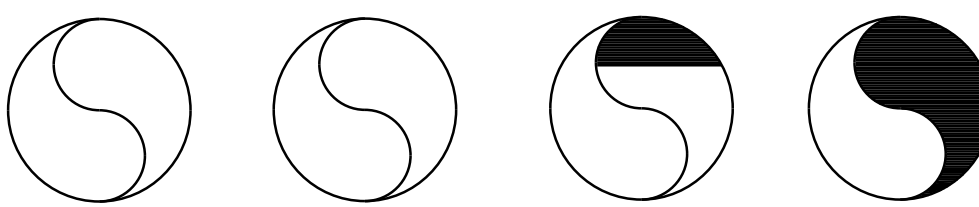


FIG. 12.1 – *Étapes de la peinture d'un dessin*

à peindre puis on place le $\mathcal{R}^{o}_{\smile}{}^{o}t$ dans la zone à peindre ou sur son bord de façon à ce que l'arrière du $\mathcal{R}^{o}_{\smile}{}^{o}t$ par où s'échappe la peinture soit dans la zone à peindre. La peinture s'étale ligne par ligne au dessus, à gauche, à droite et en dessous du point arrière du $\mathcal{R}^{o}_{\smile}{}^{o}t$. La zone couverte est la plus grande où se trouve l'arrière du $\mathcal{R}^{o}_{\smile}{}^{o}t$ et qui est délimitée par une ligne noire continue. Le $\mathcal{R}^{o}_{\smile}{}^{o}t$ peut peindre sans se déplacer, que son crayon soit levé ou baissé.

Exemple 18 (Yin Yang) Le programme ci-après dessine le symbole chinois du Yin Yang :

```
from LibRobot import *
# Dessiner le symbole chinois du Yin Yang
tailleQuadrillage = 3
def cercle():
   vd(); vd(); vd(); vd()
# Demi cercle inférieur
lg(6 * tailleQuadrillage)
pqtd(); vg(); vg()
# Demi cercle supérieur
lg(6 * tailleQuadrillage)
vd(); vd()
# Cercle du pourtour
lg(12 * tailleQuadrillage)
cercle()
# Petit cercle blanc
lg(2 * tailleQuadrillage)
lc(); pqtd(); avf(2); pqtg(); bc(); cercle()
# Peindre la zone noire
lc(); pqtd(); avf(4); peindre()
# Petit cercle noir
avf(2); pqtg(); bc(); cercle(); pqtg(); peindre()
st()
```

□

Une erreur fréquente consiste à laisser un trou dans le périmètre de la zone à peindre par lequel la peinture fuira. Un trou d'un seul pixel suffit pour provoquer une fuite. En cas de fuite, on peut arrêter la peinture en tapant sur l'une des touches esc ou § ou #. Le *Robot* continue de dessiner. Taper au moins une seconde fois sur l'une de ces touches pour l'arrêter définitivement.

12.2 Couleur

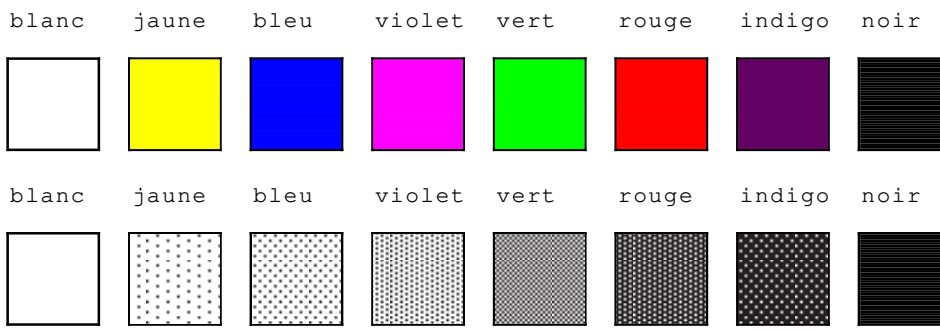
Pour dessiner et peindre, le *Robot* dispose de crayons de couleurs blanc, jaune, bleu, violet, vert, rouge, indigo ou noir. La commande `cc` permet de choisir la couleur du crayon, que l'on donne en paramètre. Par exemple après la commande `cc(rouge)` tous les tracés et la peinture sont en rouge. La commande `cc(noir)` permet de revenir aux tracés et peinture noirs. Les noms de couleurs `blanc`, `jaune`, `bleu`, `violet`, `vert`, `rouge`, `indigo` et `noir` sont des constantes déclarées par `LibRobot`, qui désignent les numéros de couleur 0, 1, 2, ..., 7 (dans cet ordre) : `cc(rouge)` est donc la même chose que `cc(5)`, mais

beaucoup plus lisible. Comme on l'a vu au chapitre 9, la commande `ec(n)` choisit l'épaisseur du crayon ; elle est indépendante de sa couleur.

Sur un écran couleur, le *Robot* dessine et peint avec les couleurs de la première ligne ci-dessous.

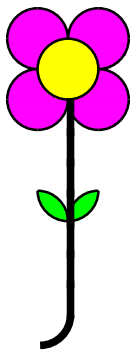
Quand on ne dispose pas d'un écran couleur mais d'un écran à plusieurs niveaux de gris dont chaque pixel peut être plus ou moins foncé, les couleurs sont représentées par des gris, dans l'ordre blanc, jaune, bleu, violet, vert, rouge, indigo et noir du plus clair au plus foncé.

Quand on dispose seulement d'un écran noir et blanc, les couleurs sont représentées par des grisés constitués par des motifs de points blancs et de points noirs juxtaposés. L'ordre est à nouveau blanc, jaune, bleu, violet, vert, rouge, indigo et noir du plus clair au plus foncé. Plus les grisés sont foncés plus les petits points utilisés pour les représenter sur l'écran sont denses comme on le voit sur la deuxième ligne ci-dessous :



Quand on peint, les bords du motif doivent être tracés sans fuite possible en n'importe quelle couleur sauf blanc.

Exemple 19 (Fleur en couleur) Le programme ci-dessous permet de peindre la fleur ci-contre en couleur :



```
from LibRobot import *
# Dessiner et peindre une fleur stylisée
tailleGrille = 12
epaisseurTige = 3
epaisseurBord = 1
eb(); lg(tailleGrille)
# Bas de la tige
ec(epaisseurTige); pqtd(); vg()
av(); av(); av()
# Feuille droite :
# dessiner le pourtour
ec(epaisseurBord); vd(); pqtd(); vd()
# peindre l'intérieur
cc(vert); phtg(); peindre(); p3htd()
# Feuille gauche :
# dessiner le pourtour
cc(noir); vg(); pqtg(); vg()
# peindre l'intérieur
cc(vert); phtd(); peindre(); p3htg()
# Haut de la tige
cc(noir); ec(epaisseurTige)
av(); av(); av(); av()
# Coeur :
# dessiner le pourtour du coeur
ec(epaisseurBord); pqtd()
vg(); vg(); vg(); vg()
# pétales
cc(noir); pqtd(); vg(); vg(); vg()
cc(violet); phtd(); peindre(); phtd()
cc(noir); pqtd(); vg(); vg(); vg()
cc(violet); phtd(); peindre(); phtd()
cc(noir); pqtd(); vg(); vg(); vg()
cc(violet); phtd(); peindre(); phtd()
cc(noir); pqtd(); vg(); vg(); vg()
cc(violet); phtd(); peindre(); phtd()
# peindre l'intérieur du coeur
pqtd(); cc(jaune); peindre()
st()
```

□

Exercice 23 *Écrire des programmes Python pour dessiner et colorier en noir et blanc :*

- *les pavages du carré de la figure 12.2 ci-dessous ;*
- *les mosaïques de la salle des Abencérages (XIV*ème *siècle) de l'Ahlambra, la résidence des rois maures à Grenade en Andalousie telles qu'elles ont été relevées par le peintre Escher en 1936 [26] (figure 12.3 de la page 145).*

□

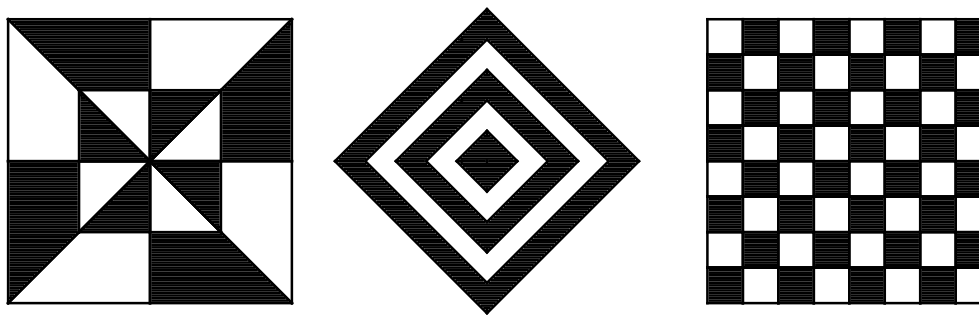


FIG. 12.2 – *Pavages du carré*

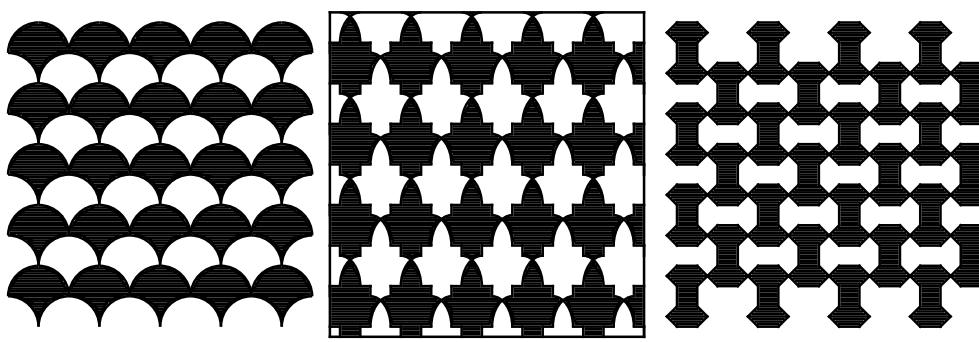


FIG. 12.3 – *Mosaïques de l'Alhambra*

Exercice 24 *Écrire des programmes Python pour dessiner et colorier en couleur :*

- *la balle et le carrelage en trois couleurs de la figure 12.4 (page 146) ;*
- *la mosaïque de l'Ahlambra [26] de la figure 12.5 (page 146) ;*

– *le pavage de carrés disjoints de la figure 12.6 (page 147) ;*

– *les pavages des figures 10.13 (page 127), 10.14 (page 128), 10.15 (page 129) et 10.16 (page 130) utilisant le motif ornemental de N. J. van de Vecht comme il est peint à la figure 10.10 (page 125).*

□

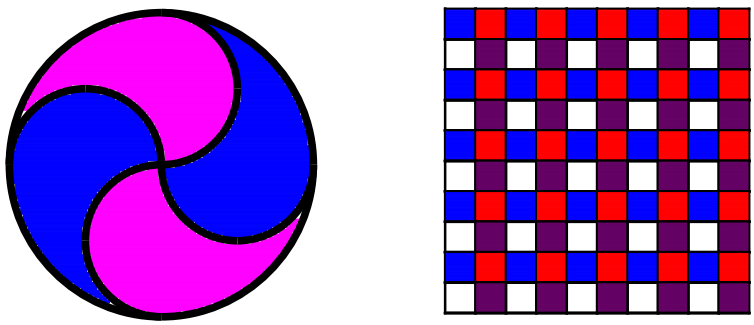


FIG. 12.4 – *Balle, pavage en trois couleurs*

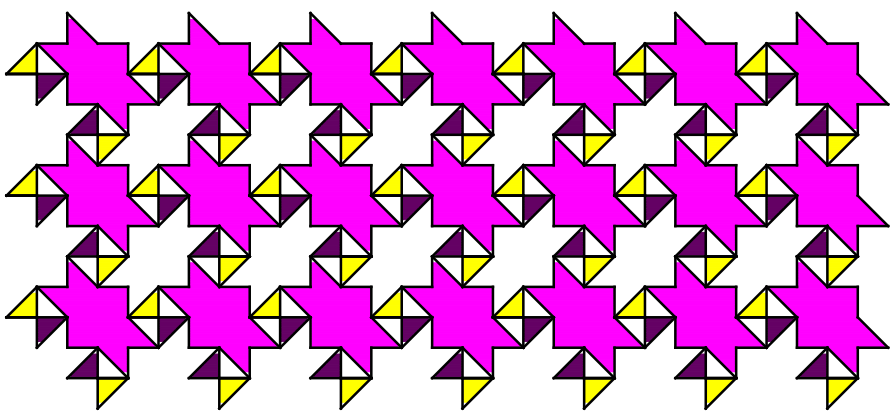


FIG. 12.5 – *Mosaïque de l'Alhambra en couleur*

Exercice 25 *Quel est le motif utilisé pour réaliser le pavage noir et blanc de la figure 12.7 (page 148) ?* □

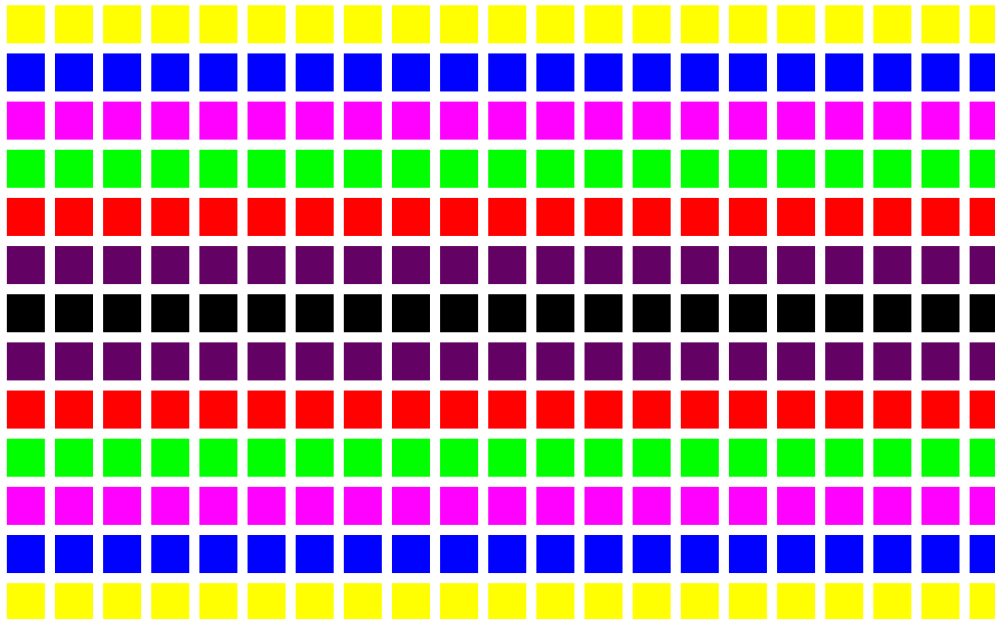


FIG. 12.6 – *Pavage de carrés disjoints en couleur*

Corrigé 23 *Nous donnons un programme, les autres se trouvent dans le dossier* `livre/12_peinture` *:*

```
from LibRobot import *

# Mosaïque de l'Alhambra
tailleGrille = 3  # pixels
largeurMotif = 4  # fois la taille de la grille
hauteurMotif = 6  # fois la taille de la grille
tailleLigne = 7  # motifs d'une ligne impaire
nombreDeLignes = 12  # pair

# Dessiner un motif en forme de 8, en partant et en
# arrivant à l'extrémité de la pointe inférieure
# gauche, avec le robot orienté vers le nord.
def motif():
    for i in range(2):
        # Côté gauche/droit (au dessus/dessous de la
        # pointe inférieure).
        phtd(); av(); phtg(); avf(hauteurMotif - 4)
        phtg(); av(); pqtd(); av()
        # Dessus/dessous
```

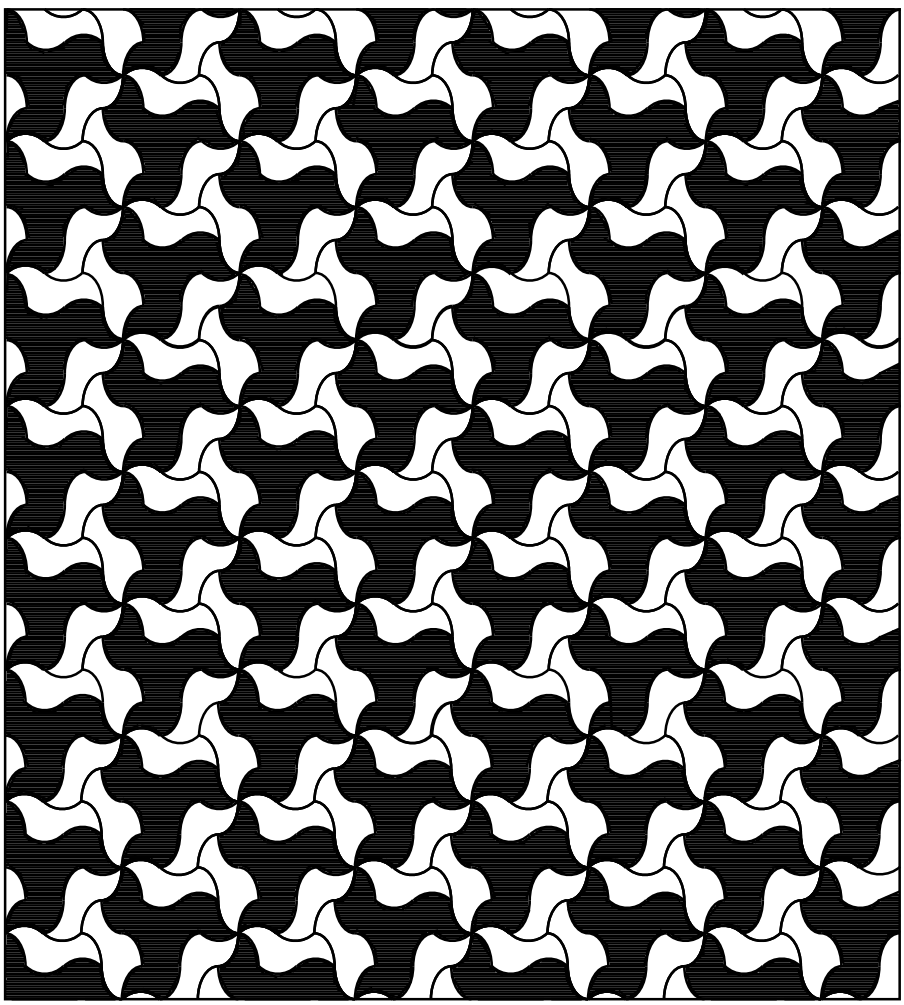


FIG. 12.7 – *Pavage du plan en noir et blanc*

```
      phtd(); avf(largeurMotif - 2); phtd()
      # Haut/bas du côté droit/gauche (en dessus/dessous
      # de la pointe supérieure/inférieure).
      av(); phtd()
   # Colorier et tourner le robot dans l'orientation
   # initiale.
   pqtg(); peindre(); pqtd()

# Passer de la pointe inférieure gauche d'un motif à
# celle du motif suivant sur la même ligne
# horizontale à droite.
def motifSuivant():
   lc(); pqtd(); avf(2 * largeurMotif); pqtg(); bc()

# Dessiner une ligne horizontale comportant
# tailleLigne motifs.
def ligneImpaire():
   for i in range(tailleLigne):
      motif(); motifSuivant()

# Dessiner une ligne comportant (tailleLigne - 1)
# motifs.
def lignePaire():
   for i in range(tailleLigne - 1):
      motif(); motifSuivant()

# Revenir sur la pointe inférieure gauche du premier
# motif d'une ligne.
def retourDebutDeLigne():
   lc(); pqtg()
   avf(2 * largeurMotif * tailleLigne - largeurMotif)
   pqtd(); bc()

# Passer de la pointe inférieure gauche d'un motif à
# celle du motif à la verticale sur la ligne
# suivante.
def ligneSuivante():
   lc(); avf(-(hauteurMotif - 2)); bc()

# Paver avec un nombre de lignes égal à
# nombreDeLignes.
def pavage():
   for i in range(nombreDeLignes // 2):
      ligneImpaire(); retourDebutDeLigne()
      ligneSuivante()
      lignePaire(); retourDebutDeLigne()
      ligneSuivante()

# Placer le robot à la pointe inférieure gauche du
```

```
# motif gauche de la première ligne.
lc(); lg(tailleGrille); ag(); eh(); pdt()
avf(hauteurMotif); pqtg(); av(); pqtg(); bc()
pavage(); st()
```

□

Corrigé 24 *Le motif du pavage du plan en noir et blanc de la figure 12.7 (page 148) est donné à la figure 12.8 (page 150).* □

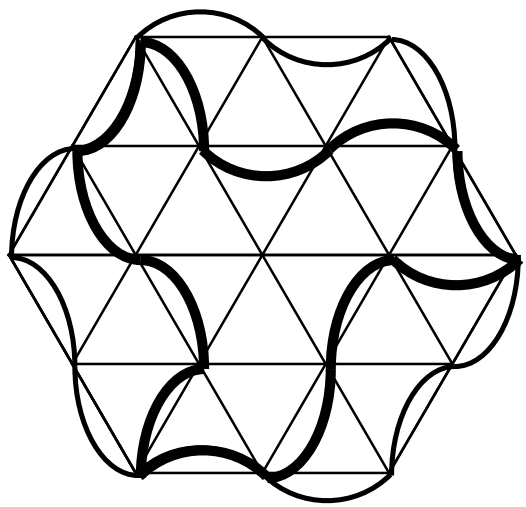


FIG. 12.8 – *Motif du pavage de la figure 12.7*

Corrigé 25 *Nous donnons un programme, les autres se trouvent dans le dossier* `livre/12_peinture` *:*

```
from LibRobot import *

# Dessiner et peindre une balle en couleur
epaisseurBord = 3
demiRayon = 30

def quartDeFigure():
    lg(demiRayon); vg(); vg()
    lg(2 * demiRayon); vg(); ce(); pdt()

# Placer le robot dans la zone à peindre et peindre
def peindreQuartDeFigure():
```

```
        lg(demiRayon // 2)
        lc(); ce(); pqtd(); av(); bc(); peindre()
    ec(epaisseurBord)
    for i in range(4):
        quartDeFigure()
    for i in range(2):
        cc(bleu); peindreQuartDeFigure()
        cc(violet); peindreQuartDeFigure()
    st()
```

□

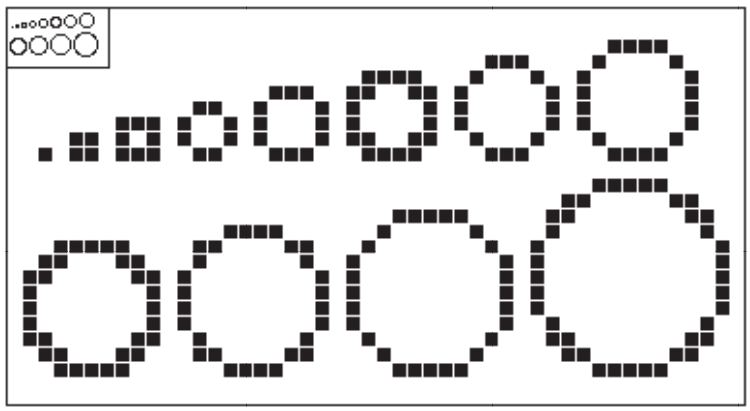
Addenda

On peut imaginer que pour peindre, le $\mathcal{R}^o\underline{b}^o t$ place un germe sur l'écran, en arrière de son point d'arrêt. Si un germe est placé sur un pixel de l'écran déjà colorié alors ce germe disparaît. En particulier si le germe initial placé par le $\mathcal{R}^o\underline{b}^o t$ est sur un pixel non blanc, la peinture s'arrête immédiatement. Si le germe est placé sur un pixel blanc de l'écran, alors le germe disparaît, le pixel blanc sur lequel il se trouvait est peint de la couleur du crayon et quatre nouveaux germes sont placés à l'est, à l'ouest, au nord et au sud du pixel qui vient d'être peint. La peinture est terminée quand tous les germes ont disparu.

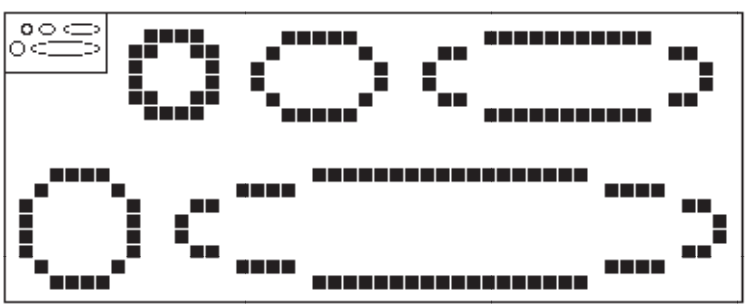
Cet algorithme de peinture est donc basé sur la représentation physique du dessin sur l'écran et non pas sur la représentation logique qui serait celle obtenue en mémorisant toutes les commandes exécutées par le $\mathcal{R}^o\underline{b}^o t$. De ce fait il se produit parfois des fuites logiquement inexplicables mais qui sont dues à la différence des représentations logiques et physiques.

Prenons l'exemple d'un cercle. Mathématiquement un cercle est une ligne continue constituée par un ensemble infini de points sans dimension. Sur l'écran d'un ordinateur le cercle sera représenté par une ligne discontinue constituée par un ensemble fini de "points" de la dimension d'un pixel. Les notions mathématique et informatique de cercle sont donc complètement différentes.

L'approximation est acceptable quand la taille des pixels est très faible par rapport au diamètre du cercle de la même façon que le compas permet de tracer des cercles qui donnent une bonne image de l'objet mathématique. Cependant pour des cercles très petits, on arrive à quelques aberrations visibles sur la figure ci-après où les cercles représentés dans le petit cadre en haut à gauche de la figure ont été agrandis huit fois. On notera en particulier que les très petits cercles sont carrés !



Sur les écrans ayant des pixels rectangulaires, les cercles ont des formes d'ellipses. Le programme doit donc corriger la déformation en dessinant logiquement des ellipses pour obtenir des cercles sur l'écran. Les cercles sont fermés en ce sens que tout pixel peint sur le cercle a au moins un pixel voisin peint dans l'une des huit directions de la rose des vents. Ce n'est pas le cas pour les ellipses très allongées que l'on peut voir sur la figure ci-dessous :

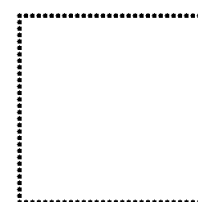


Les trous dans le périmètre des ellipses allongées sont des points de fuite de la peinture quand on veut peindre l'intérieur de l'ellipse. Pour éviter ces fuites il faudrait utiliser des algorithmes de coloriage plus complexes basés sur la structure logique et non pas physique du dessin.

13

Déclarations de procédures avec paramètres

Nous avons appris au chapitre 5 à déclarer et à appeler des procédures sans paramètres nous permettant de répéter plusieurs fois la même séquence de commmandes du *Robot* à des endroits différents dans le programme. Au chapitre 9 nous avons utilisé des commandes du *Robot* qui sont des appels de procédures avec paramètres, l'usage d'un ou plusieurs paramètres permettant à la procédure de faire des séquences de commandes similaires mais différentes selon la valeur des paramètres. Nous allons maintenant voir comment déclarer des procédures avec paramètres en Python. Commençons par un exemple simple :



```
from LibRobot import *
# Dessiner un carré en pointillé
# Tracer un trait pointillé de n points,
# en supposant n > 1.
def traitPointille(n):
    lg(8); ec(2); lc()
    for l in range(n - 1):
        dp(); av()
    dp(); bc()
# Tracer un carré pointillé avec p points par côté
# (p > 1).
def carre(p):
    for c in range(4):
```

```
        traitPointille(p); pqtd()
carre(30); cacherRobot(); st()
```

La procédure `traitPointille` permet de tracer un trait en pointillé constitué par une succession de `n` pixels noirs séparés par (`n`−1) pixels blancs, le nombre `n` de points étant passé en paramètre de cette procédure. Dans la déclaration '**def** `traitPointille(n) :`', le paramètre `n` est écrit entre les parenthèses qui suivent le nom de la procédure, parenthèses qui restent vides `()` pour les procédures sans paramètres. Il n'est pas nécessaire d'indiquer que `n` est un nombre entier : Python ne le demande pas, et c'est seulement au moment de l'exécution, en calculant l'expression `n - 1`, qu'il utilise la valeur de `n`. La procédure `carre` permet de dessiner un carré en pointillé, le nombre `p` de points par côté étant passé en paramètre. L'appel `carre(30)` indique qu'il faut exécuter la procédure `carre` avec un paramètre `p` égal à 30. De ce fait la procédure `traitPointille` est appelée quatre fois avec un paramètre `n` égal à `p` soit 30. On dit que `p` est le *paramètre formel* (dont on ne connaît pas encore la valeur au moment de la déclaration de la procédure et que l'on désigne par un identificateur) et que 30 est le *paramètre effectif* (c'est-à-dire la valeur du paramètre formel pour cet appel de la procédure).

De manière générale, les déclarations et appels de procédures avec paramètres ont la forme suivante :

Déclaration de procédure avec paramètres :

```
# Spécification : ce que fait la procédure
def nomDeLaProcedure(paramFormel1 [a], paramFormel2, ...):
    ... Instructions du corps de la procédure [b] (indentées) ...
```

[a] Un paramètre formel est un identificateur.

[b] Les expressions figurant dans ces instructions peuvent utiliser les paramètres formels `paramFormel1`, ... de la procédure.

Appel de procédure avec paramètres :

```
nomDeLaProcedure(paramEffectif1 [a], paramEffectif2, ... [b])
```

[a] Un paramètre effectif est une expression entière (ou une couleur, etc.).

[b] Les paramètres effectifs sont donnés dans l'ordre où apparaissent les paramètres formels correspondants.

On remarquera que les paramètres, formels dans la déclaration de procédure comme effectifs dans l'appel de procédure, sont placés entre parenthèses et séparés par des virgules. Par exemple pour dessiner un rectangle de hauteur 3 et de largeur 4 avec la procédure `rectangle` déclarée par '**def** `rectangle(hauteur, largeur) :`' dans le programme `MacintoshSE.py` du dossier `livre/13_decl_procedure_avec_parame`, on écrit `rectangle(3, 4)`. Si l'on oublie les parenthèses en écrivant `rectangle 3, 4`, Python refuse le programme avant de l'exécuter avec le message '`SyntaxError: invalid syntax`' (erreur de syntaxe). Un paramètre effectif peut être une expression, comme dans `demiCercleDroit(2 * tailleGrille)` (page 158) ou `avf(-3)` : elle est calculée avant l'appel de la procédure. Attention à bien placer la parenthèse fermante : `demiCercleDroit(2) * tailleGrille` serait compris comme la multiplication par `tailleGrille` du résultat de l'appel `demiCercleDroit(2)`, ce qui n'a pas de sens. Après avoir dessiné le demi-cercle, Python s'arrête avec le message : '`TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'`' (types d'opérandes non permis pour `*` : `NoneType` et entier). `NoneType` est le type de la valeur **None** ('rien' en anglais) que rend un appel de procédure.

Le nombre de paramètres effectifs doit être égal au nombre de paramètres formels. Si l'on écrit `rectangle(3)`, Python s'arrête, quand le $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ arrive à cet appel, avec le message '`TypeError: rectangle() missing 1 required positional argument: 'largeur'`' (il manque à `rectangle()` un paramètre obligatoire : `largeur`) et si l'on écrit `rectangle(3, 4, 5)`, avec le message '`TypeError: rectangle() takes 2 positional arguments but 3 were given`' (`rectangle()` prend 2 paramètres mais 3 ont été donnés).

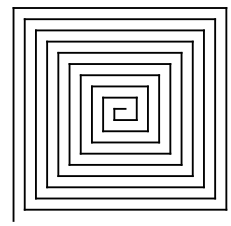
Une procédure avec paramètres formels peut être appelée plusieurs fois dans un programme avec des paramètres effectifs qui peuvent être différents à chaque fois.

Les paramètres formels d'une procédure sont des noms *locaux* à la procédure : ils ne sont utilisables que dans le corps de la procédure (ailleurs, Python répondrait par exemple '`NameError: name 'hauteur' is not defined`'). Par conséquent des procédures différentes peuvent avoir des paramètres formels portant le même nom bien qu'il s'agisse de paramètres différents.

Ces différents points sont illustrés dans l'exemple ci-dessous.

Exemple 20 (Spirale segmentée) Le programme ci-dessous dessine une spirale constituée de paires de segments de droite de longueurs successives 1, 2, 3, ..., 19 :

```
from LibRobot import *
# Dessiner une spirale segmentée
# Tourner à gauche puis avancer le robot tout droit
# de l fois la taille de la grille pour former un
# quart de tour.
def quartDeTour(l):
   pqtg(); avf(l)
# Tracer deux quarts de tour successifs pour former
# un demi-tour.
def demiTour(l):
   quartDeTour(l)
   quartDeTour(l)
# Tracer une spirale de n demi-tours
def spirale(n):
   for l in range(1, n + 1):
      demiTour(l)
lg(6); spirale(19); st()
```



□

Exercice 26 *Écrire des programmes Python comportant des procédures avec paramètres pour réaliser les dessins suivants :*

– *la croix à trois branches transversales, l'hélice coloriée, le micro-ordinateur, le logo en couleur, la chenille (où le nombre d'anneaux du corps de la chenille est passé en paramètre de la procédure de dessin), la pirogue (où le nombre de rameurs est un paramètre) représentés à la page 157 ;*

– *les carrés inscrits les uns dans les autres ou la version peinte de ce dessin représenté à la figure 13.8 (page 158) ;*

– *l'arabesque inspirée par la coupole du mausolée du Sultan Qayt-Bay représentée à la figure 13.1 (page 157) ;*

– *la péniche de la figure 1.13 (page 10) où le nombre de hublots est donné en paramètre ;*

– *le pavage de Golomb de la figure 8.6 (page 107).*

□

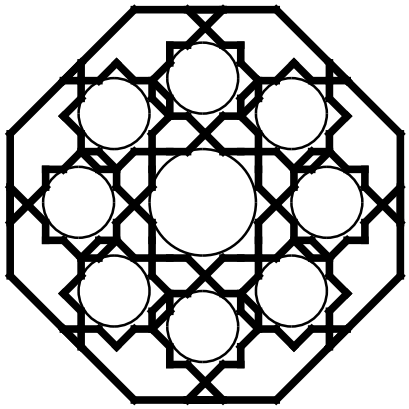


FIG. 13.1 – *Arabesque*

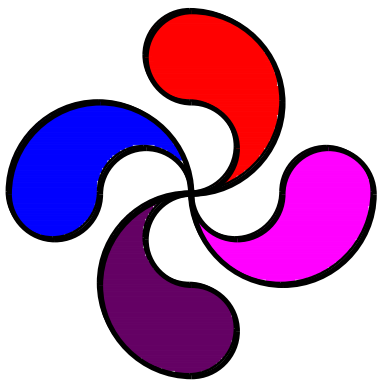


FIG. 13.2 – *Hélice coloriée*

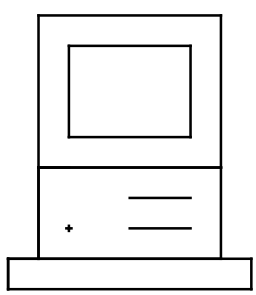


FIG. 13.3 – *Ancien micro-ordinateur*

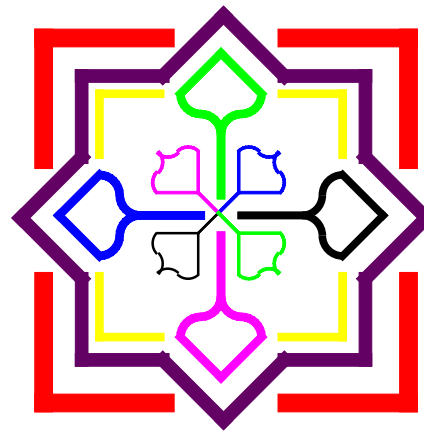


FIG. 13.4 – *Logo en couleur*

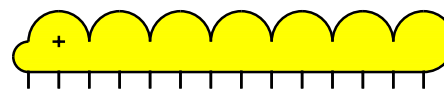


FIG. 13.5 – *Chenille*

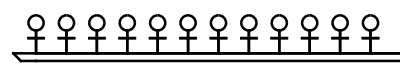


FIG. 13.6 – *Pirogue*

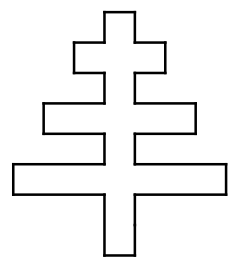


FIG. 13.7 – *Croix à trois branches*

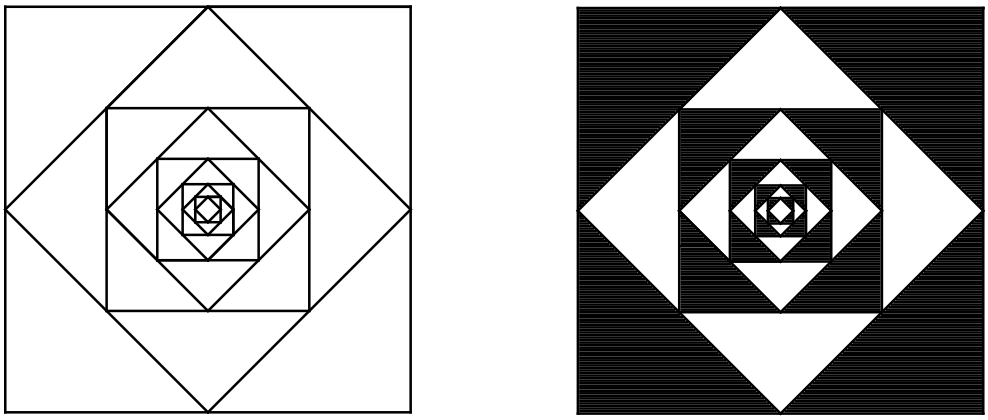


FIG. 13.8 – *Motif de carrés inscrits de Horemis*

Corrigé 26 *Nous donnons un programme, les autres se trouvant dans le dossier* `livre/13_decl_procedure_avec_parame` *:*

```
from LibRobot import *

# Dessiner une hélice à quatre pales en couleur
tailleGrille = 18
epaisseurDuBord = 2

def demiCercleGauche(r):
    lg(r); vg(); vg()

def demiCercleDroit(r):
    lg(r); vd(); vd()

def pale(c):
    # Dessiner le pourtour de la pale
    ec(epaisseurDuBord); cc(noir)
    demiCercleGauche(tailleGrille)
    demiCercleDroit(tailleGrille)
    demiCercleDroit(2 * tailleGrille)
    # Peindre l'intérieur de la pale
    pqtd(); lg(tailleGrille)
    lc(); avf(3); cc(c); peindre(); avf(-3); bc()

def helice():
    pale(bleu); pale(indigo); pale(violet); pale(rouge)

helice(); st()
```

□

Addenda

On distingue en Python, les définitions des identificateurs de leurs utilisations. Les *occurrences de définition* figurent dans une déclaration de constante ou de procédure (**def**) comme nom de constante, de procédure ou de paramètre formel. Les *occurrences d'utilisation* de ces identificateurs sont les noms de constante ou de paramètre formel utilisés dans une expression entière ou les noms de procédure utilisés dans des appels.

Il se peut qu'un même identificateur ait plusieurs occurrences de définition dans un programme. De ce fait il se pose le problème d'établir la correspondance entre les occurrences d'utilisation et de définition d'un même identificateur. Par exemple l'identificateur peut être utilisé pour nommer une constante du programme et des paramètres formels dans des procédures. En Python, ces procédures peuvent être imbriquées les unes dans les autres (en déclarant avec **def** une procédure `p2` à l'intérieur du corps d'une procédure `p1`, avec une indentation de plus), comme dans l'exemple ci-après :

```
from LibRobot import *
n = 17
def p1(n):
    def p2(n):
        ...
        ... n ...  #1
        ...
    ...
    p2(n)  #2
    ...
...
p1(n)  #3
...
```

On dit que le programme a une *structure de blocs*. Ces blocs sont formés par les procédures imbriquées les unes dans les autres, le programme étant considéré comme le bloc le plus externe.

La *règle de visibilité* des identificateurs en Python dit que l'occurrence d'utilisation d'un identificateur se réfère à l'occurrence de définition du bloc le plus interne où figure cette occurrence d'utilisation et qui contient une occurrence de définition de l'identificateur.

Dans l'exemple ci-dessus,

- l'occurrence d'utilisation de `n` à la ligne marquée `#1` se réfère au paramètre formel de la procédure `p2` ;
- l'occurrence d'utilisation de `n` à la ligne marquée `#2` se réfère au paramètre formel de la procédure `p1` ;
- l'occurrence d'utilisation de `n` à la ligne marquée `#3` se réfère à la constante 17.

La structure de blocs, que l'on retrouve dans Python, a été introduite par ALGOL 60 [29]. Ce langage fut créé par un groupe d'experts européens et américains réunis dans le groupe de travail WG2.1 de l'IFIP (International Federation of Information Processing) qui regroupe toutes les associations nationales d'informaticiens. C'est l'ancêtre d'une famille de langages parmi lesquels on compte PASCAL, ADA et MODULA.

14

Repère cartésien

Pour repérer la position du $\mathcal{R}\!\mathcal{o}\!\mathcal{b}\!\mathcal{o}t$ sur l'écran, nous utilisons un *repère cartésien* tel qu'il est représenté à la figure 14.1 ci-dessous :

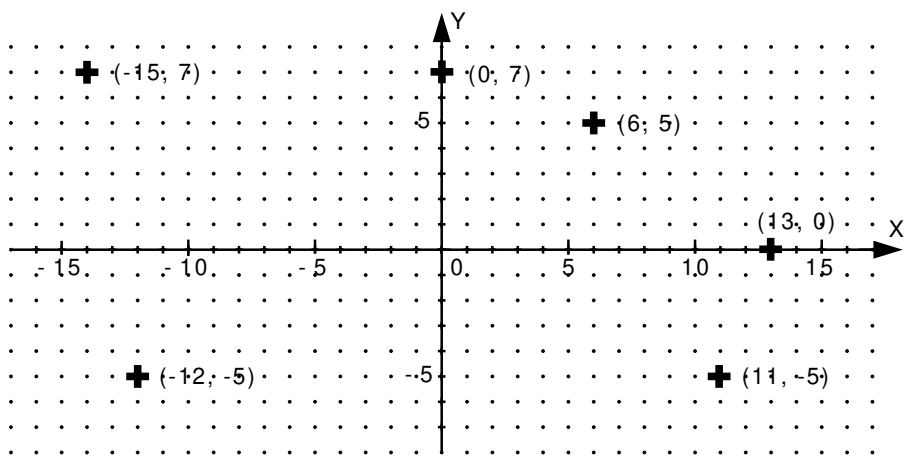


FIG. 14.1 – *Coordonnées dans un repère cartésien*

L'*origine* 0 du repère est située au centre de la grille. L'axe horizontal OX est appelé l'axe des *abscisses*. L'axe vertical OY est appelé l'axe des *ordonnées*. Ces deux axes sont gradués. La longueur des graduations sur l'axe des abscisses est égale à la longueur du côté horizontal d'un carreau de la grille comptée en pixels. De même, la longueur des graduations sur l'axe des ordonnées est égale à la longueur du côté vertical d'un carreau de la grille comptée en pixels.

Un point de la grille est repéré par ses *coordonnées cartésiennes* qui sont deux entiers (X,Y), le premier étant son *abscisse* et le second son *ordonnée*. Par exemple le point de la grille situé au centre de la grille a pour coordonnées (0,0). Les points situés à droite de l'origine O ont une abscisse positive. Les points situés à gauche de l'origine O ont une abscisse négative (précédée du signe moins $-$). De même, les points situés au dessus de l'origine O ont une ordonnée positive tandis que les points situés en dessous de l'origine O ont une ordonnée négative. A titre d'exemple, quelques points de la grille sont repérés sur la figure 14.1 (page 161) par leurs coordonnées.

L'abscisse X d'un point A est la distance à l'origine de sa projection sur l'axe OX (c'est-à-dire du point d'intersection de l'axe OX avec la droite parallèle à l'axe OY passant par A). De même, l'ordonnée Y d'un point A est la distance à l'origine de sa projection sur l'axe OY (c'est-à-dire du point d'intersection de l'axe OY avec la droite parallèle à l'axe OX passant par A). Par exemple sur la figure 14.2, ci-dessous, on a $X = 6$ et $Y = 7$ [1].

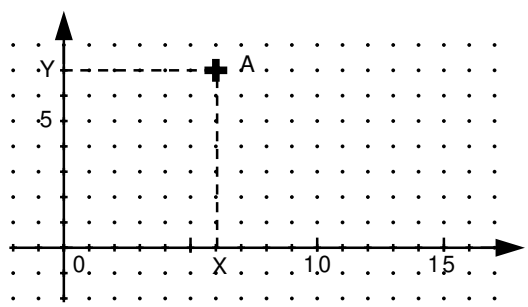


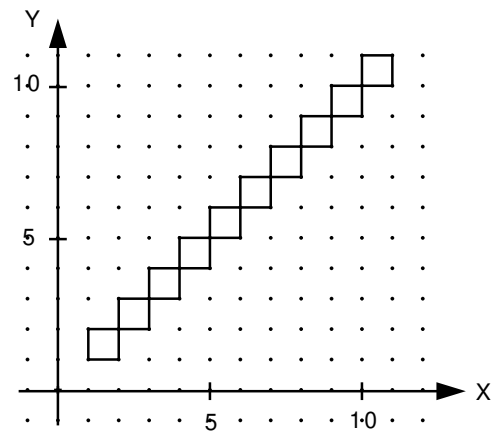
FIG. 14.2 – *Point A d'abscisse $X = 6$ et d'ordonnée $Y = 7$*

La commande `deplacer(x, y)` place le $\mathcal{R}\text{o}\underline{b}\text{o}t$ sur le point de la grille de coordonnées cartésiennes (x,y). L'orientation et la position du crayon du $\mathcal{R}\text{o}\underline{b}\text{o}t$ restent inchangées.

Par exemple, la commande `deplacer(0, 0)` place le $\mathcal{R}\text{o}\underline{b}\text{o}t$ à l'origine du repère cartésien au centre de la grille [2]. Un exemple est donné ci-après :

1. On trouvera dans le dossier `livre/14_repere_cartesien` un programme Python, `Apprendre_Repere.py`, permettant d'apprendre à repérer des points du plan avec un repère cartésien.
2. La commande `ce()` a l'effet supplémentaire de recentrer si nécessaire l'origine du repère exactement au centre de l'écran.

```
from LibRobot import *
# Ligne de carrés (deplacer)
# Carré de côté 1
def carre():
    av(); pqtd()
    av(); pqtd()
    av(); pqtd()
    av(); pqtd()
for i in range(1, 11):
    deplacer(i, i)
    carre()
st()
```



La commande `translater(x, y)` permet de translater le $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ de (x,y) sans changer son orientation [3]. Un exemple de translation d'un rectangle est donné à la figure 14.3 ci-dessous.

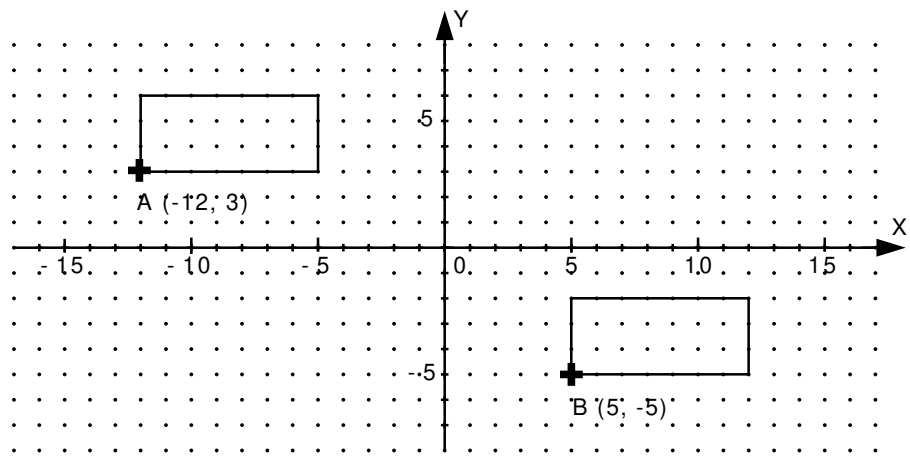


FIG. 14.3 – *Translation (17, -8) de A en B*

3. On trouvera dans le dossier `livre/14_repere_cartesien` un programme Python, `Apprendre_Translation.py`, permettant d'apprendre à translater des points du plan repérés par des coordonnées cartésiennes.

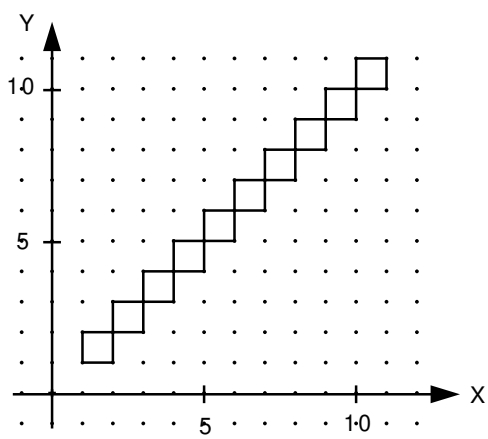
Une translation du $\mathcal{R}\!\mathit{obot}$ de (x,y) consiste à augmenter (ou diminuer si x est négatif) son abscisse de x et à augmenter (ou diminuer si y est négatif) son ordonnée de y. Si le centre de la grille de coordonnées (0,0) est exactement au centre de l'écran, on peut définir le déplacement à l'aide de la translation comme suit :

```
def deplacer(x, y):
    ce(); translater(x, y)
```

Comme le montre cet exemple, les commandes `deplacer` et `translater` sont des procédures à deux paramètres, l'abscisse `x` puis l'ordonnée `y`, placés entre parenthèses et séparés par une virgule, comme les deux paramètres de l'appel `rectangle(3, 4)` du chapitre précédent. L'ordre compte : `deplacer(6, 7)` et `deplacer(7, 6)` ne placent pas le $\mathcal{R}\!\mathit{obot}$ au même point.

Le programme `LigneDeCarres1` de la page 163 peut être réécrit en utilisant une commande de translation au lieu d'une commande de déplacement comme suit :

```
from LibRobot import *
# Ligne de carrés (translater)
# Carré de côté 1
def carre():
    av(); pqtd()
    av(); pqtd()
    av(); pqtd()
    av(); pqtd()
for i in range(10):
    translater(1, 1)
    carre()
st()
```



Avec la commande `translater(x, y)`, le déplacement du $\mathcal{R}\!\mathit{obot}$ est relatif à sa position de départ tandis qu'avec la commande `deplacer(x, y)`, le déplacement du $\mathcal{R}\!\mathit{obot}$ est relatif à l'origine du repère cartésien.

Exercice 27 *Écrire des programmes Python utilisant les commandes* `deplacer` *et* `translater` *pour réaliser les dessins suivants :*

- *les pavages obtenus par répétition de divers motifs inscrits dans un carré de dimension* (6×6) *de la figure 14.6 (page 167) ;*
- *le pavage de quadrilatères de la figure 14.4 (page 165) ;*

_ *la mosaïque de l'Alhambra relevée par Escher [26] de la figure 14.5 (page 166).*
□

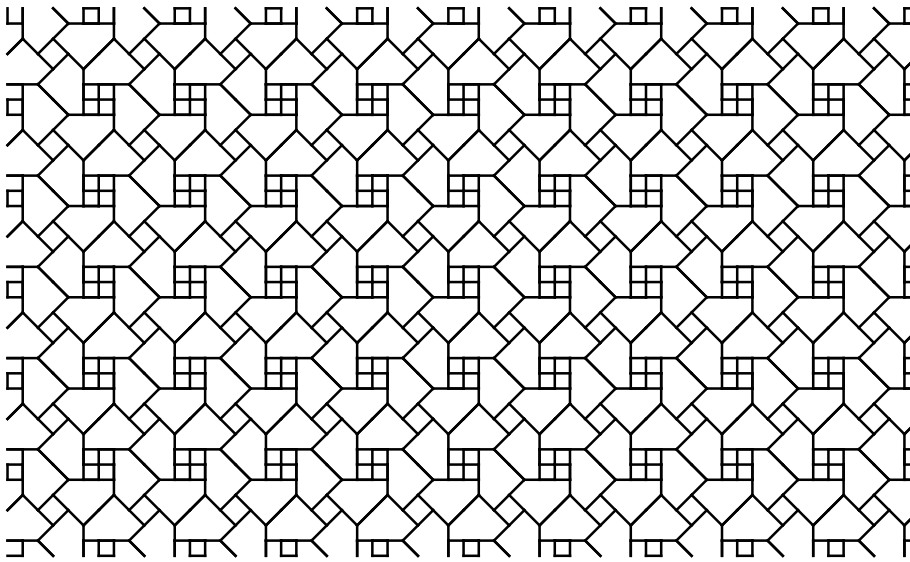


FIG. 14.4 – *Pavage de quadrilatères*

Corrigé 27 *Nous donnons le programme de dessin du pavage de quadrilatères de la figure 14.4 (page 165), les autres programmes se trouvant dans le dossier* `livre/14_repere_cartesien`. *Il s'agit d'un pavage de l'écran avec des motifs inscrits dans des carrés* (6×6), *le motif de base étant dessiné à côté de la procédure qui le dessine.*

```
from LibRobot import *

# Pavage de quadrilatères produits par un motif
# inscrit dans un carré (6 x 6)
tailleGrilleX = 6
tailleGrilleY = 6
motifX = 6  # Largeur du motif
motifY = 6  # Hauteur du motif

# Dessiner un motif dans un carré (motifX x motifY),
# le robot partant et arrivant au centre du motif,
# orienté vers le nord.
def motif():
```

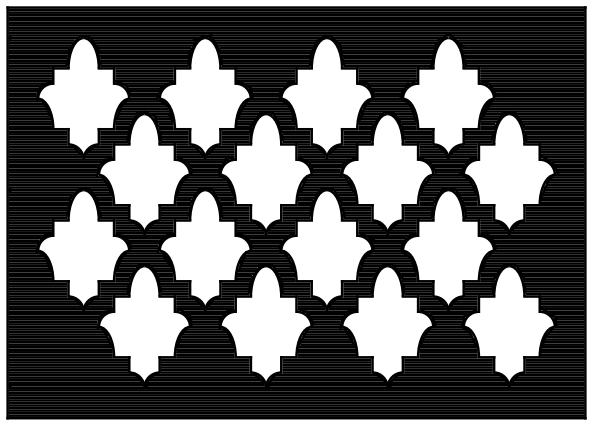


FIG. 14.5 – *Mosaïque de l'Alhambra*

```
    for i in range(4):
        lc(); pqtg(); av(); bc()
        p3htd(); av(); av(); pqtg(); av(); pdt(); av()
        phtg(); av(); av(); pdt(); av(); pqtd(); av()
        pqtd(); av()
        lc(); p3htd(); av(); av(); av(); bc()
        phtd()
# Paver avec y lignes de x motifs
def pavage(x, y):
    # Décentrer le dessin à gauche de la moitié de la
    # longueur d'une ligne et vers le haut de la
    # moitié de la hauteur du pavage.
    translater(-(motifX * (x // 2)), motifY * (y // 2))
    # Dessiner (y - 1) lignes de x motifs (avec
    # translation en bas à gauche de la ligne
    # suivante).
    for i in range(y - 1):
        # Dessiner une ligne de x motifs (sans
        # translation à droite après le dernier motif
        # de la ligne).
        for j in range(x - 1):
            motif()
            # Passer au motif suivant à droite dans la
            # même ligne.
            translater(motifX, 0)
        motif()
        # Passer au début de la ligne suivante.
        translater(-(motifX * (x - 1)), -motifY)
```

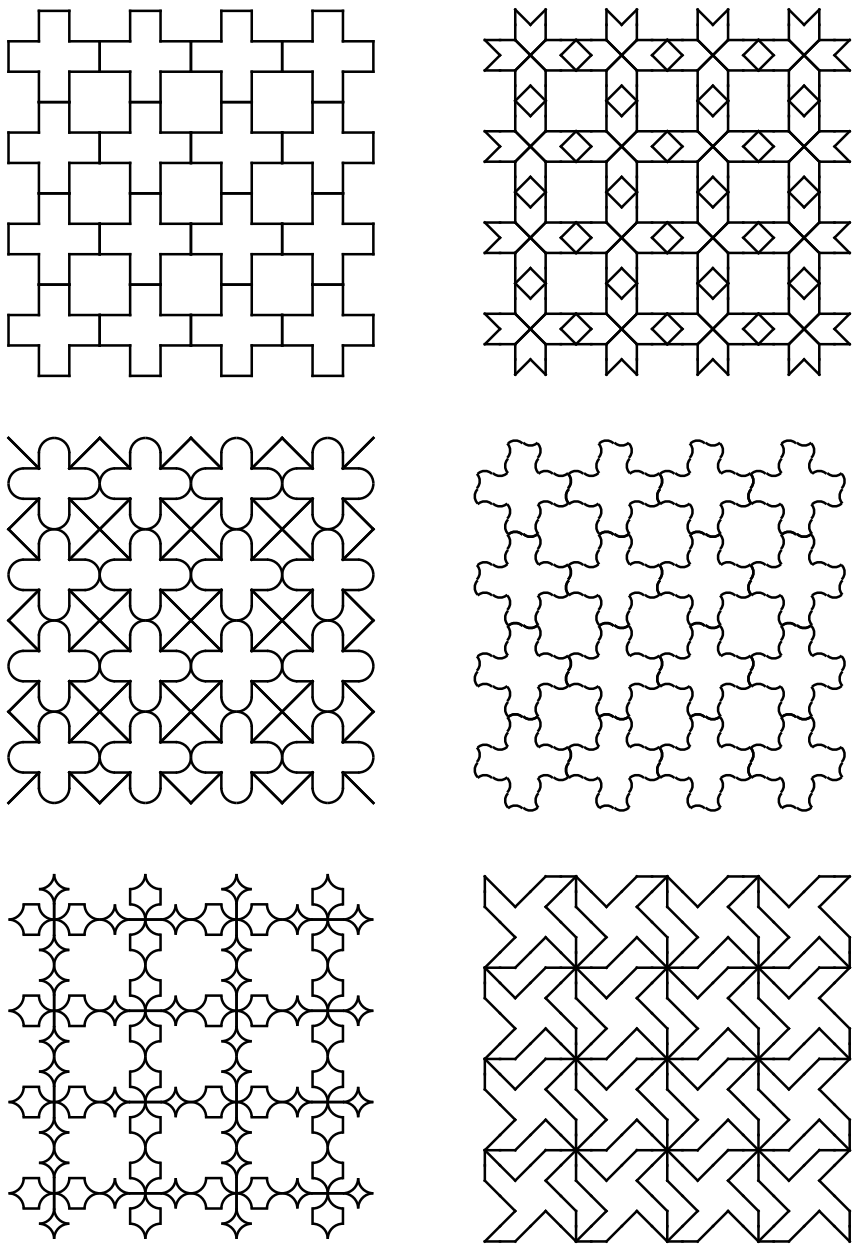


FIG. 14.6 – *Pavages de motifs* (6×6)

```
    # Dessiner la dernière ligne de x motifs (sans
    # translation en bas à gauche de la ligne
    # suivante).
    for j in range(x - 1):
       motif()
       translater(motifX, 0)
    motif()
lgX(tailleGrilleX); lgY(tailleGrilleY)
pavage(10, 6)
st()
```

□

Addenda

Suivant les traces d'Apollonius [a] qui avait donné informellement les équations des coniques (cercles, ellipses, paraboles et hyperboles), Fermat et Descartes ont été les premiers à "mettre en équations" des problèmes géométriques. Descartes a introduit la notion de système de coordonnées planes, chaque point du plan étant défini par son abscisse x et son ordonnée y obtenues en projetant sur des axes OX et OY, rectangulaires ou obliques.

Une courbe algébrique est alors définie par la relation $f(x,y) = 0$ qui existe entre les coordonnées de chacun de ses points. Par exemple l'équation cartésienne du cercle de centre (a,b) est $x^2 + y^2 - 2ax - 2by + c = 0$ avec le rayon R tel que $R^2 = a^2 + b^2 - c$ en supposant que $a^2 + b^2 \geq c$.

[a] Apollonius de Perge, astronome et mathématicien grec, 295 – 230 av. J.-C.

Descartes a exposé sa méthode dans un ouvrage publié en 1637 et intitulé "La Géométrie" qui, avec "La Dioptrique" et "Les Météores", constituaient les appendices d'un des textes les plus célèbres de la philosophie, le fameux "Discours de la méthode pour bien conduire sa raison et chercher la vérité dans les sciences" où il expose sa philosophie méthodique et rationelle appelée depuis le *cartésianisme*. L'idée d'équation apparaît plus clairement chez Fermat, qui a découvert indépendamment de Descartes, vers 1629, le principe de base de la *géométrie analytique*. Il ne l'a publié qu'en 1679 dans son "Isagoge (Introduction aux lieux plans et solides)".

Descartes n'avait étudié sérieusement que les coniques du second degré. Newton [a] étudia les cubiques du troisième degré et les courbes de degré supérieur. On lui doit aussi d'avoir employé systématiquement les coordonnées négatives dont Descartes se méfiait. La Hire [b] introduisit en 1679 la considération d'un système de coordonnées spatiales, ce qui revient à construire un troisième axe de coordonnées OZ, perpendiculaire aux axes OX et OY définissant le plan. Un point dans l'espace est alors déterminé par trois nombres réels (x,y,z) et l'équation $f(x,y,z) = 0$ définit une surface gauche. Möbius [c] et Plücker [d] étendirent la notion de coordonnées en introduisant les coordonnées homogènes (X,Y,Z,T) telles que $x = \frac{X}{T}$, $y = \frac{Y}{T}$ et $z = \frac{Z}{T}$. Ceci permet d'introduire les points à l'infini $(X,Y,Z,0)$ et les points imaginaires $X = A_1 + iA_2$, $Y = B_1 + iB_2$, $Z = C_1 + iC_2$ et $T = D_1 + iD_2$ où $i^2 = -1$. On peut alors considérer par exemple des cercles imaginaires tels que $a^2 + b^2 < c$.

---

[a] Isaac Newton, mathématicien, physicien et astronome anglais, 1642–1727.
[b] Philippe de la Hire, astronome et mathématicien français, 1640–1718
[c] August Ferdinand Möbius, astronome et mathématicien allemand, 1790–1868
[d] Julius Plücker, mathématicien et physicien allemand, 1801–1868

15

Test “if”

En réponse à nos questions, notre $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ peut indiquer dans quelle direction il est orienté. N'étant pas doué de la parole, ses réponses sont *booléennes* c'est-à-dire **True** (prononcer “trou”, *vrai* en anglais) ou **False** (prononcer “faulse”, *faux* en anglais), écrits en Python avec une majuscule. On peut imaginer qu'il donne ses réponses à l'aide d'un voyant lumineux qu'il allume pour répondre **True** (vrai) et qu'il laisse éteint pour répondre **False** (faux).

Pour poser les questions sur l'orientation du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$, on utilise les commandes de test suivantes :

Commandes de test d'orientation du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$:	
`tn()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers le nord (n) et **False** sinon.
`tne()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers le nord-est (ne) et **False** sinon.
`te()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers l'est (e) et **False** sinon.
`tse()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers le sud-est (se) et **False** sinon.
`ts()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers le sud (s) et **False** sinon.
`tsw()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers le sud-ouest (sw) et **False** sinon.
`tw()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers l' ouest (w) et **False** sinon.
`tnw()`	**True** si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ est tourné vers le nord-ouest (nw) et **False** sinon.

TAB. 15.1 – *Tests d'orientation du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$*

En Python, l'instruction de test “if” permet de n'exécuter une séquence d'instructions que si une certaine *expression booléenne* est vraie (**True**). Pour commencer nous allons utiliser des expressions booléennes simples, à savoir les commandes de test d'orientation du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}^{o}t$ (`tn()`, `tne()`, `te()`, `tse()`, `ts()`,

`tsw()`, `tw()`, `tnw()`). Dans sa forme la plus simple, l'instruction de test "if" s'écrit comme suit :

Test "if" à une seule alternative :

if ␣ expression booléenne [a] :
... Instructions (indentées) exécutées seulement si l'expression booléenne vaut **True** ...

[a] A résultat **True** (vrai) ou **False** (faux).

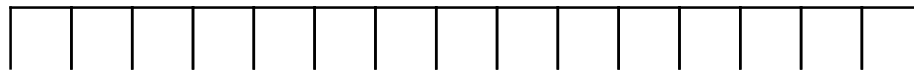
On peut traduire en français par `si ... alors`. Le mot **if** se prononce 'if' (si en anglais). Comme pour le corps d'une procédure ou d'une boucle **for**, la ligne du **if** se termine par un deux-points ' `:`' et les instructions qui dépendent du test sont indentées (décalées de quatre blancs vers la droite par rapport au **if**) : elles forment un *bloc* qui se termine à la première ligne moins indentée. Si l'on oublie le deux-points, Python refuse le programme avec le message '`SyntaxError: expected ':'`' (deux-points attendu) et si l'on oublie d'indenter, avec le message '`IndentationError: expected an indented block after 'if' statement on line 5`' (un bloc indenté était attendu après l'instruction **if** de la ligne 5, si le **if** est à la ligne 5 du programme).

Quand il n'y a qu'une seule instruction à exécuter, on peut l'écrire sur la même ligne, après le deux-points, comme dans '**if** `tw() : pqtd()`'. Mais attention, dans '**if** `tw() : pqtd() ; av()`' les deux instructions `pqtd()` et `av()` dépendent du test. Une erreur grave est de se tromper d'indentation. Par exemple dans :

```
if tw():
   pqtd()
av()
```

seule l'instruction `pqtd()` dépend du test : l'instruction `av()` qui n'est pas indentée est toujours exécutée, que le $\mathcal{R}o\underline{b}ot$ soit tourné vers l'ouest ou non. Python ne peut pas signaler cette erreur puisque le programme est correct (mais ne fait pas ce que l'on voulait). Notons aussi que l'instruction de test "if" est *une* instruction : si d'autres instructions la suivent, elles s'écrivent sur les lignes suivantes, avec la même indentation que le **if**.

Dans l'exemple ci-dessous, le $\mathcal{R}o\underline{b}ot$ avance deux fois et tourne à droite d'un quart de tour puis d'un quart de tour supplémentaire s'il est orienté à l'ouest :



```
from LibRobot import *

# Dessiner une frise en peigne
longueur = 15

ag()
for i in range(3 * longueur):
   avf(2); pqtd()
   if tw():
      pqtd()
st()
```

Dans sa première forme ci-dessus l'instruction de test ne fait rien quand l'expression booléenne vaut **False**. Si l'on veut donner le choix de la séquence d'instructions à exécuter dans les deux alternatives, on utilisera la forme suivante :

Test "if" à deux alternatives :

if ␣ expression booléenne [a] **:**
... Instructions (indentées) exécutées si l'expression booléenne vaut **True** ...
else :
... Instructions (indentées) exécutées si l'expression booléenne vaut **False** ...

[a] A résultat **True** (vrai) ou **False** (faux).

On peut traduire en français par `si ... alors ... sinon ...` (**else** se prononce 'elsse' et signifie 'autrement' en anglais).

Le **else** s'écrit exactement sous le **if** (avec la même indentation) et il est suivi d'un deux-points. Comme précédemment, une alternative qui ne comporte qu'une seule instruction peut s'écrire sur la même ligne que le **if** ou le **else**, après le deux-points. C'est une erreur que d'oublier le deux-points après le **else** ou de mal aligner le **else** sous le **if**, mais elle n'est pas très grave car elle est signalée par Python avant l'exécution ('`SyntaxError: expected ':''` ou '`IndentationError: unindent does not match any outer indentation level`', le retrait ne correspond à aucun niveau d'indentation englobant). Python signale aussi une erreur ('`SyntaxError: invalid syntax`') quand on oublie d'indenter l'une des instructions placées entre le **if** et le **else**.

Dans le dessin de la frise en dents de scie ci-dessous, le $\mathcal{R}^{o}\!\!\smile^{o}t$ tourne de trois quarts de tour à droite s'il est orienté au nord-est et à gauche dans les autres cas (le seul autre cas possible étant d'ailleurs qu'il soit tourné vers le sud) :

```
from LibRobot import *

# Dessiner une frise en dents de scie
nombreDeDents = 31

ag(); phtd()
for i in range(2 * nombreDeDents):
   av()
   if tne():
      p3htd()
   else:
      p3htg()
st()
```

Quand il y a plus de deux cas à tester on peut utiliser plusieurs alternatives et ne rien faire quand aucune des expressions booléeennes ne vaut **True** comme dans :

Test "if" à n alternatives :

if ␣ expression booléenne 1**:**
    ... Instructions (indentées) exécutées si l'expression booléenne 1 vaut **True** ...
elif ␣ expression booléenne 2**:**
    ... Instructions (indentées) exécutées si l'expression booléenne 2 vaut **True** ...
...
elif ␣ expression booléenne n**:**
    ... Instructions (indentées) exécutées si l'expression booléenne n vaut **True** ...

On peut aussi prévoir une dernière alternative qui est exécutée quand toutes les expressions booléeennes testées valent **False** :

Test "if" à $(n+1)$ alternatives :

if ␣ expression booléenne 1**:**
... Instructions (indentées) exécutées si l'expression booléenne 1 vaut **True** ...
elif ␣ expression booléenne 2**:**
... Instructions (indentées) exécutées si l'expression booléenne 2 vaut **True** ...
...
elif ␣ expression booléenne n**:**
... Instructions (indentées) exécutées si l'expression booléenne n vaut **True** ...
else:
... Instructions (indentées) exécutées si les expressions booléennes 1, ..., n valent **False** ...

Le mot **elif** est une contraction de **else if** (sinon si). Les expressions booléennes sont évaluées dans l'ordre et seules les instructions de la première alternative dont l'expression vaut **True** sont exécutées. Cette forme de test multiple est illustrée ci-dessous dans le programme construisant la frise suivante :

```
from LibRobot import *

# Dessiner une frise de vaguelettes
longueur = 15

ag()
for i in range(2 * longueur):
   if tn():
      vd(); pqtd()
   elif ts():
      vg(); pqtg()
   elif te():
      pqtd()
   else:
      pqtg()
   av()
st()
```

Pour terminer nous allons introduire de nouvelles commandes du $\mathcal{R}^{o}\!\!\smile^{o}t$ qui permettent de faire pivoter le $\mathcal{R}^{o}\!\!\smile^{o}t$ sur place vers une orientation donnée.

Par exemple la commande `pn()` qui permet de faire pivoter le $\mathcal{R}$obot vers le nord peut être définie à l'aide de la procédure Python suivante :

```
# Faire pivoter le robot sur place vers le nord
def pn():
    if tne():
        phtg()
    elif te():
        pqtg()
    elif tse():
        p3htg()
    elif ts():
        pdt()
    elif tsw():
        p3htd()
    elif tw():
        pqtd()
    elif tnw():
        phtd()
```

Le programme `OrientationAuNord.py` du dossier `livre/15_if` essaie cette procédure dans les huit directions. Nous pourrions définir de la même façon les commandes de pivotement du $\mathcal{R}$obot sur place dans l'une des huit directions principales de la rose des vents qui sont données dans la table 15.2 ci-dessous.

Commandes de pivotement du $\mathcal{R}$obot sur place :	
`pn()`	fait pivoter le $\mathcal{R}$obot sur place vers le nord (n).
`pne()`	fait pivoter le $\mathcal{R}$obot sur place vers le nord-est (ne).
`pe()`	fait pivoter le $\mathcal{R}$obot sur place vers l'est (e).
`pse()`	fait pivoter le $\mathcal{R}$obot sur place vers le sud-est (se).
`ps()`	fait pivoter le $\mathcal{R}$obot sur place vers le sud (s).
`psw()`	fait pivoter le $\mathcal{R}$obot sur place vers le sud-ouest (sw).
`pw()`	fait pivoter le $\mathcal{R}$obot sur place vers l' ouest (w).
`pnw()`	fait pivoter le $\mathcal{R}$obot sur place vers le nord-ouest (nw).

TAB. 15.2 – *Pivotement du $\mathcal{R}$obot sur place dans une direction donnée*

Exercice 28 *Écrire des programmes Python comportant des instructions de test "if" pour réaliser les frises de la figure 15.1 ci-dessous.* □

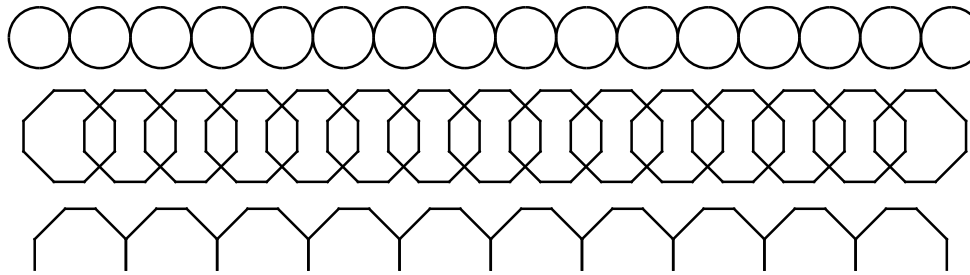


FIG. 15.1 – *Frises de cercles et (semi-) octogones*

Corrigé 28 *Nous donnons le programme pour dessiner la frise d'octogones de la figure 15.1 (page 177), les autres se trouvant dans le dossier* `livre/15_if` *:*

```
from LibRobot import *

# Dessiner une frise d'octogones
nombreDOctogones = 15

ag(); pqtd()
for i in range(8 * nombreDOctogones):
   if te():
      lc(); avf(2); bc()
   av(); phtd()
st()
```

□

Addenda

L'instruction de test `si ...alors ...sinon ...`fut introduite vers la fin des années 1950 dans le langage de programmation ALGOL 60. Ce langage, conçu par un groupe d'experts internationaux, eut une très grande influence dans la conception de langages de programmation ultérieurs notamment ALGOL W, PASCAL, ADA, MODULA et bien d'autres. Une autre forme d'instruction de test par cas fut introduite dans le langage de programmation ALGOL W (également conçu par N. Wirth) et reprise en PASCAL sous le nom de "case". En OCaml, l'instruction de test par cas s'appelle le *filtrage* ("match" en anglais). Python possède aussi, depuis sa version 3.10, une instruction `match`, mais nous ne l'utiliserons pas : en Python, on écrit habituellement un test par cas avec une suite de **if** ... **elif** ... **else**, en comparant la valeur d'une expression à chacun des cas possibles.

Par exemple `valOrientation()` est égal au numéro de l'orientation du $\mathcal{Robot}$ (0 pour le nord, 1 pour le nord-est, 2 pour l'est, etc.). La bibliothèque `LibRobot` déclare les constantes `nord`, `nordEst`, `est`, `sudEst`, `sud`, `sudOuest`, `ouest` et `nordOuest`, égales à 0, 1, ..., 7. On pourra écrire :

```
if valOrientation() == nord:           # nord
    ...
elif valOrientation() == nordEst:      # nord-est
    ...
elif valOrientation() == est:          # est
    ...
elif valOrientation() == sudEst:       # sud-est
    ...
elif valOrientation() == sud:          # sud
    ...
elif valOrientation() == sudOuest:     # sud-ouest
    ...
elif valOrientation() == ouest:        # ouest
    ...
elif valOrientation() == nordOuest:    # nord-ouest
    ...
else:                                  # impossible
    pass
```

On notera que la comparaison s'écrit avec deux signes égal '==', le signe '=' seul servant à déclarer des constantes. Avec '**if** `valOrientation() = nord :`', Python répondrait '`SyntaxError: cannot assign to function call here. Maybe you meant '==' instead of '='?`' (on ne peut pas donner de valeur à un appel de fonction, vouliez-vous dire '==' ?). On peut aussi comparer deux nombres avec `!=` (différent de, $\neq$), `<`, `<=` ($\leq$), `>` et `>=` ($\geq$). Le dernier cas **else** regroupe toutes les autres valeurs, comme le mot **otherwise** ('autrement' en anglais) de certaines extensions de PASCAL. L'instruction **pass** ('passer' en anglais) ne fait rien ; on peut aussi omettre ce dernier cas. Plusieurs cas semblables peuvent être regroupés comme dans l'exemple ci-dessous, où l'on teste si le reste de la division du numéro d'orientation par 2 est nul :

```
if valOrientation() % 2 == 0:  # N, E, S, W
    ...
else:                          # autres cas
    ...
```

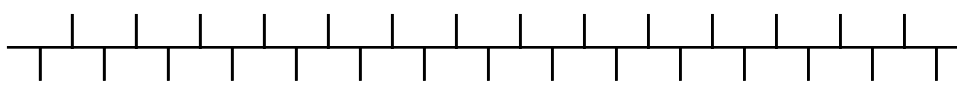
On notera qu'avec des tests **if**, les cas peuvent être des noms de constantes comme `nord` aussi bien que des nombres comme `0`, ce qui rend le programme plus lisible. Ce n'était pas le cas avec le filtrage d'OCaml où '`| nord ->`' ne signifie pas 'si l'orientation est le nord' mais déclare un nouvel identificateur `nord` qui convient pour toutes les valeurs (il en est de même avec le `match` de Python).

16

Expressions booléennes

16.1 Parité

Les tests que nous avons faits jusqu'à présent dans les instructions "if" concernaient l'orientation du $\mathcal{R}^{o}_{b}{}^{o}t$. En Python il est possible de faire des tests concernant les valeurs des paramètres et des compteurs de boucles "for". Il peut être utile de faire un test sur le compteur dans le corps d'une boucle pour exécuter des actions différentes à chaque itération. Prenons l'exemple du dessin de la frise en forme de double peigne ci-dessous :



Les dents du peigne étant dessinées de gauche à droite, les $1^{\text{ère}}$, $3^{\text{ème}}$, $5^{\text{ème}}$, ... sont dirigées vers le bas alors que les $2^{\text{ème}}$, $4^{\text{ème}}$, $6^{\text{ème}}$, ... sont dirigées vers le haut. Dans le programme de dessin on utilise un compteur de boucle `i` qui vaut 1 quand on dessine la $1^{\text{ère}}$ dent, 2 quand on dessine la $2^{\text{ème}}$, ... Par conséquent quand la valeur du compteur de boucle est *impaire* (c'est-à-dire 1, 3, 5, ...) il faut dessiner une dent vers le bas tandis que si la valeur du compteur est *paire* (c'est-à-dire 2, 4, 6, ...) il faut dessiner une dent vers le haut. Un nombre est impair quand le reste de sa division par 2 est égal à 1. Pour tester que la valeur du compteur `i` est impaire on utilise donc l'expression booléenne `i % 2 == 1` qui vaut **True** (vrai) quand le reste `i % 2` de la division entière de `i` par 2 est égal à 1 et **False** (faux) quand ce reste est égal à 0, c'est-à-dire quand `i` est pair. On remarquera que le test d'égalité s'écrit avec deux signes égal `==`, le signe `=` seul étant réservé à la déclaration

des constantes comme `nombreDeDents = 30`.

```
from LibRobot import *

# Dessiner une frise en forme de double peigne
nombreDeDents = 30

ag(); pe()
for i in range(1, nombreDeDents + 1):
    # Trait horizontal
    av()
    if i % 2 == 1:
        # Dent vers le bas
        pqtd(); av(); pdt(); av(); pqtd()
    # Dent vers le haut
    else:
        pqtg(); av(); pdt(); av(); pqtg()
st()
```

16.2 Comparaison d'entiers

Si `i` est un compteur de boucle ou un paramètre entier de procédure, l'expression booléenne `i > 10` est **True** (vraie) si la valeur de `i` est strictement plus grande que 10 et **False** (fausse) dans le cas contraire c'est-à-dire si la valeur de `i` est inférieure ou égale à 10. Les symboles mathématiques $\leq$ (inférieur ou égal) et $\geq$ (supérieur ou égal) se notent `<=` et `>=` en Python (car les symboles mathématiques $\leq$ et $\geq$ ne se trouvent pas sur les claviers d'ordinateurs alors que `<`, `>` et `=` sont toujours présents). Par exemple les expressions booléennes `10 <= 10` et `10 <= 20` valent **True** (vrai) tandis que `10 <= 9` vaut **False** (faux). En Python, comme nous l'avons vu, on note l'égalité avec deux signes égal `==` puisque le symbole mathématique usuel `=` sert à déclarer les constantes. Le symbole mathématique $\neq$ (différent) se note `!=`. Par exemple `10 == 10` et `10 != 20` valent **True** (vrai) tandis que `10 != 10` et `10 == 20` valent **False** (faux) [1]. On peut utiliser les expressions booléennes de comparaison d'entiers ou de valeurs de paramètres entiers dans le test de l'instruction "if". Par exemple la commande `avf(n)` qui fait avancer le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{o}t$ `n` fois si la valeur de `n` est positive, le fait reculer si la valeur de `n` est négative

1. Si l'on écrit par erreur **if** `i = 10 :` au lieu de **if** `i == 10 :`, Python refuse le programme avec le message `SyntaxError : invalid syntax. Maybe you meant '==' or ' :=' instead of '=' ?`

et qui ne change pas sa position quand la valeur de `n` est nulle peut se définir comme suit :

```
# Avancer n fois si n est positif, reculer -n fois si
# n est négatif, ne pas bouger si n est nul.
def avf(n):
   if n > 0:
      for i in range(n):
         av()
   elif n < 0:
      pdt()
      for i in range(-n):
         av()
      pdt()
```

On remarquera que le paramètre effectif négatif de l'appel `avf(-5)` s'écrit simplement avec son signe, entre les parenthèses de l'appel. Pour reculer, le *Robot* fait demi-tour, avance `-n` fois (qui est positif) et refait demi-tour.

De manière plus générale, on peut utiliser les opérateurs `<` (strictement inférieur), `<=` (inférieur ou égal), `==` (égal), `!=` (différent), `>=` (supérieur ou égal) et `>` (strictement supérieur) pour comparer les valeurs de deux expressions entières comme par exemple `i <= l` dans le programme suivant qui trace le serpentin ci-dessous. La commande `lg` accepte aussi bien un nombre entier, comme dans `lg(20)` ou `lg(i)`, qu'un nombre avec une virgule, écrite avec un point comme dans `lg(20.5)` (nous y reviendrons au chapitre 19, consacré aux expressions rationnelles) :

```
from LibRobot import *

# Dessiner un serpentin
def dessinSerpentin(l):
   for i in range(1, 2 * l + 1):
      if i <= l:
         lg(i)
      else:
         lg(2 * l + 1 - i)
      if i % 2 == 1:
         # Demi-cercle supérieur
         vd(); vd()
      # Demi-cercle inférieur
      else:
```

```
        vg(); vg()
ag(); dessinSerpentin(13); st()
```

16.3 Négation, conjonction et disjonction

La *négation* d'une expression booléenne E s'écrit **not** E, le mot anglais **not** signifiant 'non'. Si la valeur de E est **True** alors celle de **not** E est **False** tandis que si la valeur de E est **False** alors celle de **not** E est **True**. Par exemple **not** `(5 % 2 == 1)` vaut **False** puisque `5 % 2 == 1` vaut **True**, `5` étant impair. Les parenthèses ne sont pas indispensables mais rendent l'expression plus lisible (nous verrons pourquoi dans l'Addenda). Ceci peut se résumer par la table de vérité suivante :

E	**not** E
True	**False**
False	**True**

La *conjonction* de deux expressions booléennes E_1 et E_2 s'écrit E_1 **and** E_2, ce qui se lit 'E_1 et E_2'. La valeur de E_1 **and** E_2 est **True** (vrai) si et seulement si la valeur de E_1 et celle de E_2 sont **True** (vrai). Si l'une des expressions booléennes E_1 ou E_2 vaut **False** (faux) alors la conjonction E_1 **and** E_2 vaut également **False** (faux). Par exemple `5 >= 0` **and** `5 % 2 == 1` vaut **True** puisque `5` est positif et impair. Ceci peut se résumer par la table de vérité suivante :

E_1	E_2	E_1 **and** E_2
True	**True**	**True**
True	**False**	**False**
False	**True**	**False**
False	**False**	**False**

La *disjonction* de deux expressions booléennes E_1 et E_2 s'écrit E_1 **or** E_2, ce qui se lit 'E_1 ou E_2'. La valeur de E_1 **or** E_2 est **True** (vrai) si et seulement si la valeur de E_1 ou celle de E_2 est **True** (vrai). Si les valeurs de E_1 et de E_2 sont **False** (faux) alors la disjonction E_1 **or** E_2 vaut également **False** (faux). Par exemple `4 >= 0` **or** `4 % 2 == 1` vaut **True** puisque `4` est positif et bien que `4` soit pair. Ceci peut se résumer par la table de vérité suivante :

E_1	E_2	E_1 **or** E_2
True	**True**	**True**
True	**False**	**True**
False	**True**	**True**
False	**False**	**False**

La conjonction et la disjonction peuvent se réaliser simplement avec un circuit électrique comme on le voit à la figure 16.1 ci-dessous avec la convention que **True** représente l'interrupteur fermé et l'ampoule allumée (le courant passe) et **False** représente l'interrupteur ouvert et l'ampoule éteinte (le courant ne passe pas).

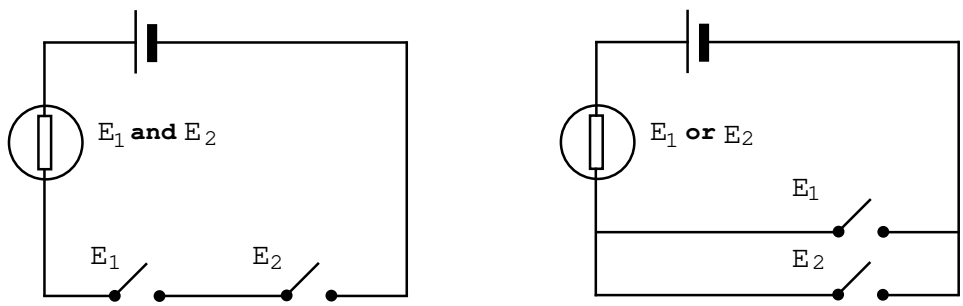


FIG. 16.1 – *Circuits électriques réalisant la conjonction et* la disjonction logiques

Une disjonction est utilisée dans le programme suivant pour tester si le $\mathcal{R}^{o}\!\!\mathcal{b}^{o}t$ est orienté au nord ou à l'est :

```
from LibRobot import *

# Dessiner une frise en or
longueur = 31

ag()
for i in range(longueur):
    if tn() or te():
        vd()
    else:
        avf(-1); vg(); pqtg()
st()
```

16.4 Expressions booléennes

La table ci-dessous résume les règles d'écriture et les règles de calcul de la valeur des expressions booléennes en Python :

Expressions booléennes :	
True	Valeur 'vrai'.
False	Valeur 'faux'.
Expr. entière$_1$ `<=` Expr. entière$_2$	**True** si et seulement si la valeur de l'Expr. entière$_1$ est inférieure ou égale à celle de l'Expr. entière$_2$.
Expr. entière$_1$ `<` Expr. entière$_2$	**True** si et seulement si la valeur de l'Expr. entière$_1$ est strictement inférieure à celle de l'Expr. entière$_2$.
Expr. entière$_1$ `==` Expr. entière$_2$	**True** si et seulement si la valeur de l'Expr. entière$_1$ est égale à celle de l'Expr. entière$_2$.
Expr. entière$_1$ `!=` Expr. entière$_2$	**True** si et seulement si la valeur de l'Expr. entière$_1$ est différente de celle de l'Expr. entière$_2$.
Expr. entière$_1$ `>` Expr. entière$_2$	**True** si et seulement si la valeur de l'Expr. entière$_1$ est strictement supérieure à celle de l'Expr. entière$_2$.
Expr. entière$_1$ `>=` Expr. entière$_2$	**True** si et seulement si la valeur de l'Expr. entière$_1$ est supérieure ou égale à celle de l'Expr. entière$_2$.

not Expr. bool.	**True** si et seulement si la valeur de l'Expr. bool. est égale à **False**.
Expr. bool.$_1$ **and** Expr. bool.$_2$	**True** si et seulement si les valeurs de l'Expr. bool.$_1$ et de l'Expr. bool.$_2$ sont égales à **True**.
Expr. bool.$_1$ **or** Expr. bool.$_2$	**True** si et seulement si la valeur de l'Expr. bool.$_1$ ou celle de l'Expr. bool.$_2$ est égale à **True**.

On peut donner un nom à une expression booléenne pour rendre les programmes plus lisibles. Par exemple :

```
# n est-il impair ?
def estImpair(n):
    return n % 2 == 1

# n est-il pair ?
def estPair(n):
    return n % 2 == 0
```

Ces déclarations ressemblent à des déclarations de procédures avec paramètres mais leur corps n'est pas une suite d'instructions pour le *Robot* : l'instruction **return** ('retourner' en anglais) est suivie d'une expression booléenne dont la valeur est calculée à chaque appel et retournée comme résultat de l'appel. On dit que `estImpair` et `estPair` sont des *fonctions booléennes* car leur évaluation retourne toujours un booléen (une valeur de type `bool`, c'est-à-dire **True** ou **False**). Par exemple `estImpair(5)` vaut **True** et `estPair(5)` vaut **False**. On peut les utiliser dans un test comme **if** `estImpair(i) :` ... ou dans une expression booléenne comme **not** `estImpair(i)`. Les tests d'orientation `tn()`, `te()`, ... du *Robot* sont également des fonctions booléennes, déclarées dans la bibliothèque `LibRobot`.

Exercice 29 *Écrire des programmes Python utilisant des tests de parité pour réaliser les dessins suivants :*

- *la serpentine pyramidale et la grille de la figure 16.2 ci-dessous ;*
- *le pavage octogonal et le pavage d'hexagones entrelacés de la figure 16.3 (page 188).*

□

Exercice 30 *Écrire des programmes Python utilisant des expressions booléennes de comparaison de valeurs entières pour réaliser les dessins suivants (le deuxième est difficile) :*

- *les chiffres digitaux de la figure 1.6 (page 8) où le chiffre à dessiner est passé en paramètre de la procédure de dessin ;*
- *le pavage de roues dentées de la figure 16.4 ci-dessous ;*
- *le tapis égyptien de la figure 16.5 (page 189).*

□

Exercice 31 *Écrire des programmes Python utilisant des négations, conjonctions ou disjonctions d'expressions booléennes pour réaliser les dessins suivants (le deuxième est difficile) :*

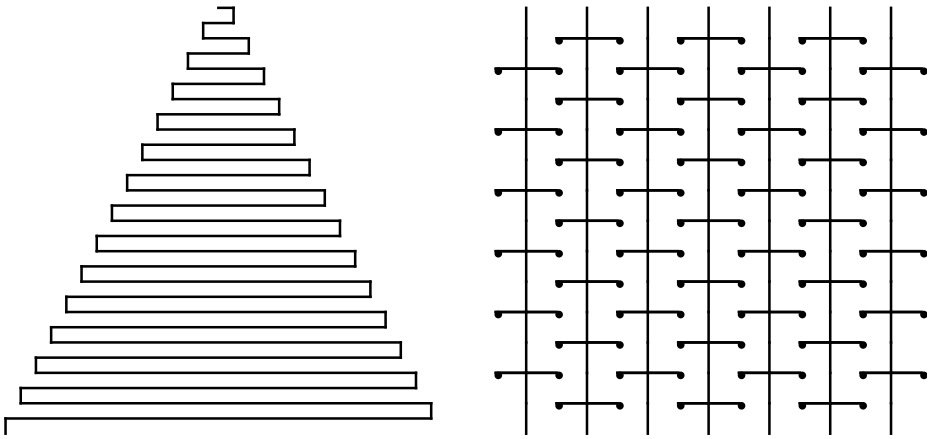


FIG. 16.2 – *Serpentine pyramidale et grille de croix*

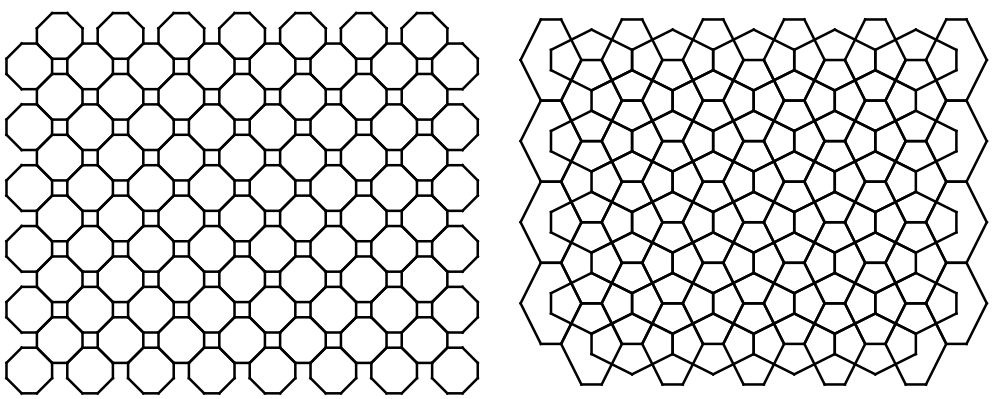


FIG. 16.3 – *Pavages d'octogones juxtaposés et d'hexagones entrelacés*

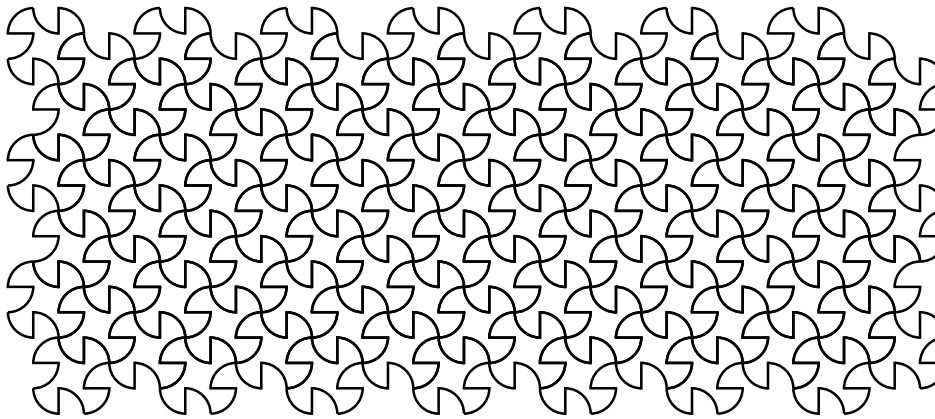


FIG. 16.4 – *Pavage de roues dentées*

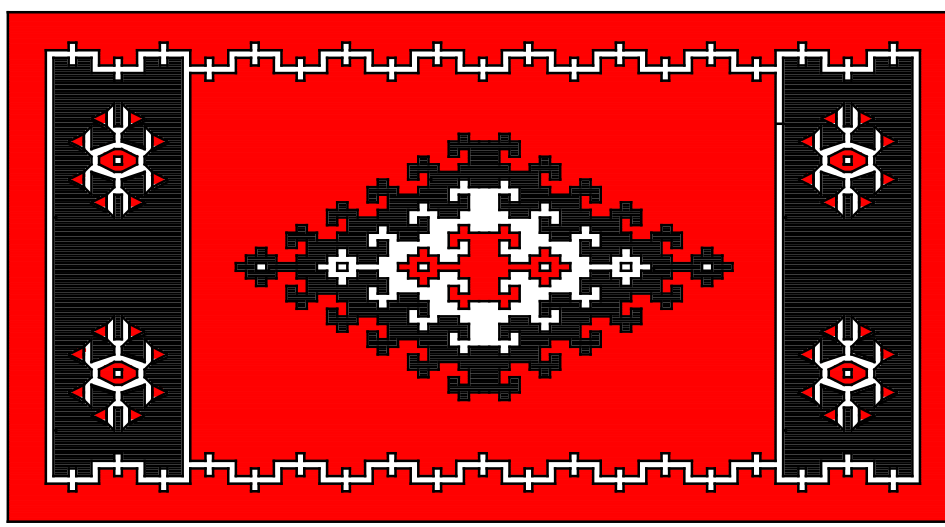


FIG. 16.5 – *Tapis égyptien*

_ *le carré zébré de la figure 16.6 (page 190) ;*

_ *la mosaïque de l'Alhambra représentée à la figure 16.7 (page 190).*

□

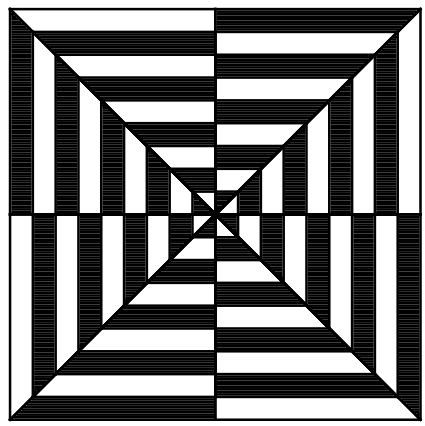


FIG. 16.6 – *Carré zébré*

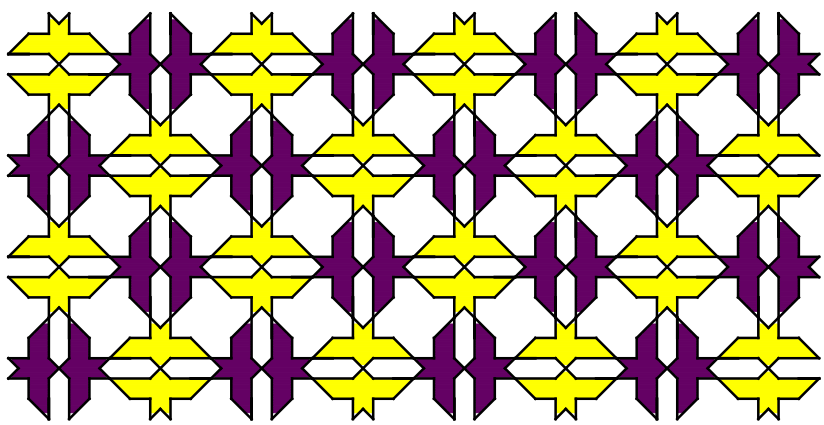


FIG. 16.7 – *Mosaïque de l'Alhambra*

Corrigé 29 *Nous donnons un programme, les autres se trouvant dans le dossier* `livre/16_expressions_booleennes` *:*

```
from LibRobot import *
# Dessiner une serpentine pyramidale
```

```
tailleGrille = 6
hauteur = 28

lg(tailleGrille); eh(); pqtd()
for i in range(1, hauteur + 1):
   avf(i)
   if i % 2 == 1:
      pqtd(); av(); pqtd()
   else:
      pqtg(); av(); pqtg()
st()
```

□

Corrigé 30 *Nous donnons un programme, le troisième se trouvant dans le dossier* `livre/16_expressions_booleennes` *:*

```
from LibRobot import *

# Dessiner un pavage de roues dentées
tailleGrille = 10
nombreDeMotifsParLigne = 7
nombreDeLignes = 14

# Avancer le robot tout droit, jusqu'à deux carreaux
# du bord nord du cadre.
def enHaut():
   eh(); pdt(); avf(2)

# Dessiner le motif du pavage
def roueDentee():
   for i in range(4):
      av(); pqtd(); vd(); pqtg(); vg(); pqtg()

def pavage():
   for l in range(1, nombreDeLignes + 1):
      lc(); enHaut(); ag()
      avf(l); pqtg(); av()
      # Décalage en début de ligne
      if l % 5 == 2:
         avf(2)
      elif l % 5 == 3:
         avf(4)
      elif l % 5 == 4:
         av()
      elif l % 5 == 0:
         avf(3)
      # Dessiner une ligne de motifs
      pqtg(); bc()
```

```
        for j in range(nombreDeMotifsParLigne - 1):
            roueDentee()
            translater(5, 0)
        roueDentee()
    lg(tailleGrille); pavage(); st()
```

□

Corrigé 31 *Nous donnons un programme, le premier se trouvant dans le dossier* `livre/16_expressions_booleennes` *:*

```
from LibRobot import *
# Dessiner une mosaïque de l'Alhambra
tailleDeGrille = 8  # doit être pair
nombreDeLignes = 4
nombreDeColonnes = 8
def demiMotif():
    phtd(); av(); phtd(); av(); pqtg(); av()
    lg(tailleDeGrille // 2)
    p3htd(); av(); pqtg(); av()
    lg(tailleDeGrille)
    p3htd(); av(); pqtg(); av(); phtd(); av()
    p3htd(); avf(2)
    lg(tailleDeGrille // 2)
    phtg(); av(); pqtd(); av()
    lg(tailleDeGrille); phtg(); avf(2)
    # Peindre
    avf(-1); phtg()
    if tsw() or tne():
        cc(jaune)
    else:
        cc(indigo)
    peindre()
    cc(noir); phtd(); av()
    # Remettre le robot dans l'orientation initiale
    pqtd()
def motif():
    demiMotif()
    lc(); pqtd(); avf(5); pqtd(); av(); bc()
    demiMotif()
def motifDouble():
    motif()
    # Motif suivant
    lc(); phtg(); avf(2); p3htd(); bc()
```

```
    motif()
lg(tailleDeGrille); eh(); ag()
translater(1, -3)
for j in range(1, nombreDeLignes + 1):
    for i in range(nombreDeColonnes // 2):
        motifDouble()
        # Motif double suivant
        lc(); phtd(); avf(2); p3htg(); bc()
    # Ligne suivante
    if j % 2 == 1:
        lc(); pdt(); avf(6); bc()
    else:
        lc(); avf(4); pdt(); bc()
st()
```

□

Addenda

En Python les priorités des opérateurs entiers et booléens sont définies comme suit :

Priorité des opérateurs entiers et booléens :		
`f(x)` (appel)	appels de fonctions	priorité la plus forte
$-$ (unaire)	opposé	
$*$, //, %	opérateurs multiplicatifs	
$+$, $-$ (binaires)	opérateurs additifs	
<, <=, ==, !=, >=, >	opérateurs de relation	
not	négation	
and	conjonction	
or	disjonction	priorité la plus faible

Par exemple l'expression booléenne '**True and False or False**' s'évalue comme '**(True and False) or False**' et est donc égale à **False**.

Ces priorités sont proches des conventions mathématiques usuelles. Par exemple '`0 <= i` **and** `i < 100`' s'évalue comme '`(0 <= i)` **and** `(i < 100)`', ce qui est bien ce que l'on veut dire. Il n'en était pas de même dans des langages plus anciens comme PASCAL, où cette expression était incorrecte et où il fallait écrire toutes les parenthèses. Python permet même d'écrire '`0 <= i < 100`' comme en mathématiques.

Comme **not** est moins prioritaire que les opérateurs de relation, '**not** `i % 2 == 1`' s'évalue comme '**not** `(i % 2 == 1)`', ce qui est bien ce que l'on veut dire. Il n'en est pas de même dans d'autres langages comme PASCAL ou OCaml, où **not** s'applique à ce qui le suit immédiatement, à la manière d'un appel de fonction, et où il faut donc écrire les parenthèses. Dans le doute, on peut toujours ajouter des parenthèses.

Enfin, Python évalue les conjonctions et les disjonctions de gauche à droite et s'arrête dès que le résultat est connu : dans E_1 **and** E_2, si E_1 vaut **False** alors E_2 n'est pas calculée puisque la conjonction vaut forcément **False** ; de même dans E_1 **or** E_2, si E_1 vaut **True** alors E_2 n'est pas calculée.

Le terme "booléen" ou "booléien" (`bool` en Python) vient de Boole [a], créateur de la logique mathématique moderne.

[a] George Boole, mathématicien britannique, 1815–1864.

17

Boucles “while”

17.1 Le robot est-il au bord du cadre ?

Nous avons vu dans le chapitre précédent que les fonctions booléennes `tn()`, `tne()`, `te()`, `tse()`, `ts()`, `tsw()`, `tw()`, `tnw()` peuvent être utilisées dans les instructions de test “if” pour connaître l'orientation du $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$. Une autre fonction booléenne, `tb()`, permet de tester si le $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ est au bord du cadre. Plus précisément `tb()` vaut **True** (vrai) si l'exécution supposée de la commande `av()` fait sortir le $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ de son cadre de déplacement. L'expression booléenne `tb()` vaut **False** (faux) si l'exécution supposée de la commande `av()` ne déplace le $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ qu'à l'intérieur de son cadre de déplacement. Par exemple, on peut définir une commande `avSansSortir()` qui ordonne au $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ d'exécuter un `av()` si cela ne le fait pas sortir du cadre :

```
# Avancer si cela ne fait pas sortir le robot du cadre
def avSansSortir():
    if not tb():
        av()
```

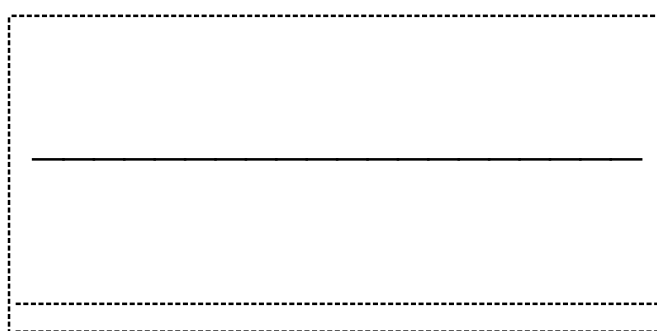
Les parenthèses vides de **not** `tb()` sont indispensables. Si on les oublie en écrivant **not** `tb`, Python ne signale aucune erreur, mais le programme ne fait pas ce que l'on veut : `tb` sans parenthèses désigne la fonction elle-même, et non la valeur calculée par son appel. Or Python considère qu'une fonction, comme toute valeur qui n'est pas nulle ou vide, équivaut à **True**. L'expression **not** `tb` vaut donc toujours **False** et le $\mathcal{R}^{o}\underset{\frown}{b}^{o}t$ n'avance jamais, qu'il soit au bord du cadre ou non. C'est une erreur difficile à trouver puisque rien ne la signale !

17.2 Boucle “while”

Une boucle “while” permet de répéter l'exécution d'une séquence d'instructions zéro ou plusieurs fois en testant à chaque itération s'il faut s'arrêter ou continuer. Le mot anglais “while” qui se prononce très approximativement ‘oua···i···le’ signifie ‘tant que’. La boucle “while” permet de répéter l'exécution d'une séquence d'instructions tant que la valeur d'une expression booléenne est **True** (vrai). Par exemple, le programme suivant permet de tracer un trait horizontal au milieu de l'écran en orientant le $\mathcal{R}\!\mathcal{o}\!\mathcal{b}\!^{o}t$ vers l'est et en répétant des `av()` tant que le $\mathcal{R}\!\mathcal{o}\!\mathcal{b}\!^{o}t$ ne sort pas du cadre :

```
from LibRobot import *
# Tracer un trait horizontal au milieu de l'écran
ag(); pe()
while not tb():
   av()
st()
```

Le résultat de l'exécution de ce programme est présenté en réduction dans la figure ci-dessous, les cadres de déplacement du $\mathcal{R}\!\mathcal{o}\!\mathcal{b}\!^{o}t$ et d'affichage des messages d'erreurs étant en pointillé :



Une boucle “while” a toujours la forme suivante :

Boucle“while” :

while ␣ Expression booléenne ␣ **:**
... Instructions répétées (indentées) ...

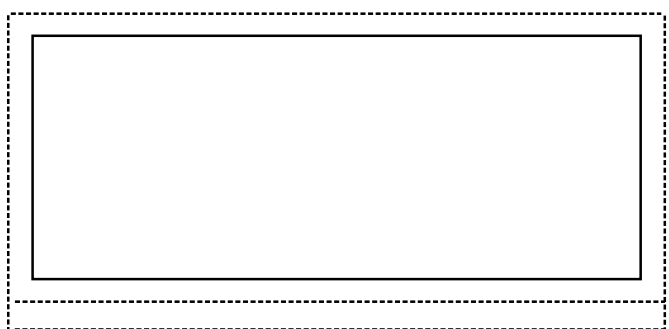
La séquence d'instructions qui est répétée tant que la valeur de l'Expression booléenne vaut **True** (vrai) s'appelle le *corps de la boucle*. Comme pour la

boucle "for", la ligne du **while** se termine par deux-points : et les instructions du corps de la boucle sont indentées (décalées vers la droite), la boucle se terminant à la première instruction qui n'est plus indentée. Si l'Expression booléenne vaut **False** quand on commence l'exécution de la boucle "while" alors cette exécution se termine immédiatement sans que le corps de la boucle ne soit exécuté. Si l'Expression booléenne vaut toujours **True** à chaque itération alors l'exécution de la boucle "while" ne se termine jamais ! On peut tout de même arrêter l'ordinateur en coupant le courant. Des méthodes plus recommandables sont expliquées ci-après.

Une erreur grave est d'écrire un corps de boucle qui ne change pas la valeur de l'Expression booléenne. Par exemple si l'on écrit **while not** `tb()` : `pqtd()`, le $\mathcal{R}^{o}\underline{b}^{o}t$ tourne sur place sans jamais s'approcher du bord, et l'ordinateur boucle éternellement.

On utilise de préférence les boucles "for" (dont l'exécution se termine toujours) en réservant l'usage des boucles "while" au cas où le nombre d'itérations n'est pas calculable par une expression entière avant de commencer à exécuter le corps de la boucle.

Exemple 21 (Cadre) Le programme ci-dessous permet de tracer un cadre autour d'un dessin :



```
from LibRobot import *
# Tracer un cadre autour du dessin
# Rejoindre le bord en avançant tout droit
def auBord():
    while not tb():
        av()
def cadre():
    # Rejoindre le bord supérieur sans rien tracer
```

```
    lc(); auBord()
    # Tracer la moitié droite du bord supérieur
    pqtd(); bc(); auBord()
    # Tracer le bord droit
    pqtd(); auBord()
    # Tracer le bord inférieur
    pqtd(); auBord()
    # Tracer le bord gauche
    pqtd(); auBord()
    # Tracer le bord supérieur
    pqtd(); auBord()
    # Placer le robot au centre, orienté au nord
    ce(); pn()
cadre(); st()
```

□

Tous les programmes que nous avions écrits jusqu'alors se terminaient. Ceci veut dire que toute exécution du programme prend un temps fini, qui semble parfois long, mais on peut être sûr qu'en attendant suffisamment longtemps, on pourra toujours observer un résultat. Si par exemple on a levé le crayon du $\mathcal{R}\!\mathcal{o}\!\mathcal{b}^{o}t$ et oublié de le baisser par la suite, le résultat est une feuille blanche mais l'exécution du programme se termine tout de même.

Avec la boucle "while", on peut écrire des programmes dont l'exécution ne se termine pas. C'est le cas pour le programme suivant :

```
from LibRobot import *
# Faire tourner le robot sans fin. Le corps de la boucle
# comprend une commande du robot : on peut arrêter son
# exécution en tapant Échap (esc), § ou # dans la fenêtre
# du robot.
lc()
while True:
   vd()
st()
```

Le $\mathcal{R}\!\mathcal{o}\!\mathcal{b}^{o}t$ tourne au centre de l'écran, tant que l'ordinateur marche et que l'on n'intervient pas. On peut l'arrêter en tapant, dans la fenêtre du $\mathcal{R}\!\mathcal{o}\!\mathcal{b}^{o}t$, sur l'une des touches esc, § ou #. L'exécution du programme prend fin car avant d'exécuter une commande, le $\mathcal{R}\!\mathcal{o}\!\mathcal{b}^{o}t$ regarde si l'utilisateur n'a pas tapé sur l'une de ces touches pour lui demander de s'arrêter.

Par contre si l'on exécute le programme Python suivant :

```
from LibRobot import *
```

```
# Boucler sans fin. Le corps de la boucle ne comprend
# aucune commande du robot : on ne peut donc pas arrêter
# son exécution avec Échap (esc), § ou #. Il faut taper
# Ctrl+C dans la fenêtre de terminal.
lc()
while True:
    pass
```

l'exécution boucle sans s'arrêter même si l'on tape sur les touches d'arrêt esc , § ou # puisque le *Robot* ne reçoit aucune commande pendant que l'ordinateur exécute cette boucle, dont le corps est réduit à **pass**, une instruction qui ne fait rien (Python exige au moins une instruction dans le corps d'une boucle et **pass**, ‘passer’ en anglais, sert à cela).

Pour arrêter l'exécution de ce programme, il faut revenir à la fenêtre de terminal dans laquelle on l'a lancé, par exemple avec la commande :

```
./robot.sh livre/17_while/Boucle2.py
```

maintenir la touche contrôle Ctrl enfoncée et taper sur la touche C . Le système d'exploitation de l'ordinateur interrompt alors le programme, quoi qu'il soit en train de faire (et la fenêtre du *Robot* se ferme).

Si le programme a été lancé en faisant glisser son icône sur l'application `LeRobotPython.app`, une fenêtre de terminal s'ouvre, que l'on peut sélectionner avec la souris pour taper Ctrl+C.

Exercice 32 *Écrire des programmes Python utilisant des boucles "while" pour réaliser les dessins suivants :*

- *la frise de la figure 17.1 ci-dessous ;*
- *la grille des points d'arrêt du Robot de la figure 17.2 (page 201), sans utiliser bien entendu la commande* `dg()` *;*
- *le quadrillage de la figure 17.2 (page 201) ;*
- *le grillage de la figure 17.2 (page 201) obtenu en superposant deux quadrillages dont un est tourné d'un huitième de tour.*

Dans tous les cas le dessin doit être le plus grand possible et s'inscrire à l'intérieur du cadre fixé pour les déplacements du Robot sur l'écran de l'ordinateur.
□

Exercice 33 *Définir les commandes* `ag()`, `ad()`, `eh()` *et* `eb()` *par des procédures comportant des boucles "while" (on utilisera la fonction booléenne* `valBc()` *qui vaut* **True** *si et seulement si le crayon du Robot est baissé).* □

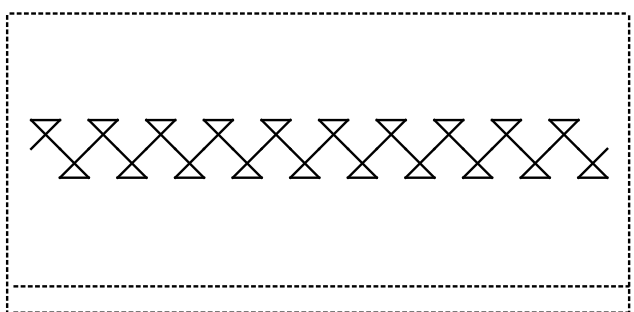


FIG. 17.1 – *Frise de triangles (dans un écran réduit)*

Corrigé 32 *Nous donnons un programme, les autres se trouvant dans le dossier* `livre/17_while` *:*

```
from LibRobot import *

# Dessiner une frise de triangles d'un bord à l'autre

# Rejoindre le bord en avançant tout droit
def auBord():
    lc()
```

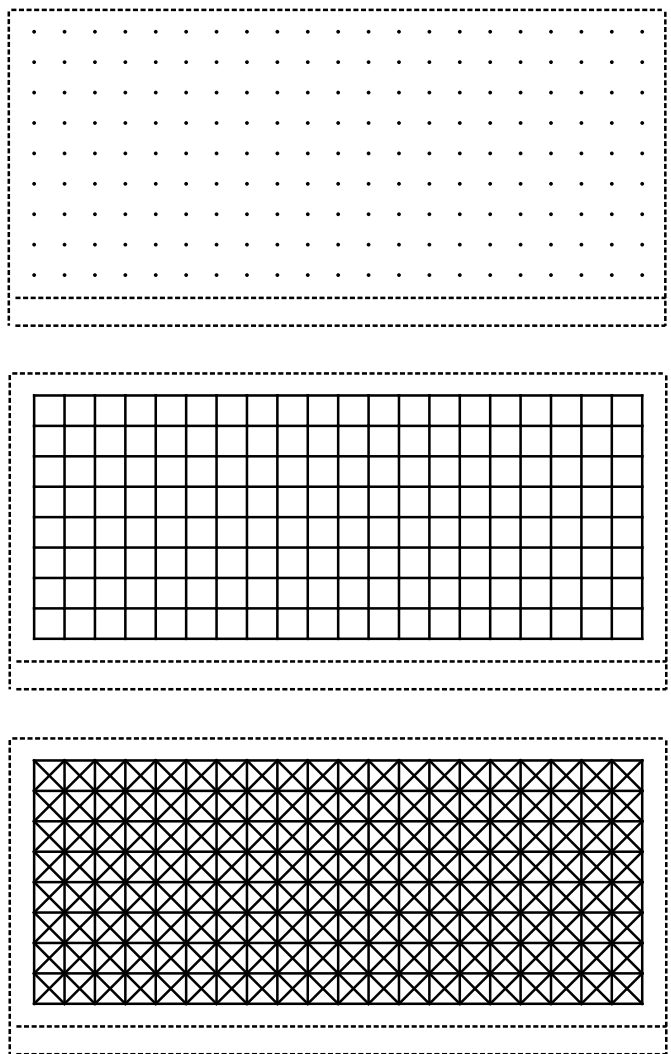


FIG. 17.2 – *Points d'arrêt, quadrillage et grillage sur un écran réduit*

```
    while not tb():
        av()
    bc()
def motif():
    if not tb():
        av(); p3htg(); av(); p3htg(); av()
        if not tb():
            av(); p3htd(); av(); p3htd(); av()
def frise():
    pne()
    while not tb():
        motif()
pw(); auBord(); frise(); st()
```

□

Corrigé 33 *Nous donnons le corrigé pour la commande* `ag`*, les autres se trouvant dans le programme* `AgAdEhEb` *du dossier* `livre/17_while` *:*

```
# Définition de la commande ag
def auBordOuest():
    lc(); pw()
    while not tb():
        av()
def alOuest():
    if tn():
        auBordOuest(); pn()
    elif tne():
        auBordOuest(); pne()
    elif te():
        auBordOuest(); pe()
    elif tse():
        auBordOuest(); pse()
    elif ts():
        auBordOuest(); ps()
    elif tsw():
        auBordOuest(); psw()
    elif tw():
        auBordOuest()
    else:
        auBordOuest(); pnw()
def ag():
    if valBc():
        alOuest(); bc()
```

```
else:
    alOuest()
```

□

Addenda

Dans d'autres langages de programmation, comme PASCAL, on trouve une autre forme de répétition non bornée, la boucle “repeat” (qui signifie répéter en anglais). Elle a la forme suivante :

repeat
... Instructions répétées (séparées par des ‘;’) ...
until ␣ Expression booléenne

Le corps de la boucle est répété (`repeat`) jusqu'à (`until`) ce que la valeur de l'Expression booléenne soit égale à `true` (vrai). Il est donc toujours exécuté au moins une fois.

La boucle “repeat” n'existe pas en Python, mais on peut toujours l'exprimer à l'aide d'une boucle “while”. Une boucle “repeat” :

repeat
  Corps de la boucle
until Expression booléenne

peut s'écrire en Python avec une boucle “while” comme suit :

Corps de la boucle
while not Expression booléenne **:**
  Corps de la boucle

mais il faut alors écrire deux fois le corps de la boucle. On préfère souvent écrire en Python une boucle “while” dont le test vaut toujours **True** et dont on sort avec l'instruction **break** (‘casser’ en anglais), qui termine immédiatement l'exécution de la boucle :

while True:
  Corps de la boucle
  if Expression booléenne **:**
    break

De la même façon, une boucle “while” :

while Expression booléenne **:**
  Corps de la boucle

pourrait se réécrire avec une boucle “repeat” comme suit :

if Expression booléenne **then**
  repeat
    Corps de la boucle
  until not (Expression booléenne)

L'usage de l'une ou l'autre forme d'itération ne dépend que du fait que le corps de la boucle peut ne jamais être exécuté (boucle “while”) ou doit toujours l'être au moins une fois (boucle “repeat”). Les concepteurs de Python ont jugé que la boucle “while” suffisait.

18

Codage

La mémoire de l'ordinateur ne contient que des zéros et des uns. Il faut donc pouvoir représenter toutes les informations que l'on veut mémoriser par des suites de zéros et de uns. Cette traduction de l'information en binaire s'appelle le *codage binaire*. Les chiffres 0 et 1 utilisés pour représenter l'information s'appellent des *bits*. Nous allons commencer par étudier d'autres formes de codages plus anciennes.

18.1 Code Morse

Le télégraphe électrique, ancêtre de notre téléphone, fut inventé en 1837 par l'américain Samuel Morse [3]. Le télégraphiste émetteur dispose d'un manipulateur qui est un simple levier à ressort permettant d'envoyer des signaux courts (appelés points) et des signaux longs (appelés traits) séparés par des silences ou blancs (correspondant à l'absence de signal). Le signal est transmis le long d'un fil électrique, le retour se faisant par la terre. Chaque mouvement du manipulateur ferme le circuit et provoque un mouvement bref ou long du récepteur qui reproduit les signaux sur une bande de papier. Pour envoyer un message, on commence par l'abréger en *style télégraphique* puis on traduit chaque signe (lettre, chiffre, signe de ponctuation, ...) en une suite de points et de traits séparés par des blancs selon le *code Morse* de la figure 18.1 (page 206). On laisse sept blancs à la fin de chaque mot. Divers autres codes sont utilisés pour indiquer le début et la fin du message, signaler les erreurs, etc. Le télégraphe de Morse permettait de transmettre environ 25 mots à la minute.

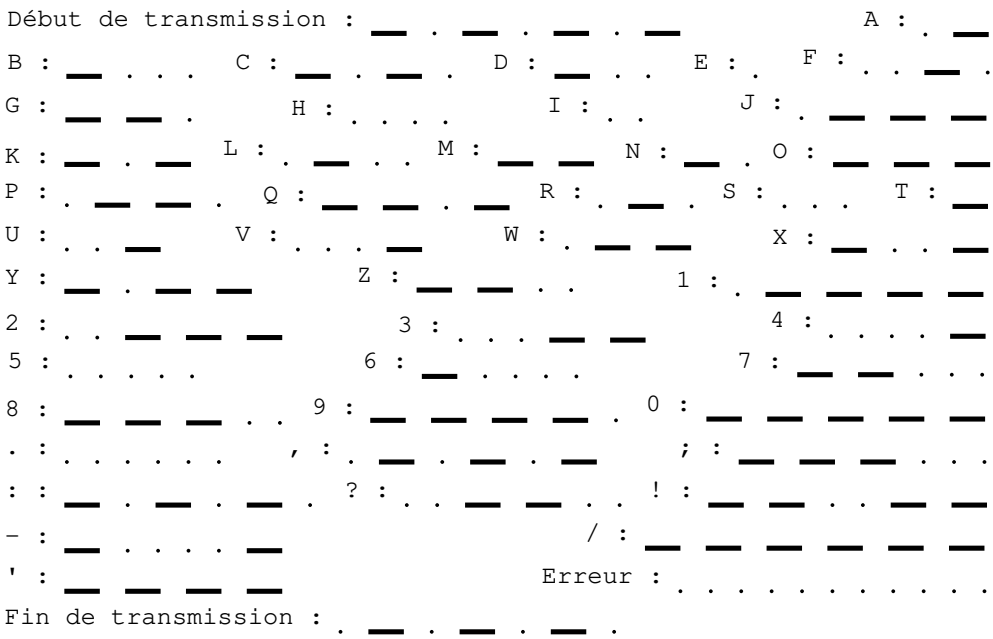


FIG. 18.1 – *Code Morse*

18.2 Numération

Notre codage des entiers naturels (positifs ou nuls) est une numération positionnelle et décimale. C'est une numération décimale car nous utilisons les dix chiffres 0, 1, ..., 9 d'origine indo-arabe. C'est une numération positionnelle car chaque chiffre dans un nombre représente une valeur qui dépend de sa position dans le nombre. Par exemple dans 1789, le 1 représente 1000, le 7 représente 700, le 8 représente 80 et le 9 représente 9. On a donc $1789 = 1000+700+80+9 = (1\times10^3)+(7\times10^2)+(8\times10^1)+(9\times10^0)$. Pour les chiffres après la virgule on considère les inverses. Par exemple $0.147 = (1\times10^{-1})+(4\times10^{-2})+(7\times10^{-3}) = \frac{1}{10}+\frac{4}{100}+\frac{7}{1000} = \frac{100}{1000}+\frac{40}{1000}+\frac{7}{1000} = \frac{147}{1000}$. Les mathématiciens anciens et modernes ont inventé bien d'autres façons de coder les nombres. La figure 18.2 (page 208) montre différentes façons de coder le nombre 123 456 789, certaines remontant aux civilisations antiques [16]. La supériorité du codage positionnel et décimal des entiers naturels ne tient pas au choix de la base dix mais à la facilité avec laquelle on peut faire les opérations arithmétiques usuelles, par exemple en utilisant un boulier

(longtemps en usage et maintenant disparu des écoles communales)

18.3 Codage binaire

Toutes les informations rangées dans la mémoire de l'ordinateur ou sur disque doivent être représentées par des suites de zéros et de uns. Sur les micro-ordinateurs, tels qu'ils sont conçus actuellement, la longueur de ces suites de bits doit être multiple de 8. Selon leur longueur on leur donne des noms spéciaux comme *octet* pour une suite de 8 bits, *demi-mot* pour une suite de 16 bits, *mot* pour 32 bits et *double-mot* pour 64.

Pour montrer que toute information peut être représentée par une suite de bits (à condition de connaître le code) nous allons prendre l'exemple des entiers puis celui des caractères et des textes tapés au clavier.

18.3.1 Entiers naturels

Pour représenter les entiers positifs par une suite de bits, il suffit de les coder en base 2. Par exemple :

$$\begin{aligned} 147 &= 128 + 16 + 2 + 1 \\ &= (1 \times 2^7) + (0 \times 2^6) + (0 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) \\ &\quad +(1 \times 2^1) + (1 \times 2^0) \end{aligned}$$

s'écrit :

$$10010011$$

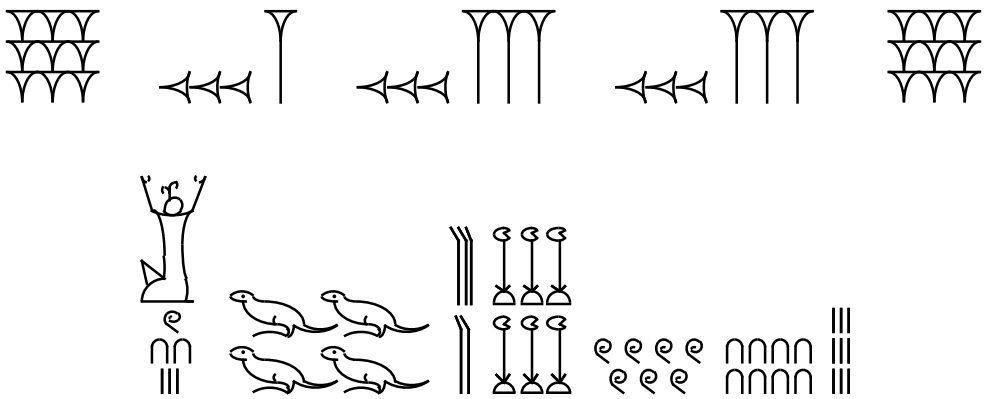
en base deux. De manière plus générale, le nombre qui s'écrit :

$$a_n a_{n-1} \ldots a_1 a_0$$

en base deux (où $a_n \ldots a_0$ valent 0 ou 1) vaut :

$$(a_n \times 2^n) + (a_{n-1} \times 2^{n-1}) + \ldots + (a_1 \times 2^1) + (a_0 \times 2^0)$$

(où 2^n est égal à 2 multiplié n fois par lui-même si $n > 0$ et $2^0 = 1$). Avec cette convention on peut donc coder sur un octet les entiers naturels compris entre 0 (00000000) et 255 (11111111).



𓍢𓍢𓍢𓎆𓎆𓎆𓎆𓏤𓏤𓏥𓐝𓐝𓐝𓆼𓆼𓆼

⌈MCCXXXIV⌉ L̅V̅I DCCLXXXIX

75BCD15

123456789

726746425

2212102202021220 0

11101011011110011010001010 1

FIG. 18.2 – *Codages de 123 456 789 en numérations babylonienne, égyptienne*, grecque, romaine, hexadécimale, décimale, octale, ternaire et binaire

18.3.2 Caractères (code ASCII)

Les programmes Python que nous écrivons sont des suites de caractères. À chaque fois que nous tapons sur une touche du clavier le code correspondant du caractère est entré dans la mémoire de l'ordinateur. Les caractères correspondant aux chiffres, aux lettres minuscules ou majuscules et aux caractères usuels comme +, ∗, (,), . . .s'appellent les caractères "imprimables". Certains autres caractères qui ne peuvent pas s'écrire s'appellent les caractères "spéciaux." C'est le cas par exemple des "caractère de contrôle" et "caractère d'échappement" que l'on obtient respectivement en tapant les touches Ctrl et Esc. Ces caractères (imprimables ou spéciaux) sont souvent représentés sur un octet (8 bits) selon un code international qui s'appelle le code ASCII. Les caractères imprimables dont le code ASCII est compris entre 32 et 126 sont donnés dans la table 18.1 ci-dessous. Par exemple, le code ASCII 65 du caractère `'A'` s'obtient dans cette table en ajoutant le numéro 64 de la ligne où se trouve le caractère `'A'` au numéro 1 de la colonne de ce caractère.

+	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
32	␣	!	"	#	$	%	&	’	(	)	*	+	,	-	.	/
48	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
64	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
96	‘	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
112	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	

TAB. 18.1 – *Caractères de codes* ASCII *compris entre 32 et 126*

Quand nous disons que le caractère `'A'` a pour code ASCII 65, il faut comprendre que ce caractère est représenté dans la mémoire de l'ordinateur par l'octet 01000001 qui représente également le nombre 65 écrit en base deux. On remarquera que le caractère zéro a pour code 48. L'entier 0 (codé en machine sur un octet par 00000000) a donc un code différent du caractère zéro `'0'` (codé en machine par 00110000). La table 18.2 (page 210) montre les codes ASCII de quelques caractères spéciaux.

Les lettres accentuées comme `'é'` ou `'ç'` ne figurent pas dans le code ASCII, conçu pour la langue anglaise. Elles sont aujourd'hui codées selon le code international UNICODE qui permet de représenter les caractères de toutes les écritures du monde. Dans le format UTF-8 employé par Python pour le texte

Code ASCII	8	13	27	127
Touche du clavier	←	↩	Esc	Del

TAB. 18.2 – *Code* ASCII *de quelques caractères spéciaux*

des programmes, les caractères ASCII sont codés sur un octet comme ci-dessus et les autres caractères sur deux, trois ou quatre octets. Par exemple la lettre `'é'` est codée par les deux octets 11000011 10101001.

18.3.3 Texte

Un texte (comme par exemple un programme Python) est une suite de caractères. Le code d'un texte en machine est la suite des codes des caractères qui le composent. Par exemple le texte `"Python"` sera représenté en machine par la suite des codes ASCII des caractères `'P'`, `'y'`, `'t'`, `'h'`, `'o'`, `'n'` soit, d'après la table 18.1 (page 209), la suite 80 (01010000), 121 (01111001), 116 (01110100), 104 (01101000), 111 (01101111), 110 (01101110). Le code du texte `"Python"` est donc 01010000 01111001 01110100 01101000 01101111 01101110.

18.3.4 Chaînes de caractères

Un programme permet de faire manipuler par l'ordinateur des booléens, des entiers mais également des textes, tous codés en binaire. Pour ne pas confondre le texte du programme et les textes manipulés par le programme, nous appelerons ces derniers des "chaînes de caractères".

Usage et notation en Python :

En Python on peut manipuler des textes aussi longs que l'on veut que l'on appelle des "chaînes de caractères". Elles sont de type `str` (abréviation de `string`, prononcer stri-ngue, 'ficelle' en anglais). Une chaîne de caractères s'écrit entre guillemets droits, comme par exemple :

```
"Ceci est une chaîne de caractères"
```

S'il est facile de distinguer un identificateur ou un entier dans le texte d'un programme, il est plus difficile de reconnaître un texte dans ce texte ! L'usage des guillemets permet donc de délimiter la partie du texte constituant la

chaîne de caractères de celle du reste du programme. Mais ceci pose problème si la chaîne de caractères contient un guillemet. C'est pourquoi on convient de faire précéder d'une barre oblique inversée `\` les guillemets d'une chaîne de caractères, comme dans :

```
"Le langage \"Python\"".
```

Une chaîne de caractères réduite à un seul guillemet s'écrit donc `"\""` et une chaîne réduite à une seule barre oblique inversée s'écrit `"\\"`. En Python, une chaîne de caractères peut aussi s'écrire entre apostrophes, comme `'Le langage "Python"'`, ce qui évite d'avoir à placer une barre oblique inversée devant les guillemets. Il n'y a pas en Python de type particulier pour les caractères (de l'anglais character) : le caractère `'A'` est simplement la chaîne de caractères `"A"`, de longueur 1.

Concaténation de chaînes de caractères :

Si c_1 et c_2 sont des chaînes de caractères alors c_1 `+` c_2 est la chaîne de caractères obtenue en concaténant (c'est-à-dire en mettant bout à bout) les chaînes de caractères c_1 et c_2. On peut concaténer plusieurs chaînes c_1 `+` c_2 `+` $\ldots$ `+` c_n (le signe `+`, qui additionne deux nombres, met donc bout à bout deux chaînes de caractères). Par exemple :

```
"Le " + "langage " + "\"" + "Python" + "\""
```

est égal à la chaîne de caractères qui s'écrit également :

```
"Le langage \"Python\""
```

Écriture sur le dessin et sous le dessin du robot :

Que son crayon soit levé ou baissé, le $\mathcal{R}^{o}_{\smile}{}^{o}t$ peut écrire des chaînes de caractères, sans se déplacer et sans changer son orientation. Il peut écrire dans la fenêtre de dessin ou dans le cadre d'affichage des messages situé sous la fenêtre de dessin.

La commande `ecrire(`c`)` indique au $\mathcal{R}^{o}_{\smile}{}^{o}t$ qu'il faut écrire la chaîne de caractères c à droite du point où il se trouve dans sa fenêtre de dessin. La commande `message(`c`)` indique au $\mathcal{R}^{o}_{\smile}{}^{o}t$ qu'il faut écrire la chaîne de caractères c dans le cadre prévu en dessous de la fenêtre de déplacement. Ceci a pour effet d'effacer le message précédent affiché dans ce cadre. Pour laisser à l'utilisateur le temps de lire le message, on peut, si nécessaire, utiliser la procédure `marquerUnePause()` qui attend pour continuer que l'utilisateur

tape sur une touche ou clique sur le bouton de la souris. La commande `effacerMessage()` efface ce message. Les commandes `policeTexte`, `styleTexte` et `tailleTexte` de la bibliothèque `LibRobot` permettent de choisir une police, un style ou une taille de caractères autre que le choix standard (sur certains ordinateurs, elles restent sans effet).

Si n est un entier alors `str(`n`)` (de l'anglais `string`, 'chaîne') est la chaîne de caractères qui dénote cet entier dans la notation décimale usuelle (la bibliothèque `LibRobot` propose aussi `entierEnChaine(`n`)` qui fait la même chose). Si b est un booléen alors `booleenEnChaine(`b`)` est `"Vrai"` si b est égal à **True** et `"Faux"` si b est égal à **False** (alors que `str(`b`)` donne `"True"` ou `"False"`). Par exemple si `x` est un paramètre entier de procédure égal à 17 alors la commande :

```
ecrire("La valeur de x est " + str(x) + ".")
```

écrit la concaténation des chaînes de caractères `"La valeur de x est "`, `"17"` et `"."`, c'est-à-dire :

```
La valeur de x est 17.
```

à la droite du $\mathcal{R}\text{o}\underline{\text{b}}\text{o}t$. On remarquera que la conversion `str(x)` est indispensable : sans elle, Python refuserait d'ajouter un entier à une chaîne de caractères et arrêterait le programme avec le message `TypeError : can only concatenate str (not "int") to str`. Si `b` est un paramètre booléen de procédure égal à **True** alors la commande :

```
message("b = " + booleenEnChaine(b))
```

écrit la concaténation des chaînes de caractères `"b = "` et `"Vrai"`, c'est-à-dire :

```
b = Vrai
```

dans le cadre des messages.

18.4 Musique

Ce paragraphe est destiné aux musiciens qui connaissent le solfège, c'est-à-dire la manière de coder la musique. Le $\mathcal{R}\text{o}\underline{\text{b}}\text{o}t$ peut jouer de la musique, de manière il est vrai assez limitée, puisqu'il ne peut jouer du piano qu'à un seul doigt. Pour coder la musique pour le $\mathcal{R}\text{o}\underline{\text{b}}\text{o}t$ nous disposons des procédures avec paramètres. Il nous suffit donc d'établir la correspondance avec la théorie classique de la musique [13] pour définir comment écrire de la musique pour le $\mathcal{R}\text{o}\underline{\text{b}}\text{o}t$.

18.4.1 Figures de notes

La procédure `note(h, d)` permet de faire entendre sur le haut-parleur de l'ordinateur une note de hauteur `h` et de figure `d`. La hauteur de la note est codée par un entier `h` comme indiqué à la figure 18.3 (page 214). La figure de la note indique sa durée qui est codée par une constante `d` qui peut être `ronde`, `ronde_pointee`, `blanche`, `blanche_pointee`, `blanche_en_triolet`, `noire`, `noire_pointee`, `noire_en_triolet`, `croche`, `croche_pointee`, `croche_en_triolet`, `double_croche`, `double_croche_pointee` ou `triple_croche`. Ces constantes, déclarées dans la bibliothèque `LibRobot`, sont des nombres écrits avec un point (comme nous l'expliquerons au chapitre 19) qui donnent la durée de la note en nombre de temps : une `noire` vaut `1.0`, une `croche` vaut `0.5`, une `ronde` vaut `4.0`, etc. La durée relative d'une `ronde_pointee` est une fois et demie celle d'une `ronde`. La durée relative d'une `ronde` est deux fois celle d'une `blanche`, trois fois celle d'une `blanche_en_triolet`, ... On peut définir d'autres figures de notes, par exemple `quadruple_croche = 0.0625`.

18.4.2 Mouvement

Le mouvement, c'est-à-dire le degré de vitesse ou de lenteur avec lequel doit être exécuté le morceau de musique détermine la durée absolue des différentes figures de notes. Ce mouvement est défini par la procédure `metronome(`n`)` qui indique le nombre n d'oscillations du métronome par minute, sachant qu'une noire est exécutée à chaque battement. Par exemple `metronome(60)` indique qu'il faut jouer 60 noires à la minute. Les mouvements sont généralement définis par des termes italiens (adagio, allegro, prestissimo,...) que l'on place au début d'un morceau et au dessus de la portée. Ils sont définis dans la bibliothèque `LibRobot` par les déclarations de constantes suivantes :

```
largo = 50
lento = 56
larghetto = 60
adagio = 70
andante = 85
andantino = 95
moderato = 110
allegretto = 115
allegro = 130
vivace = 150
```

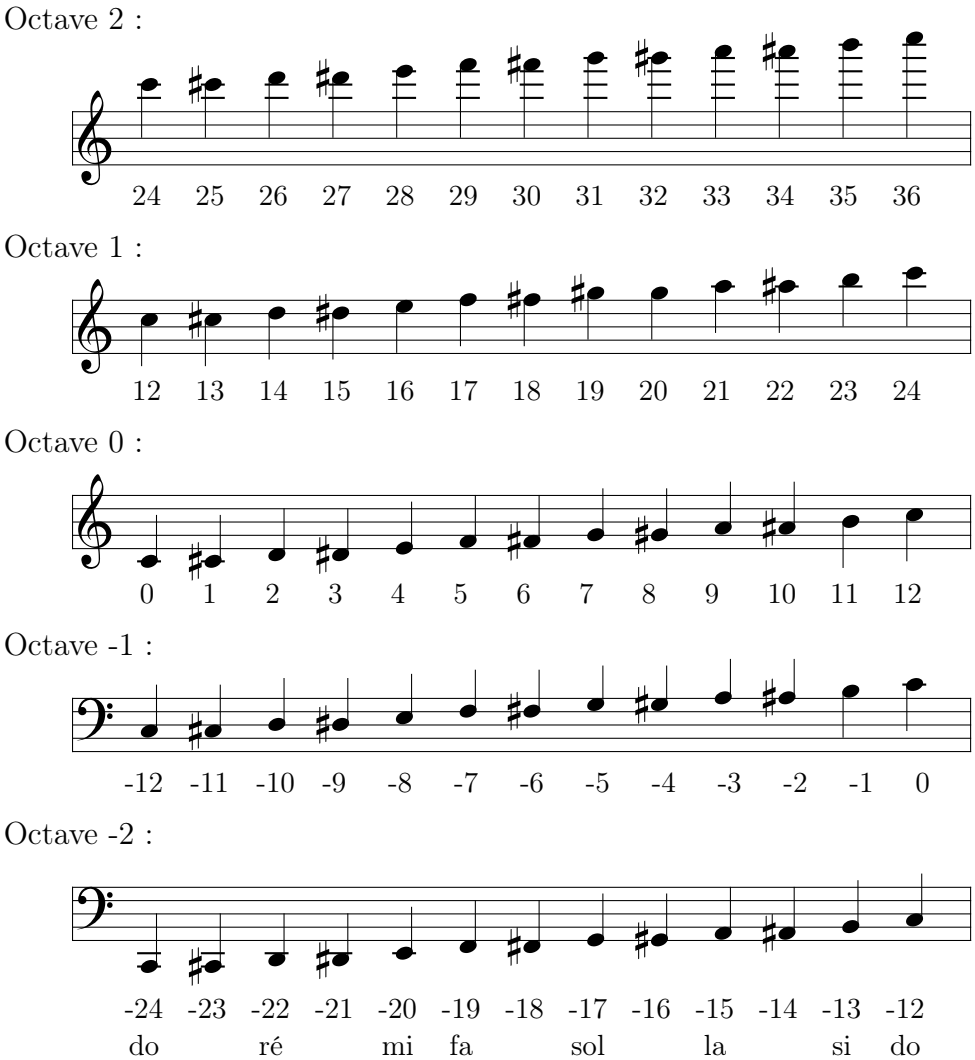



FIG. 18.3 – *Codage de la hauteur des notes de musique par des entiers*

```
presto = 175
prestissimo = 200
```

18.4.3 Notes

Le numérotage des notes de la figure 18.3 (page 214) n'est pas facile à mémoriser. C'est pourquoi nous allons coder les notes par une expression entière portant sur des constantes dont il est facile de se souvenir.

Une note est définie par son nom n (à savoir do, ré, mi, fa, sol, la, si), son éventuelle altération a (c'est-à-dire un signe qui modifie le son de la note à laquelle il est affecté qui peut être le bémol ♭ (qui abaisse le son de la note d'un demi-ton), le dièse ♯ (qui élève le son de la note d'un demi-ton) ou le bécarre ♮ (qui détruit l'effet du dièse ou du bémol)) et son octave o (qui indique à quelle série de notes portant le même nom appartient cette note). Nous codons les octaves du grave à l'aigu par des entiers -3, -2, ..., 3, 4 , l'octave 0 commençant par le do au centre du clavier du piano. Les constantes suivantes sont déclarées dans la bibliothèque `LibRobot` [1] :

```
# Notes
ut = 0
d0 = 0
re = 2
mi = 4
fa = 5
sol = 7
la = 9
si = 11
# Altérations
bemol = -1
becarre = 0
diese = 1
# Octaves
octave = 12
```

Le numéro de la note tel qu'il est défini à la figure 18.3 (page 214) est donné par la valeur de l'expression entière $o * octave + n + a$. Si la note n'est pas altérée, son code est simplement $o*octave+n$. Par exemple le do du milieu du

1. On remarquera que la note do n'a pas de constante `do`. Dans d'autres langages, comme OCaml, **do** est un mot réservé utilisé dans les boucles "for" et "while", qui ne peut pas servir de nom de constante. Ce n'est pas le cas en Python, mais nous avons gardé les mêmes noms pour la note do : `ut`, `d0` (lettre d suivie du chiffre zéro) ou `do0`, au choix.

clavier du piano correspond à $o = 0$, $n =$`d0` soit $0*$`octave`$+0 = 0$. Le ré dièse, une octave plus haut a pour code $1*$`octave`$+$`re`$+$`diese`$= 1 * 12 + 2 + 1 = 15$. Le ré bémol voisin a pour code $1*$`octave`$+$`re`$+$`bemol`$= 1 * 12 + 2 + (-1) = 13$. Quand la hauteur d'une note est donnée par une expression, on écrit simplement cette expression comme premier paramètre de `note`, comme dans `note(re + octave + diese, noire)`.

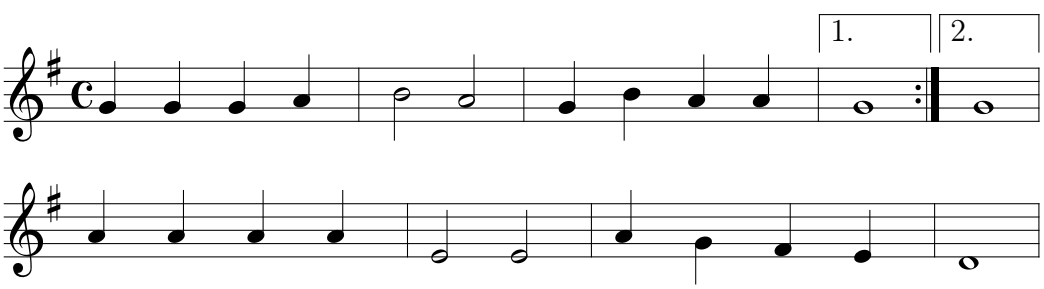
18.4.4 Silences

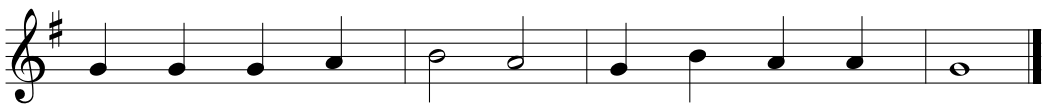
La procédure `silence(`s`)` permet d'interrompre le son pour une durée relative fixée par la figure de silence s qui peut être l'une des constantes `pause`, `demi_pause_pointee`, `demi_pause`, `soupir`, `soupir_pointe` [2], `demi_soupir`, `demi_soupir_pointe`, `quart_de_soupir` ou `huitieme_de_soupir`.

18.4.5 Nuances

Les nuances sont les différents degrés de force par lesquels peuvent passer un ou plusieurs sons, un trait ou un morceau entier. La procédure `nuance(`n`)` définit la nuance avec laquelle doit être jouée la suite du morceau. Les termes n de nuances possibles sont, du plus faible au plus fort, les suivants : `pianississimo`, `pianissimo`, `piano`, `mezzo_piano` (ou `mezza_voce`), `mezzo_forte`, `forte`, `fortissimo` ou `fortissisimo`.

Exemple 22 (Au clair de la lune) "Au clair de la lune", qui aurait été composée par le musicien Jean-Baptiste Lulli, est sans doute la plus célèbre des chansons enfantines françaises :



2. Il n'est pas d'usage de pointer le soupir.

Le programme ci-dessous permet de la faire jouer par l'ordinateur. Pour gagner de la place, nous avons écrit deux notes par ligne, avec en commentaire les syllabes correspondantes des paroles (celles du premier et du second vers sont séparées par une barre oblique `/`) :

```
from LibRobot import *

# Au clair de la lune (chanson attribuée à
# Jean-Baptiste Lulli)
def strophe():
    metronome(moderato)  # mesure 4 / 4
    nuance(piano)
    for i in range(2):
        note(sol, noire); note(sol, noire)  # Au clair/Prê-te
        note(sol, noire); note(la, noire)  # de la/moi ta
        note(si, blanche); note(la, blanche)  # lu-ne/plu-me
        note(sol, noire); note(si, noire)  # Mon a-/Pour é-
        note(la, noire)
        note(la, noire)  # -mi Pier-/-crire un
        note(sol, ronde)  # -rot./mot.
    note(la, noire); note(la, noire)  # Ma chan-
    note(la, noire); note(la, noire)  # -delle est
    note(mi, blanche); note(mi, blanche)  # mor-te.
    note(la, noire); note(sol, noire)  # Je n'ai
    note(fa + diese, noire); note(mi, noire)  # plus de
    note(re, ronde)  # feu.
    note(sol, noire); note(sol, noire)  # Ou-vre
    note(sol, noire); note(la, noire)  # moi ta
    note(si, blanche); note(la, blanche)  # por-te,
    note(sol, noire); note(si, noire)  # Pour l'a-
    note(la, noire); note(la, noire)  # -mour de
    note(sol, ronde)  # Dieu.
    silence(pause)

def chanson():
    for j in range(3):
        strophe()

chanson()
```

□

Exercice 34

_ *Écrire des procédures* `debutTransmission`, `a`, ..., `z`, `c0`, ..., `c9` *(chiffres),* `point`, `virgule`, `pointVirgule`, `deuxPoints`, `pointDInterrogation`, `pointDExclamation`, `traitDUnion`, `apostrophe`, `barreDeDivision`, `erreur`, `finTransmission`, `finDeMot` *et* `aLaLigne` *(passer à la ligne suivante) puis les utiliser pour coder des messages en morse, chaque signe du message en clair étant codé en appelant l'une de ces procédures. Par exemple "*message codé*" sera traduit par la suite d'instructions "* `m(); e(); s(); s(); a(); g(); e(); finDeMot(); aLaLigne(); c(); o(); d(); e(); finDeMot()`". On pourra ajouter le son en utilisant les procédures suivantes du Robot :*

- `bip()` *produit un son court ;*
- `biiip()` *produit un son long ;*
- `delai(`*t*`)` *fait attendre pendant* t *secondes (*t *peut être un nombre écrit avec un point, comme dans* `delai(0.5)`*).*

_ *Avant l'invention de l'électricité, on utilisait en France le télégraphe aérien inventé en 1793 par le citoyen Claude Chappe. Le télégraphe est constitué par trois bras de bois articulés à l'extrémité d'un poteau installé sur une tour au sommet d'une colline ou sur un clocher d'église. Dès qu'il aperçoit à la lunette un signe d'activité au poste le plus proche, le* télégraphier *s'empresse de manœuvrer à son tour les bras de sa machine pour transmettre (sans le comprendre) le message au poste suivant. De proche en proche le message arrive à destination. Il s'agit en quelque sorte d'une version perfectionnée des signaux de fumée des Indiens ! Après une première expérience le 12 juillet 1793, la première ligne fut ouverte entre Paris et Lille le 17 août 1794 [3]. Les signaux du télégraphe optique de Chappe pouvaient être décodés selon plusieurs codes. Le code alphabétique est donné à la figure 18.4 (page 220). Écrire un programme qui permet d'afficher successivement, un par un, les signaux et envoyer le message "Paris est tranquille et les bons citoyens sont contents" que Bonaparte fit diffuser immédiatement après le coup d'État du 18 brumaire (9-10 novembre 1799).*

Pour laisser apparaître le signal un certain temps on utilisera la procédure `delai(`*t*`)` *qui fait attendre pendant t secondes. Pour effacer l'écran entre chaque signal, on utilisera la procédure* `cf(blanc)` *qui peint le fond de l'écran en blanc (ou dans une autre couleur selon le paramètre) et place le Robot dans les conditions initiales (visible au centre de l'écran, orienté vers le nord,*

crayon noir d'épaisseur 1 baissé, avec une grille de déplacement (12 × 12)).

_ *Écrire un programme Python de traduction des lettres de l'alphabet en alphabet phonétique selon le code international de la table 18.3 (page 219).*

_ *Écrire un programme Python de cryptage et de décryptage de messages au moyen d'un code secret simple, obtenu par exemple par permutation des lettres de l'alphabet.*

□

Exercice 35

_ *Quelle est la représentation binaire sur 16 bits du nombre qui s'écrit 127 en base 10 ?*

_ *Quelle est l'écriture décimale du nombre entier positif représenté en binaire sur 16 bits par 0011010101010101 ?*

_ *Quel est le texte représenté en code* ASCII *par les octets suivants : 01000100 01100101 01110011 01100011 01100001 01110010 01110100 01100101 0111-0011 ?*

□

A	Alpha	B	Bravo	C	Charlie	D	Delta	E	Echo
F	Foxtrot	G	Golf	H	Hotel	I	India	J	Juliet
K	Kilo	L	Lima	M	Mike	N	November	0	Oscar
P	Papa	Q	Quebec	R	Romeo	S	Sierra	T	Tango
U	Uniform	V	Victor	W	Whiskey	X	Xray	Y	Yankee
Z	Zulu								

TAB. 18.3 – *Alphabet phonétique international*

Exercice 36 *Les chansons sont des sources inépuisables d'exercices [6], [36], [37], [38]. Écrire par exemple des programmes Python pour jouer* Greensleeves *(figure 18.5, page 221) et* O Tannenbaum *(figure 18.6, page 221). On pourra également dérouler la partition musicale ligne par ligne sur l'écran en même temps qu'elle est jouée par l'ordinateur en représentant la portée et les notes comme à la figure 18.7 (page 222).* □

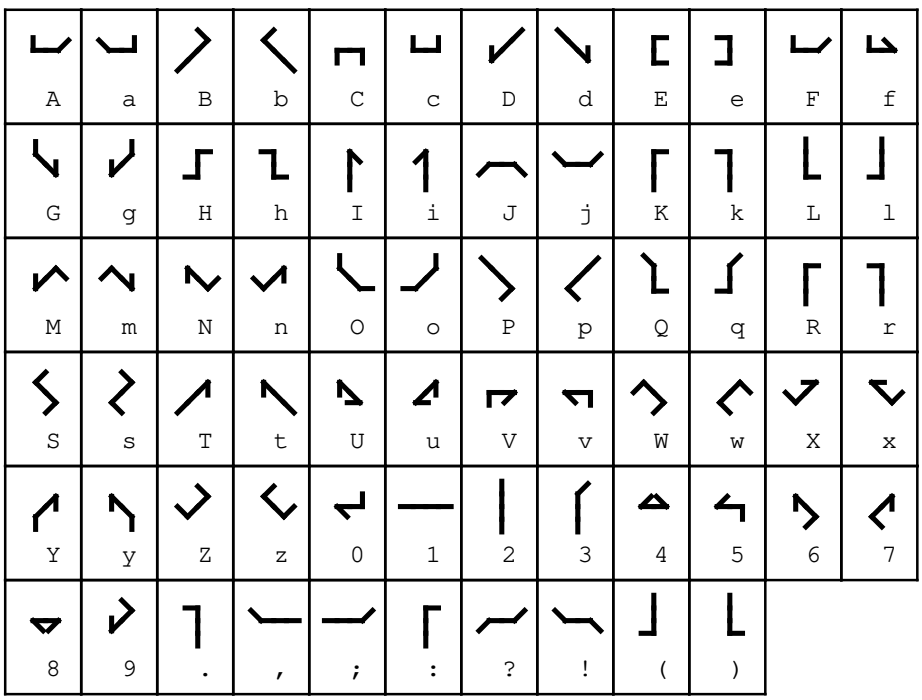


FIG. 18.4 – *Alphabet du télégraphe optique de Chappe*

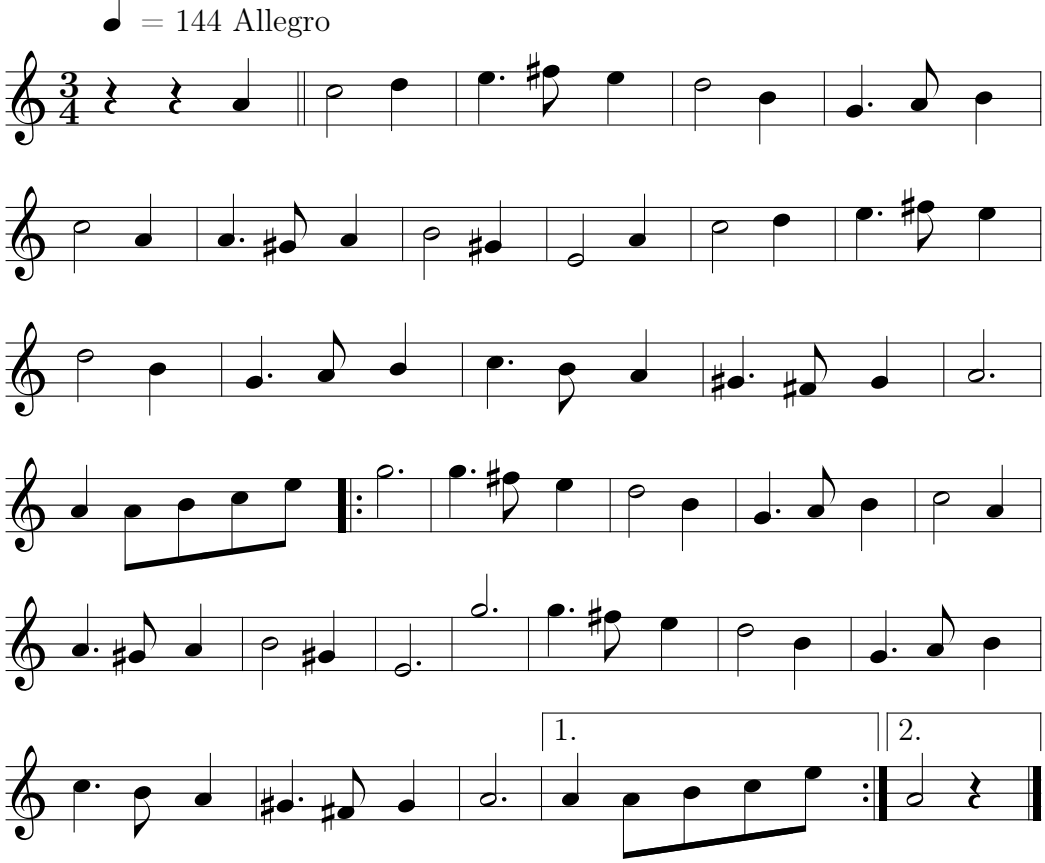



FIG. 18.5 – *Greensleeves (chanson irlandaise)*

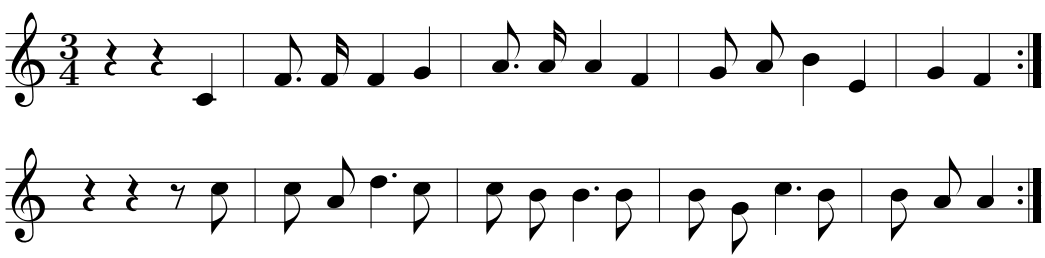


FIG. 18.6 – *O Tannenbaum (chanson allemande)*

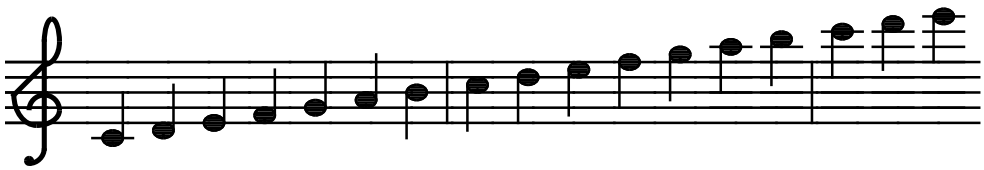



FIG. 18.7 – *Portée et notes représentées sur l'écran de l'ordinateur*

Corrigé 34 *Nous donnons un programme, les autres se trouvant dans le dossier* `livre/18_codage` *:*

```
from LibRobot import *

# Écrire et faire entendre des signes en code Morse

# Passer au début de la ligne suivante
def aLaLigne():
    lc(); ag(); ps(); av(); pe(); av(); bc()

# Point
def pt():
    dp(); bip(); lc(); av(); bc(); delai(0.1)

# Blanc
def bl():
    lc(); av(); bc(); delai(0.1)

# Trait
def tr():
    av(); biiip(); lc(); av(); bc(); delai(0.1)

# Alphabet
def a():
    pt(); tr(); bl(...)

def z():
    tr(); tr(); pt(); pt(); bl()

# Chiffres
def c1():
    pt(); tr(); tr(); tr(); tr(); bl(...)

def c0():
    tr(); tr(); tr(); tr(); tr(); bl()

# Ponctuation
def point():
    pt(); pt(); pt(); pt(); pt(); pt(); bl()

def virgule():
    pt(); tr(); pt(); tr(); pt(); tr(); bl()
```

```
def pointVirgule():
   tr(); tr(); tr(); pt(); pt(); pt(); bl()
def deuxPoints():
   tr(); pt(); tr(); pt(); tr(); pt(); bl()
def pointDInterrogation():
   pt(); pt(); tr(); tr(); pt(); pt(); bl()
def pointDExclamation():
   tr(); tr(); pt(); pt(); tr(); tr(); bl()
def traitDUnion():
   tr(); pt(); pt(); pt(); pt(); tr(); bl()
def apostrophe():
   tr(); tr(); tr(); tr(); bl()
def barreDeDivision():
   tr(); tr(); tr(); tr(); tr(); tr(); bl()
# Fin de mot
def finDeMot():
   lc(); av(); bc(); delai(0.6)
def erreur():
   for i in range(11):
      pt()
   bl()
def debutTransmission():
   ec(2); eh(); aLaLigne()
   tr(); pt(); tr(); pt(); tr(); pt(); tr(); bl()
def finTransmission():
   pt(); tr(); pt(); tr(); pt(); tr(); pt(); bl()
debutTransmission()
a(); b(); c(); d(); e(); aLaLigne()
f(); g(); h(); i(); j(); k(); l(); aLaLigne()
m(); n(); o(); p(); q(); r(); s(); aLaLigne()
t(); u(); v(); w(); x(); y(); aLaLigne()
z(); c1(); c2(); c3(); aLaLigne()
c4(); c5(); c6(); c7(); c8(); aLaLigne()
c9(); c0(); point(); virgule(); aLaLigne()
pointVirgule(); deuxPoints(); pointDInterrogation()
pointDExclamation(); aLaLigne()
traitDUnion(); apostrophe(); barreDeDivision()
aLaLigne(); erreur(); finTransmission(); st()
```

On trouvera dans le même dossier diverses améliorations possibles de ce programme, notamment un retour automatique à la ligne (`CodeMorse2`*). Les*

procédures `a`, ..., `z` *et* `c0`, ..., `c9` *ont des noms en minuscules car l'usage en Python est de commencer les noms de procédures par une minuscule, les noms qui commencent par une majuscule étant habituellement réservés à autre chose.* □

Corrigé 35

_ *La représentation binaire sur 16 bits de 127 est 0000000001111111.*

_ *Le nombre représenté par le demi-mot 0011010101010101 s'écrit 13653 en base 10.*

_ *La suite d'octets 01000100 01100101 01110011 01100011 01100001 01110010 01110100 01100101 01110011 soit, en notation décimale 68 101 115 99 97 114 116 101 115, représente la chaîne de caractères* `'Descartes'`.
□

Corrigé 36 *Nous donnons un programme, de nombreuses autres chansons se trouvant programmées dans le dossier* `livre/18_codage` *et dans son sous-dossier* `Chansons` *:*

```
from LibRobot import *
# O Tannenbaum (chanson allemande)
def refrain():
    note(d0, noire)  # O
    note(fa, croche_pointee); note(fa, double_croche)
    # Tan-nen-
    note(fa, noire)  # -baum,
    note(sol, noire)  # o
    note(la, croche_pointee); note(la, double_croche)
    # Tan-nen-
    note(la, noire_pointee)  # -baum,
    note(fa, croche); note(sol, croche)  # wie treu
    note(la, croche); note(si + bemol, noire)  # sind dei-
    note(mi, noire)  # -ne
    note(sol, noire); note(fa, noire)  # Blät-ter !
def couplet():
    silence(demi_soupir)
    note(d0 + octave, croche)  # Du
    note(d0 + octave, croche); note(la, croche)
    # grünst nicht
    note(re + octave, noire_pointee)  # nur
    note(d0 + octave, croche); note(d0 + octave, croche)
    # zur Som-
```

```
    note(si + bemol, croche)  # -mer-
    note(si + bemol, noire_pointee)  # -zeit
    note(si + bemol, croche); note(si + bemol, croche)
    # Nein, auch
    note(sol, croche)  # im
    note(d0 + octave, noire_pointee)  # Win-
    note(si + bemol, croche); note(si + bemol, croche)
    # -ter wenn
    note(la, croche); note(la, noire)  # es schneit.
# Chanson allemande
def chanson():
    metronome(adagio); nuance(mezza_voce)
    refrain(); couplet(); refrain()
chanson()
```

□

Addenda

En complément nous expliquons deux représentations des entiers relatifs, la seconde étant la plus employée.

Représentation signée des entiers relatifs : Pour coder un entier relatif (positif ou négatif) sur n bits, on peut réserver un bit pour le signe et coder la valeur absolue de l'entier sur les $n-1$ bits restants. Prenons la convention de placer le signe en tête en choisissant 0 pour le signe $+$ et 1 pour le signe $-$.

Le code de -10 sur un octet sera 10001010 soit 1 pour le signe suivi de 0001010 pour la valeur absolue 10 de -10. Avec cette convention on peut coder sur un octet les entiers compris entre -127 (11111111) et 127 (01111111). L'entier nul est représenté par 00000000 $(+0)$ ou bien par 10000000 (-0).

Représentation en complément à deux des entiers relatifs : Dans la représentation en complément à deux sur n bits, les nombres positifs strictement inférieurs à 2^{n-1} sont représentés comme précédemment en base 2 sur $n-1$ bits, le premier bit de signe étant nul. Un nombre négatif $-x$ tel que $x \leq 2^{n-1}$ est représenté par un premier bit de signe égal à 1 suivi de la représentation du nombre positif $p < 2^{n-1}$ tel que $-x = -2^{n-1} + p$ codé sur $n-1$ bits. Par exemple pour coder - 1 sur un octet, on écrit $-1 = -128 + 127$ et l'on code 127 sur 7 bits ce qui donne 1111111 que l'on fait précéder de 1 pour trouver 11111111. Avec cette convention on peut également coder $-128 = -128 + 0$ ce qui donne le code 10000000. Cette méthode permet de coder sur un octet tous les entiers relatifs compris entre -128 et +127. Sur 16 bits, on pourra coder les entiers compris entre -32768 et 32767, ce qui correspondait aux entiers des premiers micro-ordinateurs. Sur 32 bits on pourra coder les entiers compris entre $-2\,147\,483\,648$ et $2\,147\,483\,647$. Cette représentation conduit à des algorithmes d'addition et soustraction très simples. De plus l'algorithme pour le codage d'un nombre négatif $-x$ est également simple. On code le nombre positif x sur n bits puis on inverse les bits 0 en 1 et les bits 1 en 0 puis on ajoute 1. Par exemple pour coder -127, on code 127 en 01111111 puis on inverse tous les bits ce qui donne 10000000 et on ajoute 1 pour trouver 10000001. Les entiers de type `int` de Python ne sont pas limités à 32 ou 64 bits : Python utilise autant de bits que nécessaire (dans la limite de la mémoire de l'ordinateur), si bien que `2 ** 100` (2 multiplié 100 fois par lui-même) vaut exactement $1\,267\,650\,600\,228\,229\,401\,496\,703\,205\,376$.

19

Expressions rationnelles

19.1 Nombres rationnels

Les paramètres de procédures et les constantes que nous avons utilisés jusqu'à présent ont des valeurs entières. En Python, on dit qu'ils sont de *type* `int` (de l'anglais integer, entier). Contrairement à beaucoup d'autres langages, Python n'impose pas de limite à la taille des entiers : `2 ** 100` (2 à la puissance 100) vaut 1 267 650 600 228 229 401 496 703 205 376, ce qui laisse de la marge pour compter les côtés d'un polygone ou les carreaux d'une grille. Ils sont codés en base deux dans la mémoire de l'ordinateur, sur autant de bits qu'il le faut.

Nous pouvons également utiliser en Python des nombres décimaux avec une virgule comme `3.14159` appelés flottants (de type `float`). Ils s'écrivent selon la convention anglo-saxonne avec un point à la place de notre virgule. Les mathématiciens appellent ces nombres des *nombres rationnels*. En Python, on distingue les nombres entiers des nombres rationnels par la présence du point. Par exemple `1000` est un entier alors que `1000.0` est un nombre rationnel, que l'on peut aussi écrire `1000.` sans chiffre après le point. De même `.5` est permis, mais il est plus lisible d'écrire `0.5`.

Les nombres entiers et rationnels sont codés de manières différentes dans la mémoire de l'ordinateur. Les nombres rationnels sont de type `float` [1].

1. De l'anglais *floating point number*, nombre à virgule flottante. L'ordinateur garde en mémoire un nombre fixe de chiffres significatifs et la position de la virgule (qui "flotte" parmi ces chiffres). Ce ne sont pas des nombres réels puisque les nombres de type `float` ne peuvent pas avoir une infinité de chiffres différents de zéro après la virgule !

19.2 Paramètres de type “float”

En Python, on n'indique pas le type des paramètres d'une procédure : un même paramètre peut recevoir, d'un appel à l'autre, un entier ou un rationnel, et c'est seulement pendant l'exécution, quand la valeur est utilisée, que Python vérifie qu'elle convient (on dit que Python est un langage à *typage dynamique*). Par exemple, dans la procédure :

```
def arc(l) : lg(l) ; vd()
```

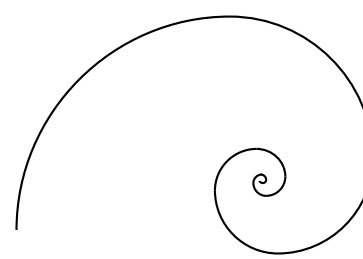
le paramètre formel `l` (lire la lettre ℓ et non le chiffre 1) est passé à la commande `lg` du $\mathcal{Robot}$ qui attend un nombre, entier ou rationnel.

Le paramètre effectif correspondant peut donc être indifféremment un entier, comme dans `arc(100)`, ou un nombre rationnel, écrit avec un point, comme dans `arc(66.666666667)` : Python transforme de lui-même l'entier `100` en rationnel `100.0` quand c'est nécessaire. Si l'on veut faire cette transformation soi-même, on utilise la fonction `float` : si `n` est un entier, `float(n)` est le nombre rationnel correspondant (par exemple `float(3)` vaut `3.0`). C'est rarement utile.

Les procédures de commande du $\mathcal{Robot}$ `lg`, `lgX` et `lgY` (ainsi que `lgRT`, `rt` et `tl` que nous verrons au chapitre suivant) acceptent des paramètres rationnels. C'est pourquoi on peut écrire `lg(20)` aussi bien que `lg(20.5)`. Les commandes `avf`, `cf`, `ec`, `deplacer` ou `tracer` ont au contraire des paramètres entiers : `avf(2.5)` arrête le programme avec le message ‘`TypeError: 'float' object cannot be interpreted as an integer`’ (un objet de type `float` ne peut pas être interprété comme un entier).

Exemple 23 (Spirale arithmétique) La spirale ci-dessous est constituée d'arcs de cercles de rayons successifs 100 ; $100/1.5 = 66.666\ldots$; $66.666\ldots/1.5 = 44.444\ldots$; $44.444\ldots/1.5 = 29.629\ldots$; etc :

```
from LibRobot import *
# Dessiner une spirale arithmétique
# Tracer un arc de cercle de rayon l
def arc(l):
   lg(l); vd()
arc(100); arc(66.666666667)
arc(44.444444444); arc(29.62962963)
arc(19.75308642); arc(13.16872428)
```



```
arc(8.7791495199); arc(5.8527663466)
arc(3.9018442311); arc(2.6012294874)
arc(1.7341529916); arc(1.1561019944)
st()
```

□

19.3 Expressions rationnelles

L'addition, la soustraction, la multiplication et la division de nombres rationnels se notent `+`, `-`, `*` et `/`, c'est-à-dire comme pour les entiers. Par exemple `(((2 * (10.3 + 0.7)) - 1.0) / 3)` vaut `7.0`

Quand une opération porte sur un entier et un rationnel, Python transforme l'entier en rationnel et le résultat est un rationnel : `2 * 3.5` vaut `7.0`. Quand les deux opérandes sont entiers, `+`, `-` et `*` donnent un entier. La division, elle, demande de l'attention : `/` donne *toujours* un rationnel, même entre deux entiers. Ainsi `7 / 2` vaut `3.5` et `6 / 2` vaut `3.0`. La division entière, qui donne un entier, se note `//` : `7 // 2` vaut `3`. Grâce à ces deux notations, le lecteur du programme sait toujours quelle division est faite. Comme `6 / 2` est un rationnel, `avf(6 / 2)` est refusé par la commande `avf` qui attend un entier ; il faut écrire `avf(6 // 2)`.

Si r est un nombre, `sqrt(`r`)` est sa racine carrée $\sqrt{r}$ (de l'anglais square root, racine carrée), r `**` `2` est son carré r^2 (que l'on peut aussi écrire r `*` r), `abs(`r`)` est sa valeur absolue (pour un entier comme pour un rationnel), `int(`r`)` est sa partie entière obtenue en supprimant les chiffres après la virgule (on dit qu'elle est tronquée) et `round(`r`)` est l'entier le plus proche (de l'anglais round, arrondir). Les résultats de `int` et de `round` sont des entiers. Par exemple `int(0.3)` = `int(0.5)` = `int(0.9)` = `0` tandis que `round(0.3)` = `0` et `round(0.5)` = `round(0.9)` = `1` [2]. La fonction `sqrt` est fournie par la bibliothèque du $\mathcal{R}^{o}\!\!\smile^{o}t$ (ligne **from** `LibRobot` **import** `*`), les autres font partie de Python. Comme pour les procédures, l'argument d'une fonction est toujours placé entre parenthèses : `sqrt(x + 1)`.

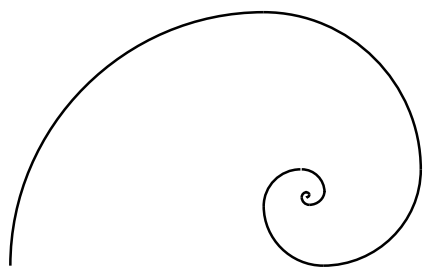
Dans une expression rationnelle, on peut utiliser la longueur horizontale `valLgX()` et la longueur verticale `valLgY()` d'un carreau de la grille de dépla-

2. Le `round` d'origine de Python arrondit les demis vers l'entier pair le plus proche (`round(0.5)` vaut `0` et `round(2.5)` vaut `2`). La bibliothèque du $\mathcal{R}^{o}\!\!\smile^{o}t$ le remplace par un `round` qui arrondit les demis à l'opposé de zéro, comme Pascal et OCaml : `round(2.5)` vaut `3` et `round(-2.5)` vaut `-3`.

cement du $\mathcal{R}^{o}\!\!\smile^{o}t$. Par exemple `lgX(valLgX() * 2)` double la taille horizontale d'un carreau de la grille.

Exemple 24 (Spirale d'or) La spirale d'or ci-dessous est constituée d'arcs de cercles successifs dont les rayons ont pour rapport le *nombre d'or* $\Phi = \frac{1}{2}(1+\sqrt{5}) \simeq 1.618033989$. Pour les tenants d'esthétique traditionnelle, la spirale d'or constituerait une figure géométrique très harmonieuse à l'œil.

```
from LibRobot import *
# Dessiner une spirale d'or
nombreDor = (1 + sqrt(5)) / 2
lg(100)
for i in range(12):
   vd()
   lg(valLgX() / nombreDor)
st()
```



□

19.4 Erreurs d'arrondi

Pour les nombres réels, qui ont un nombre infini de chiffres après la virgule, comme $\frac{1}{3} = 0.3333333333333333\ldots$ ou $\pi = 3.1415926535897932\ldots$ l'ordinateur ne peut garder dans sa mémoire que quelques chiffres après la virgule, ce qui conduit, comme pour une calculette, à des erreurs d'arrondi dans les calculs. Par exemple en mathématiques on a :

$$\left(\left(\left(\left(\left(\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{2}}}}\right)^2\right)^2\right)^2\right)^2\right)^2 = \left(\left(\left(\left(\sqrt{\sqrt{\sqrt{\sqrt{2}}}}\right)^2\right)^2\right)^2\right)^2 =$$

$$\left(\left(\left(\sqrt{\sqrt{\sqrt{2}}}\right)^2\right)^2\right)^2 = \left(\left(\sqrt{\sqrt{2}}\right)^2\right)^2 = \left(\sqrt{2}\right)^2 = 2$$

tandis que l'ordinateur trouve que :

```
((((sqrt(sqrt(sqrt(sqrt(sqrt(2))))) ** 2)
     ** 2) ** 2) ** 2) ** 2 = 1.9999999999999964
```

De même, il trouve que `0.1 + 0.2` vaut `0.30000000000000004` (le programme `ErreursDarrondi` du dossier `livre/19_expressions_rationnelles` fait ces deux calculs).
On retrouve des problèmes d'arrondi similaires avec la grille de déplacement du $\mathcal{R}^{o}_{o}t$. De manière idéale, les coordonnées cartésiennes du $\mathcal{R}^{o}_{o}t$ devraient être réelles. L'ordinateur en conserve en mémoire une approximation rationnelle. Sur l'écran elles doivent être arrondies à l'entier le plus proche pour correspondre à un pixel de l'écran.

Exercice 37 *Écrire des programmes Python pour réaliser les motifs de la figure ci-dessous (seul le motif central qui utilise les fonctions trigonométriques introduites dans l'addenda du chapitre 21 est difficile).* □

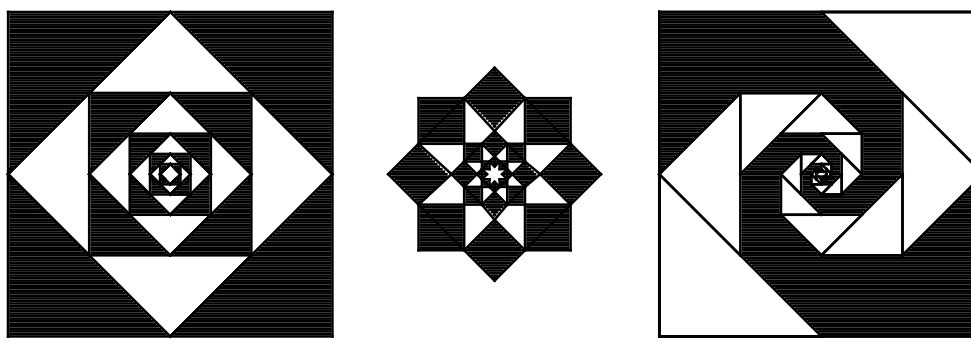


FIG. 19.1 – *Motifs de Horemis et Beard ; spirale de Baravelle*

Corrigé 37 *Nous donnons un programme, les autres se trouvant dans le dossier* `livre/19_expressions_rationnelles` *:*

```
from LibRobot import *

# Motif de S. Horemis, 'Optical and geometrical
# patterns and designs', Dover, New York, 1970.
n = 6
deuxPuissanceN = 64  # 2 à la puissance n

eb(); ag(); lg(deuxPuissanceN)
for i in range(n):
   for j in range(4):
      avf(4); pqtd()
   avf(2); phtd()
   for j in range(4):
      avf(2); avf(-1); pqtd(); peindre(); pqtg()
      av(); pqtd()
   av(); phtd()
   lg(valLgX() / 2)
st()
```

□

Addenda

Les nombres rationnels sont souvent qualifiés de réels par abus de langage. Ils peuvent se noter en utilisant la *notation scientifique*, qui consiste à faire suivre la partie fractionnaire par un facteur d'échelle formé de la lettre e ou `E` (qui se lit "dix à la puissance") suivie d'un exposant.
Par exemple $-3.0\text{e}2 = -300.0$, $3.0\text{e}0 = 3.0$ et $3.0\text{e}{-2} = 0.03$. Un nombre rationnel ne contient pas de blanc. Par conséquent $-3.0 \text{␣e␣} 2$ est erroné. En Python, les nombres de type `float` sont représentés avec 15 à 16 chiffres significatifs en base 10, le plus petit rationnel strictement positif (sans perte de chiffres significatifs) étant de l'ordre de $2.2\text{e}{-308}$ et le plus grand de l'ordre de $1.8\text{e}308$. Au-delà, le résultat d'un calcul vaut `inf` (l'infini) : `1e308 * 10` vaut `inf`.

La valeur d'une expression arithmétique peut être de type `int` ou `float`, selon les règles suivantes :

- les constantes entières (sans point) et les opérateurs $+$, $-$, $*$, `//`, `%`, $**$ (avec un exposant positif) et `abs` [a] appliqués à des entiers donnent un résultat entier de type `int`, de même que `int` et `round` qui s'appliquent à un rationnel ;
- les constantes rationnelles (avec un point ou un exposant `e` ou `E`), la division `/` et la fonction `sqrt` donnent un résultat rationnel de type `float`, de même que `float` qui s'applique à un entier ;
- les opérateurs $+$, $-$, $*$, $**$ et `abs` appliqués à des rationnels, ou à un entier et un rationnel, donnent un résultat rationnel.

Une expression qui mélange les deux, comme `1 + 2.5`, est donc permise : l'entier est transformé en rationnel et le résultat `3.5` est rationnel.

En Python, le type des paramètres n'est pas déclaré. Python transforme de lui-même un entier en rationnel quand il le faut, mais jamais un rationnel en entier : c'est au programmeur de le demander avec `int` ou `round`, par exemple `avf(round(x))`. On peut indiquer le type attendu d'un paramètre formel lors de la déclaration de la procédure, comme dans **def** `arc(l : float) : lg(l) ; vd()`, ce qui n'est pas nécessaire (Python ne le vérifie pas) mais peut aider le lecteur du programme.

---

[a] Nous utilisons des typographies différentes pour les *mots-clés* (comme **def** ou **for**) et les *identificateurs prédéfinis* (comme `sqrt` ou `abs`). Les identificateurs introduits dans un programme Python (comme les noms de constantes, de procédures, etc.) doivent être différents des mots-clés mais peuvent être identiques aux identificateurs prédéfinis. Cette pratique est déconseillée car les programmes qui changent le sens habituel des identificateurs prédéfinis sont illisibles.

20

Rotations et translations

20.1 Dimensions de la grille de déplacement du robot en coordonnées polaires

Les dimensions horizontales X et verticales Y d'un carreau de la grille de déplacement du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ peuvent être définies en coordonnées cartésiennes par les commandes `lgX(`X`)` et `lgY(`Y`)` comme indiqué à la figure 20.1 ci-dessous.

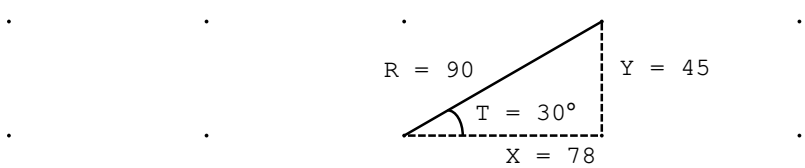


FIG. 20.1 – *Grille de déplacement du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ définie par* `lgX(`X`)` ; `lgY(`Y`)` ou `lgRT(`R`, `T`)`

On peut également définir les dimensions d'un carreau du quadrillage en *coordonnées polaires* par la commande `lgRT(`R`, `T`)` en donnant la longueur R de la diagonale de ce carreau mesurée en pixels et l'angle T exprimé en degrés que fait cette diagonale avec une horizontale comme illustré à la figure 20.1 ci-dessus. La commande `lgRT` a deux paramètres, qui sont des nombres entiers ou rationnels, comme dans `lgRT(50, 70)` ou `lgRT(50.5, 22.5)`.

Exemple 25 (Triangle isocèle) Le programme ci-dessous dessine un triangle isocèle ayant deux angles égaux à 70 degrés et deux côtés égaux à 50 pixels :

```
from LibRobot import *
# Dessiner un triangle isocèle
longueurCote = 50
angle = 70
lgRT(longueurCote, angle)
phtd(); av()
pqtd(); av()
p3htd(); avf(2)
st()
```

□

20.2 Rotations

La commande `rt(`t`)` fait effectuer au $\mathcal{R}\!\mathit{obot}$ une rotation de t degrés, t étant une expression entière ou rationnelle (par exemple `rt(15)`, `rt(-90)` ou `rt(22.5)`). Si la valeur de l'expression t est positive alors le pivotement sur place se fait dans le sens *contraire* des aiguilles d'une montre [1]. Si la valeur de l'expression t est négative alors le pivotement sur place se fait dans le sens des aiguilles d'une montre. Cette commande ne change pas la position du $\mathcal{R}\!\mathit{obot}$ mais elle change son orientation. La longueur du segment tracé par le prochain `av()` reste également inchangée. Pour ce faire la taille du carreau du quadrillage doit être convenablement modifiée. Ceci est illustré sur la figure 20.2 (page 237) qui montre un carreau du quadrillage en pointillés avant et après des rotations successives de 15 degrés (le $\mathcal{R}\!\mathit{obot}$ partant à chaque fois du coin inférieur gauche du carreau et la rotation étant suivie d'un `av()` pour montrer que la longueur du segment tracé ne change pas). Sur cette figure, les schémas successifs sont disposés de haut en bas et de gauche à droite.

Quand on utilise la commande `rt`, nous conseillons d'éviter les commandes `phtd()`, `pqtd()`, `p3htd()`, ..., `pne()`, `pse()`, ...en les remplaçant par `rt` avec un paramètre approprié (seules les commandes `pn()`, `ps()`, ...sont utiles pour les positionnements absolus). En utilisant exclusivement la commande `rt`,

1. Sens contraire des aiguilles d'une montre que les mathématiciens appellent *sens trigonométrique*.

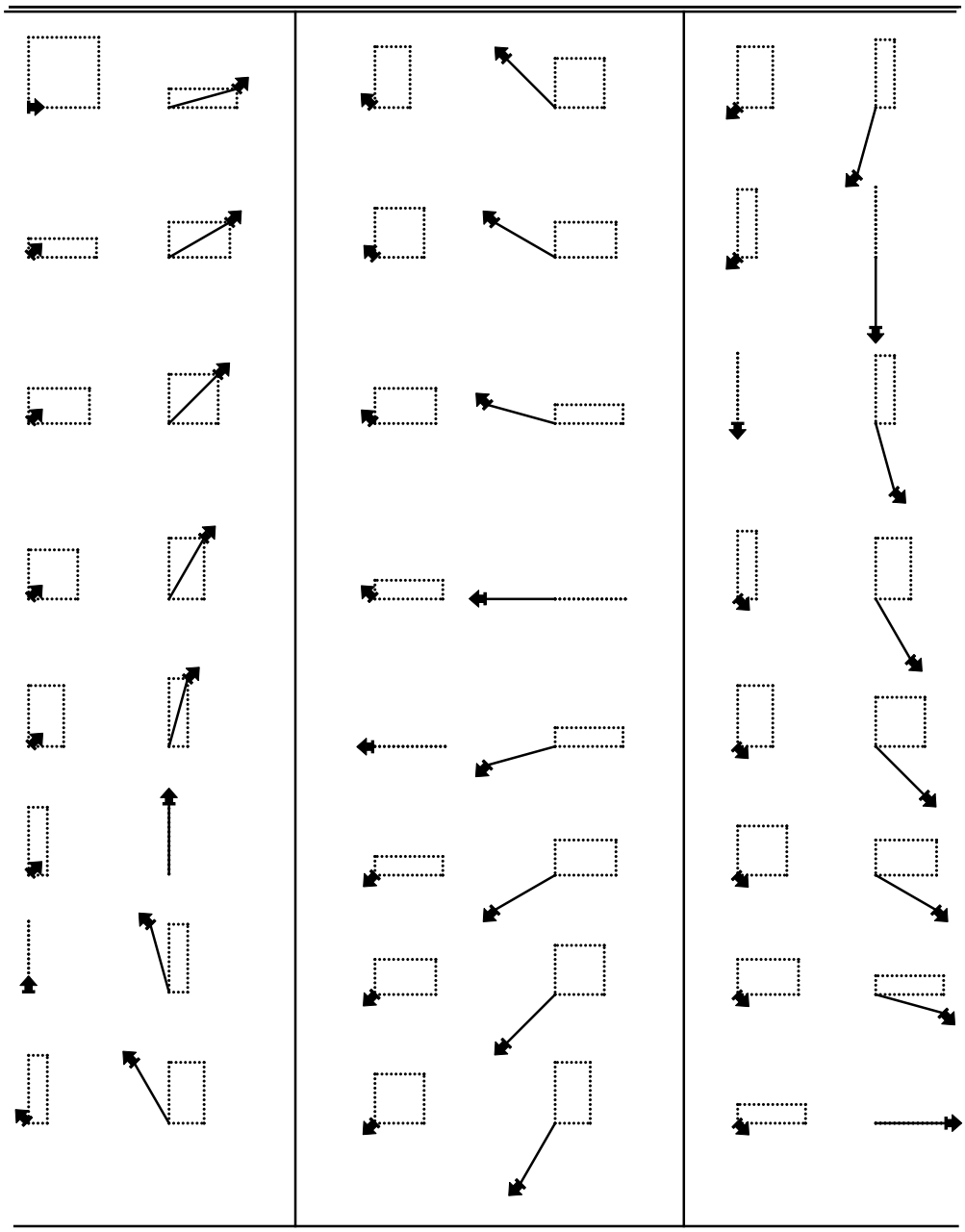
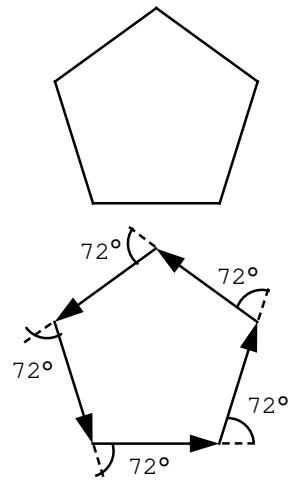


FIG. 20.2 – *Rotations et déplacements successifs* `rt(15) ; av()`

il n'est pas nécessaire de tenir compte des changements de repères qu'elle induit.

Exemple 26 (Pentagone) Le programme ci-dessous dessine un pentagone en répétant cinq fois le tracé d'un segment suivi d'une rotation de $(360/5) = 72$ degrés :

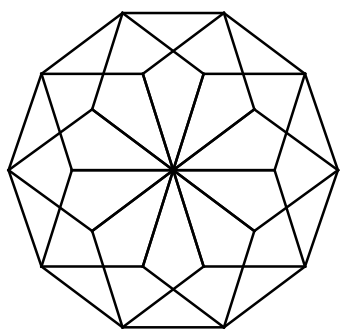
```
from LibRobot import *
# Dessiner un pentagone
longueurCote = 50
lg(longueurCote); pe()
for j in range(5):
   av()
   rt(72)
st()
```



□

Exemple 27 (Rosace de pentagones) La rosace ci-dessous est obtenue par superposition de dix pentagones tournés successivement de 36 degrés :

```
from LibRobot import *
# Dessiner une rosace de dix pentagones
longueurCote = 40
pqtd(); lg(longueurCote)
for i in range(10):
   rt(36)
   for j in range(5):
      av(); rt(72)
st()
```



□

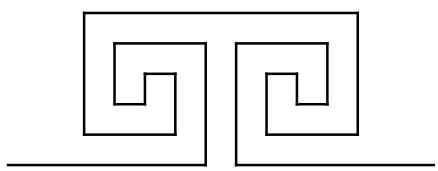
20.3 Translations

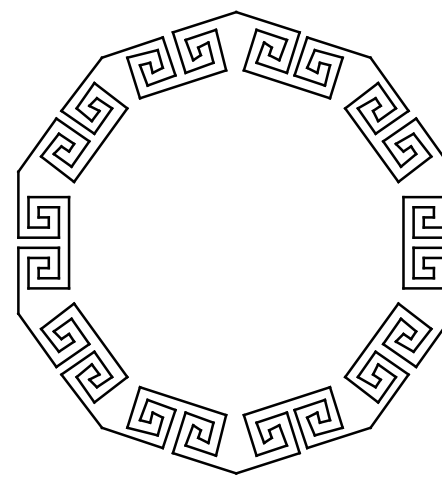
Quelle que soit l'orientation du $\mathcal{R}^{o}\!\!\mathcal{b}^{o}t$, l'expression `valLgTl()` de type `float` a pour valeur la longueur (mesurée en pixels) d'un segment qui serait tracé par un `av()`.

Pour tracer un segment d'une longueur différente, on peut utiliser la commande `tl(`E`)` qui prend en paramètre une expression E (entière ou rationnelle) dont la valeur est la longueur de ce segment comptée en pixels. Si cette valeur est positive, le segment est tracé en avançant dans la direction courante du $\mathcal{R}^{o}_{b}{}^{o}t$. Si cette valeur est négative, le tracé se fait en reculant. Le $\mathcal{R}^{o}_{b}{}^{o}t$ est déplacé à l'extrémité du segment et ne change pas son orientation. Les dimensions d'un carreau du quadrillage sont changées pour permettre ce déplacement (si le crayon est levé) ou ce tracé (si le crayon est baissé). Par conséquent, le segment peut être prolongé par un segment d'égale longueur en utilisant une commande `av()`.

Par exemple, la séquence de commandes `tl(20) ; rt(90) ; av() ; rt(90) ; av() ; rt(90) ; av()` trace un carré de côté 20 pixels.

Exemple 28 (Grecque polygonale) La frise polygonale ci-dessous est obtenue en remplaçant le tracé d'un côté du polygone par celui de la grecque :





```
from LibRobot import *

# Dessiner une grecque polygonale
fg = 2  # Facteur de grossissement
n = 10  # Nombre de côtés

def motif():
   # Partie gauche
   tl(13 * fg); rt(90); tl(8 * fg); rt(90)
   tl(6 * fg); rt(90); tl(4 * fg); rt(90)
   tl(2 * fg); rt(90); tl(2 * fg); rt(-90)
   tl(2 * fg); rt(-90); tl(4 * fg); rt(-90)
```

```
    tl(6 * fg); rt(-90); tl(8 * fg); rt(-90)
    tl(18 * fg); rt(-90)
    # Partie symétrique droite
    tl(8 * fg); rt(-90); tl(6 * fg); rt(-90)
    tl(4 * fg); rt(-90); tl(2 * fg); rt(-90)
    tl(2 * fg); rt(90); tl(2 * fg); rt(90)
    tl(4 * fg); rt(90); tl(6 * fg); rt(90)
    tl(8 * fg); rt(90)
    tl(13 * fg)
def grecque():
    for i in range(n):
        motif(); rt(360 / n)
grecque(); st()
```

□

Exercice 38 *Écrire des programmes Python utilisant la commande* `lgRT` *pour réaliser les dessins suivants :*

- *un triangle équilatéral ;*
- *le pentagone, les croix ou la toile d'araignée de la figure 20.3 (page 240) ;*
- *les motifs géométriques d'après Horemis [17] de la figure 20.4 (page 241) ;*
- *les pavages coloriés de la figure 20.6 (page 242).*

Les très nombreux pavages donnés par Grünbaum et Shephard [15] sont la source d'innombrables exercices. □

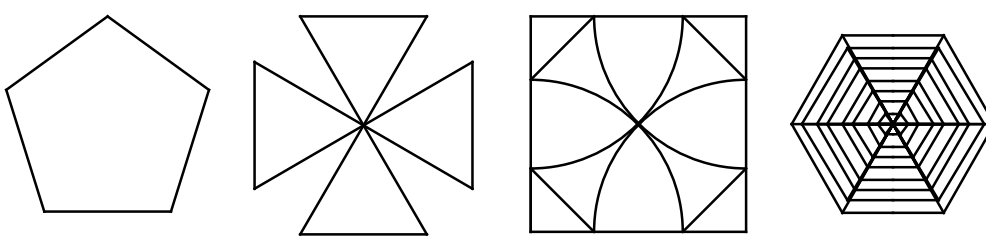


FIG. 20.3 – *Pentagone, croix et toile d'araignée*

Exercice 39 *Écrire des programmes Python utilisant la commande* `rt` *pour réaliser les dessins suivants :*

- *les étoiles, l'octogone décoré ou la courbe spirolatérale d'ordre 3 à 135 degrés de la figure 20.5 (page 241), cette dernière étant obtenue en avançant une,*

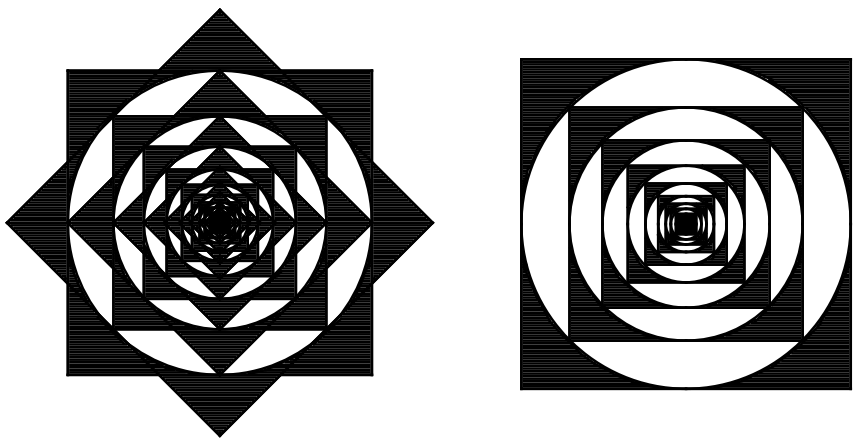


FIG. 20.4 – *Motifs géométriques d'après Horemis*

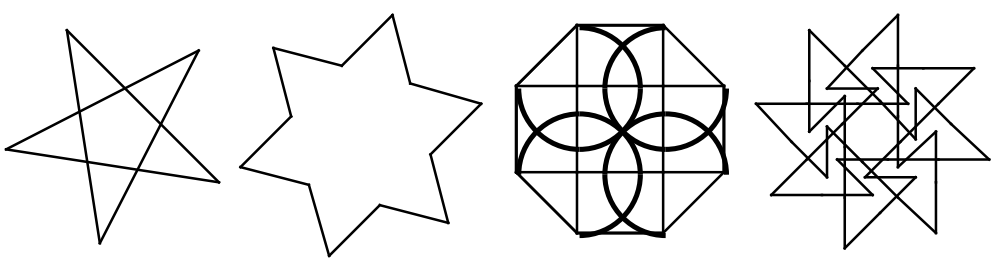


FIG. 20.5 – *Étoiles, octogone décoré et spirolatérale*

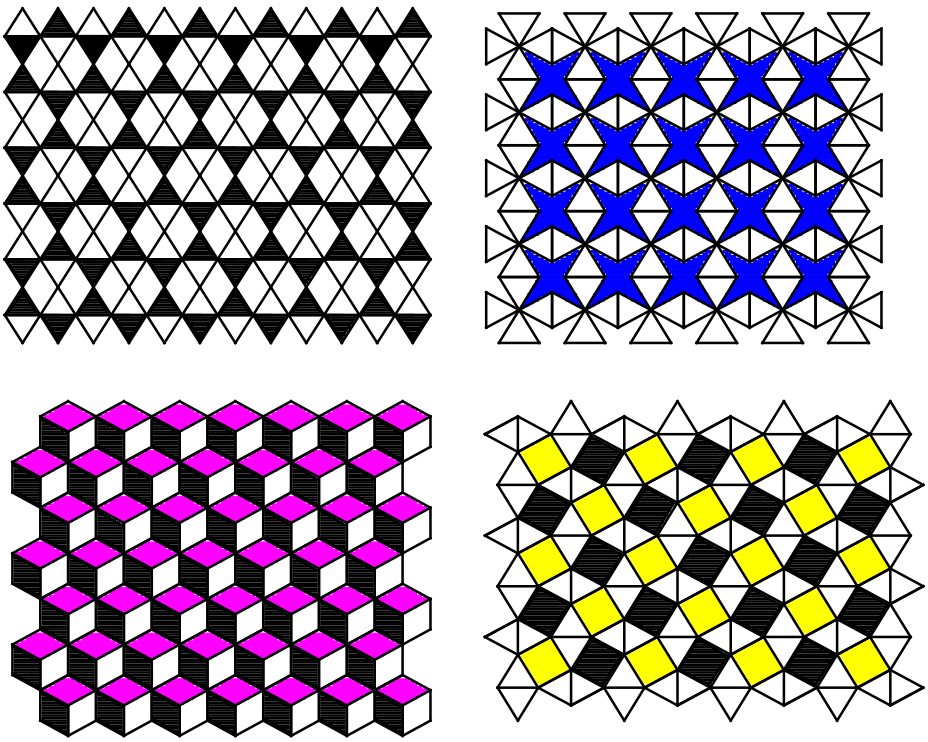


FIG. 20.6 – *Pavages de triangles, losanges et carrés*

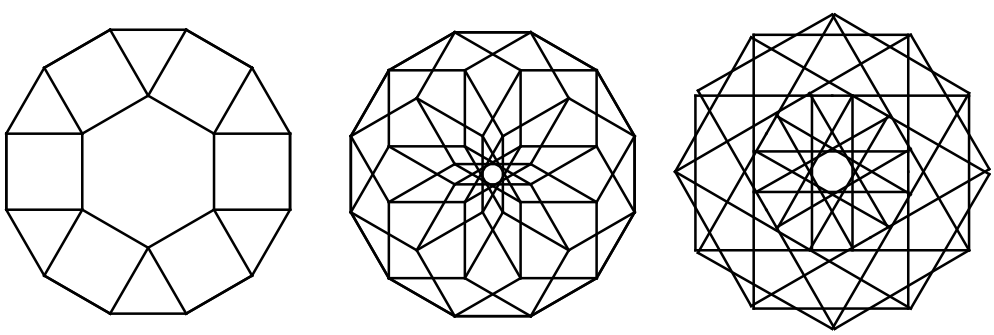


FIG. 20.7 – *Rosaces polygonales*

puis deux, puis trois fois, en tournant à chaque fois de 135 degrés et en répétant ces déplacements jusqu'à ce que la courbe soit fermée ;

– *les polygones réguliers de la figure 20.8 (page 244), le nombre de côtés étant un paramètre de la procédure de dessin ;*

– *les étoiles de la figure 20.9 (page 244), le nombre de branches étant un paramètre de la procédure de dessin ;*

– *le rapporteur de la figure 20.10 (page 244), en utilisant la commande* `ecrire(entierEnChaine(e))` *qui écrit la valeur de l'expression entière* e *à droite de la position courante du* Robot *(l'irrégularité des graduations est due aux erreurs d'arrondi en coordonnées d'écran entières qui ne permettent pas de représenter des droites de pentes quelconques) ;*

– *le pavage du plan avec des hexagones ou des étoiles de la figure 20.11 (page 245) ;*

– *le pavage à la Escher [26] du disque et du triangle de la figure 20.12 (page 245) ;*

– *le pavage du disque utilisant le pavé de Voderberg [42] de la figure 20.13 (page 246) ;*

– *les spirales utilisant le pavé de Voderberg [42] de la figure 20.14 (page 246).*
□

Exercice 40 *Écrire des programmes Python utilisant les commandes* `lgRT` *et* `rt` *pour réaliser les développements des solides suivants :*

– *solides de Platon (cube, tétraèdre, octaèdre, dodécaèdre et isocaèdre réguliers) de la figure 20.15 (page 247) ;*

– *les grand et petit rhombicuoctaèdres et le tétraèdre tronqué de la figure 20.16 (page 247).*

Les développements des solides donnés par Cundy et Rollett [12] sont la source de nombreux exercices. □

Exercice 41 *Écrire des programmes Python utilisant les commandes* `rt` *et* `tl` *pour réaliser les dessins suivants :*

– *les rosaces polygonales de la figure 20.7 (page 242) ;*

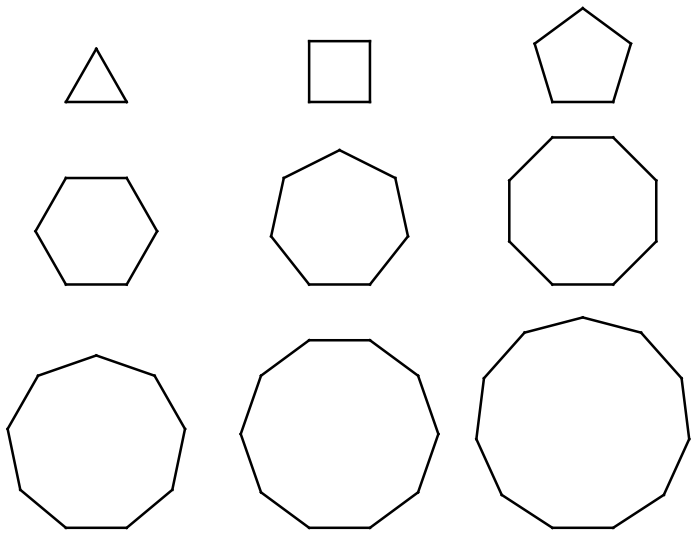


FIG. 20.8 – *Polygones réguliers*

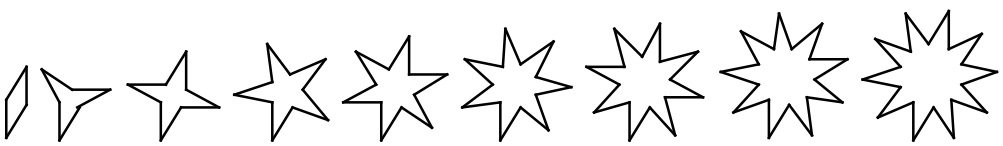


FIG. 20.9 – *Étoiles*

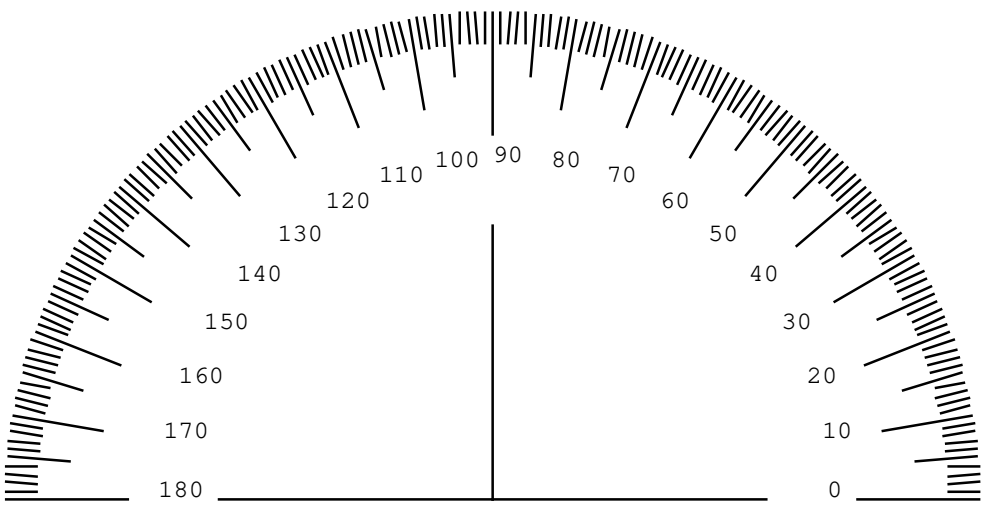


FIG. 20.10 – *Rapporteur gradué en degrés*

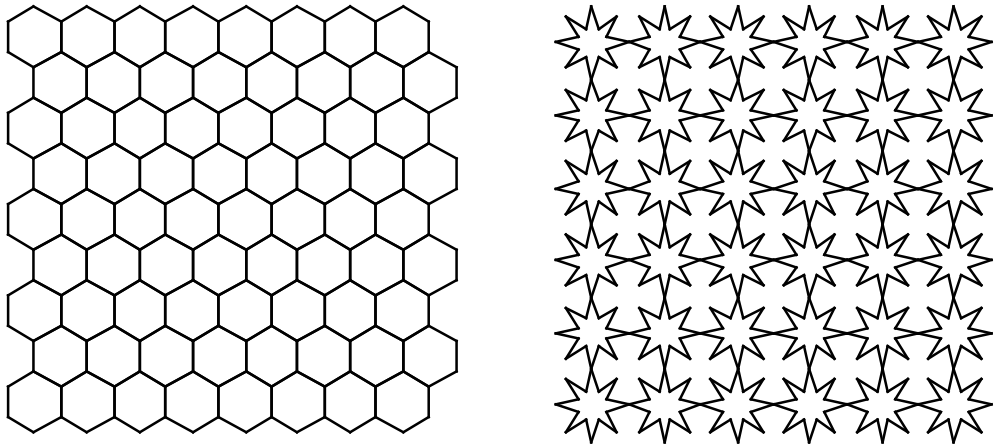


FIG. 20.11 – *Pavage du plan avec des hexagones et des étoiles*

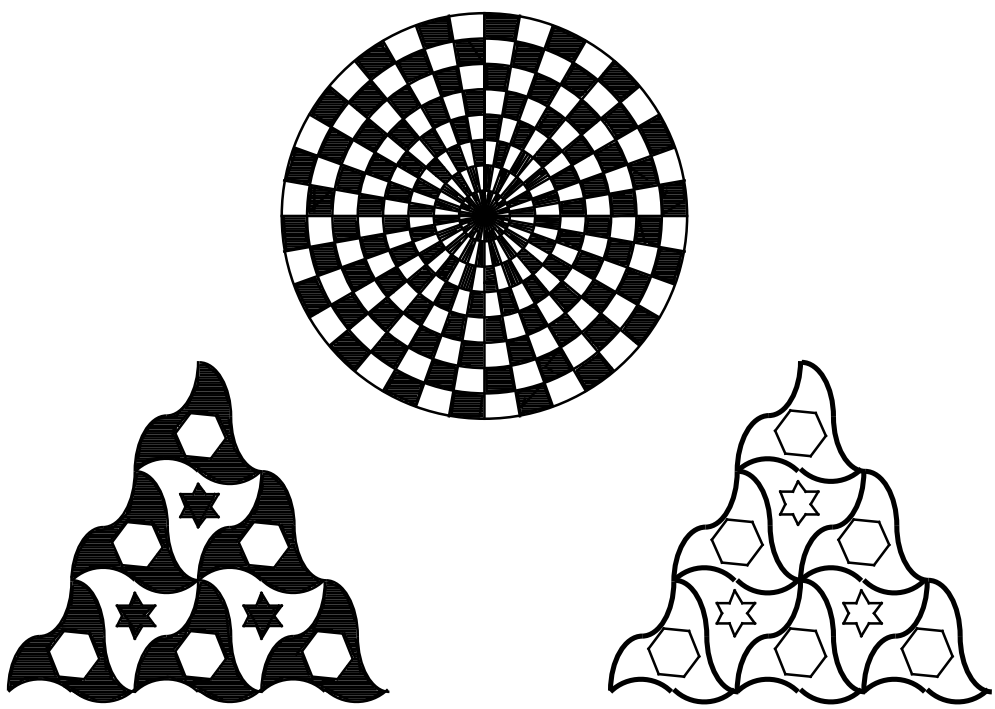


FIG. 20.12 – *Pavage à la Escher du disque et du triangle*

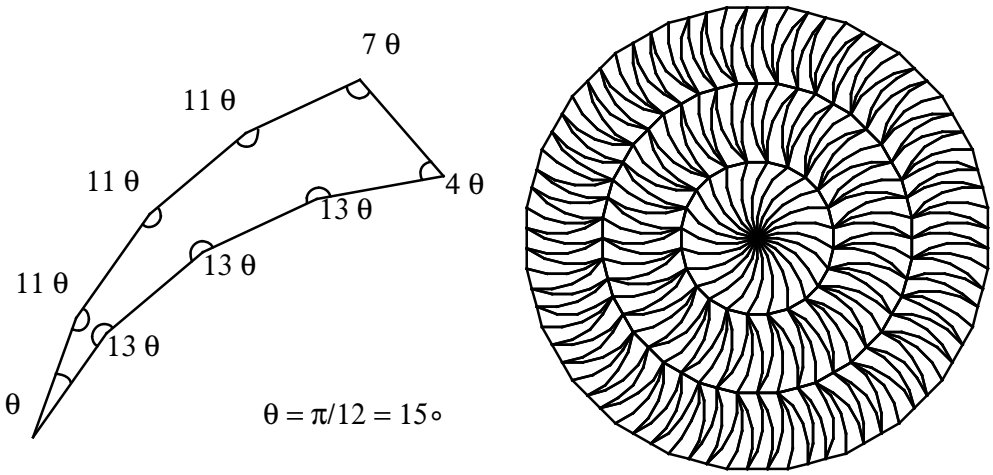


FIG. 20.13 – *Pavé et pavage du disque de Voderberg*

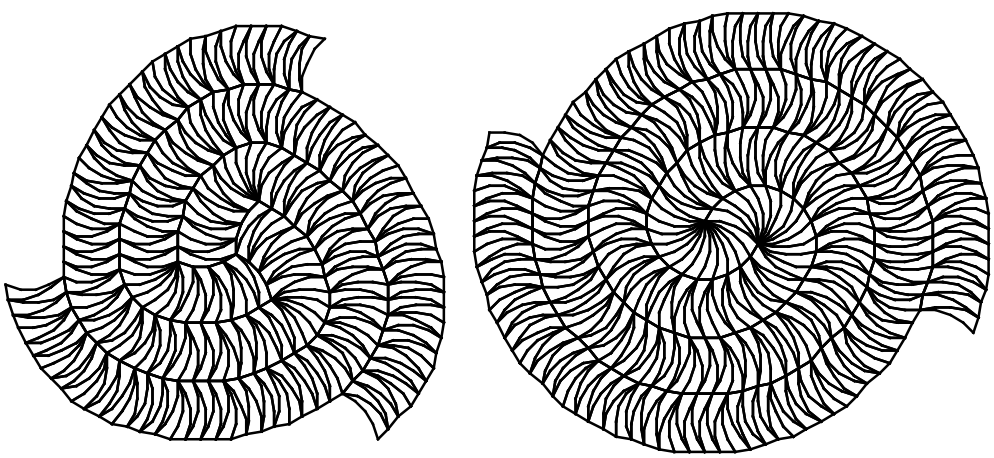


FIG. 20.14 – *Spirales de Voderberg*

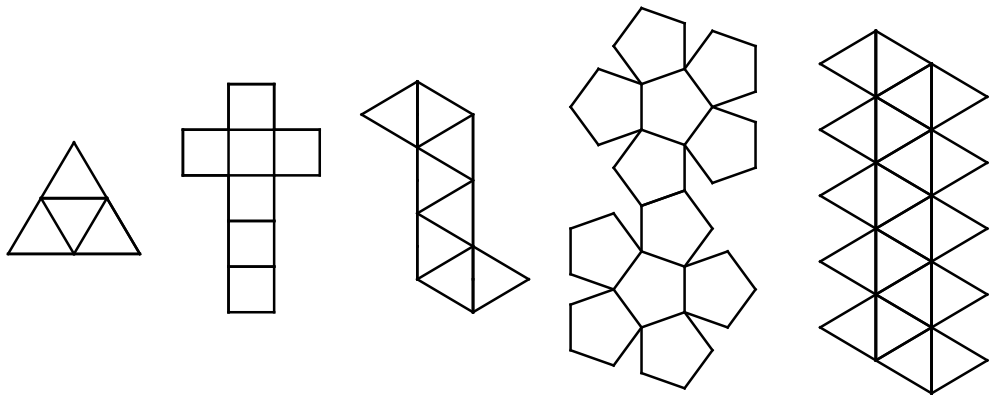


FIG. 20.15 – *Développements des solides de Platon*

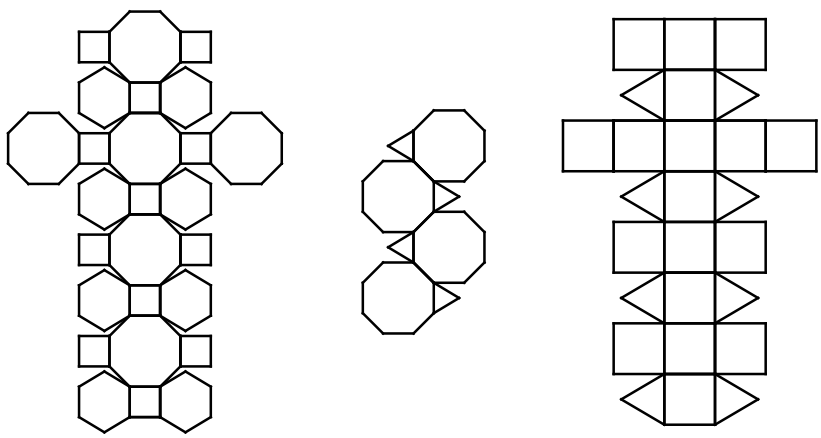


FIG. 20.16 – *Développements des grand et petit rhombicuoctaèdres* et du tétraèdre tronqué

_ *les "jolygones" de la figure 20.18 (page 249) obtenus en traçant des segments successifs dont la longueur diminue à chaque fois de 5 %, deux segments successifs faisant un angle de θ degrés ; θ étant choisi respectivement égal à 15, 60, 72, 76, 89, 92, 118, 175 et 144 degrés pour les jolygones considérés de gauche à droite et de haut en bas ;*

_ *les courbes planes de la figure 20.19 (page 249) définies par une équation polaire, comme suit :*

- *le limaçon de Pascal, $\rho = a + b\cos\theta$ avec $a \neq 0$, $b \neq 0$ et $\theta \in [0,2\pi]$ (sur l'exemple, $a = 40$ et $b = 80$),*
- *le rhodoneæ à trois pétales, $\rho = a\cos n\theta$ avec $a = 60$, $n = 3$ et $\theta \in [0,\pi]$,*
- *la courbe de Moritz, $\rho = a\cos\frac{9}{10}\theta$ avec $a = 60$ et $\theta \in [-10\pi,10\pi]$;*

_ *les courbes planes définies par les équations cartésiennes $y = \sin x, x \in [-2\pi,2\pi]$ (figure 20.17 ci-dessous) et $y = \cos x, x \in [-2\pi,2\pi]$ en utilisant les commandes* `deplacerReel` *et* `tracerReel`*. La commande* `tracerReel(x, y)`*, où* `x` *et* `y` *sont des expressions rationnelles, trace un segment (si le crayon est baissé) de la position courante du Robot au point de coordonnées cartésiennes (x, y) et déplace le Robot dans cette nouvelle position (x, y). La commande* `deplacerReel(x, y)` *fait de même sans tracer. Les fonctions* `sin` *et* `cos` *et la constante* `pi` *sont expliquées dans l'addenda du chapitre 21. De nombreux autres exemples de tracés de courbes sont donnés dans [10].*

□

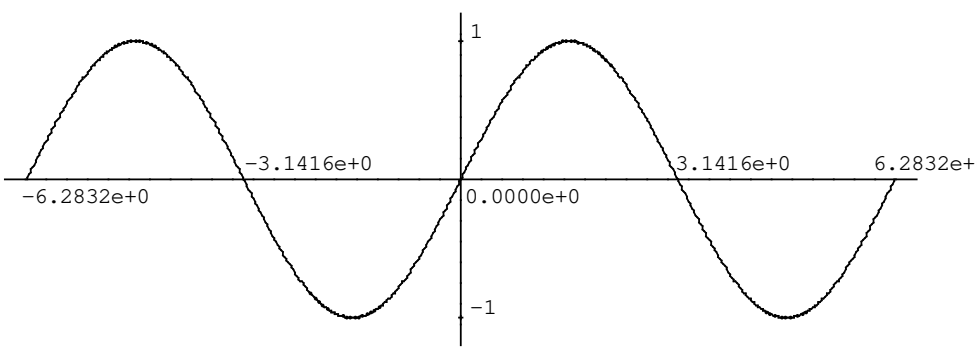


FIG. 20.17 – *Courbe cartésienne $y = \sin x, x \in [-2\pi,2\pi]$*

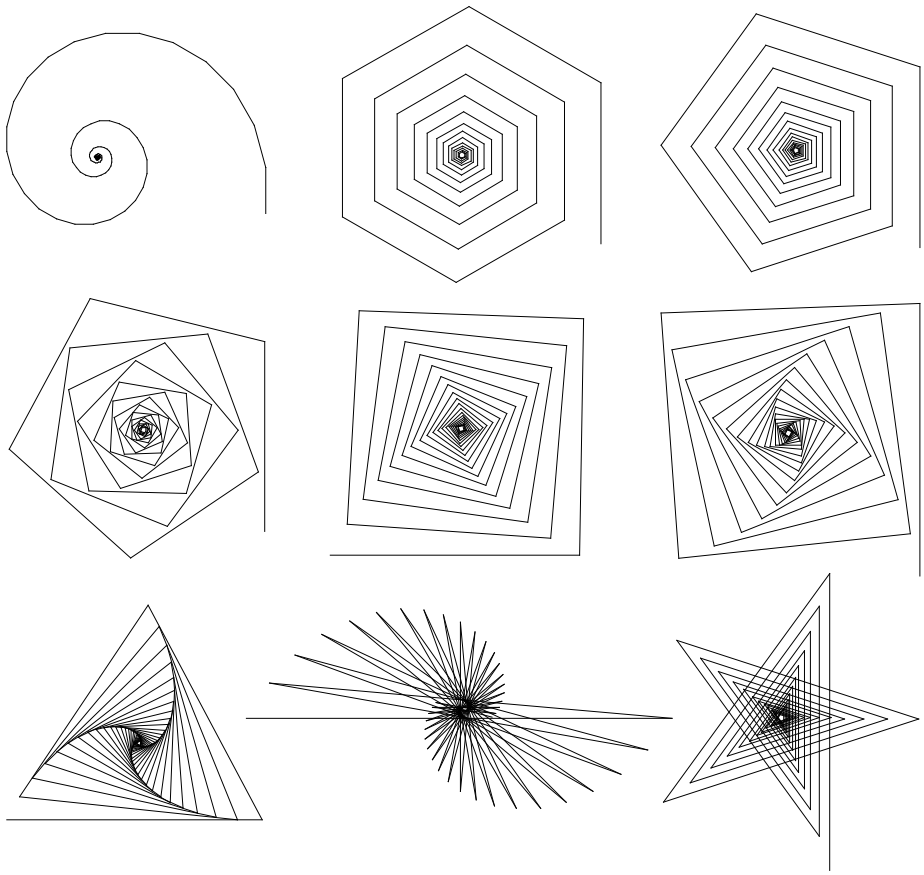


FIG. 20.18 – *Jolygones à 15, 60, 72, 76, 89, 92, 118, 175 et 144 degrés*

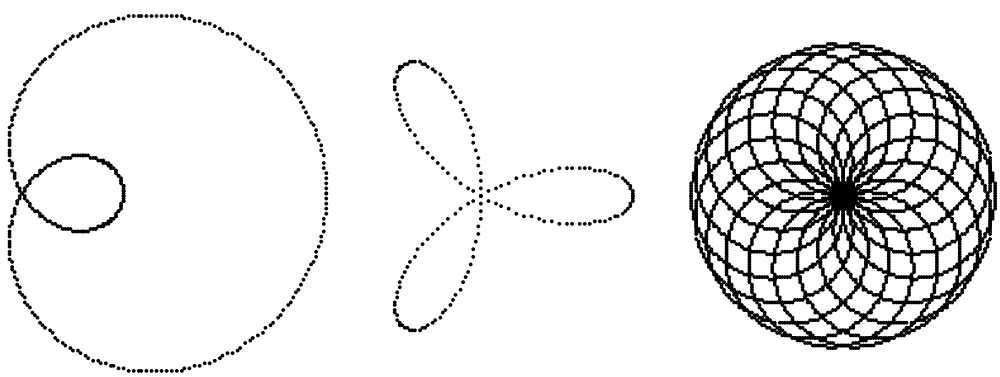


FIG. 20.19 – *Limaçon de Pascal, rhodoneæ à trois pétales, courbe de Moritz*

Corrigé 38 *Nous donnons le programme de dessin de la toile d'araignée de la figure 20.3 (page 240), les autres programmes se trouvant dans le dossier* `livre/20_grille_polaire` *:*

```
from LibRobot import *

# Dessiner une toile d'araignée
def hexagone(r):
    lgRT(r, 60)
    ce(); pw()
    for i in range(2):
        lc(); av(); av(); bc(); p3htd(); av(); phtd()
        av(); av(); phtd(); av(); ce()
        pe()
def diagonales(r):
    lgRT(r, 60)
    psw()
    for i in range(4):
        av(); ce(); pqtd()
    ce(); pw(); avf(2); ce(); pe(); avf(2)
def toile():
    for i in range(1, 9):
        hexagone(5 * i)
        diagonales(5 * i)
toile(); st()
```

□

Corrigé 39 *Nous donnons la procédure de dessin d'un polygone régulier de n côtés (figure 20.8, page 244), les autres programmes se trouvant dans le dossier* `livre/20_grille_polaire` *:*

```
from LibRobot import *

# Dessiner un polygone régulier
tailleQuadrillage = 30
nombreDeCotes = 7

# Dessiner un polygone régulier de n >= 1 côtés
def polygone(n):
    for i in range(n):
        av(); rt(360 / n)
lg(tailleQuadrillage); polygone(nombreDeCotes); st()
```

□

Corrigé 40 *Nous donnons le programme de dessin du développement du tétraèdre tronqué, les autres programmes se trouvant dans le dossier* `livre/20_grille_polaire` *:*

```
from LibRobot import *

# Dessiner le développement d'un tétraèdre tronqué
lgCote = 24

def faceEtCoin():
    # Hexagone
    pqtd(); lgRT(lgCote, 60); phtg()
    for i in range(6):
        av(); rt(60)
    # Triangle équilatéral
    pdt(); av(); pqtd(); av()

def developpementDuTetraedreTronque():
    for i in range(2):
        # Face et coin en bas
        faceEtCoin(); lc(); p3htd(); avf(2); phtg(); av()
        p3htd(); bc()
        # Face et coin en haut
        faceEtCoin(); lc(); av(); phtg(); avf(2); pqtg()
        bc()

developpementDuTetraedreTronque(); st()
```

□

Corrigé 41 *Nous donnons le programme de dessin du jolygone à 175 degrés, les autres jolygones sont obtenus en changeant la valeur de la constante* `angle`*. Les autres programmes se trouvent dans le dossier* `livre/20_grille_polaire`*.*

```
from LibRobot import *

# Dessiner un jolygone à 175 degrés
longueurInitiale = 350
angle = 175
nbrSegments = 40

ag()
lg(longueurInitiale); pqtd(); av()
for i in range(nbrSegments - 1):
    rt(angle); tl(valLgTl() * 0.95)
st()
```

□

Addenda

La commande `lgRT(r, t)` définit la longueur `r` de la diagonale d'un carreau du quadrillage (exprimée en pixels) et l'angle `t` exprimé en degrés que fait cette diagonale avec une horizontale. Elle est équivalente à :

```
lgX(r * cos(t / 180 * pi))
lgY(r * sin(t / 180 * pi))
```

mais évite l'emploi des fonctions trigonométriques. L'angle `t`, compris entre 0 et 90, définit l'orientation nord-est qui est confondue avec l'est quand `t` vaut `0` et avec le nord quand `t` vaut 90.

La commande `rt(t)` fait pivoter le $\mathcal{R}\underline{o}\underline{b}^{o}t$ sur place d'un angle de `t` degrés à gauche, dans le sens trigonométrique ou sens inverse des aiguilles d'une montre, si $\texttt{t} > 0$, et à droite, dans le sens des aiguilles d'une montre, si $\texttt{t} < 0$. La commande `tl(l)` fait avancer (si $\texttt{l} > 0$) ou reculer (si $\texttt{l} < 0$) le $\mathcal{R}\underline{o}\underline{b}^{o}t$ tout droit de `l` fois la taille d'un côté d'un pixel de l'écran.

Ces commandes `rt` et `tl` sont directement inspirées du graphisme "tortue" de LOGO. Elles changent le repère et l'orientation du $\mathcal{R}\underline{o}\underline{b}^{o}t$ pour rester compatibles avec l'emploi de coordonnées cartésiennes ou polaires.

Le langage LOGO fut conçu par Seymour Papert à la fin des années 1970 [31]. Il présente de nombreuses similitudes avec LISP [28], en particulier l'interprétation de données représentant des instructions, la portée dynamique des identificateurs et la structure de liste. L'interprétation des données comme des programmes implique quelques subtilités dans l'écriture des programmes (comme l'emploi de la fonction `quote`, [45]) qui ne sont pas à la portée de tous les débutants en informatique. La portée dynamique des identificateurs [45] est largement abandonnée et la structure de liste est presque toujours complétée par des tableaux dans les LISP modernes comme COMMON LISP qui suivent en cela les langages descendant d'ALGOL 60 [29] comme PASCAL ou ADA. Le langage Python offre lui aussi à la fois les listes et les tableaux, n'interprète pas ses données comme des instructions (sauf si on le lui demande explicitement) et utilise la portée statique des identificateurs.

21

Récursivité

En Python, une procédure peut appeler une autre procédure précédemment déclarée. Une procédure peut également s'appeler elle-même. On dit alors qu'elle est *récursive* (de l'anglais *recursive*). Elle se déclare avec **def**, comme toute autre procédure : il n'y a rien de spécial à écrire. Les procédures récursives sont utiles pour faire des dessins comme ceux de la figure 21.1 (page 254). Ces dessins récursifs ont la particularité de se retrouver indéfiniment en réduction à l'intérieur d'eux-mêmes (exactement comme l'image infinie obtenue dans deux miroirs se faisant face). Comme en pratique il n'est pas possible de dessiner un trait plus petit qu'un pixel de l'écran, le tracé s'arrête pour les dessins très petits.

21.1 Récursivité simple

La structure des dessins récursifs simples se comprend par *récurrence* sur la taille du dessin. Il faut d'abord comprendre la *base*, c'est-à-dire comment faire un dessin de taille 0 (ou 1). Ensuite il faut comprendre l'*induction*, c'est-à-dire comment construire un dessin de taille n en utilisant des dessins identiques de tailles $n-1$, $n-2$, . . .strictement inférieures à n.

Prenons l'exemple des pavés récursifs de la figure 21.2 (page 255). Un pavé de taille 0 est réduit à un point (plus précisément un pixel de l'écran). Un pavé de taille n est constitué d'un pavé de taille $n-1$ entouré d'un carré de côté n fois la taille du quadrillage, comme expliqué à la figure 21.3 (page 255). La procédure récursive `pave` de dessin d'un pavé récursif reçoit en paramètre la taille `n` du pavé. Si `n` est nul, elle dessine un point. Si `n` est strictement

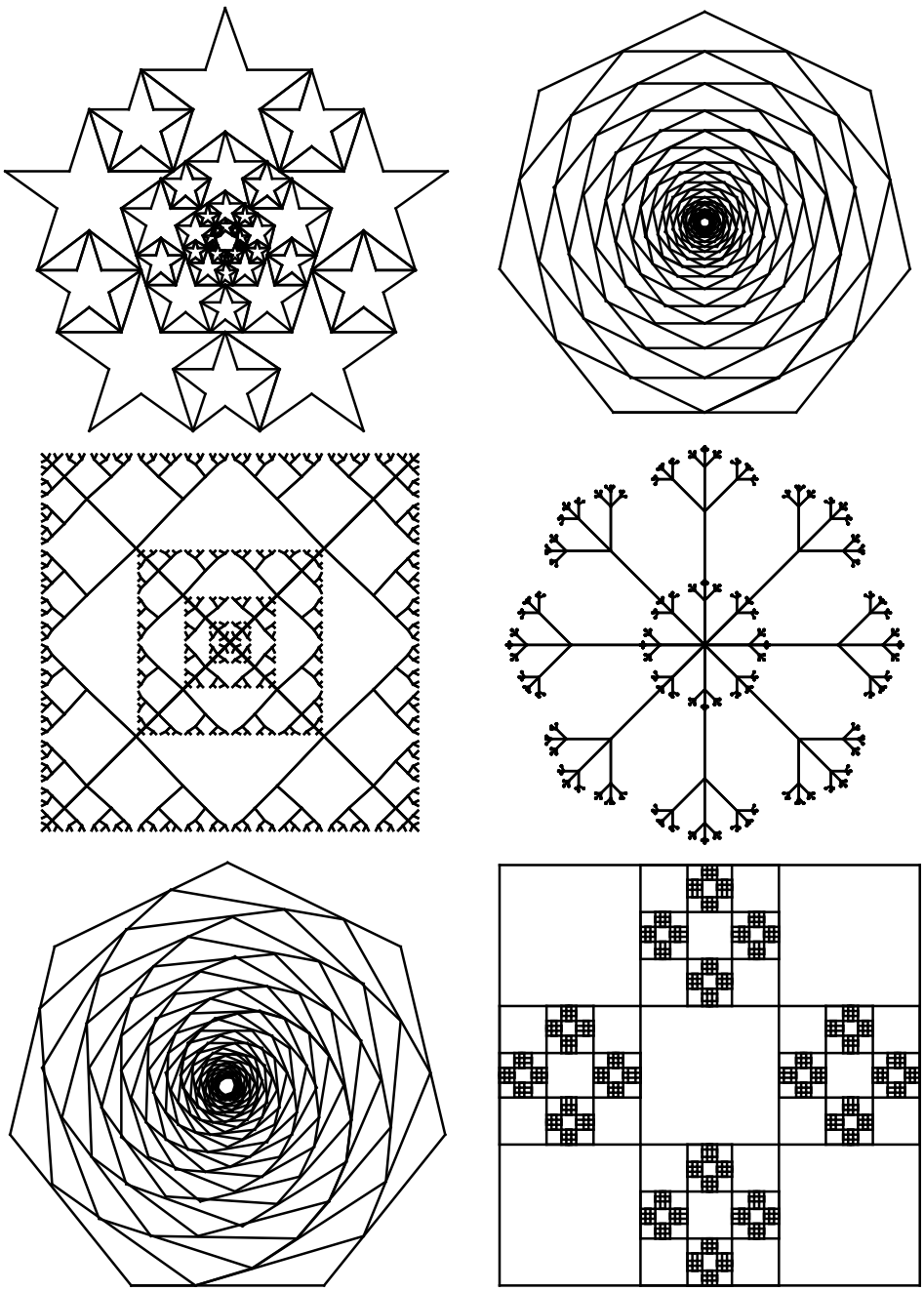


FIG. 21.1 – *Dessins récursifs*

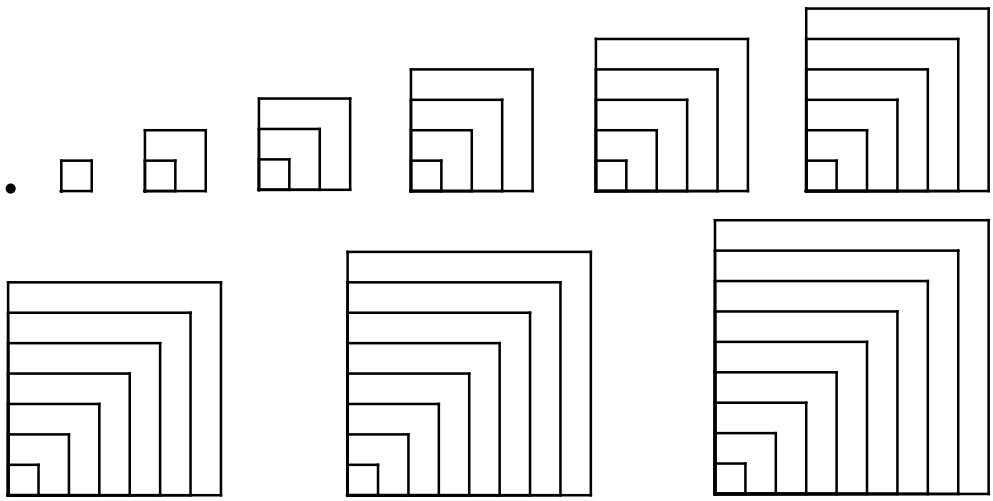


FIG. 21.2 – *Pavés récursifs*

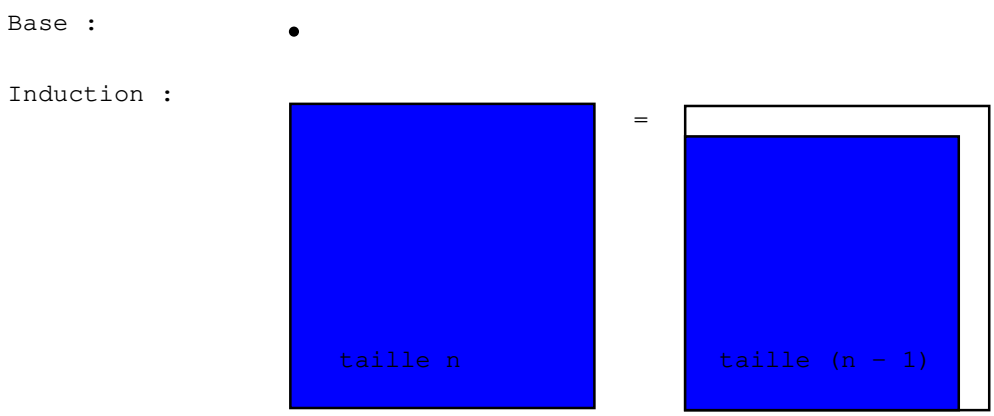


FIG. 21.3 – *Règle de construction récurrente des pavés récursifs*

positif alors elle dessine un carré de côté `n` puis s'appelle récursivement pour dessiner un pavé de taille `n - 1`.

```
from LibRobot import *
# Dessiner des pavés récursifs
def pave(n):
    if n == 0:
        # Dessiner un carré de côté 0
        dp()
    # Dessiner un carré de côté n
    else:
        avf(n); pqtd(); avf(n); pqtd(); avf(n); pqtd()
        avf(n); pqtd()
        # Paver l'intérieur
        pave(n - 1)
eb(); ag(); pave(10); st()
```

Le nom `pave` peut être utilisé dans le corps de la procédure `pave` elle-même : quand Python exécute l'appel `pave(n - 1)`, la déclaration de `pave` est terminée et le nom `pave` est donc connu. Une procédure récursive est donc déclarée et appelée exactement comme les procédures que nous avons vues jusqu'à présent.

Le dessin des pavés récursifs peut se faire *itérativement*, c'est-à-dire en utilisant une boucle "for" ou "while". Ce serait plus difficile avec des dessins comme le tableau d'un tournoi de tennis représenté à la figure 21.4 ci-dessous.

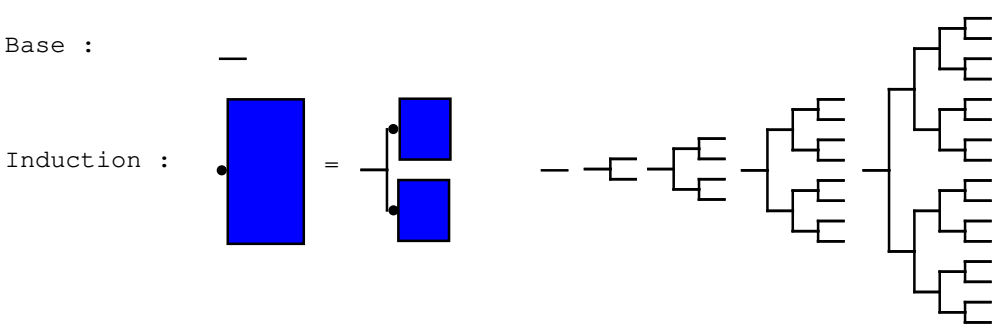


FIG. 21.4 – *Règle de construction récurrente d'un tableau* de tournoi de tennis

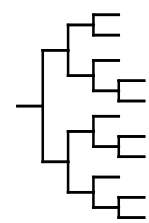
Un tableau de tournoi de tennis ne comprenant qu'un seul inscrit comporte un trait horizontal prévu pour écrire le nom de cet inscrit qui est également le vainqueur. Un tableau de tournoi de tennis comprenant `n` inscrits s'obtient en traçant un trait horizontal prévu pour écrire le nom du vainqueur en face de la pointe d'une accolade reliant les sous-tableaux servant à désigner les finalistes. Chacun des sous-tableaux contient à peu près le même nombre de joueurs c'est-à-dire respectivement `n // 2` et `n - n // 2` joueurs (rappelons que `//` est la division entière). La procédure récursive de dessin du tableau d'un tournoi de `n` joueurs est donnée ci-dessous et illustrée dans le cas où `n` vaut 11.

```
from LibRobot import *

# Dessiner le tableau d'un tournoi de tennis
l = 5  # Longueur d'un trait horizontal

# Dessiner un tableau pour un tournoi de n joueurs
def tableau(n):
    # Pointe de l'accolade, pour le vainqueur
    avf(l)
    if n > 1:
        # Branche supérieure de l'accolade
        pqtg(); avf(n); pqtd()
        # Sous-tableau supérieur, contenant la
        # moitié des joueurs
        tableau(n // 2)
        # Revenir au centre de l'accolade
        lc(); pqtg(); avf(-n); pqtd(); bc()
        # Branche inférieure de l'accolade
        pqtd(); avf(n); pqtg()
        # Sous-tableau inférieur, contenant le
        # reste des joueurs
        tableau(n - n // 2)
        # Revenir au centre de l'accolade
        lc(); pqtd(); avf(-n); pqtg(); bc()
    # Revenir à l'extrémité de la pointe
    avf(-l)

lg(3); pe(); tableau(16); st()
```



21.2 Invariants

Quand on écrit une procédure récursive il est important de réfléchir à l'*invariant* de cette procédure c'est-à-dire aux hypothèses que doivent satisfaire les paramètres effectifs de la procédure et le $\mathcal{R}\mathcal{o}\mathcal{b}^{o}t$ pour que le dessin soit correct. Prenons l'exemple d'un arbre binaire (voir la figure 21.5 ci-dessous), une structure de base de la programmation, que nous avons déjà rencontrée sous la forme du tableau de tournoi de tennis.

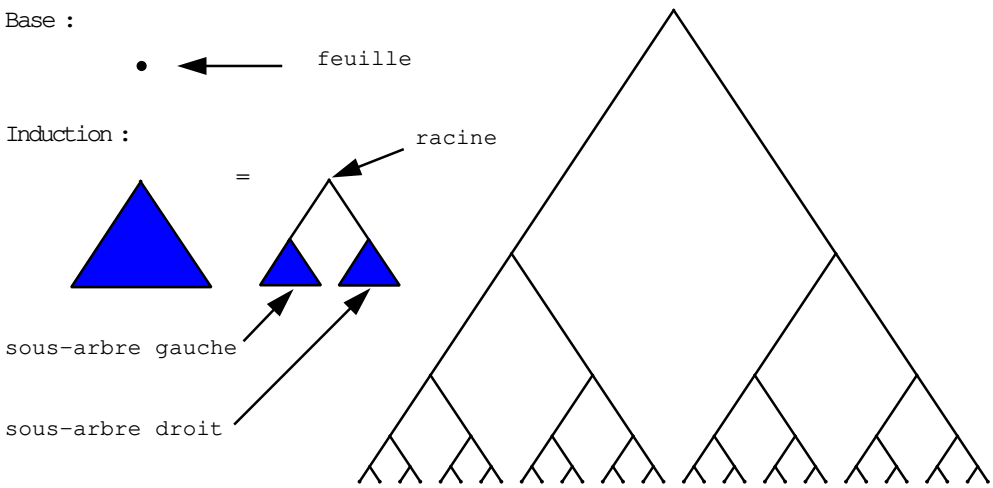


FIG. 21.5 – *Règle de construction récurrente d'un arbre binaire*

La procédure de dessin d'un arbre binaire est la suivante :

```
from LibRobot import *

# Dessiner un arbre binaire

# Dessiner un arbre à l feuilles, la racine en haut,
# les feuilles en bas, en supposant que l >= 1,
# l'arbre étant réduit à un point quand l = 1. Avant
# et après le dessin, le robot doit être placé à la
# racine de l'arbre, orienté au nord avec son crayon
# baissé.
def arbre(l):
   if l == 1:
      # Dessiner un arbre réduit à un seul point (sans
      # changer la position et l'orientation du robot)
      dp()
   # Le robot étant placé à la racine de l'arbre,
```

```
    # orienté au nord avec son crayon baissé, dessiner
    # la branche sud-ouest
    else:
       p3htg(); avf(l)
       # Orienter le robot au nord
       p3htd()
       # Dessiner le sous-arbre gauche ayant l / 2 >= 1
       # feuilles, le robot étant placé à la racine du
       # sous-arbre, orienté au nord avec son crayon
       # baissé
       arbre(l // 2)
       # Sachant que le robot est placé à la racine du
       # sous-arbre gauche, orienté au nord et crayon
       # baissé, revenir à la racine de l'arbre, sans
       # rien dessiner
       lc(); phtd(); avf(l)
       # Le robot étant placé à la racine de l'arbre,
       # orienté au nord-est avec son crayon levé,
       # dessiner la branche sud-est
       bc(); pqtd(); avf(l)
       # Orienter le robot au nord
       p3htg()
       # Dessiner le sous-arbre droit ayant l - l / 2 >= 1
       # feuilles, le robot étant placé à la racine du
       # sous-arbre, orienté au nord avec son crayon
       # baissé
       arbre(l - l // 2)
       # Placer le robot à la racine de l'arbre, orienté
       # au nord avec son crayon baissé
       lc(); phtg(); avf(l); phtd(); bc()
lgX(2); lgY(3); eh(); arbre(32); st()
```

Le premier invariant est que le paramètre `l` de la procédure `arbre` doit être supérieur ou égal à 1. C'est vrai pour l'appel principal puisque `l` $= 32$. Un appel quelconque ne fait des appels supplémentaires que si `l` > 1. Comme `l` ≥ 2, on a (`l // 2`) ≥ 1 et l'invariant est vrai quand on appelle récursivement la procédure pour dessiner le sous-arbre gauche. Comme `l` $= (2 * ($`l // 2`$))$ si `l` est pair et `l` $= (2 * ($`l // 2`$)) + 1$ si `l` est impair, on a `l` $\geq (2 * ($`l // 2`$))$ donc (`l` $-($`l // 2`$)) \geq ($`l // 2`) de sorte que (`l` $-($`l // 2`$)) \geq 1$ puisque (`l // 2`) ≥ 1. L'invariant est donc également vrai quand on appelle récursivement la procédure `arbre` pour dessiner le sous-arbre droit. Comme l'invariant est vrai pour l'appel principal et qu'il reste vrai pour tous les appels suivants, il est toujours vrai.

On vérifierait de la même façon l'invariant concernant le $\mathcal{R}^{o}_{\smile}{}^{o}t$, à savoir qu'avant et après le dessin de l'arbre binaire, le $\mathcal{R}^{o}_{\smile}{}^{o}t$ se trouve placé à la racine de l'arbre, orienté vers le nord avec son crayon baissé.

21.3 Non-terminaison

Quand on utilise des procédures récursives, il peut arriver qu'elles s'appellent les unes les autres sans fin, exactement comme dans le cas de boucles "while" dont l'exécution ne se termine pas. L'exemple le plus simple est celui de la procédure `boucler` ci-dessous.

```
from LibRobot import *

# Un programme qui ne se termine pas
def boucler():
   boucler(); st()

boucler(); st()
```

Un appel de `boucler()` provoque immédiatement un nouvel appel de `boucler()`, et ainsi de suite indéfiniment. À chaque appel, l'ordinateur utilise un peu de sa mémoire (appelée la *pile*) pour se souvenir de ce qui reste à faire ensuite (ici le `st()` qui suit l'appel). Pour éviter de remplir toute la mémoire, Python limite le nombre d'appels en cours à environ 1000. Au-delà, il se produit une erreur (`RecursionError`) que la bibliothèque du $\mathcal{R}^{o}_{\smile}{}^{o}t$ signale par le message '`Erreur ligne 5 de NonTerminaison.py : recursivite sans fin (pile pleine)`' avant l'arrêt de l'exécution du programme (la ligne 5 est celle de l'appel `boucler()` dans le corps de la procédure). En général l'erreur est moins flagrante et ne se produit que pour certaines valeurs des paramètres. Cette limite signifie aussi qu'une procédure récursive correcte ne peut pas s'appeler elle-même plus d'environ 1000 fois de suite : c'est largement suffisant pour nos dessins, puisqu'en pratique le tracé s'arrête bien avant, au niveau du pixel.

21.4 Récursivité mutuelle

Considérons les serpentines de la figure 21.6 (page 261). Il y en a de deux sortes. Les serpentines $\mathtt{E}_n$ ont une hauteur n et se dessinent à partir de leur coin inférieur ouest, en traçant un trait horizontal d'ouest en est puis un trait vertical pour passer à l'est de l'étage supérieur. Les serpentines $\mathtt{W}_n$ ont une

hauteur n et se dessinent à partir de leur coin inférieur est, en traçant un trait horizontal d'est en ouest puis un trait vertical pour passer à l'ouest de l'étage supérieur.

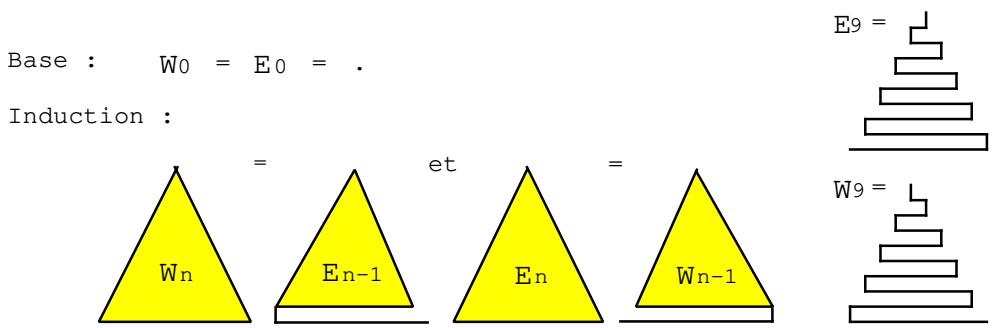


FIG. 21.6 – *Règle de construction récurrente d'une serpentine*

Les procédures `serpentineE` et `serpentineW` dans le programme ci-dessous dessinent respectivement une serpentine E_n et W_n, leur paramètre formel `n` ayant pour valeur effective la hauteur n de la serpentine à dessiner.

```
from LibRobot import *
# Dessiner des serpentines
# Construire une serpentine de hauteur n en partant de
# son coin inférieur ouest
def serpentineE(n):
    if n > 0:
        # Construire la base de largeur n, d'ouest en est
        pe(); avf(n); pn(); av()
        # Construire la serpentine supérieure de hauteur
        # n - 1 en partant de son coin inférieur est
        serpentineW(n - 1)
# Construire une serpentine de hauteur n en partant de
# son coin inférieur est
def serpentineW(n):
    if n > 0:
        # Construire la base de largeur n, d'est en ouest
        pw(); avf(n); pn(); av()
        # Construire la serpentine supérieure de hauteur
        # n - 1 en partant de son coin inférieur ouest
        serpentineE(n - 1)
lg(6); eb(); ad(); serpentineW(18); eb(); ag()
serpentineE(18); st()
```

Une serpentine `E`$_0$ ou `W`$_0$ de hauteur nulle étant vide, les procédures `serpentineE` et `serpentineW` ne font rien quand la valeur n de leur paramètre `n` est négative ou nulle. Pour dessiner une serpentine `E`$_n$ ou `W`$_n$ dont la hauteur $n \geq 1$ est la valeur effective de leur paramètre formel `n`, les procédures `serpentineE` et `serpentineW` dessinent la base puis au-dessus une serpentine `W`$_{n-1}$ ou `E`$_{n-1}$ dont la hauteur est la valeur de `n - 1`. Comme la procédure `serpentineE` appelle la procédure `serpentineW` et que la procédure `serpentineW` appelle la procédure `serpentineE`, on dit que ces procédures sont *mutuellement récursives*.

Quand on utilise des procédures mutuellement récursives en Python, il faut respecter la règle que *tout identificateur doit être déclaré avant d'être utilisé*. Pourtant le corps de `serpentineE` appelle `serpentineW`, qui est déclarée après elle. C'est permis parce que Python ne cherche le sens du nom `serpentineW` qu'au moment où il exécute cet appel, c'est-à-dire pendant l'exécution du programme principal `serpentineW(18)`, alors que les deux procédures sont déjà déclarées. Il suffit donc de déclarer les deux procédures l'une après l'autre, *avant* le premier appel de l'une d'elles. Chacune peut alors appeler l'autre (et elle-même). Si au contraire on appelait `serpentineE` entre les deux déclarations, l'exécution s'arrêterait avec le message '`NameError: name 'serpentineW' is not defined`' (le nom `serpentineW` n'est pas défini).

De manière générale, les déclarations de procédures mutuellement récursives ont la forme suivante :

Déclaration de procédures mutuellement récursives :

```
# Spécification de la première procédure
def nomDeLaProcedure1 [a](paramFormel1^1 [b], paramFormel1^2, ...):
    ... Instructions du corps de la procédure [c d] (indentées) ...

# Spécification de la seconde procédure
def nomDeLaProcedure2(paramFormel2^1, paramFormel2^2, ...):
    ... Instructions du corps de la procédure [e] (indentées) ...

# Instructions du programme (appels des procédures)
```

[a] Un nom de procédure est un identificateur.
[b] Un paramètre formel est un identificateur.
[c] Y compris les appels des procédures `nomDeLaProcedure`$_1$ et `nomDeLaProcedure`$_2$.
[d] Les expressions figurant dans ces instructions peuvent utiliser les paramètres formels `paramFormel`$_1^1$, `paramFormel`$_1^2$, ... de la procédure `nomDeLaProcedure`$_1$.
[e] Les expressions figurant dans ces instructions peuvent utiliser les paramètres formels `paramFormel`$_2^1$, `paramFormel`$_2^2$, ... de la procédure `nomDeLaProcedure`$_2$.

On peut de même déclarer trois procédures mutuellement récursives ou plus, les unes après les autres.

Exercice 42 *L'exercice consiste à dessiner des* fractales *[2] [27], c'est-à-dire des dessins que l'on retrouve à l'infini, en réduction à l'intérieur d'eux-mêmes.*

Écrire une procédure Python récursive pour construire une courbe de Von Koch[1]*, comme indiqué à la de la figure 21.7 (page 265). La courbe K_0 est un segment de droite. La courbe K_1 est obtenue en divisant le segment K_0 en trois segments égaux et en remplaçant celui du milieu par un triangle équilatéral sans base. La courbe K_n est obtenue de même en divisant en trois chacun des segments de la courbe K_{n-1}. Écrire un programme de dessin d'un flocon de Von Koch, obtenu en fragmentant ainsi vers l'extérieur les côtés d'un triangle équilatéral. Faire de même pour un anti-flocon de Von Koch pour lequel la fragmentation se fait vers l'intérieur du triangle équilatéral*

1. Von Koch, mathématicien suédois, 1870–1924.

initial. Utiliser diverses fragmentations d'un segment pour obtenir d'autres types de flocons comme ceux de la figure 21.10 (page 266).

_ *Programmer les dessins des motifs récursifs de la figure 21.1 (page 254).*

_ *Écrire un programme Python de dessin des carrés (figure 21.11 page 267), triangles (figure 21.12 page 268) et pentagones (figure 21.13 page 269) de Sierpinski*[2]*.*

_ *Dessiner les végétaux de la figure 21.14 (page 270).*

_ *Dessiner les carrés de Sedgewick [39] de la figure 21.15 (page 271).*

_ *Dessiner la courbe du dragon de la figure 21.16 (page 272) selon les règles de construction de la figure 21.8 (page 265). La courbe du dragon d'ordre n est celle que l'on observe en regardant par la tranche une feuille de papier qui a été repliée n fois sur elle-même puis dépliée à angles droits.*

_ *Dessiner la courbe de Hilbert*[3] *de la figure 21.17 (page 273) selon les règles de construction de la figure 21.9 (page 265).*

_ *Dessiner la courbe de Peano*[4] *de la figure 21.19 (page 275) selon les règles de construction de la figure 21.18 (page 274).*

_ *Écrire une procédure Python qui écrit en base 2 un nombre passé en paramètre en remarquant que l'écriture binaire $\beta(n)$ d'un nombre n est définie par :*

- $\beta(-n) = -\beta(n)$ *si* $-n < 0$ *;*
- $\beta(0) = 0$ *;*
- $\beta(1) = 1$ *;*
- $\beta(n) = \beta(n \texttt{ // } 2)\beta(n \texttt{ \% } 2)$ *si* $n > 1$.

Faire de même en bases 3, 8 et 16 (en utilisant les digits `A`*,* `B`*,* `C`*,* `D`*,* `E`*,* `F` *pour dénoter respectivement les nombres 10, 11, 12, 13, 14, 15 et "*`ecrire("d") ; translater(1, 0)`*" pour écrire le digit* `d`*).*
□

Corrigé 42 *Nous donnons deux programmes, les autres se trouvant dans le dossier* `livre/21_recursivite` *:*

■ **from** `LibRobot` **import** `*`

2. Waclaw Sierpinski, mathématicien polonais, 1882–1969.
3. David Hilbert, mathématicien allemand, 1862–1943.
4. Giuseppe Peano, mathématicien italien, 1858–1932.

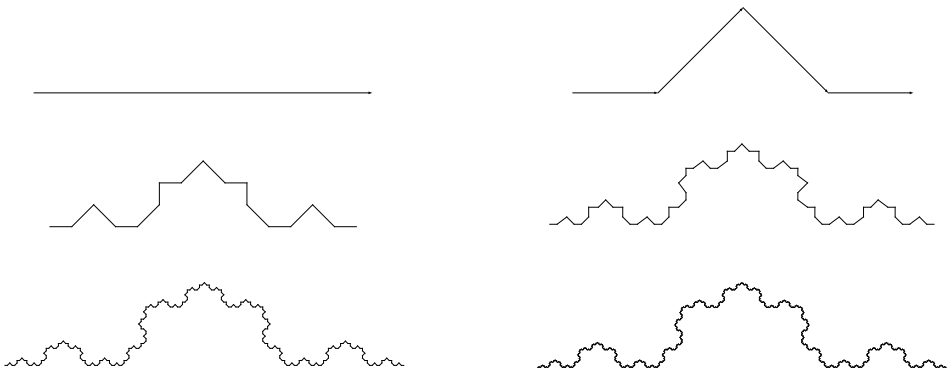


FIG. 21.7 – *Courbe de Von Koch*

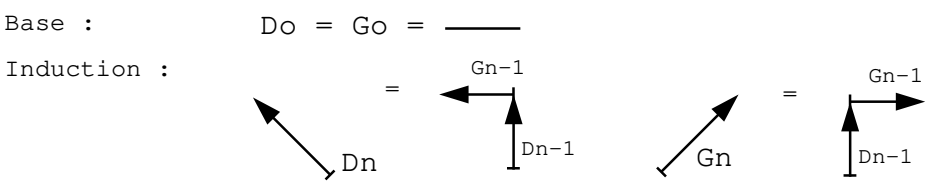


FIG. 21.8 – *Règles de construction de la courbe du dragon*

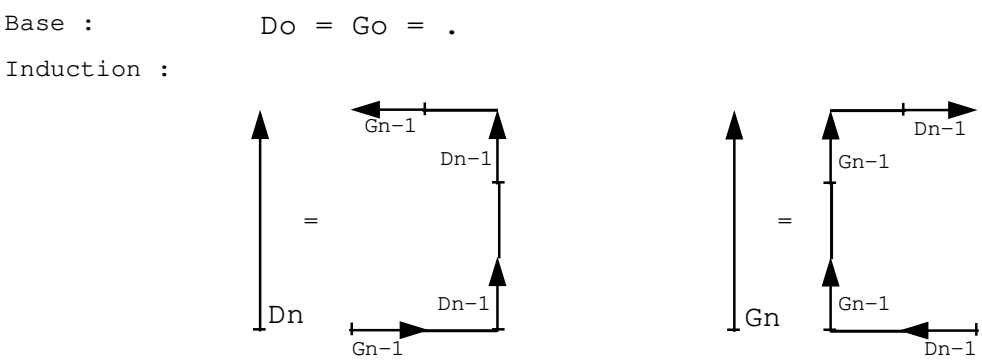


FIG. 21.9 – *Règles de construction des courbes de Hilbert*

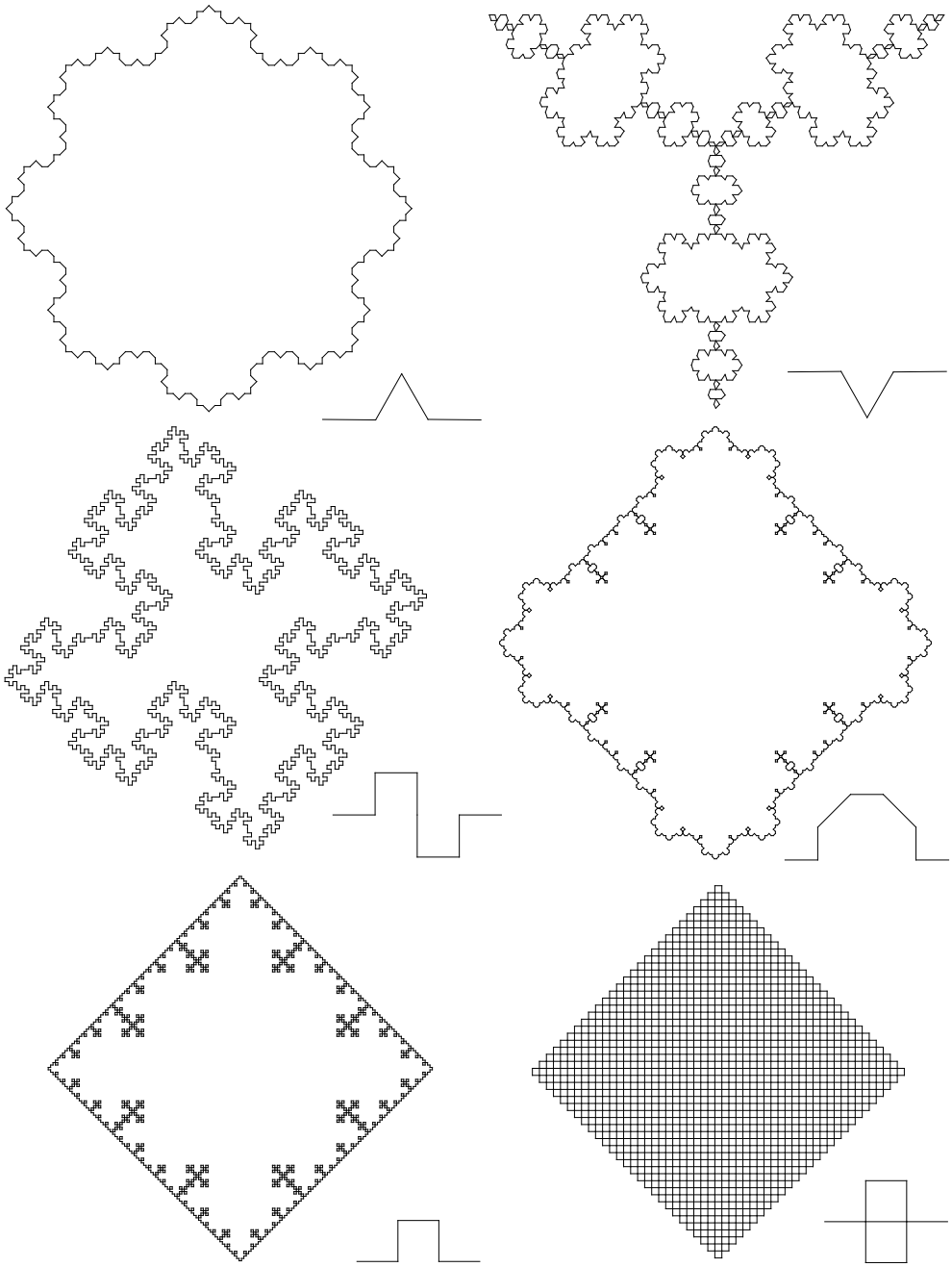


FIG. 21.10 – *Flocons de Von Koch*

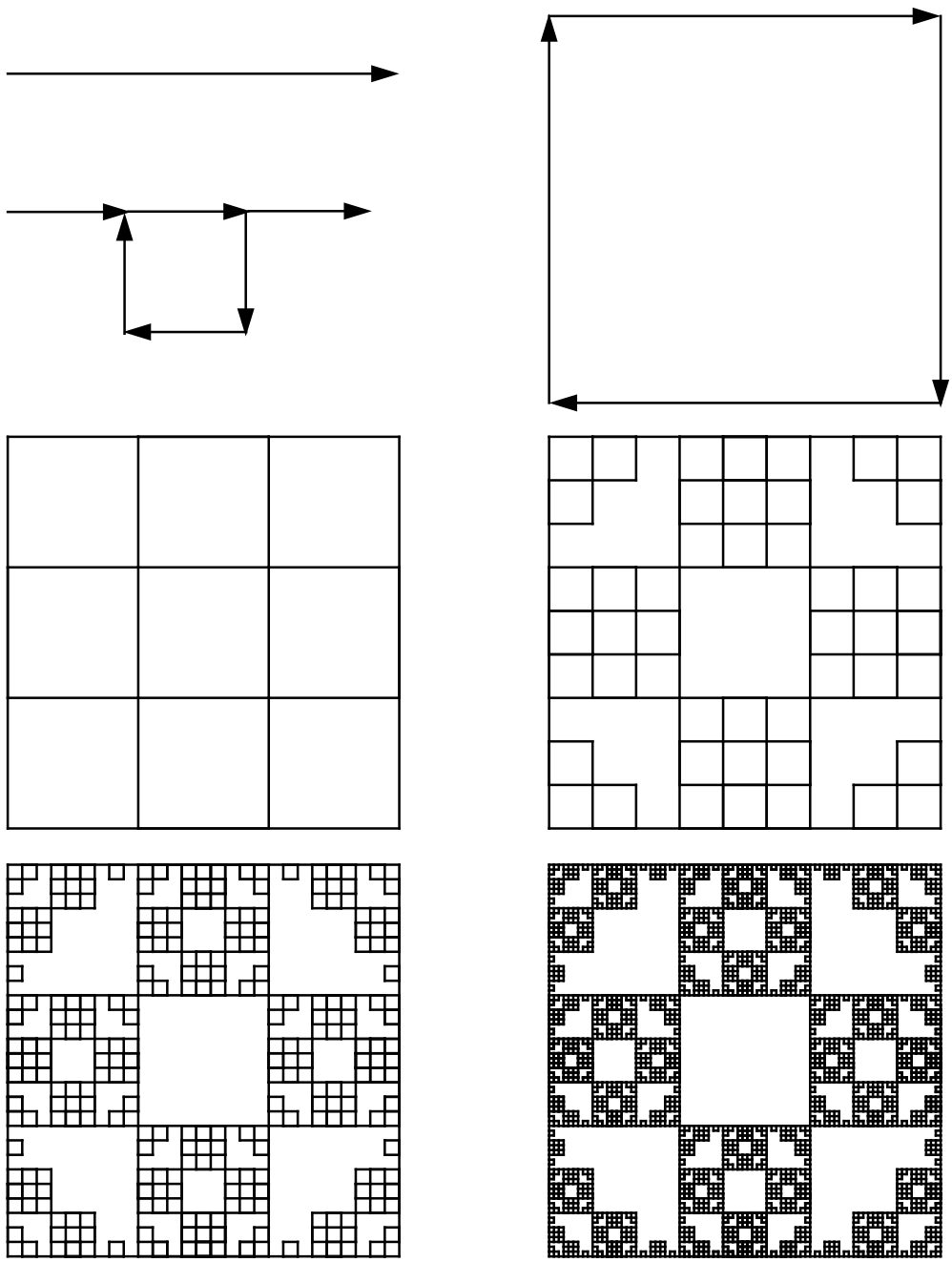


FIG. 21.11 – *Carrés de Sierpinski*

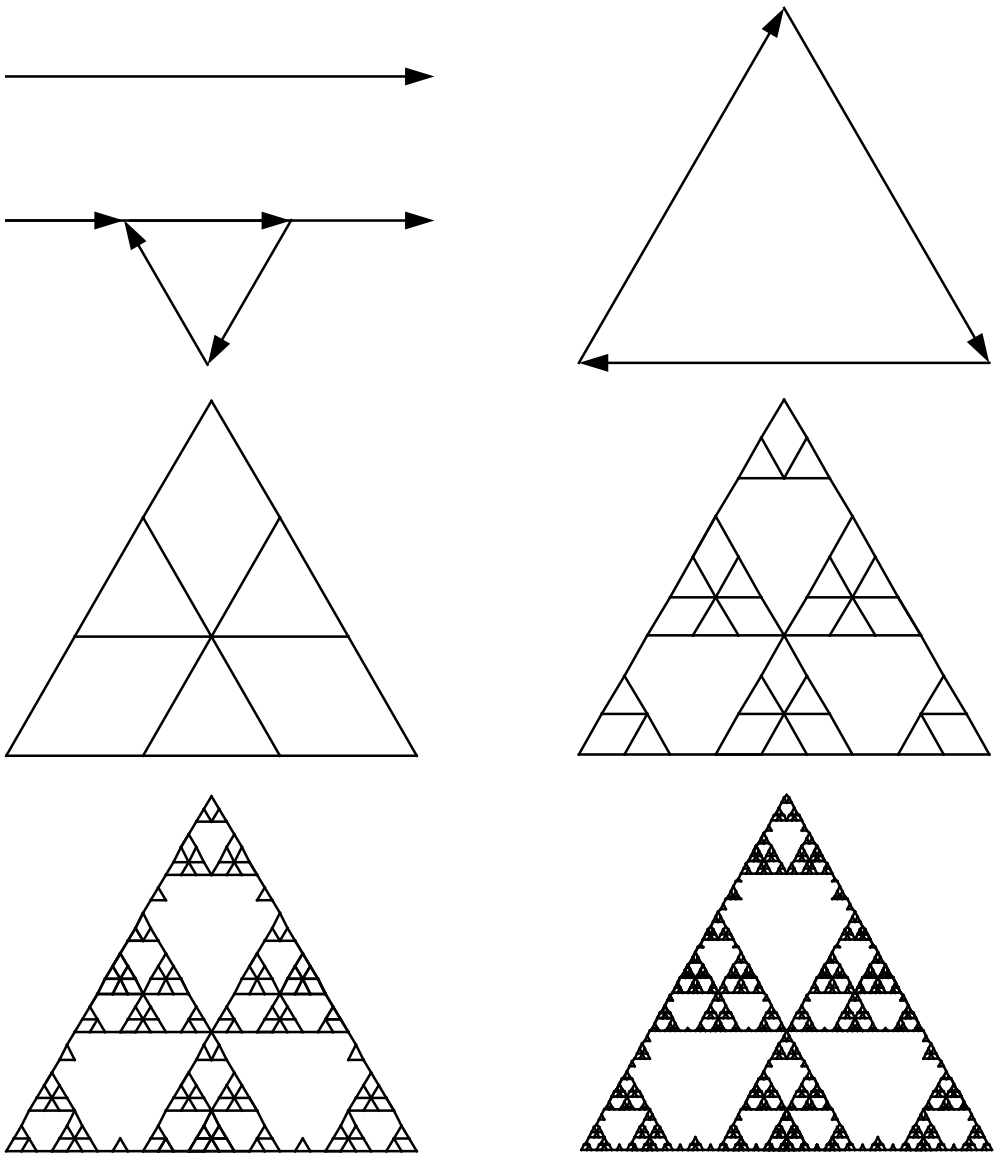


FIG. 21.12 – *Triangles de Sierpinski*

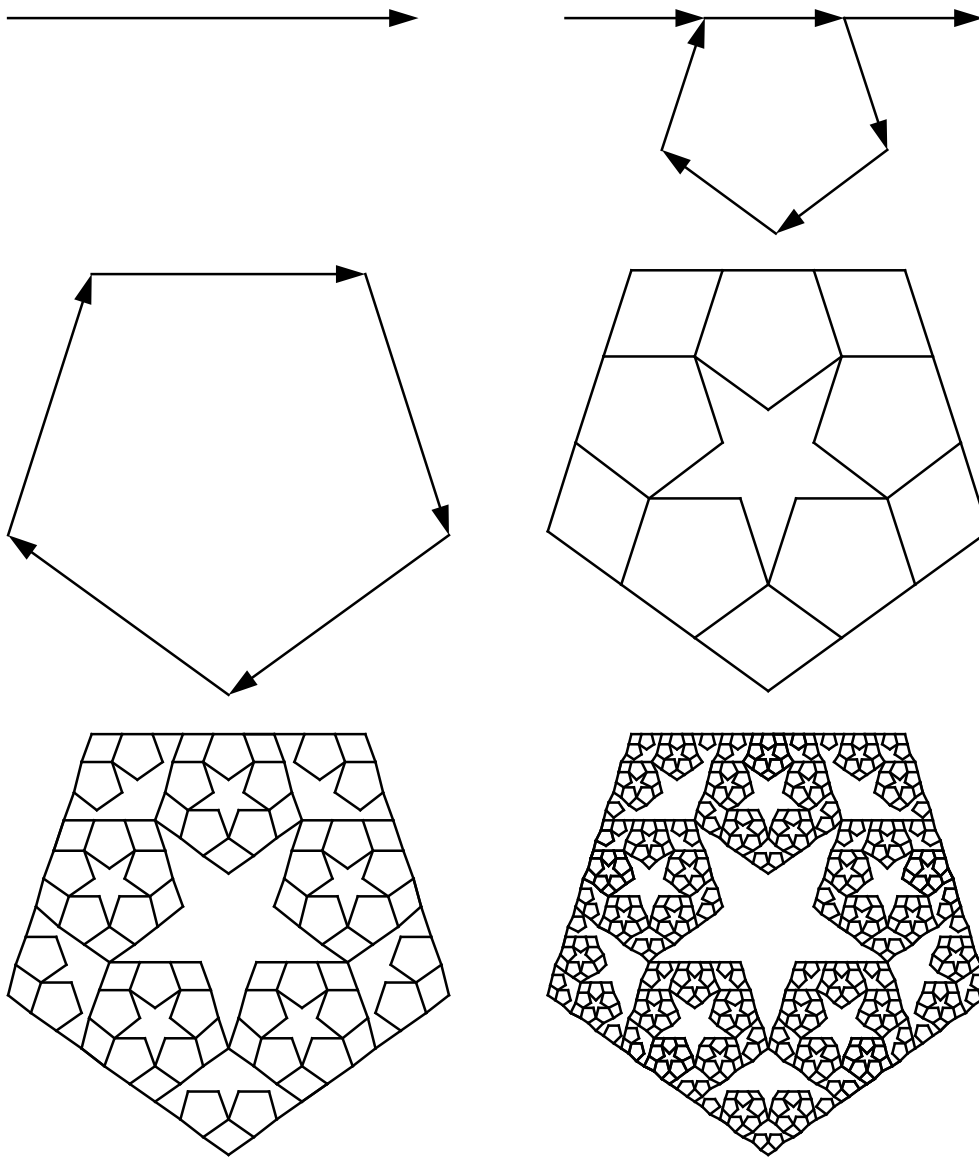


FIG. 21.13 – *Pentagones de Sierpinski*

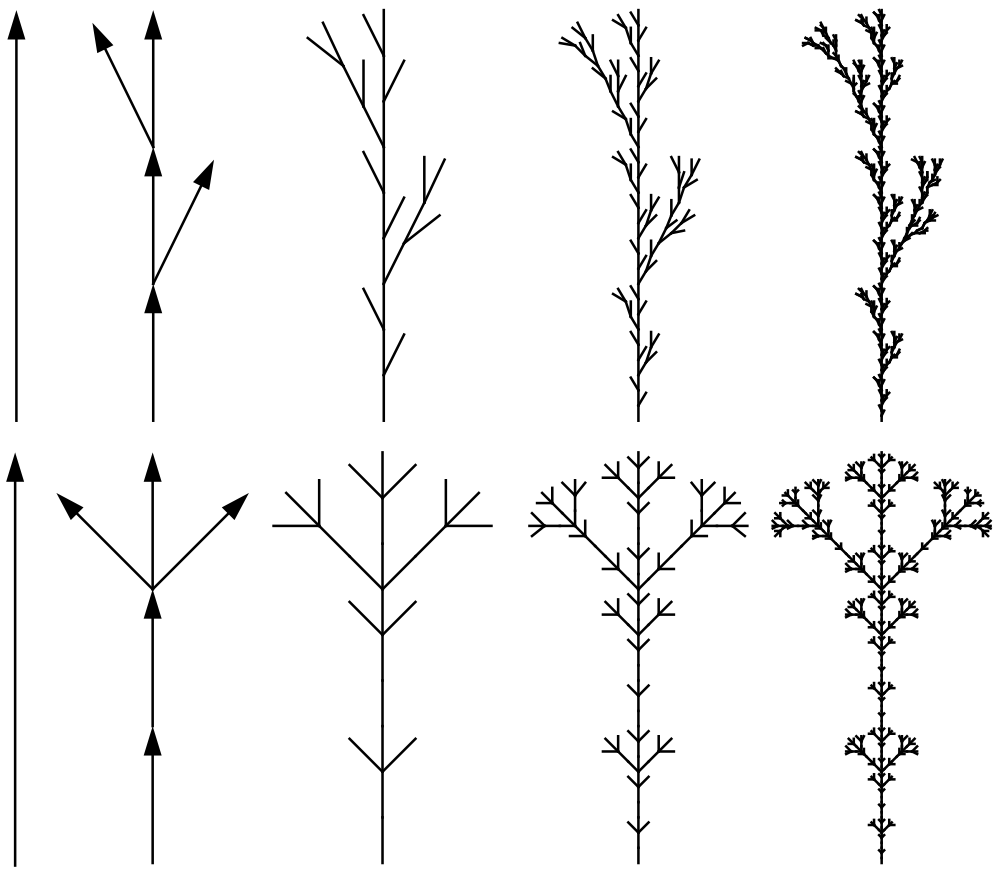


FIG. 21.14 – *Végétaux fractals*

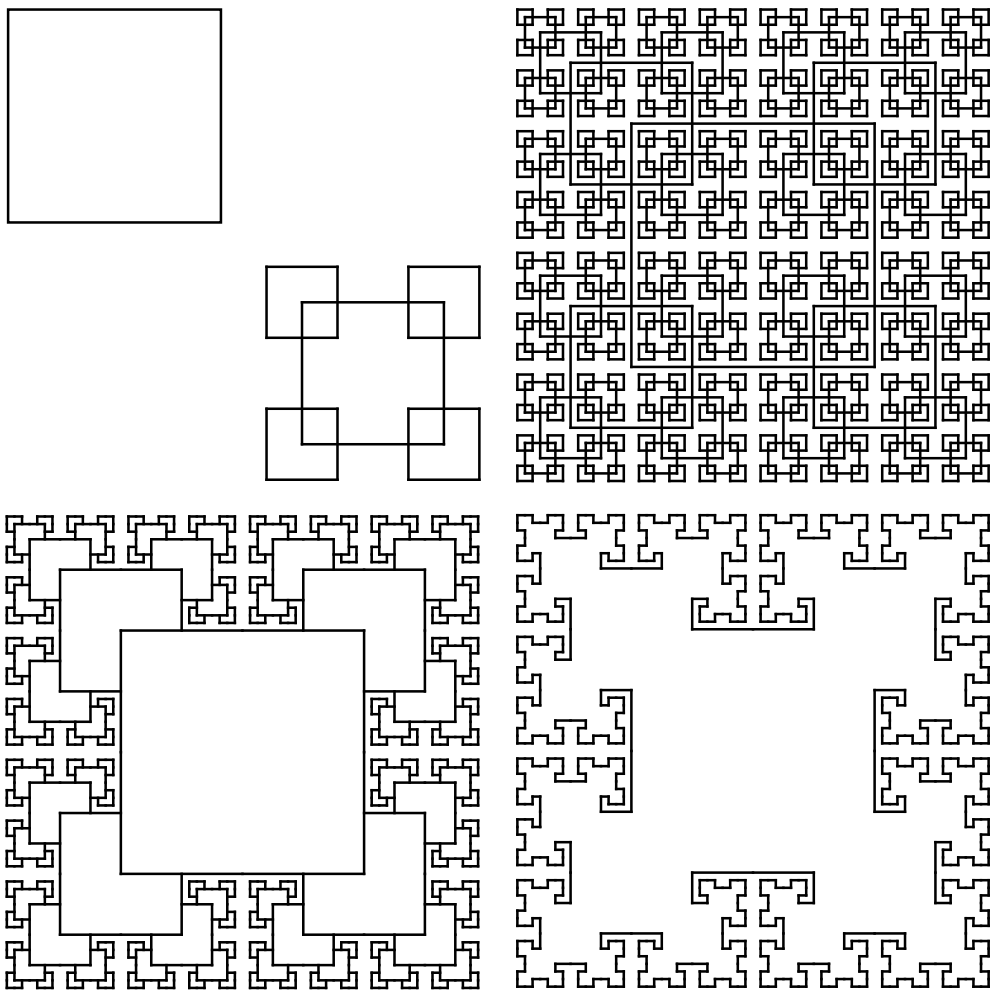


FIG. 21.15 – *Étoile fractale de Sedgewick*

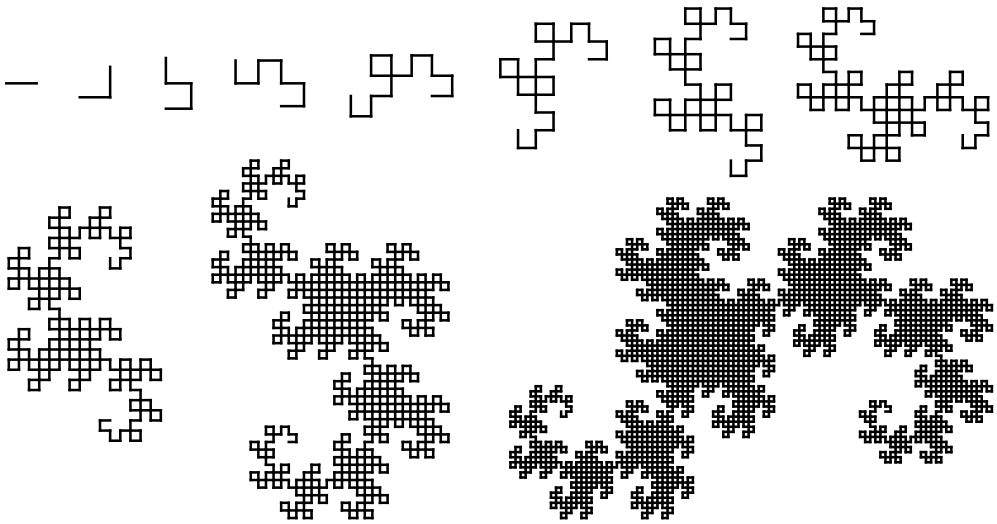


FIG. 21.16 – *Courbes du dragon d'ordre 1, 2, ..., 8, 10 et 12*

```
# Dessiner une courbe de Von Koch
def segment(n):
    if n == 0:
        av()
    else:
        segment(n - 1)
        phtg()
        segment(n - 1)
        pqtd()
        segment(n - 1)
        phtg()
        segment(n - 1)
lg(6); ag(); pe(); segment(3); st()
```

-
```
from LibRobot import *
# Écrire un nombre en base 2
def decimalEnBinaire(n):
    if n < 0:
        ecrire("-"); translater(1, 0)
        decimalEnBinaire(-n)
    elif n == 0:
        ecrire("0"); translater(1, 0)
    elif n == 1:
        ecrire("1"); translater(1, 0)  # n >= 2
```

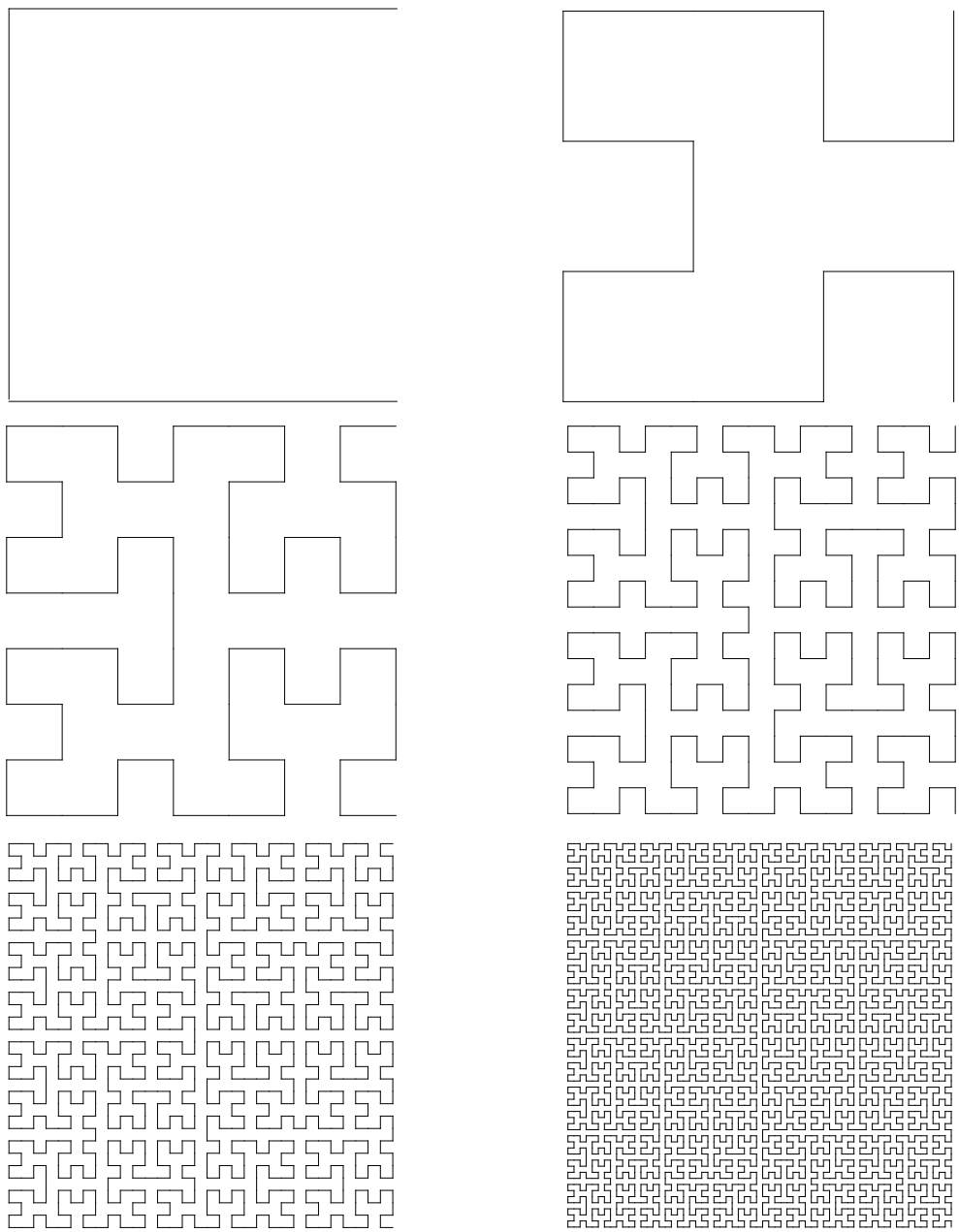


FIG. 21.17 – *Courbes de Hilbert d'ordres 1 à 6*

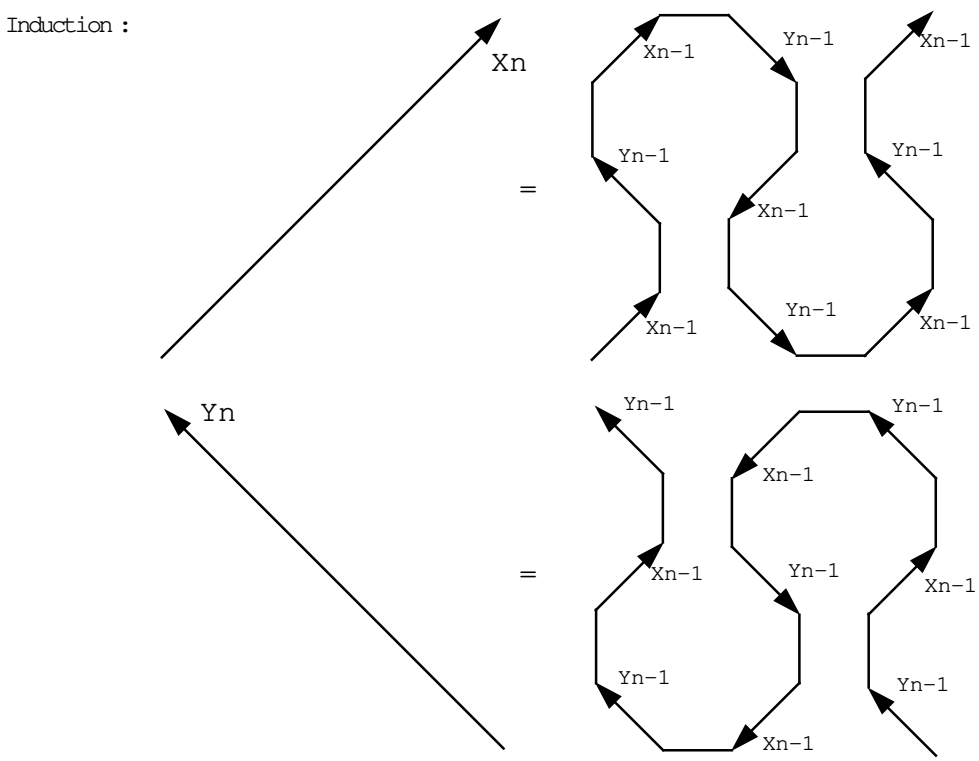


FIG. 21.18 – *Règles de construction des courbes de Peano*

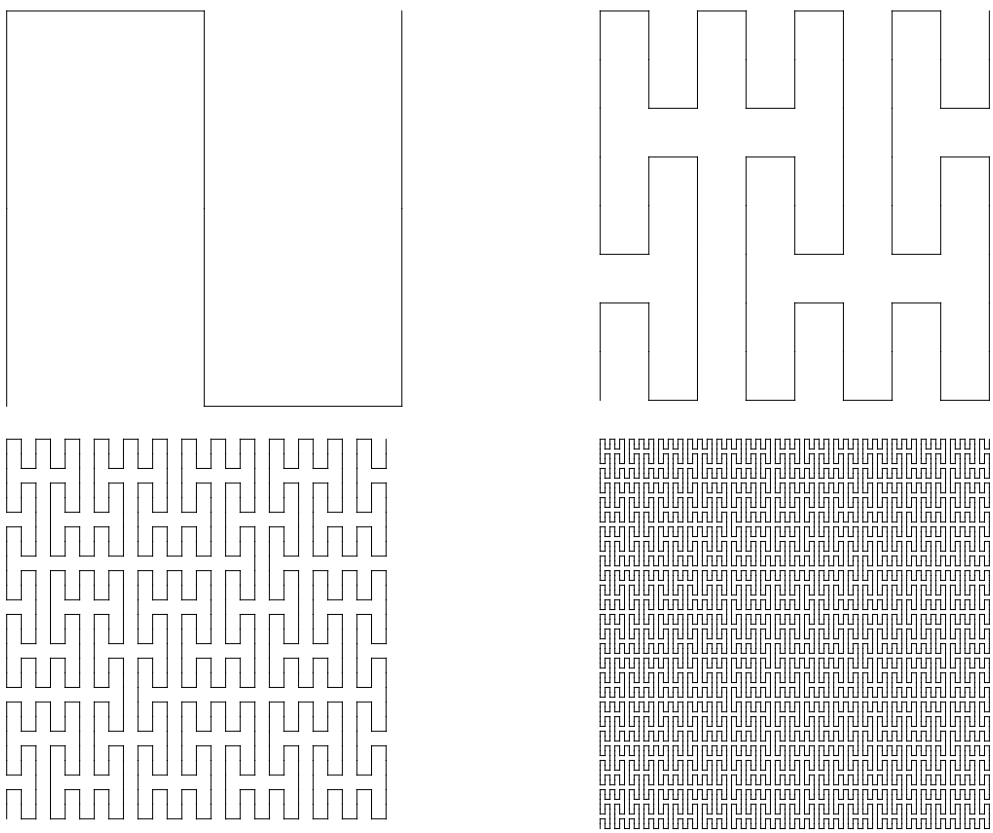


FIG. 21.19 – *Courbes de Peano*

```
    else:
        decimalEnBinaire(n // 2)
        decimalEnBinaire(n % 2)
lg(7); decimalEnBinaire(123456789); st()
```

□

Addenda

Les langages ALGOL60 [29] puis LISP [28] furent les premiers à introduire l'usage de la récursivité. La polémique initiale sur l'efficacité des procédures récursives est maintenant dépassée, les techniques de compilation étant bien au point. L'idée de récursivité fut à la base de nombreux progrès en informatique, que ce soit les structures de données récursives (comme les arbres), les algorithmes récursifs (comme les algorithmes rapides de tri) ou plus récemment, la programmation fonctionnelle, dont OCaml est un exemple (Python en reprend certaines idées).

La façon la plus simple de comprendre une procédure récursive est d'imaginer qu'à chaque appel l'ordinateur exécute une nouvelle *copie* de la procédure.

Pour montrer que cette succession d'appels se termine, il suffit de montrer qu'il existe une fonction entière de la valeur des paramètres qui est positive et décroit strictement lors d'appels successifs. Par exemple la procédure `segment` du programme `SegmentDeVonKoch` (page 264) doit être appelée avec un paramètre `n` positif ou nul. Si c'est le cas, les appels récursifs sont de la forme `segment(n - 1)` avec `n` > 0 de sorte que lors d'appels successifs les paramètres effectifs décroissent strictement en restant positifs. Ceci n'est pas possible indéfiniment. Par conséquent la procédure `segment` ne boucle pas quand elle est appelée avec un paramètre initial positif ou nul.

Les *fonctions* sont aux expressions ce que les procédures sont aux instructions. En Python, une déclaration de fonction s'écrit comme une déclaration de procédure, mais son corps contient une instruction **return** *e* (de l'anglais *return*, rendre) qui termine l'exécution de la fonction et rend comme résultat la valeur de l'expression *e*. Les procédures sont simplement des fonctions sans **return** et dont on ne s'intéresse pas au résultat, qui est la valeur spéciale **None** (rien).

Par exemple, une constante est une fonction sans paramètres de sorte qu'une déclaration comme :

```
tailleGrille = 15
```

peut être remplacée par une déclaration de fonction :

```
def tailleGrille(): return 15
```

qui s'utilise en écrivant `tailleGrille()`, comme `valLgX()`. Comme second exemple considérons le calcul de 2^n avec $n \geq 0$ en observant pour la base que $2^0 = 1$ et pour l'induction que $2^n = 2 * 2^{n-1}$:

```
def deuxPuissance(n):
  if n == 0:
    return 1
  else:
    return 2 * deuxPuissance(n - 1)
```

Après cette déclaration, l'expression `deuxPuissance(2)` vaut 4 et l'expression `deuxPuissance(deuxPuissance(2))` vaut 16 (le programme `DeuxPuissance` du dossier `livre/21_recursivite` dessine la valeur de `deuxPuissance(16)`). Ici **if**. . .**else** choisit entre deux instructions **return**, donc entre les deux valeurs que peut rendre la fonction.

En Python on peut utiliser les fonctions trigonométriques `sin` (sinus), `cos` (cosinus), `tan` (tangente) et `atan` (arc tangente). Les angles sont exprimés en radians (un angle de d degrés vaut `d / 180 * pi` radians, la constante `pi` valant $\pi \simeq 3.14159$). Comme pour les fonctions `exp` (exponentielle) et `log` (logarithme népérien), le paramètre est un nombre (entier ou rationnel) et le résultat est de type `float`. Ces fonctions et la constante `pi` sont fournies par la bibliothèque du $\mathcal{R}\underline{o}\underline{b}ot$ (elles viennent du module `math` de Python). Pour la puissance, on utilise `a ** x` qui vaut $a^x = e^{x \ln a}$. On pourrait définir soi-même la fonction tangente par :

```
def tg(x): return sin(x) / cos(x)
```

Quand `cos(x)` vaut `0.0`, la division provoque une erreur et l'arrêt du programme avec le message 'ZeroDivisionError: float division by zero' (division d'un rationnel par zéro). À cause des erreurs d'arrondi, cela n'arrive pas pour `tg(pi / 2)` : `cos(pi / 2)` vaut `6.123233995736766e-17` et non `0.0`, de sorte que `tg(pi / 2)` vaut `1.633123935319537e+16`, un très grand nombre au lieu de l'infini.

22

Variables

Nous avons utilisé des noms pour désigner des valeurs : les constantes, les paramètres des procédures et les compteurs des boucles “for”. Jusqu'à présent, une fois fixée, la valeur désignée par un tel nom ne changeait plus. Les *variables* permettent au contraire de mémoriser des résultats intermédiaires dans les calculs et de les modifier au cours de l'exécution du programme.

22.1 Variables et affectation

Une variable est utilisée pour conserver une valeur dans la mémoire de l'ordinateur. En Python, une variable est créée par une *déclaration de variable*, qui indique le nom de la variable (qui est un identificateur) et sa valeur initiale, placée après le signe `=` :

```
b = True
i = 0
r = 0.0
s = ""
```

Cela s'écrit exactement comme les déclarations de constantes que nous avons utilisées jusqu'à présent : en Python, une constante n'est rien d'autre qu'une variable dont on ne change jamais la valeur. C'est Python qui décide quelle partie de la mémoire et quel codage sera utilisé pour représenter la valeur de la variable, selon le type de cette valeur : `b` est une variable booléenne, `i` une variable entière, `r` une variable rationnelle et `s` une variable contenant une chaîne de caractères (ici la chaîne vide). En Python, une variable a donc toujours une valeur, fixée dès sa création. On peut utiliser cette valeur dans une expression (comme `2 * i + 1`) en la notant simplement par le nom de

la variable. Il ne faut pas utiliser une variable avant de l'avoir créée : si le programme commence par `i = i + 1` sans avoir d'abord donné une valeur initiale à `i`, l'exécution s'arrête avec le message '`NameError: name 'i' is not defined`' (le nom `i` n'est pas défini). L'instruction d'*affectation* permet de changer la valeur d'une variable. Par exemple l'instruction :

```
b = False
```

affecte la valeur "faux" à la variable booléenne `b`. De manière générale, une instruction d'affectation a la forme suivante :

Instruction d'affectation :

`nomDeVariable` = expression [a]

[a] L'expression peut utiliser l'ancienne valeur de la variable.

Le symbole `=` se lit "devient égal à". Il ne faut pas le confondre avec la comparaison `==` ("est égal à") utilisée dans les conditions des **if** et des **while**. L'exécution de cette instruction consiste à évaluer l'"expression" pour calculer sa valeur qui devient celle de la variable nommée `nomDeVariable`. Le type de la variable est celui de la valeur qu'elle contient :

Types des valeurs :

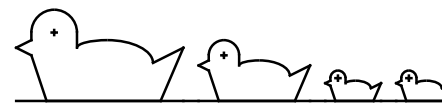
Exemple de valeur	Type	Nom du type
True, **False**	`bool`	booléen
`0`, `1`, `-5`	`int`	entier
`0.0`, `1.5`	`float`	rationnel
`""`, `"abc"`	`str`	chaîne de caractères

En Python, le type d'une variable n'est pas fixé une fois pour toutes : si `i` est une variable entière, l'affectation `i = 1.5` est acceptée et `i` devient une variable rationnelle. C'est rarement une bonne idée, car le lecteur du programme s'y perd et l'erreur n'apparaît que plus tard, par exemple dans `avf(i)` qui exige un entier. Mieux vaut garder à chaque variable le même type et faire les *conversions* explicitement : par exemple `r = float(i)` ou `i = int(r)`.

22.2 Sauvegarde de l'état du robot

Une première utilisation des noms est la sauvegarde de l'état du $\mathcal{R}\!\mathcal{o}\!\mathcal{b}\!\mathcal{o}t$. Imaginons par exemple que nous voulions concevoir une procédure pour faire

un dessin qui nécessite divers changements de la taille de la grille. Pour que l'appel de cette procédure dans un programme ne modifie pas la taille de la grille utilisée dans le programme, il faut sauvegarder cette taille au début de la procédure puis la restituer à la fin. Il suffit de lui donner un nom par une *déclaration locale* `nom = expression` placée dans le corps de la procédure, comme dans l'exemple suivant :



```
from LibRobot import *
# Dessiner des canards

# Dessiner un canard sur une
# grille de taille l sans changer
# la grille originelle
def canard(l):
    # Mémoriser les dimensions de la grille originelle
    lX = valLgX(); lY = valLgY()
    # Dessiner un canard avec une grille de taille l
    lg(l)
    avf(19)  # Ligne de flottaison
    lgX(3 * l); lgY(7 * l); phtg()  # Queue
    av(); pdt(); lgX(4 * l); lgY(2 * l); av()
    lgX(6 * l); lgY(3 * l); p3htd(); vg()  # Dos
    lgX(4 * l); lgY(l); vg(); pdt()
    lg(l); avf(2); lg(3 * l); vg(); vg()  # Tête
    lc(); pqtg(); av(); dc(); pdt(); av()  # Oeil
    pqtg(); bc()
    lgX(2 * l); lgY(l); av(); phtd(); av()  # Bec
    pqtg(); av()
    lgX(2 * l); lgY(6 * l); av()  # Poitrail
    phtg()
    # Placer le robot à l'arrière du canard
    lg(l); avf(18)
    # Restituer la grille originelle
    lgX(lX); lgY(lY)
lg(6); pe(); canard(3); av(); canard(2); av()
canard(1); av(); canard(1); st()
```

Les déclarations `lX = valLgX() ; lY = valLgY()` calculent les valeurs de `valLgX()` et de `valLgY()` au début de la procédure et leur donnent les noms `lX` et `lY`, qui ne peuvent être utilisés que dans la suite du corps de la procédure `canard`. On dit que ce sont des variables *locales* à la procédure : elles n'existent pas en dehors de son corps.

22.3 Compter

Supposons que nous voulions compter le nombre d'itérations dans une boucle "while". On utilise une variable entière :

```
i = 0
```

Avant d'exécuter la boucle "while", la variable doit être initialisée à 0 (elle peut avoir été modifiée depuis sa déclaration) :

```
i = 0
```

Après chaque tour de boucle, la valeur de la variable doit être augmentée de 1 :

```
i = i + 1
```

Supposons que la valeur de la variable `i` soit égale à 5 *avant* l'affectation `i = i + 1`. La valeur de l'expression `i + 1` est $5+1=6$ de sorte que la valeur de la variable `i` *après* l'affectation sera égale à 6. Comme cette instruction est très fréquente, Python permet de l'abréger en `i += 1` ("augmenter `i` de 1"). De même `i -= 1` diminue `i` de 1 et `i *= 2` le double.

Par exemple, ceci peut nous servir pour déterminer la taille de la fenêtre dans laquelle le $\mathcal{R}\circ\mathcal{b}\circ t$ se déplace sur l'écran de l'ordinateur :

```
from LibRobot import *

# Calcul de la taille de la fenêtre de déplacement du
# robot
largeur = 0
hauteur = 0

def largeurFenetreRobot():
    global largeur
    lg(1); lc(); ce(); ag(); pe(); largeur = 0
    while not tb():
        largeur += 1; av()

def hauteurFenetreRobot():
    global hauteur
    lg(1); lc(); ce(); eb(); pn(); hauteur = 0
    while not tb():
        hauteur += 1; av()

largeurFenetreRobot(); hauteurFenetreRobot()
message("(" + str(largeur) + " x " + str(hauteur) + ")")
marquerUnePause()
```

Les variables `largeur` et `hauteur` sont déclarées dans le programme, en dehors des procédures ; on dit qu'elles sont *globales*. Une procédure peut utiliser leur valeur sans rien dire de spécial. Par contre, pour les *modifier*,

il faut le déclarer au début du corps de la procédure par **global** `largeur`. Sans cette déclaration, Python considérerait que l'affectation `largeur = 0` crée une nouvelle variable locale `largeur`, sans rapport avec la variable globale, qui garderait la valeur 0 (et `largeur += 1` sans `largeur = 0` avant provoquerait l'erreur '`UnboundLocalError: local variable 'largeur' referenced before assignment`', la variable locale `largeur` étant utilisée avant d'avoir reçu une valeur).

L'opérateur `+` appliqué à deux chaînes de caractères les met bout à bout et la fonction `str` donne la chaîne de caractères qui représente un nombre (par exemple `str(25)` vaut `"25"`). On ne peut pas mettre bout à bout une chaîne et un nombre : `"(" + largeur` provoque l'erreur '`TypeError: can only concatenate str (not "int") to str`'.

22.4 Entrée d'une valeur au clavier

On peut obtenir une valeur en la tapant au clavier de l'ordinateur. Pour ce faire la fonction `lireEntier()` attend que l'on tape un nombre entier et rend sa valeur. La question posée par le programme (écrite avec `write`, voir ci-dessous) s'affiche dans la barre du bas de la fenêtre du *Robot*, et la réponse que l'on tape au clavier s'inscrit à sa suite (on peut aussi la taper dans le terminal d'où le programme a été lancé). On écrit par exemple :

```
write("n = ")
n = lireEntier()
```

pour donner à la variable `n` la valeur tapée. On peut corriger les fautes de frappe avec la touche ⟵ d'effacement. Il faut terminer l'entrée de la valeur par la frappe de la touche ↩ de retour à la ligne. Si ce que l'on a tapé n'est pas un entier, le programme le redemande. Sur un clavier français, on peut taper les chiffres avec ou sans la touche des majuscules.

De même `lireReel()` rend un nombre rationnel (de type `float`), `lireChaine()` rend la chaîne de caractères tapée (sans apostrophes ni guillemets) et `lireCar()` rend le caractère de la première touche frappée dans la fenêtre du *Robot* (sans attendre la touche de retour à la ligne). En Pascal, une seule procédure `readln` permettait de lire une valeur de n'importe quel type. En Python, chaque fonction de lecture rend une valeur d'un type bien précis (la fonction `input` de Python rend toujours une chaîne de caractères, qu'il faudrait convertir en entier avec `int` ; `lireEntier` le fait pour nous). Par contre certaines fonctions, comme `write`, `str` ou `abs`, acceptent des paramètres de

types différents : on dit qu'elles sont *polymorphes* (alors que d'autres fonctions, comme `lireEntier` ou `sqrt`, sont *monomorphes* en ce sens que leurs paramètres et leur résultat ont toujours le même type).

L'instruction `readln()` permet d'attendre que la touche ⏎ de retour à la ligne soit enfoncée. Elle est donc utile pour laisser le temps de lire ce que le programme a écrit avant la fin de son exécution.

22.5 Affichage de la valeur d'une expression

Pour afficher des textes dans le terminal (et dans la barre du bas de la fenêtre du $\mathcal{R}$o$\mathcal{b}$ot), on peut utiliser l'instruction `write(`t_1`, `t_2`, ...)` qui écrit t_1, t_2, ... les uns à la suite des autres, sur la même ligne. L'instruction `writeln(`t_1`, `t_2`, ...)` fait de même puis passe à la ligne suivante. Les t_i peuvent être des chaînes de caractères ou des expressions entières ou rationnelles, dont la valeur est écrite : si `x` vaut 3, `writeln("x = ", x)` écrit `x = 3`. On peut aussi transformer soi-même la valeur d'une expression E en chaîne de caractères avec `str(`E`)`, comme le fait le programme ci-dessous. Par exemple ce programme lit un entier n puis écrit sa factorielle $n! = (1{*}2{*}\ldots{*}(n{-}1){*}n)$. Pour redemander la valeur de n tant qu'elle est incorrecte, il utilise une fonction récursive `lireN` (voir le chapitre 21) qui rend la valeur lue :

```
from LibRobot import *

# Lire un entier n et écrire sa factorielle n!
nmin = 0
nmax = 20

# Lire au clavier la valeur de n, en recommençant
# tant que l'on n'a pas nmin <= n <= nmax
def lireN():
   write("n = ")
   n = lireEntier()
   if nmin <= n and n <= nmax:
      return n
   else:
      writeln("n doit être compris entre ", str(nmin), " et ",
            str(nmax), " ! Recommencer.")
      return lireN()

n = lireN()
# Calculer f = n!
f = 1
for i in range(1, n + 1):
```

```
    f = f * i
# Écrire f
writeln("n! = ", str(f))
readln()
```

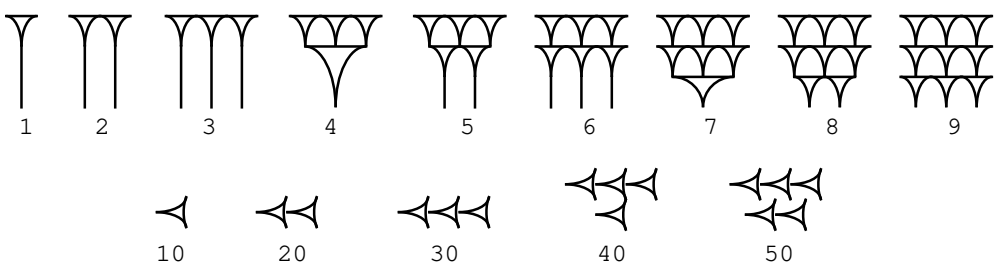
La variable `n` de la fonction `lireN` est locale à cette fonction : c'est une autre variable que la variable `n` du programme, qui reçoit la valeur rendue par `lireN()`. La valeur maximale de n est fixée à 20, comme dans la version OCaml de ce programme où 21! dépasse le plus grand entier. En Python, les entiers n'ont pas de limite : on pourrait calculer exactement 100!, qui s'écrit avec 158 chiffres.

Exercice 43

_ *Écrire un programme qui dessine un cadre le plus près possible du bord de la fenêtre de déplacement du* $\mathcal{R}\text{o}\underset{\smile}{\text{b}}\text{o}t$.

_ *Écrire un programme qui esquisse les dessins à compléter de la figure 1.4 (page 7).*

_ *La numération babylonienne [16], qui date d'environ 2000 ans av. J.-C., est en base 60. C'est donc une numération sexagésimale tout comme celle que nous utilisons encore pour noter l'heure. Les unités, de 1 à 59, sont notées de manière additive en accumulant des "clous" (pour 1) et des "chevrons" (pour 10) regroupés et superposés comme indiqué sur la figure ci-dessous.*

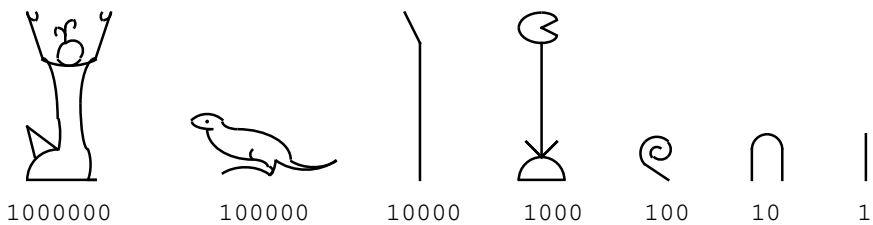


Au delà de 60, la notation est positionnelle, la valeur d'un chiffre occupant la $p^{\text{ème}}$ *position (comptée de droite à gauche à partir de zéro) étant multipliée par* 60^p. *L'absence d'unités sexagésimales en position médiale (que nous noterions par 0) fut d'abord marquée par un espace puis, vers le* $3^{\text{ème}}$ *siècle av. J.-C., par "deux clous inclinés". Les Babyloniens ont donc mis 15 siècles pour inventer une notation pour le zéro sans toutefois en comprendre la valeur calculatoire ! Par exemple le nombre* $3657 = 1 \times 60^2 + 0 \times 60^1 + 57 \times 60^0$ *est représenté par la juxtaposition des chiffres* 1, 0 *et* $57 = 50 + 7$ *:*

"zéro" babylonien 3657

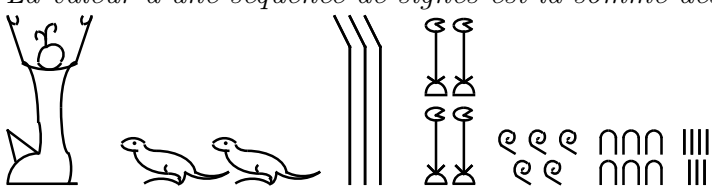
Écrire un programme Python de conversion d'un nombre positif quelconque dans la numération babylonienne.

_ *La numération égyptienne (3ème millénaire av. J.-C.) est décimale et additive [16]. Elle utilise des hiéroglyphes pour noter les puissances de 10 (le trait vertical pour 1, l'anse pour 10, la spirale pour 100, le lotus pour 1000, l'index pour 10 000, le têtard pour 100 000 et le génie pour 1 000 000) :*



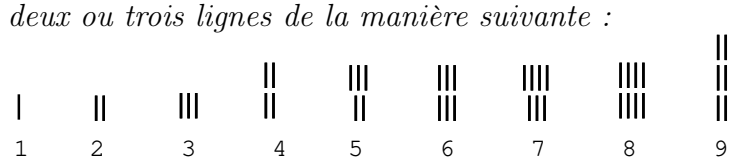
1000000 100000 10000 1000 100 10 1

La valeur d'une séquence de signes est la somme des valeurs de ces signes :



1 234 567

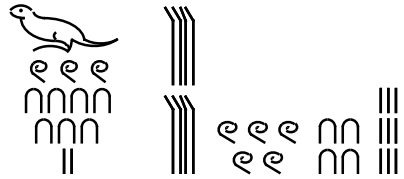
On écrit de gauche à droite ou de droite à gauche, la lecture se faisant dans le sens où sont orientés les hiéroglyphes. Les signes identiques se regroupent sur une, deux ou trois lignes de la manière suivante :



1 2 3 4 5 6 7 8 9

Entre -2000 et -1600 ans av. J.-C., une représentation hybride (additive et partiellement positionnelle) est apparue. Un nombre inférieur à 1000 situé avant, au dessus ou au dessous du signe du génie du million, du têtard, de l'index ou du lotus représente le produit de ce nombre et de la valeur de

ce hiéroglyphe. Par exemple, le nombre 37 280 549 peut se décomposer en $((372 * 100000) + 80549)$ *et s'écrit :*



37 280 549

Écrire un programme Python de conversion d'un nombre dans la numération égyptienne.

_ *La numération acrophonique grecque [16] fut utilisée à partir du Vème siècle av. J.-C. à Athènes. C'est une numération additive décimale utilisant la base auxiliaire 5 pour diminuer les répétitions. Les chiffres supérieurs à un sont notés par des lettres (ou combinaisons de lettres) correspondant chacune à l'initiale (ou à l'abréviation) du nom du nombre correspondant :*

Écrire un programme Python de conversion d'un nombre inférieur à 100 000 dans la numération grecque. Pour représenter les grands nombres, utiliser la numération acrophonique pour les monnaies :

Oboles :

1

Drachmes (1 drachme = 6 oboles) :

1 5 10 50 100 500 1000 5000

Talents (1 talent = 6 000 drachmes) :

1 5 10 50 100 500 1000 5000

Par exemple le nombre 239 686 966 se représente par :

6657 Talents 5827 Drachmes 4 Oboles

239 686 966

_ *La numération romaine [16] est additive et soustractive (tout signe placé à gauche d'un signe de valeur supérieure s'en soustrait). Ses chiffres sont bien connus :*

1 5 10 50 100 500 1000

Pour les grands nombres, la barre horizontale multiplie par 1 000 la valeur du nombre qu'elle surligne et le rectangle sans base multiplie par 100 000 la valeur du nombre qu'il enferme, comme dans l'exemple 135 792 468 ci-dessous :

1 3 5 7 9 2 4 6 8

Écrire un programme Python de conversion d'un nombre dans la numération romaine.

_ *Écrire un programme de dessin de la courbe de Sierpinski à un ordre quelconque de la figure 22.1 (page 290).*

_ *Un carré magique d'ordre n est un tableau carré de n^2 nombres tel que la somme des nombres de chaque ligne, de chaque colonne et de chaque diagonale est la même. Des exemples sont donnés à la figure 22.2 (page 290). Notons $C_{i,j}$ le nombre figurant à la $i^{ème}$ ligne et la $j^{ème}$ colonne du carré comptées à partir de 1 et numérotées de bas en haut pour les lignes et de gauche à droite pour les colonnes. Écrire un programme Python qui construit des carrés magiques d'ordre n impair en commençant par placer 1 dans $C_{i,j} = C_{\frac{n+1}{2},n}$ puis affecte les nombres 2, 3, ..., n^2 aux éléments $C_{i,j}$ obtenus en augmentant i et j de 1 modulo n à chaque fois pendant $n-1$ pas, et en diminuant j de 1 tout en laissant i inchangé à chaque $n^{ème}$ pas.*

□

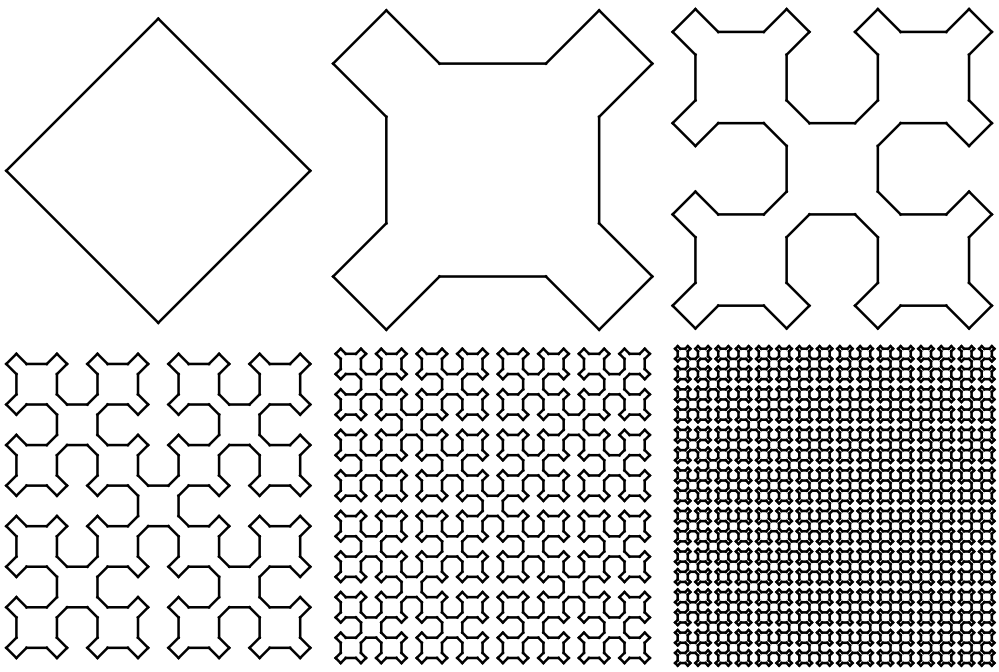


FIG. 22.1 – *Courbes de Sierpinski d'ordres 0 à 5*

8	1	6
3	5	7
4	9	2

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

30	39	48	1	10	19	28
38	47	7	9	18	27	29
46	6	8	17	26	35	37
5	14	16	25	34	36	45
13	15	24	33	42	44	4
21	23	32	41	43	3	12
22	31	40	49	2	11	20

FIG. 22.2 – *Carrés magiques d'ordres 3, 5 et 7*

Corrigé 43 *Nous donnons un programme (sans le corps de la procédure technique* `ecrireCentre`*). Les autres se trouvent dans le dossier* `livre/22_variables_E_S` *:*

```
from LibRobot import *

# Construire un carré magique d'ordre impair

# Lire au clavier l'ordre n (impair) du carré
def lireOrdre():
   write("Ordre du carré magique (impair), n = ")
   n = lireEntier()
   if n > 0 and n % 2 == 1:
      return n
   else:
      return lireOrdre()

# Placer le robot au coin inférieur gauche du carré : un
# pas au-dessus et à droite du coin de la fenêtre
def origine():
   eb(); ag(); translater(1, 1)

# Écrire le texte s centré sur la position du robot
# (procédure technique, voir le programme complet dans
# le dossier livre/22_variables_E_S)
def ecrireCentre(s):
   ...

# Afficher l'élément c au centre de la case (i, j)
def afficher(i, j, c):
   origine(); translater(2 * i - 1, 2 * j - 1)
   ecrireCentre(str(c))

n = lireOrdre()
# Dessiner la matrice dans la fenêtre du robot
lg(20); cacherRobot()
for k in range(n + 1):
   origine(); translater(2 * k, 0); pn(); avf(2 * n)
   origine(); translater(0, 2 * k); pe(); avf(2 * n)
# Remplir le carré magique
# Ligne i et colonne j de la case à remplir
i = (n + 1) // 2
j = n
afficher(i, j, 1)
for c in range(2, n * n + 1):
   if c % n != 1:
      i = i % n + 1
      j = j % n + 1
   else:
```

```
        j -= 1
    afficher(i, j, c)
st()
```

□

Addenda

Une variable sert à donner un *nom* à une mémoire qui conserve une *valeur* représentée selon un code déterminé par le *type* de la variable. Pour un codage binaire, on peut imaginer que la mémoire est une suite de lampes éteintes (pour représenter le bit 0) ou allumées (pour le bit 1). Dans beaucoup de langages, comme PASCAL, la valeur initiale d'une variable est *indéterminée* puisqu'elle dépend de l'état de la mémoire au moment du début de l'exécution du programme, ce qui est une source d'erreurs. En Python, une variable reçoit une valeur initiale dès sa création. L'instruction d'affectation permet de conserver une nouvelle valeur en modifiant physiquement l'état de cette mémoire. La nouvelle valeur est donnée par une expression (qui peut utiliser l'ancienne valeur). La notion informatique de variable est donc très différente de la notion mathématique homonyme. En mathématiques une variable désigne une valeur inconnue fixée une fois pour toutes, comme les constantes d'un programme Python dont on ne change jamais la valeur. Pour un mathématicien, une variable informatique serait la suite des valeurs qu'elle prend à sa création puis après chaque affectation.

On peut déclarer des variables *locales* à une procédure (ou fonction), en écrivant `x = ...` dans le corps de la procédure, comme la variable `n` de la fonction `lireOrdre` du programme `CarreMagique`. Dans ce cas la variable n'est utilisable que dans la suite du corps de la procédure. À l'exécution, elle est créée à chaque exécution du corps de la procédure. Pour une procédure récursive, il s'agit à chaque appel d'une nouvelle variable sans aucun lien avec les instances correspondant aux appels récursifs antérieurs.

Dans le corps de la procédure on peut également utiliser des variables *globales* dont la déclaration figure dans le programme, en dehors des procédures, comme `largeur` et `hauteur` dans le programme `CalculerTailleFenetreRobot` ou `i` et `j` dans le programme `CarreMagique`. Pour qu'une procédure puisse les modifier par affectation, il faut les nommer après le mot-clé **global** au début de son corps. Ces variables globales existent pendant toute la durée de l'exécution du programme. Par conséquent en cas d'appels récursifs d'une procédure, il s'agit toujours de la même variable globale.

Enfin des variables locales et globales peuvent porter le même nom (comme la variable `n` de la fonction `lireOrdre` et la variable `n` du programme `CarreMagique`). Dans ce cas c'est la variable locale qui est visible dans la procédure, la variable globale continuant à exister mais étant inaccessible. En Python, une variable est locale à une procédure dès qu'elle reçoit une valeur par affectation dans le corps de cette procédure (sauf si elle est déclarée **global**).

Bibliographie

[1] J. W. Backus, R.J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herick, R. A. Nelson, D. Sayre, P. B. Sheridan, H. Stern, I. Ziller, R. A. Hugues & R. Nutt. *The Fortran automatic coding system*, Proceedings Western Joint Computer Conference, Volume 11, pages 188–198, 1957.

[2] M. Barnsley, R. Devaney, B. Mandelbrot, H.-O. Peitgen, D. Saupe & R. Voss. *The science of fractal images*, Springer-Verlag, Berlin, 312 pages, 1988.

[3] C. Bertho. *Télégraphes et téléphones, de Valmy au microprocesseur*, Le Livre de Poche, Paris, 541 pages, 1981.

[4] P. Benoit. "Calcul, algèbre et marchandise", dans [40], pages 197–221, 1989.

[5] C. Böhm. "Macchina calcolatrice digitale a programma con programma preordinato fisso con tastiera algebrica ridotta atta a comporre formule mediante la combinazione dei singoli elementi simbolici", Dépôt de brevet N° 13567, Milan, 1^{er} octobre 1952, 26 pages.

[6] M. Boutet de Monvel. *Chansons de France pour les petits enfants*, Gautier-Languereau, Paris, 77 pages, 1981.

[7] Commission ministérielle de terminologie de l'informatique. *Glossaire des termes officiels de l'informatique*, AFNOR, Tour Europe, 92080 Paris La Défense Cedex 7, 21 pages, 1989 ($2^{\text{ème}}$ édition).

[8] S. Conchon & J.-C. Filliâtre. *Apprendre à programmer avec OCaml. Algorithmes et structures de données*, Eyrolles, Paris, 2014.

[9] P. Cousot & R. Cousot. "Abstract interpretation : a unified lattice model for static analysis of programs by construction or approximation of fixpoints", Conference Record of the 4th ACM Symposium on Principles of Programming Languages, Los Angeles, Californie, U.S.A., p. 238–252, 1977.

[10] P. Cousot. *Introduction à l'algorithmique numérique et à la programmation en Pascal*, McGraw-Hill, Paris, 621 pages, 1988.

[11] H. S. M. Coxeter, M. Emmer, R. Penrose & M. L. Teuber. *M. C. Escher : Art and Science*, North-Holland, Amsterdam, 402 pages, 1986.

[12] H. M. Cundy & A. R. Rollett. *Mathematical models*, Tarquin publications (Stradbroke, Diss, Norfolk, G.B.), 286 pages, 1981 (3ème édition).

[13] A. Danhauser. *Théorie de la musique (édition revue et corrigée par H. Rabaud)*, Éditions Henry Lemoine, Paris, 128 pages, 1929.

[14] S. W. Golomb. "A geometric proof of a famous identity", *Mathematical gazette*, Vol. 69, p. 198–200, 1965.

[15] B. Grünbaum & G. C. Shephard. *Tilings and patterns*, W. H. Freeman & Cie, New York, 700 pages, 1987.

[16] G. Guitel. *Histoire comparée des numérations écrites*, Flammarion, Paris, 857 pages, 1975.

[17] S. Horemis. *Optical and geometrical patterns and designs*, Dover, New York, 1970.

[18] K. E. Iverson. *A Programming Language*, J. Wiley & Sons, 1962.

[19] K. Jensen & N. Wirth. *Pascal, user manual and report*, Springer-Verlag (Heidelberg), 167 pages, 1976 (2ème édition).

[20] B. W. Kernighan & D. M. Ritchie. *The C programming language*, Prentice-Hall (Englewood Cliffs, New Jersey, U.S.A.), 228 pages, 1978.

[21] D. E. Knuth. *The art of computer programming, Volume 1 / Fundamental algorithms*, Addison-Wesley (Reading, Massachusetts, USA), 634 pages, 1973 (2ème édition).

[22] D. E. Knuth. *The TEXbook*, Addison-Wesley (Reading, Massachusetts, USA), 483 pages, 1987 (12ème édition).

[23] L. Lamport. *LATEX a document preparation system*, Addison-Wesley (Reading, Massachusetts, USA), 242 pages, 1986.

[24] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Rémy, K. Sivaramakrishnan & J. Vouillon. *The OCaml system : documentation and user's manual*, Inria, https://ocaml.org/manual/.

[25] P. Lévy. “L’invention de l’ordinateur”, dans [40], pages 515–535, 1989.

[26] J. L. Locher. *The world of M. C. Escher*, Abrams (New York, USA), 1971 ; traduction française : J. L. Locher, *Le monde de M. C. Escher*, Éditions du Chêne (Paris), 293 pages, 1986 (12ème édition).

[27] B. Mandelbrot. *Les objets fractals* suivi de *Survol du langage fractal*, Flammarion, Nouvelle collection scientifique (Paris), 1989 (3ème édition).

[28] J. McCarthy, P. W. Abrahams, D. J. Edwards, T. P. Hart & M. I. Levin. *LISP 1.5 Programmer’s manual*, MIT Press, 1965.

[29] P. Naur *et al.* (Eds.). “Revised report on the algorithmic language Algol 60”, *Communications of the Association for Computing Machinery*, volume 6, pages 1–17, 1963.

[30] F. C. Odds. “Spirolaterals”, *Mathematics teacher*, volume 66, pages 121–124, 1973.

[31] S. Papert. *Mindstorms : children, computers and powerful ideas*, Harvester Press, 1980.

[32] Python Software Foundation. *The Python Language Reference*, documentation officielle de Python 3, https://docs.python.org/3/reference/.

[33] Python Software Foundation. *Le tutoriel Python*, documentation officielle de Python 3 (traduction française), https://docs.python.org/fr/3/tutorial/.

[34] J. Ritter. “Babylone -1800”, dans [40], pages 17–37, 1989.

[35] H. Rutishauser. “Automatische Rechenplanfertigung bei programmgesteuerten Rechenmaschinen”, *Mitteilungen aus dem Inst. für angew. Math. an der ETH Zürich*, No. 3, Birkhäuser, Bâle, 1952, 45 pages.

[36] R. Sabatier. *Le livre des chansons de France*, Gallimard, Paris, 157 pages, 1984.

[37] R. Sabatier. *Deuxième livre des chansons de France et d’ailleurs*, Gallimard, Paris, 165 pages, 1986.

[38] C. Sabatier & R. Sabatier. *Troisième livre des chansons de France*, Gallimard, Paris, 165 pages, 1987.

[39] R. Sedgewick. *Algorithms*, Addison-Wesley, Reading, Massachusetts, U.S.A., 650 pages, 1988 (2ème édition), traduit en français par J.-M. Moreau : R. Sedgewick, *Algorithmes*, InterÉditions, Paris, 1990.

[40] M. Serres. *Éléments d’histoire des sciences*, Bordas, Paris, 576 pages, 1989.

[41] S. Thévenet, A. Garioud & N. Pitot. *Mathématiques CM2*, Bordas, Paris, 160 pages, 1986.

[42] H. Voderberg. "Zur Zerlegung der Umgebung eines ebenen Bereiches in kongruente", *Jahresber. Deutsch. Math.-Verein.*, volume 46, pages 229–231, 1936 ; "Zur Zerlegung der Ebene in kongruente Bereiche in Form einer Spirale", Ibid., volume 47, pages 159–160, 1937

[43] P. Weis & X. Leroy. *Le langage Caml*, InterÉditions, Paris, 1993 ; 2ème édition, Dunod, 1999.

[44] M. V. Wilkes, D. J. Wheeler & S. Gill. "The preparation of programs for an electronic digital computer : with special reference to the EDSAC and the use of a library of subroutines", Addison-Wesley, Cambridge, Massachusetts, 170 pages, 1951.

[45] L. B. Wilson & R. G. Clark. *Comparative programming languages*, Addison-Wesley publishing company, Reading, Massachusetts, 379 pages, 1988.

[46] N. Wirth. "The programming language Pascal", *Acta Informatica*, Vol. 1, Springer-Verlag (Heidelberg), pages 35–63, 1971.

Résumé des commandes du robot

Les commandes du $\mathcal{R}^{o}\!\!\smile^{o}t$ sont celles de la bibliothèque `LibRobot` (un programme commence par **from** `LibRobot` **import** `*`). Une commande s'écrit suivie de parenthèses, vides quand elle n'a pas de paramètre, comme `av()` ; les paramètres s'écrivent entre les parenthèses, séparés par des virgules, comme `ec(2)`, `tl(-3.5)` ou `deplacer(2, 3)`. Un nombre réel (de type `float`) s'écrit avec un point, comme `20.5`, un entier (de type `int`) sans point ; on peut donner un entier là où un réel est attendu, comme `lg(20)`. Une chaîne de caractères est de type `str`, un booléen de type `bool` (**True** ou **False**). Les fonctions `val`... rendent la valeur courante d'un réglage du $\mathcal{R}^{o}\!\!\smile^{o}t$.

Grille des points d'arrêt du $\mathcal{R}^{o}_{b}{}^{o}t$:

`dg()` dessine la grille des points d'arrêt du $\mathcal{R}^{o}_{b}{}^{o}t$ (le dessin de la grille peut être interrompu par Esc, § ou #).

`lg(`n`)` définit la nouvelle longueur n (un réel $n \geq 0$) des côtés d'un carreau de la grille (12×12 pixels par défaut).

`lgX(`n`)` définit la nouvelle longueur n (réelle) du côté horizontal (X) d'un carreau de la grille (12 pixels par défaut).

`lgY(`n`)` définit la nouvelle longueur n (réelle) du côté vertical (Y) d'un carreau de la grille (12 pixels par défaut).

`lgRT(`r`, `t`)` définit les nouvelles longueurs des côtés d'un carreau de la grille par les coordonnées polaires r et t (deux réels : la longueur $r \geq 0$ de la diagonale du carreau est exprimée en pixels et l'angle t que fait cette diagonale avec le côté horizontal est exprimé en degrés, en général $0 \leq t \leq 90$).

`valLgX()` est égal à la valeur (de type `float`) de la longueur du côté horizontal (X) d'un carreau de la grille (comptée en pixels).

`valLgY()` est égal à la valeur (de type `float`) de la longueur du côté vertical (Y) d'un carreau de la grille (comptée en pixels).

`valLgR()` est égal à la valeur (de type `float`) de la longueur de la diagonale (R) d'un carreau de la grille (comptée en pixels).

`valLgT()` est égal à la valeur (de type `float`) de l'angle (T) de la diagonale d'un carreau de la grille avec son côté horizontal (compté en degrés).

`valX()` est égal à la valeur (de type `float`) de l'abscisse absolue (X) du $\mathcal{R}^{o}_{b}{}^{o}t$ (comptée en pixels à partir du bord gauche de la fenêtre).

`valY()` est égal à la valeur (de type `float`) de l'ordonnée absolue (Y) du $\mathcal{R}^{o}_{b}{}^{o}t$ (comptée en pixels à partir du bas de la fenêtre), de sorte qu'après `x = round(valX())` et `y = round(valY())`, plus tard `lc() ; tracer(x, y)` replace le $\mathcal{R}^{o}_{b}{}^{o}t$ à cette place sur l'écran.

Pivotements relatifs sur place :

`phtd()` fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place d'un huitième de tour à droite.

`pqtd()` fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place d'un quart de tour à droite.

`p3htd()` fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place de 3 huitièmes de tour à droite.

`phtg()` fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place d'un huitième de tour à gauche.

`pqtg()` fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place d'un quart de tour à gauche.

`p3htg()` fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place de 3 huitièmes de tour à gauche.

`pdt()` fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place d'un demi-tour.

Pivotements absolus sur place :	
`pn()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers le nord (N).
`pne()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers le nord-est (NE).
`pe()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers l'est (E).
`pse()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers le sud-est (SE).
`ps()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers le sud (S).
`psw()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers le sud-ouest (SW).
`pw()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers l'ouest (W).
`pnw()`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ sur place vers le nord-ouest (NW).
`pivoter(`o`)`	fait pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ dans l'orientation indiquée par la valeur de l'entier o (entre 0 et 7) égale à `nord` = 0, `nordEst` = 1, `est` = 2, `sudEst` = 3, `sud` = 4, `sudOuest` = 5, `ouest` = 6 ou `nordOuest` = 7.
`rt(`t`)`	change le repère et l'orientation du $\mathcal{R}^{o}_{b}{}^{o}t$ pour qu'il effectue une rotation sur place d'un angle de t degrés (t réel, dans le sens trigonométrique si $t > 0$ et dans le sens des aiguilles d'une montre si $t < 0$, comme `rt(15)` ou `rt(-30)`).
`valOrientation()`	est égal à la valeur (de type `int`) de l'orientation du $\mathcal{R}^{o}_{b}{}^{o}t$ codée par `nord` = 0, `nordEst` = 1, `est` = 2, `sudEst` = 3, `sud` = 4, `sudOuest` = 5, `ouest` = 6 et `nordOuest` = 7. Les directions `nord`, `est`, `sud` et `ouest` sont parallèles aux bords de l'écran, le `nord` en haut. Les directions `nordEst`, `sudEst`, `sudOuest` et `nordOuest` correspondent à l'orientation des diagonales d'un carreau de la grille.
Crayon :	
`bc()`	baisse le crayon pour dessiner lors des déplacements.
`lc()`	lève le crayon pour éviter de dessiner lors des déplacements.
`dc()`	dessine une petite croix (de l'épaisseur du crayon), sans déplacer le $\mathcal{R}^{o}_{b}{}^{o}t$.
`dp()`	dessine un point (de l'épaisseur du crayon), sans déplacer le $\mathcal{R}^{o}_{b}{}^{o}t$.
`ec(`n`)`	définit la nouvelle épaisseur n (un entier) du crayon (1 pixel par défaut).
`valEc()`	est égal à la valeur (de type `int`) de l'épaisseur du crayon.
`valBc()`	est égal à la valeur booléenne **True** (vrai) si et seulement si le crayon est baissé, sinon à **False** (faux).

Déplacements (avec ou sans tracé selon que le crayon est levé ou baissé) :	
`av()`	fait avancer le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ tout droit jusqu'au prochain point d'arrêt sur la grille.
`vd()`	fait tourner le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ par un virage d'un quart de cercle à droite.
`vg()`	fait tourner le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ par un virage d'un quart de cercle à gauche.
`avf(`n`)`	fait avancer le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ tout droit n fois (n entier ; sans rien faire si $n = 0$ et en reculant si $n < 0$, comme `avf(-2)`).
`tl(`l`)`	change le repère (mais pas l'angle des directions NE-E) et fait avancer le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ par une translation de l pixels (l réel) dans la direction où il se trouve (si $l > 0$) ou dans la direction opposée (si $l < 0$).
`tracerReel(`x`, `y`)`	trace un segment de droite (si le crayon est baissé) de la position courante du $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ au point de coordonnées réelles (x,y) du repère cartésien de la grille (voir `deplacer`) et déplace le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ en ce point.
`tracer(`x`, `y`)`	fait de même jusqu'au point de coordonnées entières (x,y) comptées en pixels à partir du coin inférieur gauche de la fenêtre (comme `valX()` et `valY()`).
`valLgTl()`	est égal à la valeur (de type `float`) de la longueur du segment tracé par la dernière translation par `av()` ou `tl()` (comptée en pixels).

Positionnement au centre et sur les bords du cadre :	
`ce()`	place le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ au centre de *l'écran*, qui est aussi *le centre de la grille*, sans changer l'orientation du $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ et sans rien dessiner.
`eh()`	place le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ en haut du cadre, sans changer sa position horizontale, ni son orientation et sans rien dessiner.
`eb()`	place le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ en bas du cadre, sans changer sa position horizontale, ni son orientation et sans rien dessiner.
`ag()`	place le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ à gauche du cadre, sans changer sa position verticale, ni son orientation et sans rien dessiner.
`ad()`	place le $\mathcal{R}^{o}\!\underset{\smile}{b}^{o}t$ à droite du cadre, sans changer sa position verticale, ni son orientation et sans rien dessiner.

Repère cartésien :

`translater(`x`, `y`)` translate le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ de x carreaux de la grille horizontalement (à droite si $x > 0$ et à gauche si $x < 0$) et y carreaux de la grille verticalement (en haut si $y > 0$ et en bas si $y < 0$), sans changer son orientation et sans rien dessiner (x et y entiers).

`deplacer(`x`, `y`)` déplace le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ au point de coordonnées entières (x,y) sur la grille, sans changer son orientation ni rien dessiner, l'origine du repère cartésien étant placée au centre de la grille.

`deplacerReel(`x`, `y`)` fait de même avec des coordonnées réelles x et y (de type `float`), comme `deplacerReel(1.5, -2.5)`.

Test booléen de l'orientation du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$:

`tn()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté au nord (N).

`tne()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté au nord-est (NE).

`te()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté à l'est (E).

`tse()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté au sud-est (SE).

`ts()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté au sud (S).

`tsw()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté au sud-ouest (SW).

`tw()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté à l'ouest (W).

`tnw()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est orienté au nord-ouest (NW).

Test booléen de la position du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$:

`tb()` teste si le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ est au bord du cadre (un déplacement du $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ en avant par `av()` le faisant sortir du cadre).

Temps :

`heure()` rend trois entiers : l'heure (entre 0 et 23), la minute (entre 0 et 59) et la seconde (entre 0 et 59) présentes, de sorte qu'après `h, mi, s = heure()` les variables `h`, `mi` et `s` contiennent ces trois valeurs.

`date()` rend de même trois entiers, le jour (entre 1 et 31), le mois (entre 1 et 12) et l'année courante, comme dans `j, mo, a = date()`.

`delai(`t`)` attend pendant un délai de t secondes (t de type `float`, comme `delai(0.5)`).

`tickCount()` est égal au nombre entier de soixantièmes de seconde écoulés (pour mesurer une durée par différence de deux valeurs).

Interruption de l'exécution du programme :

Esc, § ou # tapée pendant que le $\mathcal{R}\!\!\mathcal{o}\!\!\mathcal{b}\!\!\mathcal{o}t$ se déplace (par `av()`, `vd()`, `vg()`, ...) ou s'exécute pas à pas, l'une de ces touches arrête le programme et ferme la fenêtre.

Mise au point du programme :

`st()` stoppe le $\mathcal{R}^{o}\!\!\smile^{o}t$ et rappelle dans la ligne du bas de la fenêtre les touches utilisables : [+] augmente et [−] diminue la vitesse du $\mathcal{R}^{o}\!\!\smile^{o}t$ (entre 0 et 10, la vitesse 0 faisant passer en mode pas à pas), [Esc] ou [#] ferme la fenêtre et termine le programme, toute autre touche fait redessiner tout le dessin depuis le début, à la vitesse choisie. Un programme qui se termine sans `st()` attend que l'on tape une touche ou que l'on clique avant de fermer la fenêtre.

`pp()` exécute la suite du programme en mode pas à pas, à la vitesse 0 : le $\mathcal{R}^{o}\!\!\smile^{o}t$ attend que l'on tape une touche avant chaque commande ([Esc], [§] ou [#] arrêtant le programme).

`ex()` exécute la suite du programme en mode exécution normale, à la vitesse 10.

`vt(`v`)` exécute la suite du programme à la vitesse v (un entier ; en pas à pas si $v \leq 0$, avec un maximum de 10 si $v > 10$).

`montrerRobot()` rend le $\mathcal{R}^{o}\!\!\smile^{o}t$ visible sur l'écran pendant le dessin (option par défaut).

`cacherRobot()` rend le $\mathcal{R}^{o}\!\!\smile^{o}t$ invisible sur l'écran pendant le dessin.

`interdireSorties()` interdit les sorties du $\mathcal{R}^{o}\!\!\smile^{o}t$ en dehors du cadre dans la suite du programme (option par défaut : une commande qui ferait sortir le $\mathcal{R}^{o}\!\!\smile^{o}t$ du cadre arrête alors le programme sur une erreur, en écrivant dans le terminal, par exemple, `Erreur ligne 4 de Programme.py : le robot sortirait du cadre (av)` ; la fenêtre reste affichée).

`autoriserSorties()` autorise les sorties du $\mathcal{R}^{o}\!\!\smile^{o}t$ en dehors du cadre dans la suite de l'exécution du programme.

`valVt()` est égal à la valeur (de type `int`) de la vitesse du $\mathcal{R}^{o}\!\!\smile^{o}t$ (entre 0 (mode pas à pas) et 10).

`valVisible()` est égal à la valeur booléenne **True** (respectivement **False**) si le $\mathcal{R}^{o}\!\!\smile^{o}t$ est visible (respectivement invisible) sur l'écran.

`valAutoriserSorties()` est égal à la valeur booléenne **True** (respectivement **False**) si les sorties en dehors du cadre sont autorisées (respectivement interdites).

Couleur :

Les couleurs sont des entiers : `blanc` = 0, `jaune` = 1, `bleu` = 2, `violet` = 3, `vert` = 4, `rouge` = 5, `indigo` = 6 et `noir` = 7.

`cc(`c`)` définit la couleur c du crayon, comme `cc(rouge)`.

`peindre()` peint de la couleur du crayon la région d'une même couleur (blanche en général), limitée par les traits du dessin, où se trouve le $\mathcal{R}^{o}_{b}{}^{o}t$.

`cf(`c`)` efface complètement le dessin puis remplit l'écran de la couleur de fond c. Le $\mathcal{R}^{o}_{b}{}^{o}t$ est placé au centre de l'écran dans les conditions initiales (visible, tourné vers le nord, sorties hors du cadre interdites, crayon noir d'épaisseur 1 baissé et exécution normale à la vitesse 10) ; les dimensions de la grille ne changent pas.

`valCf()` est égal à la valeur (de type `int`) de la couleur du fond.

`valCc()` est égal à la valeur (de type `int`) de la couleur du crayon.

`ecranNoirEtBlanc()` est égal à **True** si et seulement si l'écran est noir et blanc, sinon à **False** (c'est toujours **False** sur les écrans actuels).

Écriture dans la fenêtre de dessin du $\mathcal{R}^{o}_{b}{}^{o}t$:

`ecrire(`c`)` écrit la chaîne de caractères c (entre guillemets `""` et `""`, où tout guillemet doit être précédé d'une barre oblique inverse : `\"`, comme `ecrire("Bonjour")`) à droite du $\mathcal{R}^{o}_{b}{}^{o}t$ (ou à sa gauche s'il est tourné vers l'ouest), sans changer sa position ni celle de son crayon.

`entierEnChaine(`i`)` est la chaîne de caractères représentant l'entier i en base 10 (comme `str(`i`)`).

`reelEnChaine(`r`)` est la chaîne de caractères représentant le réel r sous la forme d.$dddd$`e`$\pm ee$ où d est un chiffre de la mantisse, e est un chiffre de l'exposant et `e` se lit « fois 10 à la puissance ».

`booleenEnChaine(`b`)` est la chaîne de caractères (`"Vrai"` ou `"Faux"`) représentant le booléen b.

`concat([`c_1`, `c_2`, ..., `c_n`])` est la chaîne de caractères obtenue en concaténant les chaînes c_1, c_2, …, c_n les unes derrière les autres (comme c_1 `+` c_2 `+` … `+` c_n).

`policeTexte(`p`)` choisit la police p du texte (`courier`, `times`, `helvetica`, …).

`tailleTexte(`t`)` choisit la taille t (un entier) du texte.

`styleTexte(`s`)` choisit le style du texte (`normal`, `gras`, `italique`, `souligne`, …).

Ces trois commandes sont sans effet visible : la police de la fenêtre du $\mathcal{R}^{o}_{b}{}^{o}t$ ne peut pas être changée.

`valPoliceTexte()` valeur (de type `str`) de la police courante du texte.

`valTailleTexte()` valeur (de type `int`) de la taille courante du texte.

Messages affichés dans la ligne du bas de la fenêtre du $\mathcal{R}^o\underline{b}^o t$:
`message(`c`)` efface le message de la ligne du bas de la fenêtre du $\mathcal{R}^o\underline{b}^o t$ puis y écrit la chaîne de caractères c.
`marquerUnePause()` marque une pause (en faisant clignoter le symbole ↩ dans la ligne des messages) en attendant que le bouton de la souris ou qu'une touche du clavier soit enfoncé.
`effacerMessage()` efface le message de la ligne du bas de la fenêtre du $\mathcal{R}^o\underline{b}^o t$.
`lireCar()` attend (en faisant clignoter le symbole ↩ dans la ligne des messages) qu'une touche du clavier ou que le bouton de la souris soit enfoncé, puis lit et rend le caractère (une chaîne d'un seul caractère, de type `str`) correspondant à la touche enfoncée (ou le caractère `'0'` si le bouton de la souris a été enfoncé).
`mementoCommandesClavier` est une chaîne de caractères rappelant les touches de pilotage interactif du $\mathcal{R}^o\underline{b}^o t$ (voir `interaction`), à afficher par `message(mementoCommandesClavier)`.
`valMessage()` est égal à la valeur (de type `str`) du message inscrit dans la ligne du bas de la fenêtre du $\mathcal{R}^o\underline{b}^o t$ ou à la chaîne vide `""` s'il n'y a aucun message.

Lecture et écriture de texte :

Le texte écrit par `write` et `writeln` s'affiche dans le terminal. Quand le programme attend une réponse (`lireEntier()`, ...), la question (la ligne en cours, précédée des deux dernières lignes écrites) s'affiche dans la ligne du bas de la fenêtre du *Robot*. La réponse se tape dans la fenêtre du *Robot*, où elle s'affiche à la suite de la question (⟵ pour corriger, ↩ pour terminer), ou bien dans le terminal.

`write(`c_1`, ..., `c_n`)` écrit les chaînes de caractères (ou les nombres) c_1, ..., c_n les uns à la suite des autres, comme `write("N = ", n)`.

`writeln(`c_1`, ..., `c_n`)` fait de même puis passe à la ligne (`writeln()` passe seulement à la ligne).

`lireEntier()` lit la réponse de l'utilisateur, terminée par ↩, et rend l'entier (de type `int`) qu'elle représente. Si la réponse n'est pas un nombre entier, la question `"Entrer un nombre entier : "` est posée à nouveau. Sur un clavier français, les chiffres peuvent être tapés sans la touche majuscule.

`lireReel()` fait de même pour lire un nombre réel (de type `float`).

`lireChaine()` lit la réponse de l'utilisateur, terminée par ↩, et la rend comme une chaîne de caractères (de type `str`).

`lireCaractere()` fait de même et rend le premier caractère de la réponse (une chaîne d'un seul caractère).

`readln()` attend que l'utilisateur tape ↩ (ce qu'il a tapé avant est ignoré).

`effacerTexte()` efface le texte du terminal.

Fonctions utiles :

`pi` est le réel $\pi = 3.14159\ldots$ (de type `float`).

`round(`x`)` est l'entier le plus proche du réel x, comme `round(2.5)` $= 3$ et `round(-2.5)` $= -3$ (la bibliothèque remplace le `round` de Python, qui arrondit les moitiés au nombre pair : 2 pour 2.5).

`random()` est un entier tiré au hasard entre -32767 et 32767.

Sons :

`bip()` émet un son bref.

`biiip()` émet un son long.

Musique :

`metronome(`n`)` règle le métronome à n noires par minute (`largo` = 50, `lento` = 56, `larghetto` = 60, `adagio` = 70, `andante` = 85, `andantino` = 95, `moderato` = 110, `allegretto` = 115, `allegro` = 130, `vivace` = 150, `presto` = 175, `prestissimo` = 200 ; 100 par défaut).

`nuance(`n`)` règle la nuance n (entre 1 et 9 : `pianississimo` = 1, `pianissimo` = 2, `piano` = 3, `mezza_voce` = 4, `mezzo_piano` = 4, `mezzo_forte` = 6, `forte` = 7, `fortissimo` = 8, `fortississimo` = 9).

`note(`h`,` d`)` joue une note de hauteur $h = o * \texttt{octave} + n + a$ (un entier) où $o = -3, -2, \ldots, 3$ est l'octave de la note (0 pour l'octave du do du milieu du piano, `octave` = 12), n = `do0` ou `d0` ou `ut` = 0, `re` = 2, `mi` = 4, `fa` = 5, `sol` = 7, `la` = 9, `si` = 11 est le nom de la note, a = `bemol` = -1, `becarre` = 0, `diese` = 1 est l'altération de la note et d (un réel) est la durée de la note, comptée en noires : `ronde` = 4., `ronde_pointee` = 6., `blanche` = 2., `blanche_pointee` = 3., `blanche_en_triolet` = 4/3, `noire` = 1., `noire_pointee` = 1.5, `noire_en_triolet` = 2/3, `croche` = 0.5, `croche_pointee` = 0.75, `croche_en_triolet` = 1/3, `double_croche` = 0.25, `double_croche_pointee` = 0.375 ou `triple_croche` = 0.125, comme `note(octave + sol + diese, croche)`.

`silence(`d`)` marque un silence de durée d = `pause` = 4., `demi_pause_pointee` = 3., `demi_pause` = 2., `soupir_pointe` = 1.5, `soupir` = 1., `demi_soupir_pointe` = 0.75, `demi_soupir` = 0.5, `quart_de_soupir` = 0.25 ou `huitieme_de_soupir` = 0.125.

Interaction avec l'utilisateur du $\mathcal{R}^{o}_{b}{}^{o}t$:

`interaction(coordonnees, pivotements, dessin, finsouris, sauver)`

efface l'écran puis laisse l'utilisateur déplacer le $\mathcal{R}^{o}_{b}{}^{o}t$ avec la souris, en cliquant sur un point de la grille (si `finsouris` est **True** : de carreau en carreau, bouton relâché) ou avec les flèches du clavier, et éventuellement le piloter au clavier (les cinq paramètres sont des booléens, **True** ou **False**). L'interaction se termine par la touche ⏎ (ou en enfonçant le bouton de la souris si `finsouris` est **True**) ; le programme s'arrête alors comme avec `st()`.

– Si `coordonnees` est **True**, les coordonnées du $\mathcal{R}^{o}_{b}{}^{o}t$ sont affichées pendant l'interaction.

– Si `pivotements` est **True**, l'utilisateur peut faire pivoter le $\mathcal{R}^{o}_{b}{}^{o}t$ avec les touches – (`pqtg`), + (`pqtd`), g (`phtg`) et d (`phtd`).

– Si `pivotements` et `dessin` sont **True**, il peut aussi dessiner avec les touches a (`av`), ((`vg`),) (`vd`), l (`lc`), b (`bc`), p (`dp`), c (`dc`), . (`dg`), * (changement de la couleur du crayon) et w (`peindre`), en annulant la dernière commande avec ←.

– Si `dessin` et `sauver` sont **True**, le programme demande à la fin s'il faut sauver le dessin : en répondant y, on obtient un programme `mon_dessin.py` dont l'exécution reproduit le dessin.

`x, y, c = interactionTP(coordonnees, pivotements, dessin, finsouris, sauver)`

laisse l'utilisateur déplacer le $\mathcal{R}^{o}_{b}{}^{o}t$ avec la souris (ou les flèches) et le piloter au clavier comme `interaction` (sauf *, w et ←), mais sans effacer l'écran, et continue ensuite le programme. L'interaction se termine par la touche ⏎, par Esc, par toute touche qui n'est pas une commande du $\mathcal{R}^{o}_{b}{}^{o}t$ ou, si `finsouris` est **True**, en enfonçant le bouton de la souris. À la fin, `interactionTP` rend trois valeurs, mises ici dans les variables `x`, `y` et `c` : les entiers `x` et `y` sont le déplacement horizontal et vertical du $\mathcal{R}^{o}_{b}{}^{o}t$ compté en carreaux de la grille, et `c` est la dernière touche tapée (un espace `' '` si l'interaction s'est terminée par un clic). Le paramètre `sauver` est sans effet.

Index

Abscisse, 161
Abstraction, 68
Affectation, 280, 292
Appel de procédure, 61, 65, 154
Avertissement, 49

Barre d'espacement, 39
Bibliothèque, 38
Bogue, 29
Booléen, 171
Boucle "for", 76, 77
Boucle "while", 196, 203
break, 203
Bug, 30

Caractère, 209
Chaîne de caractères, 210
Chaîne de documentation, 42
Code ASCII, 209
Coller, 58
Commentaire, 42
Compteur de boucle, 76
Compatibilité de type, 280
Compilateur, 55
Compilation, 55
Compteur de boucle, 79
Concaténation, 211
Conjonction, 184
Conversion, 280
Coordonnées cartésiennes, 161, 162
Coordonnées polaires, 235
Copier, 58
Corps de boucle, 76, 78, 196
Corps de procédure, 61, 65

Disjonction, 184
Déboguage, 29
Déboguer, 29
Déclaration, 137
Déclaration de constantes, 137
Déclaration de procédure, 61, 62, 154
Déclaration de variable, 279
Déclaration locale, 281

Entier, 226
Erreur, 20
Erreur de programmation, 44, 48
Erreur logique, 48
Erreur syntaxique, 37, 44
Expression booléenne, 171, 186, 196
Expression entière, 97, 108, 233
Expression rationnelle (réelle), 229, 233
Exécution, 31, 46
Exécution (interruption), 46
Exécution en pas à pas, 50

Faux, 171
float, 228
Fonction, 64, 276
Fonction booléenne, 186

global, 283

Grille du *Robot*, 14

Identificateur, 61, 64, 154, 159, 262
Identificateur prédéfini, 233
Identificateur réservé, 233
Indentation, 63, 78
Infographie, 11
Interprétation, 44
Interpréter, 31
Interpréteur, 26, 30, 37, 44
Interpréteur de commandes, 53
Interruption de l'exécution, 46
Invariant, 69, 84, 258
Itération, 76

Langage de commande, 26
Langage de programmation, 29
Langage machine, 31, 44

Mode conversationnel, 29
Mode interactif, 26, 29
Monomorphe, 284

Nombre flottant, 119
Notation scientifique, 232
Numération binaire, 207, 225
Numération décimale, 206
Négation, 184

Ordinateur, 29
Ordonnée, 161

Paramètre, 111
Paramètre effectif, 154
Paramètre formel, 154
Parité, 181
Pile, 260
Pixel, 35
Polymorphe, 284
Priorité des opérateurs, 193
Procédure, 60, 68, 262
Programmation, 29
Programme, 29, 137
Programme d'application, 53

Rationnel, 227
Repère cartésien, 161
return, 276
Rotation, 236
Règle de dessin, 5
Règle de syntaxe, 37
Récurrence, 253
Récursif, 253
Réel, 230

Sens trigonométrique, 236
Spécification, 10, 69
Structure de blocs, 159
Système d'exploitation, 53
Séquence, 24

Test, 172–174, 178, 186
Touche d'effacement, 20
Touche d'échappement, 46
Translation, 238
Typage dynamique, 228
Type, 227

Variable, 77, 279, 292
Variable entière, 77
Variable globale, 282
Visibilité, 159
Vrai, 171

Les auteurs de cet ouvrage utilisent le dessin comme support intuitif pour présenter sous une forme simple et originale les notions de base de la programmation en Python [structuration des programmes en procédures, itérations bornées (`for`), expressions entières, déclarations de constantes, de paramètres (passés par valeur), tests (`if,elif`), expressions booléennes, itérations non bornées (`while`), codages de l'information, expressions rationnelles, récursivité, déclarations de variables de types simples et affectations].

Du pilotage d'un "robot" qui se déplace sur l'écran pour exécuter des dessins stylisés jusqu'à la manipulation de concepts plus abstraits comme la récursivité, chacun, de l'écolier à l'adulte débutant, pourra progresser à son propre rythme jusqu'à développer des applications personnelles.

Les auteurs : Laurent Cousot est collégien ; Patrick Cousot est docteur ès sciences mathématiques, professeur d'informatique à l'École polytechnique, directeur du laboratoire d'informatique de l'École polytechnique ; Radhia Cousot est docteur ès sciences mathématiques, chargée de recherche au CNRS en informatique ; Thibault Cousot est écolier.